



阿里系和1号店资深技术专家撰写，Java并发编程领域的扛鼎之作，内容在InfoQ等社群得到高度认可

从JDK源码、JVM、CPU等多角度全面剖析与讲解Java并发编程的框架、原理和核心技术



Java

并发编程的艺术

The Art of Java Concurrency Programming

方腾飞 魏鹏 程晓明 著



机械工业出版社
China Machine Press

Java核心技术系列

Java并发编程的艺术

方腾飞 魏鹏 程晓明 著

ISBN:978-7-111-50824-3

本书纸版由机械工业出版社于2015年出版, 电子版由华章分社(北京华章图文信息有限公司, 北京奥维博世图书发行有限公司)全球范围内制作与发行。

版权所有, 侵权必究

客服热线:+ 86-10-68995265

客服信箱:service@bbbvip.com

官方网址:www.hzmedia.com.cn

新浪微博 @华章数媒

腾讯微博 @yanfabook

微信公众号 华章电子书(微信号:hzebook)

前言

为什么要写这本书

记得第一次写并发编程的文章时还是在2012年，当时花了几个星期的时间写了一篇文章《深入分析volatile的实现原理》，准备在自己的博客中发表。在同事建法的建议下，怀着试一试的心态投向了InfoQ，庆幸的是半小时后得到InfoQ主编采纳的回复，高兴之情无以言表。这也是我第一次在专业媒体上发表文章，而后在InfoQ编辑张龙的不断鼓励和支持下，我陆续在InfoQ发表了几篇与并发编程相关的文章，于是便形成了“聊聊并发”专栏。在这个专栏的写作过程中，我得到快速的成长和非常多的帮助，在此非常感谢InfoQ的编辑们。2013年，华章的福川兄找到我，问有没有兴趣写一本书，当时觉得自己资历尚浅，婉言拒绝了。后来和福川兄一直保持联系，最后允许我花两年的时间来完成本书，所以答应了下来。由于并发编程领域的技术点非常多且深，所以陆续又邀请了同事魏鹏和朋友晓明一起参与到本书的编写当中。

写本书的过程也是对自己研究和掌握的技术点进行整理的过程，希望本书能帮助读者快速掌握并发编程技术。

本书一共11章，由三名作者共同编写完成，其中第3章和第10章节由程晓明编写，第4章和第5章由魏鹏编写，其他7章由方腾飞编写。

本书特色

本书结合JDK的源码介绍了Java并发框架、线程池的实现原理，帮助读者做到知其所以然。

本书对原理的剖析不仅仅局限于Java层面，而是深入到JVM，甚至CPU层面来进行讲解，帮助读者从更底层看并发技术。

本书结合线上应用,给出了一些并发编程实战技巧,以及线上处理并发问题的步骤和思路。

读者对象

- Java开发工程师
- 架构师
- 并发编程爱好者
- 开设相关课程的大专院校师生

如何阅读本书

阅读本书之前,你必须有一定的Java基础和开发经验,最好还有一定的并发编程基础。如果你是一名并发编程初学者,建议按照顺序阅读本书,并按照书中的例子进行编码和实战。如果你有一定的并发编程经验,可以把本书当做一个手册,直接看需要学习的章节。以下是各章节的基本介绍。

第1章介绍Java并发编程的挑战,向读者说明进入并发编程的世界可能会遇到哪些问题,以及如何解决。

第2章介绍Java并发编程的底层实现原理,介绍在CPU和JVM这个层面是如何帮助Java实现并发编程的。

第3章介绍深入介绍了Java的内存模型。Java线程之间的通信对程序员完全透明,内存可见性问题很容易困扰Java程序员,本章试图揭开Java内存模型的神秘面纱。

第4章从介绍多线程技术带来的好处开始,讲述了如何启动和终止线程以及线程的状态,

详细阐述了多线程之间进行通信的基本方式和等待/通知经典范式。

第5章介绍Java并发包中与锁相关的API和组件，以及这些API和组件的使用方式与实现细节。

第6章介绍了Java中的大部分并发容器，并深入剖析其实现原理，让读者领略大师的设计技巧。

第7章介绍了Java中的原子操作类，并给出一些实例。

第8章介绍了Java中提供的并发工具类，这是并发编程中的瑞士军刀。

第9章介绍了Java中的线程池实现原理和使用建议。

第10章介绍了Executor框架的整体结构和成员组件。

第11章介绍几个并发编程的实战，以及排查并发编程造成问题的方法。

勘误和支持

由于笔者的水平有限，编写时间仓促，书中难免会出现一些错误或者不准确的地方，恳请读者批评指正。为此，特意创建一个在线支持与应急方案的站点<http://ifeve.com/book/>。你可以将书中的错误发布在勘误表页面中，同时如果你遇到任何问题，也可以访问Q&A页面，我将尽量在线上为读者提供最满意的解答。书中的全部源文件除可以从华章网站^[1]下载外，还可以从并发编程网站^[2]下载，我也会将相应的功能更新及时发布出来。如果你有更多的宝贵意见，也欢迎发送邮件至邮箱tengfei@ifeve.com，期待能够得到你的真挚反馈。

致谢

感谢机械工业出版社华章公司的杨福川、高婧雅、孙海亮，在这一年多的时间中始终支持

我的写作，你们的鼓励和帮助引导我顺利完成全部书稿。

感谢方正电子的刘老师，是他带我进入了面向对象的世界。

感谢我的主管朱老板，他在工作和生活上给予我很多的帮助和支持，还经常激励我完成本书编写。

最后感谢我的爸妈、岳父母和老婆，感谢你们的支持，并时时刻刻为我灌输信心和力量！

谨以此书献给我的儿子方熙皓，希望他能健康成长，以及众多热爱并发编程的朋友们，希望你们能快乐工作，认真生活！

方腾飞

[1] 参见华章网站www.hzbook.com。——编辑注

[2] <http://ifeve.com>。——编者注

第1章 并发编程的挑战

并发编程的目的是为了让程序运行得更快，但是，并不是启动更多的线程就能让程序最大限度地并发执行。在进行并发编程时，如果希望通过多线程执行任务让程序运行得更快，会面临非常多的挑战，比如上下文切换的问题、死锁的问题，以及受限于硬件和软件的资源限制问题，本章会介绍几种并发编程的挑战以及解决方案。

1.1 上下文切换

即使是单核处理器也支持多线程执行代码, CPU通过给每个线程分配CPU时间片来实现这个机制。时间片是CPU分配给各个线程的时间, 因为时间片非常短, 所以CPU通过不停地切换线程执行, 让我们感觉多个线程是同时执行的, 时间片一般是几十毫秒(ms)。

CPU通过时间片分配算法来循环执行任务, 当前任务执行一个时间片后会切换到下一个任务。但是, 在切换前会保存上一个任务的状态, 以便下次切换回这个任务时, 可以再加载这个任务的状态。所以任务从保存到再加载的过程就是一次上下文切换。

这就像我们同时读两本书, 当我们在读一本英文的技术书时, 发现某个单词不认识, 于是便打开中英文字典, 但是在放下英文技术书之前, 大脑必须先记住这本书读到了多少页的第多少行, 等查完单词之后, 能够继续读这本书。这样的切换是会影响读书效率的, 同样上下文切换也会影响多线程的执行速度。

1.1.1 多线程一定快吗

下面的代码演示串行和并发执行并累加操作的时间，请分析：下面的代码并发执行一定比串行执行快吗？

```
public class ConcurrencyTest {
    private static final long count = 100001;
    public static void main(String[] args) throws InterruptedException {
        concurrency();
        serial();
    }
    private static void concurrency() throws InterruptedException {
        long start = System.currentTimeMillis();
        Thread thread = new Thread(new Runnable() {
            @Override
            public void run() {
                int a = 0;
                for (long i = 0; i < count; i++) {
                    a += 5;
                }
            }
        });
        thread.start();
        int b = 0;
        for (long i = 0; i < count; i++) {
            b--;
        }
        long time = System.currentTimeMillis() - start;
        thread.join();
        System.out.println("concurrency :" + time+"ms,b="+b);
    }
    private static void serial() {
        long start = System.currentTimeMillis();
        int a = 0;
        for (long i = 0; i < count; i++) {
            a += 5;
        }
        int b = 0;
        for (long i = 0; i < count; i++) {
            b--;
        }
        long time = System.currentTimeMillis() - start;
        System.out.println("serial:" + time+"ms,b="+b+",a="+a);
    }
}
```

上述问题的答案是“不一定”，测试结果如表1-1所示。

表1-1 测试结果

循环次数	串行执行耗时 /ms	并发执行耗时	并发比串行快多少
1 亿	130	77	约 1 倍
1 千万	18	9	约 1 倍
1 百万	5	5	差不多
10 万	4	3	慢
1 万	0	1	慢

从表1-1可以发现，当并发执行累加操作不超过百万次时，速度会比串行执行累加操作要慢。那么，为什么并发执行的速度会比串行慢呢？这是因为线程有创建和上下文切换的开销。

1.1.2 测试上下文切换次数和时长

下面我们来看看有什么工具可以度量上下文切换带来的消耗。

- 使用Lmbench3^[1]可以测量上下文切换的时长。
- 使用vmstat可以测量上下文切换的次数。

Linux

下面是利用vmstat测量上下文切换次数的示例。

```
$ vmstat 1
procs -----memory----- --swap-- -----io----- --system-- -----cpu-----
 r  b    swpd   free   buff   cache  si   so    bi    bo    in   cs  us  sy  id  wa  st
 0  0        0 127876 398928 2297092    0    0     0     4     2     2   0   0  99   0   0
 0  0        0 127868 398928 2297092    0    0     0     0  595 1171   0   1  99   0   0
 0  0        0 127868 398928 2297092    0    0     0     0  590 1180   1   0 100   0   0
 0  0        0 127868 398928 2297092    0    0     0     0  567 1135   0   1  99   0   0
```

CS(Content Switch)表示上下文切换的次数, 从上面的测试结果中我们可以看到, 上下文每1秒切换1000多次。

^[1] Lmbench3是一个性能分析工具。

1.1.3 如何减少上下文切换

减少上下文切换的方法有无锁并发编程、CAS算法、使用最少线程和使用协程。

无锁并发编程。多线程竞争锁时，会引起上下文切换，所以多线程处理数据时，可以用一些办法来避免使用锁，如将数据的ID按照Hash算法取模分段，不同的线程处理不同段的数据。

CAS算法。Java的Atomic包使用CAS算法来更新数据，而不需要加锁。

使用最少线程。避免创建不需要的线程，比如任务很少，但是创建了很多线程来处理，这样会造成大量线程都处于等待状态。

协程：在单线程里实现多任务的调度，并在单线程里维持多个任务间的切换。

1.1.4 减少上下文切换实战

本节将通过减少线上大量WAITING的线程, 来减少上下文切换次数。

第一步: 用jstack命令dump线程信息, 看看pid为31177的进程里的线程都在做什么。

```
sudo -u admin /opt/ifeve/java/bin/jstack 31177 > /home/tengfei.fangtf/dump17
```

第二步: 统计所有线程分别处于什么状态, 发现300多个线程处于WAITING(onobjectmonitor)状态。

```
[tengfei.fangtf@ifeve ~]$ grep java.lang.Thread.State dump17 | awk '{print $2$3$4$5}'  
| sort | uniq -c  
39 RUNNABLE  
21 TIMED_WAITING(onobjectmonitor)  
6 TIMED_WAITING(parking)  
51 TIMED_WAITING(sleeping)  
305 WAITING(onobjectmonitor)  
3 WAITING(parking)
```

第三步: 打开dump文件查看处于WAITING(onobjectmonitor)的线程在做什么。发现这些线程基本全是JBOSS的工作线程, 在await。说明JBOSS线程池里线程接收到的任务太少, 大量线程都闲着。

```
"http-0.0.0.0-7001-97" daemon prio=10 tid=0x000000004f6a8000 nid=0x555e in  
  Object.wait() [0x0000000052423000]  
java.lang.Thread.State: WAITING (on object monitor)  
at java.lang.Object.wait(Native Method)  
- waiting on <0x000000007969b2280> (a org.apache.tomcat.util.net.AprEndpoint$Worker)  
at java.lang.Object.wait(Object.java:485)  
at org.apache.tomcat.util.net.AprEndpoint$Worker.await(AprEndpoint.java:1464)  
- locked <0x000000007969b2280> (a org.apache.tomcat.util.net.AprEndpoint$Worker)  
at org.apache.tomcat.util.net.AprEndpoint$Worker.run(AprEndpoint.java:1489)  
at java.lang.Thread.run(Thread.java:662)
```

第四步: 减少JBOSS的工作线程数, 找到JBOSS的线程池配置信息, 将maxThreads降到100。

```
<maxThreads="250" maxHttpHeaderSize="8192"
emptySessionPath="false" minSpareThreads="40" maxSpareThreads="75"
    maxPostSize="512000" protocol="HTTP/1.1"
enableLookups="false" redirectPort="8443" acceptCount="200" bufferSize="16384"
connectionTimeout="15000" disableUploadTimeout="false" useBodyEncodingForURI="true">
```

第五步:重启JBOSS,再dump线程信息,然后统计WAITING(onobjectmonitor)的线程,发现减少了175个。WAITING的线程少了,系统上下文切换的次数就会少,因为每一次从WAITING到RUNNABLE都会进行一次上下文的切换。读者也可以使用vmstat命令测试一下。

```
[tengfei.fangtf@ifeve ~]$ grep java.lang.Thread.State dump17 | awk '{print $2$3$4$5}'
| sort | uniq -c
44 RUNNABLE
22 TIMED_WAITING(onobjectmonitor)
9 TIMED_WAITING(parking)
36 TIMED_WAITING(sleeping)
130 WAITING(onobjectmonitor)
1 WAITING(parking)
```

1.2 死锁

锁是个非常有用的工具，运用场景非常多，因为它使用起来非常简单，而且易于理解。但同时它也会带来一些困扰，那就是可能会引起死锁，一旦产生死锁，就会造成系统功能不可用。让我们先来看一段代码，这段代码会引起死锁，使线程t1和线程t2互相等待对方释放锁。

```
public class DeadLockDemo {
    private static String A = "A";
    private static String B = "B";
    public static void main(String[] args) {
        new DeadLockDemo().deadLock();
    }
    private void deadLock() {
        Thread t1 = new Thread(new Runnable() {
            @Override
            public void run() {
                synchronized (A) {
                    try { Thread.currentThread().sleep(2000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                synchronized (B) {
                    System.out.println("1");
                }
            }
        });
        Thread t2 = new Thread(new Runnable() {
            @Override
            public void run() {
                synchronized (B) {
                    synchronized (A) {
                        System.out.println("2");
                    }
                }
            }
        });
        t1.start();
        t2.start();
    }
}
```

这段代码只是演示死锁的场景，在现实中你可能不会写出这样的代码。但是，在一些更为复杂的场景中，你可能会遇到这样的问题，比如t1拿到锁之后，因为一些异常情况没有释放锁（死循环）。又或者是t1拿到一个数据库锁，释放锁的时候抛出了异常，没释放掉。

一旦出现死锁，业务是可感知的，因为不能继续提供服务了，那么只能通过dump线程查看到底是哪个线程出现了问题，以下线程信息告诉我们是DeadLockDemo类的第42行和第31行引起的死锁。

```
"Thread-2" prio=5 tid=7fc0458d1000 nid=0x116c1c000 waiting for monitor entry [116c1b00
  java.lang.Thread.State: BLOCKED (on object monitor)
    at com.ifeve.book.forkjoin.DeadLockDemo$2.run(DeadLockDemo.java:42)
    - waiting to lock <7fb2f3ec0> (a java.lang.String)
    - locked <7fb2f3ef8> (a java.lang.String)
    at java.lang.Thread.run(Thread.java:695)
"Thread-1" prio=5 tid=7fc0430f6800 nid=0x116b19000 waiting for monitor entry [116b1800
  java.lang.Thread.State: BLOCKED (on object monitor)
    at com.ifeve.book.forkjoin.DeadLockDemo$1.run(DeadLockDemo.java:31)
    - waiting to lock <7fb2f3ef8> (a java.lang.String)
    - locked <7fb2f3ec0> (a java.lang.String)
    at java.lang.Thread.run(Thread.java:695)
```

现在我们介绍避免死锁的几个常见方法。

- 避免一个线程同时获取多个锁。
- 避免一个线程在锁内同时占用多个资源，尽量保证每个锁只占用一个资源。
- 尝试使用定时锁，使用lock.tryLock(timeout)来替代使用内部锁机制。
- 对于数据库锁，加锁和解锁必须在一个数据库连接里，否则会出现解锁失败的情况。

1.3 资源限制的挑战

(1) 什么是资源限制

资源限制是指在进行并发编程时，程序的执行速度受限于计算机硬件资源或软件资源。

例如，服务器的带宽只有2Mb/s，某个资源的下载速度是1Mb/s每秒，系统启动10个线程下载资源，下载速度不会变成10Mb/s，所以在进行并发编程时，要考虑这些资源的限制。硬件资源限制有带宽的上传/下载速度、硬盘读写速度和CPU的处理速度。软件资源限制有数据库的连接数和socket连接数等。

(2) 资源限制引发的问题

在并发编程中，将代码执行速度加快的原则是将代码中串行执行的部分变成并发执行，但是如果将某段串行的代码并发执行，因为受限于资源，仍然在串行执行，这时候程序不仅不会加快执行，反而会更慢，因为增加了上下文切换和资源调度的时间。例如，之前看到一段程序使用多线程在办公网并发地下载和处理数据时，导致CPU利用率达到100%，几个小时都不能运行完成任务，后来修改成单线程，一个小时就执行完成了。

(3) 如何解决资源限制的问题

对于硬件资源限制，可以考虑使用集群并行执行程序。既然单机的资源有限制，那么就让程序在多机上运行。比如使用ODPS、Hadoop或者自己搭建服务器集群，不同的机器处理不同的数据。可以通过“数据ID%机器数”，计算得到一个机器编号，然后由对应编号的机器处理这笔数据。

对于软件资源限制，可以考虑使用资源池将资源复用。比如使用连接池将数据库和Socket连接复用，或者在调用对方webservice接口获取数据时，只建立一个连接。

(4) 在资源限制情况下进行并发编程

如何在资源限制的情况下，让程序执行得更快呢？方法就是，根据不同的资源限制调整程序的并发度，比如下载文件程序依赖于两个资源——带宽和硬盘读写速度。有数据库操作时，涉及数据库连接数，如果SQL语句执行非常快，而线程的数量比数据库连接数大很多，则某些线程会被阻塞，等待数据库连接。

1.4 本章小结

本章介绍了在进行并发编程时，大家可能会遇到的几个挑战，并给出了一些解决建议。有的并发程序写得不严谨，在并发下如果出现问题，定位起来会比较耗时和棘手。所以，对于Java开发工程师而言，**笔者强烈建议多使用JDK并发包提供的并发容器和工具类来解决并发问题，因为这些类都已经通过了充分的测试和优化，均可解决了本章提到的几个挑战。**

第2章 Java并发机制的底层实现原理

Java代码在编译后会变成Java字节码，字节码被类加载器加载到JVM里，JVM执行字节码，最终需要转化为汇编指令在CPU上执行，Java中所使用的并发机制依赖于JVM的实现和CPU的指令。本章我们将深入底层一起探索下Java并发机制的底层实现原理。

2.1 volatile的应用

在多线程并发编程中synchronized和volatile都扮演着重要的角色，volatile是轻量级的synchronized，它在多处理器开发中保证了共享变量的“可见性”。可见性的意思是当一个线程修改一个共享变量时，另外一个线程能读到这个修改的值。**如果volatile变量修饰符使用恰当的话，它比synchronized的使用和执行成本更低，因为它不会引起线程上下文的切换和调度。**本文将深入分析在硬件层面上Intel处理器是如何实现volatile的，通过深入分析帮助我们正确地使用volatile变量。

我们先从了解volatile的定义开始。

1.volatile的定义与实现原理

Java语言规范第3版中对volatile的定义如下：Java编程语言允许线程访问共享变量，为了确保共享变量能被**准确和一致地更新**，线程应该确保通过**排他锁单独获得这个变量**。Java语言提供了volatile，在某些情况下比锁要更加方便。如果一个字段被声明成volatile，Java线程内存模型确保所有线程看到这个**变量的值是一致的**。

在了解volatile实现原理之前，我们先来看下与其实实现原理相关的CPU术语与说明。表2-1是CPU术语的定义。

表2-1 CPU的术语定义

术 语	英文单词	术语描述
内存屏障	memory barriers	是一组处理器指令，用于实现对内存操作的顺序限制
缓冲行	cache line	缓存中可以分配的最小存储单位。处理器填写缓存线时会加载整个缓存线，需要使用多个主内存读周期
原子操作	atomic operations	不可中断的一个或一系列操作
缓存行填充	cache line fill	当处理器识别到从内存中读取操作数是可缓存的，处理器读取整个缓存行到适当的缓存（L1，L2，L3 的或所有）
缓存命中	cache hit	如果进行高速缓存行填充操作的内存位置仍然是下次处理器访问的地址时，处理器从缓存中读取操作数，而不是从内存读取
写命中	write hit	当处理器将操作数写回到一个内存缓存的区域时，它首先会检查这个缓存的内存地址是否在缓存行中，如果存在一个有效的缓存行，则处理器将这个操作数写回到缓存，而不是写回到内存，这个操作被称为写命中
写缺失	write misses the cache	一个有效的缓存行被写入到不存在的内存区域

volatile是如何来保证可见性的呢？让我们在X86处理器下通过[工具获取JIT编译器生成的汇编指令](#)来查看对volatile进行写操作时，CPU会做什么事情。JITWatch图形界面工具

Java代码如下。

```
instance = new Singleton(); // instance是volatile变量
```

转变成汇编代码，如下。

```
0x01a3de1d: movb $0x0,0x1104800(%esi);0x01a3de24: lock addl $0x0,(%esp);
```

有volatile变量修饰的共享变量进行写操作的时候会多出第二行汇编代码，通过查IA-32架构软件开发者手册可知，**Lock前缀的指令在多核处理器下会引发了两件事情^[1]**。

- 1)将当前处理器缓存行的数据写回到系统内存。
- 2)这个写回内存的操作会使在其他CPU里缓存了该内存地址的数据无效。

为了提高处理速度，处理器不直接和内存进行通信，而是先将系统内存的数据读到内部

缓存(L1, L2或其他)后再进行操作,但操作完不知道何时会写到内存。如果对声明了volatile的变量进行写操作,JVM就会向处理器发送一条Lock前缀的指令,将这个变量所在缓存行的数据写回到系统内存。但是,就算写回到内存,如果其他处理器缓存的值还是旧的,再执行计算操作就会有问題。所以,在多处理器下,为了保证各个处理器的缓存是一致的,就会实现缓存一致性协议,每个处理器通过嗅探在总线上传播的数据来检查自己缓存的值是不是过期了,当处理器发现自己缓存行对应的内存地址被修改,就会将当前处理器的缓存行设置成无效状态,当处理器对这个数据进行修改操作的时候,会重新从系统内存中把数据读到处理器缓存里。

下面来具体讲解volatile的两条实现原则。

1)Lock前缀指令会引起处理器缓存回写到内存。Lock前缀指令导致在执行指令期间,声言处理器的LOCK#信号。在多处理器环境中,LOCK#信号确保在声言该信号期间,处理器可以独占任何共享内存^[2]。但是,在最近的处理器里,LOCK#信号一般不锁总线,而是锁缓存,毕竟锁总线开销的比较大。在8.1.4节有详细说明锁定操作对处理器缓存的影响,对于Intel486和Pentium处理器,在锁操作时,总是在总线上声言LOCK#信号。但在P6和目前的处理器中,如果访问的内存区域已经缓存在处理器内部,则不会声言LOCK#信号。相反,它会锁定这块内存区域的缓存并回写到内存,并使用缓存一致性机制来确保修改的原子性,此操作被称为“缓存锁定”,缓存一致性机制会阻止同时修改由两个以上处理器缓存的内存区域数据。

2)一个处理器的缓存回写到内存会导致其他处理器的缓存无效。IA-32处理器和Intel 64处理器使用MESI(修改、独占、共享、无效)控制协议去维护内部缓存和其他处理器缓存的一致性。在多核处理器系统中进行操作的时候,IA-32和Intel 64处理器能嗅探其他处理器访问系统内存和它们的内部缓存。处理器使用嗅探技术保证它的内部缓存、系统内存和其他处理器的缓存的数据在总线上保持一致。例如,在Pentium和P6 family处理器中,如果通过嗅探一个处理器来检测其他处理器打算写内存地址,而这个地址当前处于共享状态,那么正在嗅探的处理器将使它的缓存行无效,在下次访问相同内存地址时,强制执行缓存行填充。

2.volatile的使用优化

著名的Java并发编程大师Doug lea在JDK 7的并发包里新增一个**队列集合类Linked-**

TransferQueue, 它在使用volatile变量时, 用一种追加字节的方式来优化队列出队和入队的性能。**LinkedTransferQueue**的代码如下。

```
/** 队列中的头部节点 */
private transient f?inal PaddedAtomicReference<QNode> head;
/** 队列中的尾部节点 */
private transient f?inal PaddedAtomicReference<QNode> tail;
static f?inal class PaddedAtomicReference <T> extends AtomicReference T> {
    // 使用很多4个字节的引用追加到64个字节
    Object p0, p1, p2, p3, p4, p5, p6, p7, p8, p9, pa, pb, pc, pd, pe;
    PaddedAtomicReference(T r) {
        super(r);
    }
}
public class AtomicReference <V> implements java.io.Serializable {
    private volatile V value;
    // 省略其他代码
}
```

追加字节能优化性能？这种方式看起来很神奇, 但如果深入理解处理器架构就能理解其中的奥秘。让我们先来看看LinkedTransferQueue这个类, 它使用一个内部类类型来定义队列的头节点(head)和尾节点(tail), 而这个内部类PaddedAtomicReference相对于父类AtomicReference只做了一件事情, 就是将共享变量追加到64字节。我们可以来计算下, 一个对象的引用占4个字节, 它追加了15个变量(共占60个字节), 再加上父类的value变量, 一共64个字节。

为什么追加64字节能够提高并发编程的效率呢？因为对于英特尔酷睿i7、酷睿、Atom和NetBurst, 以及Core Solo和Pentium M处理器的L1、L2或L3缓存的高速缓存行是64个字节宽, 不支持部分填充缓存行, 这意味着, 如果队列的头节点和尾节点都不足64字节的话, 处理器会将它们都读到同一个高速缓存行中, 在多处理器下每个处理器都会缓存同样的头、尾节点, 当一个处理器试图修改头节点时, 会将整个缓存行锁定, 那么在缓存一致性机制的作用下, 会导致其他处理器不能访问自己高速缓存中的尾节点, 而队列的入队和出队操作则需要不停修改头

节点和尾节点，所以在多处理器的情况下将会严重影响到队列的入队和出队效率。 Doug lea使用追加到64字节的方式来填满高速缓冲区的缓存行，避免头节点和尾节点加载到同一个缓存行，使头、尾节点在修改时不会互相锁定。

那么是不是在使用volatile变量时都应该追加到64字节呢？不是的。在两种场景下不应该使用这种方式。

·**缓存行非64字节宽的处理器。**如P6系列和奔腾处理器，它们的L1和L2高速缓存行是32个字节宽。

·**共享变量不会被频繁地写。**因为使用追加字节的方式需要处理器读取更多的字节到高速缓冲区，这本身就会带来一定的性能消耗，如果共享变量不被频繁写的话，锁的几率也非常小，就没必要通过追加字节的方式来避免相互锁定。

不过这种追加字节的方式在Java 7下可能不生效，因为Java 7变得更加智慧，它会淘汰或重新排列无用字段，需要使用其他追加字节的方式。除了volatile，Java并发编程中应用较多的是synchronized，下面一起来看一下。

[1] 这两件事情在IA-32软件开发者架构手册的第三册的多处理器管理章节(第8章)中有详细阐述。

[2] 因为它会锁住总线，导致其他CPU不能访问总线，不能访问总线就意味着不能访问系统内存。

2.2 synchronized的实现原理与应用

在多线程并发编程中synchronized一直是元老级角色，很多人都会称呼它为重量级锁。但是，随着Java SE 1.6对synchronized进行了各种优化之后，有些情况下它就并不那么重了。本文详细介绍Java SE 1.6中为了减少获得锁和释放锁带来的性能消耗而引入的偏向锁和轻量级锁，以及锁的存储结构和升级过程。

先来看下利用synchronized实现同步的基础：Java中的每一个对象都可以作为锁。具体表现为以下3种形式。

- 对于普通同步方法，锁是当前实例对象。
- 对于静态同步方法，锁是当前类的Class对象。
- 对于同步方法块，锁是Synchronized括号里配置的对象。

当一个线程试图访问同步代码块时，它首先必须得到锁，退出或抛出异常时必须释放锁。那么锁到底存在哪里呢？锁里面会存储什么信息呢？

从JVM规范中可以看到Synchronized在JVM里的实现原理，JVM基于进入和退出Monitor对象来实现方法同步和代码块同步，但两者的实现细节不一样。代码块同步是使用monitorenter和monitorexit指令实现的，而方法同步是使用另外一种方式实现的，细节在JVM规范里并没有详细说明。但是，方法的同步同样可以使用这两个指令来实现。

monitorenter指令是在编译后插入到同步代码块的开始位置，而monitorexit是插入到方法结束处和异常处，JVM要保证每个monitorenter必须有对应的monitorexit与之配对。任何对象都有一个monitor与之关联，当且一个monitor被持有后，它将处于锁定状态。线程执行到monitorenter指令时，将会尝试获取对象所对应的monitor的所有权，即尝试获得对象的锁。

2.2.1 Java对象头

synchronized用的锁是存在Java对象头里的。如果对象是数组类型，则虚拟机用3个字宽（Word）存储对象头，如果对象是非数组类型，则用2字宽存储对象头。在32位虚拟机中，1字宽等于4字节，即32bit，如表2-2所示。

表2-2 Java对象头的长度

长 度	内 容	说 明
32/64bit	Mark Word	存储对象的 hashCode 或锁信息等
32/64bit	Class Metadata Address	存储到对象类型数据的指针
32/32bit	Array length	数组的长度（如果当前对象是数组）

Java对象头里的Mark Word里默认存储对象的HashCode、分代年龄和锁标记位。32位JVM的Mark Word的默认存储结构如表2-3所示。

表2-3 Java对象头的存储结构

锁状态	25bit	4bit	1bit 是否是偏向锁	2bit 锁标志位
无锁状态	对象的 hashCode	对象分代年龄	0	01

在运行期间，Mark Word里存储的数据会随着锁标志位的变化而变化。Mark Word可能变化为存储以下4种数据，如表2-4所示。

表2-4 Mark Word的状态变化

锁状态	25bit		4bit	1bit	2bit
	23bit	2bit		是否是偏向锁	锁标志位
轻量级锁	指向栈中锁记录的指针				00
重量级锁	指向互斥量（重量级锁）的指针				10
GC 标记	空				11
偏向锁	线程 ID	Epoch	对象分代年龄	1	01

在64位虚拟机下，Mark Word是64bit大小的，其存储结构如表2-5所示。

表2-5 Mark Word的存储结构

锁状态	25bit	31bit	1bit	4bit	1bit	2bit
			cms_free	分代年龄	偏向锁	锁标志位
无锁	unused	hashCode			0	01
偏向锁	ThreadID(54bit) Epoch(2bit)				1	01

2.2.2 锁的升级与对比

Java SE 1.6为了减少获得锁和释放锁带来的性能消耗，引入了“偏向锁”和“轻量级锁”，在Java SE 1.6中，锁一共有4种状态，级别从低到高依次是：无锁状态、偏向锁状态、轻量级锁状态和重量级锁状态，这几个状态会随着竞争情况逐渐升级。锁可以升级但不能降级，意味着偏向锁升级成轻量级锁后不能降级成偏向锁。这种锁升级却不能降级的策略，目的是为了减少获得锁和释放锁的效率，下文会详细分析。

1. 偏向锁

HotSpot^[1]的作者经过研究发现，大多数情况下，锁不仅不存在多线程竞争，而且总是由同一线程多次获得，为了让线程获得锁的代价更低而引入了偏向锁。当一个线程访问同步块并获取锁时，会在对象头和栈帧中的锁记录里存储锁偏向的线程ID，以后该线程在进入和退出同步块时不需要进行CAS操作来加锁和解锁，只需简单地测试一下对象头的Mark Word里是否存储着指向当前线程的偏向锁。如果测试成功，表示线程已经获得了锁。如果测试失败，则需要再测试一下Mark Word中偏向锁的标识是否设置成1（表示当前是偏向锁）：如果没有设置，则使用CAS竞争锁；如果设置了，则尝试使用CAS将对象头的偏向锁指向当前线程。

(1) 偏向锁的撤销

偏向锁使用了一种等到竞争出现才释放锁的机制，所以当其他线程尝试竞争偏向锁时，持有偏向锁的线程才会释放锁。偏向锁的撤销，需要等待全局安全点（在这个时间点上没有正在执行的字节码）。它会首先暂停拥有偏向锁的线程，然后检查持有偏向锁的线程是否活着，如果线程不处于活动状态，则将对象头设置成无锁状态；如果线程仍然活着，拥有偏向锁的栈会被执行，遍历偏向对象的锁记录，栈中的锁记录和对象头的Mark Word要么重新偏向于其他线程，要么恢复到无锁或者标记对象不适合作为偏向锁，最后唤醒暂停的线程。图2-1中的线程1演示了偏向锁初始化的流程，线程2演示了偏向锁撤销的流程。

偏向锁的获得和撤销流程

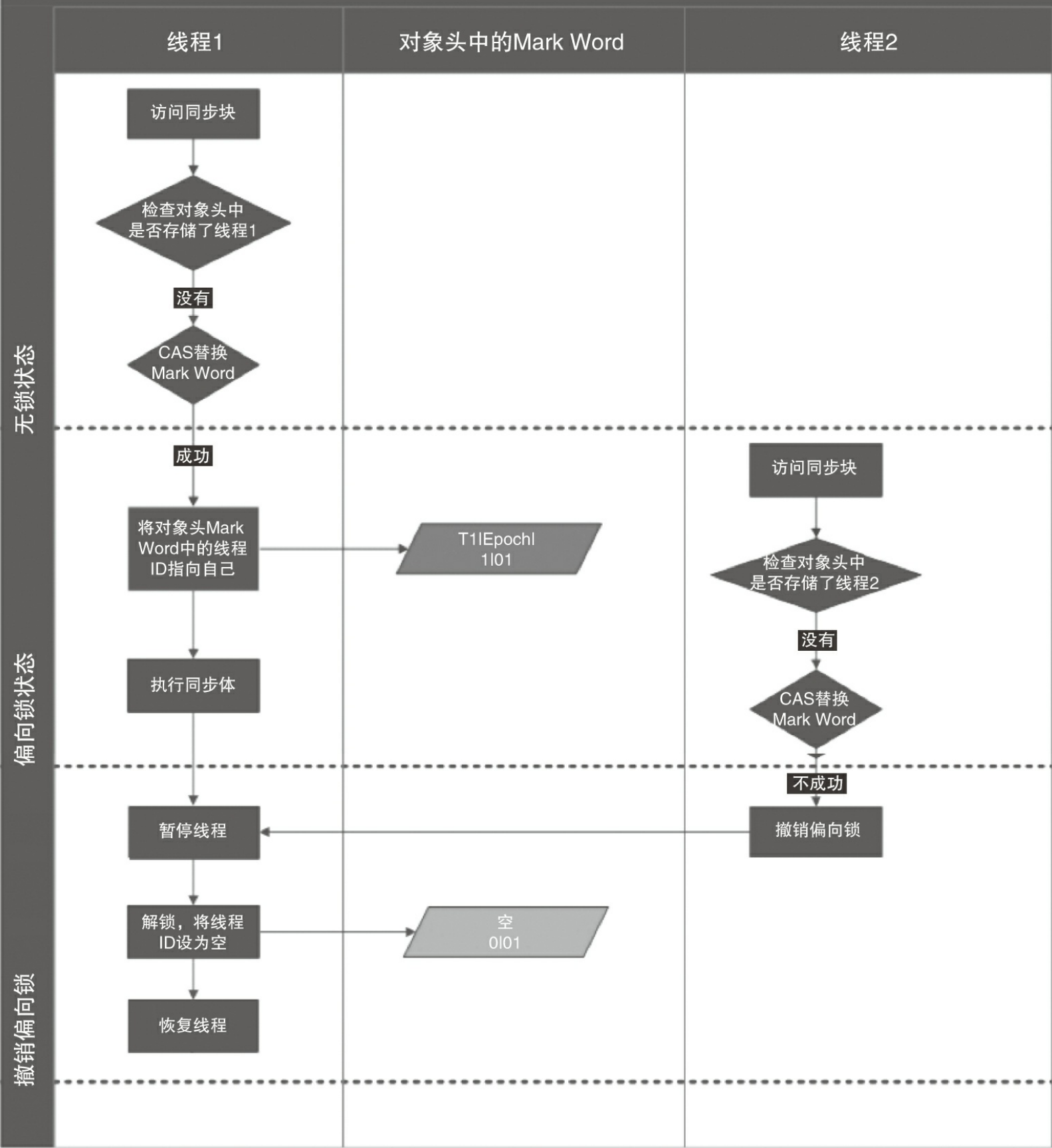


图2-1 偏向锁初始化的流程

(2) 关闭偏向锁

偏向锁在Java 6和Java 7里是默认启用的，但是它在应用程序启动几秒钟之后才激活，如有必要可以使用JVM参数来关闭延迟：`-XX:BiasedLockingStartupDelay=0`。如果你确定应用程序里所有的锁通常情况下处于竞争状态，可以通过JVM参数关闭偏向锁：`-XX:-UseBiasedLocking=false`，那么程序默认会进入轻量级锁状态。

2. 轻量级锁

(1) 轻量级锁加锁

线程在执行同步块之前，JVM会先在当前线程的栈帧中创建用于存储锁记录的空间，并将对象头中的Mark Word复制到锁记录中，官方称为Displaced Mark Word。然后线程尝试使用CAS将对象头中的Mark Word替换为指向锁记录的指针。如果成功，当前线程获得锁，如果失败，表示其他线程竞争锁，当前线程便尝试使用自旋来获取锁。

(2) 轻量级锁解锁

轻量级解锁时，会使用原子的CAS操作将Displaced Mark Word替换回到对象头，如果成功，则表示没有竞争发生。如果失败，表示当前锁存在竞争，锁就会膨胀成重量级锁。图2-2是两个线程同时争夺锁，导致锁膨胀的流程图。

轻量级锁及膨胀流程图

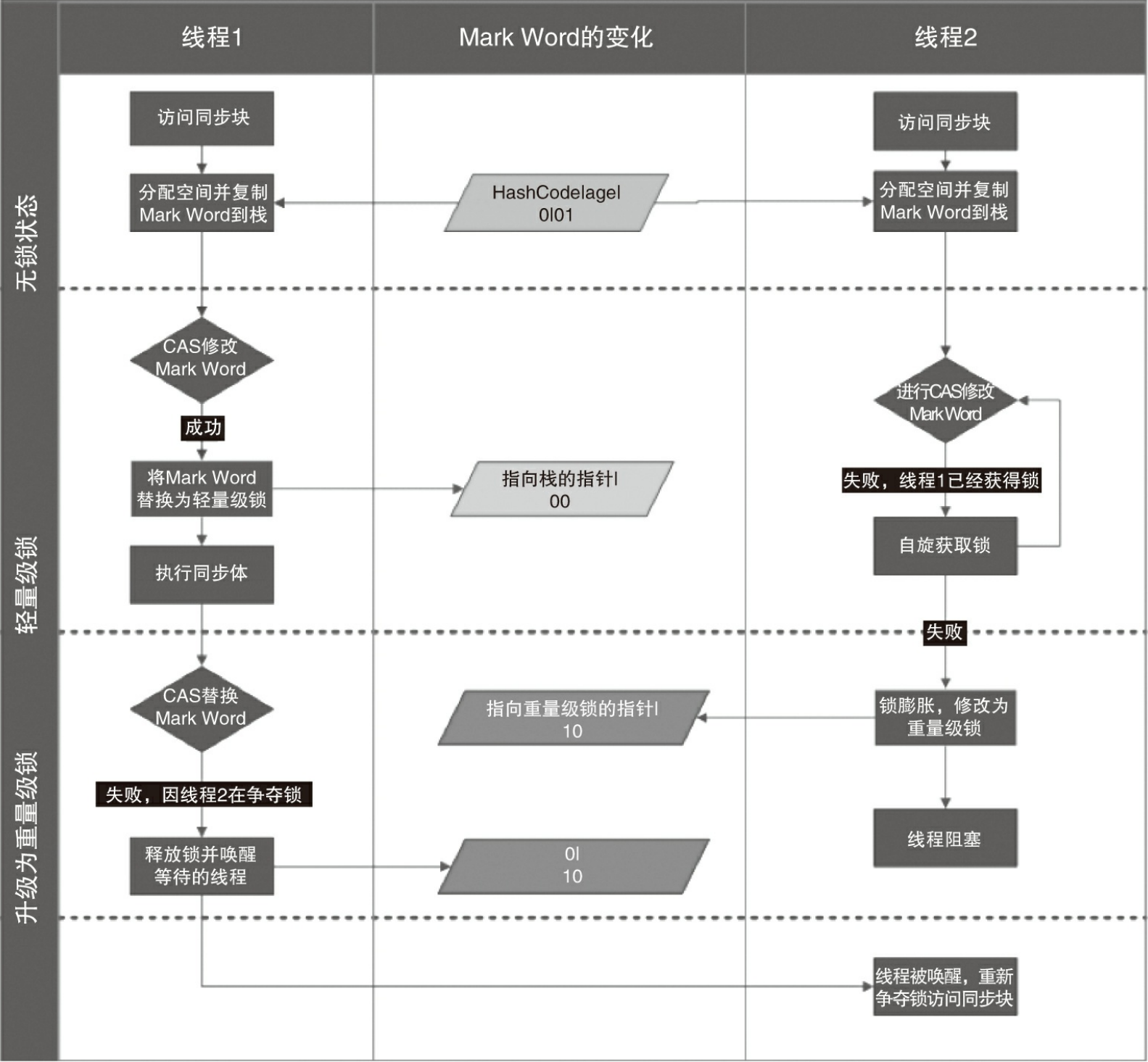


图2-2 争夺锁导致的锁膨胀流程图

因为自旋会消耗CPU, 为了避免无用的自旋(比如获得锁的线程被阻塞住了), 一旦锁升级成重量级锁, 就不会再恢复到轻量级锁状态。当锁处于这个状态下, 其他线程试图获取锁时, 都会被阻塞住, 当持有锁的线程释放锁之后会唤醒这些线程, 被唤醒的线程就会进行新一轮的夺锁之争。

3.锁的优缺点对比

表2-6是锁的优缺点的对比。

表2-6 锁的优缺点的对比

锁	优 点	缺 点	适用场景
偏向锁	加锁和解锁不需要额外的消耗，和执行非同步方法相比仅存在纳秒级的差距	如果线程间存在锁竞争，会带来额外的锁撤销的消耗	适用于只有一个线程访问同步块场景
轻量级锁	竞争的线程不会阻塞，提高了程序的响应速度	如果始终得不到锁竞争的线程，使用自旋会消耗 CPU	追求响应时间 同步块执行速度非常快
重量级锁	线程竞争不使用自旋，不会消耗 CPU	线程阻塞，响应时间缓慢	追求吞吐量 同步块执行速度较长

[1] 本节一些内容参考了HotSpot源码、对象头源码markOop.hpp、偏向锁源码biasedLocking.cpp，以及其他源码ObjectMonitor.cpp和BasicLock.cpp。

2.3 原子操作的实现原理

原子(atomic)本意是“不能被进一步分割的最小粒子”，而原子操作(atomic operation)意为“不可被中断的一个或一系列操作”。在多个处理器上实现原子操作就变得有点复杂。让我们一起来聊一聊在Intel处理器和Java里是如何实现原子操作的。

1.术语定义

在了解原子操作的实现原理前，先要了解一下相关的术语，如表2-7所示。

表2-7 CPU术语定义

术语名称	英 文	解 释
缓存行	Cache line	缓存的最小操作单位
比较并交换	Compare and Swap	CAS 操作需要输入两个数值，一个旧值（期望操作前的值）和一个新值，在操作期间先比较旧值有没有发生变化，如果没有发生变化，才交换成新值，发生了变化则不交换
CPU 流水线	CPU pipeline	CPU 流水线的工作方式就像工业生产上的装配流水线，在 CPU 中由 5 ~ 6 个不同功能的电路单元组成一条指令处理流水线，然后将一条 X86 指令分成 5 ~ 6 步后再由这些电路单元分别执行，这样就能实现在一个 CPU 时钟周期完成一条指令，因此提高 CPU 的运算速度
内存顺序冲突	Memory order violation	内存顺序冲突一般是由假共享引起的，假共享是指多个 CPU 同时修改同一个缓存行的不同部分而引起其中一个 CPU 的操作无效，当出现这个内存顺序冲突时，CPU 必须清空流水线

2.处理器如何实现原子操作

32位IA-32处理器使用基于对缓存加锁或总线加锁的方式来实现多处理器之间的原子操作。首先处理器会自动保证基本的内存操作的原子性。处理器保证从系统内存中读取或者写入一个字节是原子的，意思是当一个处理器读取一个字节时，其他处理器不能访问这个字节的内存地址。Pentium 6和最新的处理器能自动保证单处理器对同一个缓存行里进行16/32/64位的操作是原子的，但是复杂的内存操作处理器是不能自动保证其原子性的，比如跨总线宽度、

跨多个缓存行和跨页表的访问。但是，处理器提供总线锁定和缓存锁定两个机制来保证复杂内存操作的原子性。

(1)使用总线锁保证原子性

第一个机制是通过总线锁保证原子性。如果多个处理器同时对共享变量进行读改写操作($i++$ 就是经典的读改写操作)，那么共享变量就会被多个处理器同时进行操作，这样读改写操作就不是原子的，操作完之后共享变量的值会和期望的不一致。举个例子，如果 $i=1$ ，我们进行两次 $i++$ 操作，我们期望的结果是3，但是有可能结果是2，如图2-3所示。

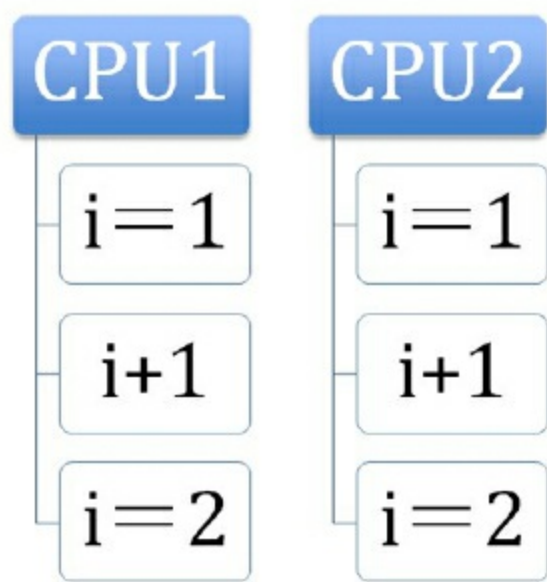


图2-3 结果对比

原因可能是多个处理器同时从各自的缓存中读取变量 i ，分别进行加1操作，然后分别写入系统内存中。那么，想要保证读改写共享变量的操作是原子的，就必须保证CPU1读改写共享变量的时候，CPU2不能操作缓存了该共享变量内存地址的缓存。

处理器使用总线锁就是来解决这个问题的。所谓总线锁就是使用处理器提供的一个LOCK#信号，当一个处理器在总线上输出此信号时，其他处理器的请求将被阻塞住，那么该处理器可以独占共享内存。

(2)使用缓存锁保证原子性

第二个机制是通过缓存锁定来保证原子性。在同一时刻，我们只需保证对某个内存地址的操作是原子性即可，但总线锁定把CPU和内存之间的通信锁住了，这使得锁定期间，其他处理器不能操作其他内存地址的数据，所以总线锁定的开销比较大，目前处理器在某些场合下使用缓存锁定代替总线锁定来进行优化。

频繁使用的内存会缓存在处理器的L1、L2和L3高速缓存里，那么原子操作就可以直接在处理器内部缓存中进行，并不需要声明总线锁，在Pentium 6和目前的处理器中可以使用“缓存锁定”的方式来实现复杂的原子性。所谓“缓存锁定”是指内存区域如果被缓存在处理器的缓存行中，并且在Lock操作期间被锁定，那么当它执行锁操作回写到内存时，处理器不在总线上声言LOCK#信号，而是修改内部的内存地址，并允许它的缓存一致性机制来保证操作的原子性，因为缓存一致性机制会阻止同时修改由两个以上处理器缓存的内存区域数据，当其他处理器回写已被锁定的缓存行的数据时，会使缓存行无效，在如图2-3所示的例子中，当CPU1修改缓存行中的i时使用了缓存锁定，那么CPU2就不能同时缓存i的缓存行。

但是有两种情况下处理器不会使用缓存锁定。

第一种情况是：当操作的数据不能被缓存在处理器内部，或操作的数据跨多个缓存行(cache line)时，则处理器会调用总线锁定。

第二种情况是：有些处理器不支持缓存锁定。对于Intel 486和Pentium处理器，就算锁定的内存区域在处理器的缓存行中也会调用总线锁定。

针对以上两个机制，我们通过Intel处理器提供了很多Lock前缀的指令来实现。例如，位测试和修改指令：BTS、BTR、BTC；交换指令XADD、CMPXCHG，以及其他一些操作数和逻辑指令（如ADD、OR）等，被这些指令操作的内存区域就会加锁，导致其他处理器不能同时访问它。

3.Java如何实现原子操作

在Java中可以通过锁和循环CAS的方式来实现原子操作。

(1)使用循环CAS实现原子操作

JVM中的CAS操作正是利用了处理器提供的CMPXCHG指令实现的。自旋CAS实现的基本思路就是循环进行CAS操作直到成功为止，以下代码实现了一个基于CAS线程安全的计数器方法safeCount和一个非线程安全的计数器count。

```
private AtomicInteger atomicI = new AtomicInteger(0);
private int i = 0;
public static void main(String[] args) {
    final Counter cas = new Counter();
    List<Thread> ts = new ArrayList<Thread>(600);
    long start = System.currentTimeMillis();
    for (int j = 0; j < 100; j++) {
        Thread t = new Thread(new Runnable() {
            @Override
            public void run() {
                for (int i = 0; i < 10000; i++) {
                    cas.count();
                    cas.safeCount();
                }
            }
        });
        ts.add(t);
    }
    for (Thread t : ts) {
        t.start();
    }
    // 等待所有线程执行完成
    for (Thread t : ts) {
        try {
            t.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    System.out.println(cas.i);
    System.out.println(cas.atomicI.get());
    System.out.println(System.currentTimeMillis() - start);
}
/**      * 使用CAS实现线程安全计数器      */
private void safeCount() {
    for (;;) {
        int i = atomicI.get();
        boolean suc = atomicI.compareAndSet(i, ++i);
        if (suc) {
            break;
        }
    }
}
/**
 * 非线程安全计数器
 */
```

```
    */  
    private void count() {  
        i++;  
    }  
}
```

从Java 1.5开始, JDK的并发包里提供了一些类来支持原子操作, 如AtomicBoolean(用原子方式更新的boolean值)、AtomicInteger(用原子方式更新的int值)和AtomicLong(用原子方式更新的long值)。这些原子包装类还提供了有用的工具方法, 比如以原子的方式将当前值自增1和自减1。

(2) CAS实现原子操作的三大问题

在Java并发包中有一些并发框架也使用了自旋CAS的方式来实现原子操作, 比如LinkedTransferQueue类的Xfer方法。CAS虽然很高效地解决了原子操作, 但是CAS仍然存在三大问题。ABA问题, 循环时间长开销大, 以及只能保证一个共享变量的原子操作。

1) ABA问题。因为CAS需要在操作值的时候, 检查值有没有发生变化, 如果没有发生变化则更新, 但是如果一个值原来是A, 变成了B, 又变成了A, 那么使用CAS进行检查时会发现它的值没有发生变化, 但是实际上却变化了。**ABA问题的解决思路就是使用版本号**。在变量前面追加版本号, 每次变量更新的时候把版本号加1, 那么A→B→A就会变成1A→2B→3A。从Java 1.5开始, JDK的Atomic包里提供了一个类AtomicStampedReference来解决ABA问题。这个类的compareAndSet方法的作用是首先检查当前引用是否等于预期引用, 并且检查当前标志是否等于预期标志, 如果全部相等, 则以原子方式将该引用和该标志的值设置为给定的更新值。

```
public boolean compareAndSet(  
    V          expectedReference,          // 预期引用  
    V          newReference,              // 更新后的引用  
    int        expectedStamp,             // 预期标志  
    int        newStamp                   // 更新后的标志  
)
```

2) 循环时间长开销大。自旋CAS如果长时间不成功, 会给CPU带来非常大的执行开销。如

果JVM能支持处理器提供的pause指令, 那么效率会有一定的提升。pause指令有两个作用: 第一, 它可以延迟流水线执行指令(de-pipeline), 使CPU不会消耗过多的执行资源, 延迟的时间取决于具体实现的版本, 在一些处理器上延迟时间是零; 第二, 它可以避免在退出循环的时候因内存顺序冲突(Memory Order Violation)而引起CPU流水线被清空(CPU Pipeline Flush), 从而提高CPU的执行效率。

3) 只能保证一个共享变量的原子操作。当对一个共享变量执行操作时, 我们可以使用循环CAS的方式来保证原子操作, 但是对多个共享变量操作时, 循环CAS就无法保证操作的原子性, 这个时候就可以用锁。还有一个取巧的办法, 就是把多个共享变量合并成一个共享变量来操作。比如, 有两个共享变量 $i=2$, $j=a$, 合并一下 $ij=2a$, 然后用CAS来操作 ij 。从Java 1.5开始, JDK提供了AtomicReference类来保证引用对象之间的原子性, 就可以把多个变量放在一个对象里来进行CAS操作。

(3) 使用锁机制实现原子操作

锁机制保证了只有获得锁的线程才能够操作锁定的内存区域。JVM内部实现了很多种锁机制, 有偏向锁、轻量级锁和互斥锁。有意思的是除了偏向锁, JVM实现锁的方式都用了循环CAS, 即当一个线程想进入同步块的时候使用循环CAS的方式来获取锁, 当它退出同步块的时候使用循环CAS释放锁。

2.4 本章小结

本章我们一起研究了volatile、synchronized和原子操作的实现原理。Java中的大部分容器和框架都依赖于本章介绍的volatile和原子操作的实现原理，了解这些原理对我们进行并发编程会更有帮助。

第3章 Java内存模型

Java线程之间的通信对程序员完全透明,内存可见性问题很容易困扰Java程序员,本章将揭开Java内存模型神秘的面纱。本章大致分4部分:Java内存模型的基础,主要介绍内存模型相关的基本概念;Java内存模型中的顺序一致性,主要介绍重排序与顺序一致性内存模型;同步原语,主要介绍3个同步原语(synchronized、volatile和final)的内存语义及重排序规则在处理器中的实现;Java内存模型的设计,主要介绍Java内存模型的设计原理,及其与处理器内存模型和顺序一致性内存模型的关系。

3.1 Java内存模型的基础

3.1.1 并发编程模型的两个关键问题

在并发编程中，需要处理两个关键问题：线程之间如何通信及线程之间如何同步（这里的线程是指并发执行的活动实体）。通信是指线程之间以何种机制来交换信息。在命令式编程中，线程之间的通信机制有两种：共享内存和消息传递。

在共享内存的并发模型里，线程之间共享程序的公共状态，通过写-读内存中的公共状态进行隐式通信。在消息传递的并发模型里，线程之间没有公共状态，线程之间必须通过发送消息来显式进行通信。

同步是指程序中用于控制不同线程间操作发生相对顺序的机制。在共享内存并发模型里，同步是显式进行的。程序员必须显式指定某个方法或某段代码需要在线程之间互斥执行。在消息传递的并发模型里，由于消息的发送必须在消息的接收之前，因此同步是隐式进行的。

Java的并发采用的是共享内存模型，Java线程之间的通信总是隐式进行，整个通信过程对程序员完全透明。如果编写多线程程序的Java程序员不理解隐式进行的线程之间通信的工作机制，很可能会遇到各种奇怪的内存可见性问题。

3.1.2 Java内存模型的抽象结构

Heap中存储

在Java中，所有实例域、静态域和数组元素都存储在堆内存中，堆内存存在线程之间共享（本章用“共享变量”这个术语代指实例域，静态域和数组元素）。stack中存储：局部变量(Local Variables)，方法定义参数(Java语言规范称之为Formal Method Parameters)和异常处理器参数(Exception Handler Parameters)不会在线程之间共享，它们不会有内存可见性问题，也不受内存模型的影响。

Java线程之间的通信由Java内存模型(本文简称为JMM)控制，JMM决定一个线程对共享变量的写入何时对另一个线程可见。从抽象的角度来看，JMM定义了线程和主内存之间的抽象关系：线程之间的共享变量存储在主内存(Main Memory)中，每个线程都有一个私有的本地内存(Local Memory)，本地内存中存储了该线程以读/写共享变量的副本。本地内存是JMM的一个抽象概念，并不真实存在。它涵盖了缓存、写缓冲区、寄存器以及其他的硬件和编译器优化。Java内存模型的抽象示意如图3-1所示。

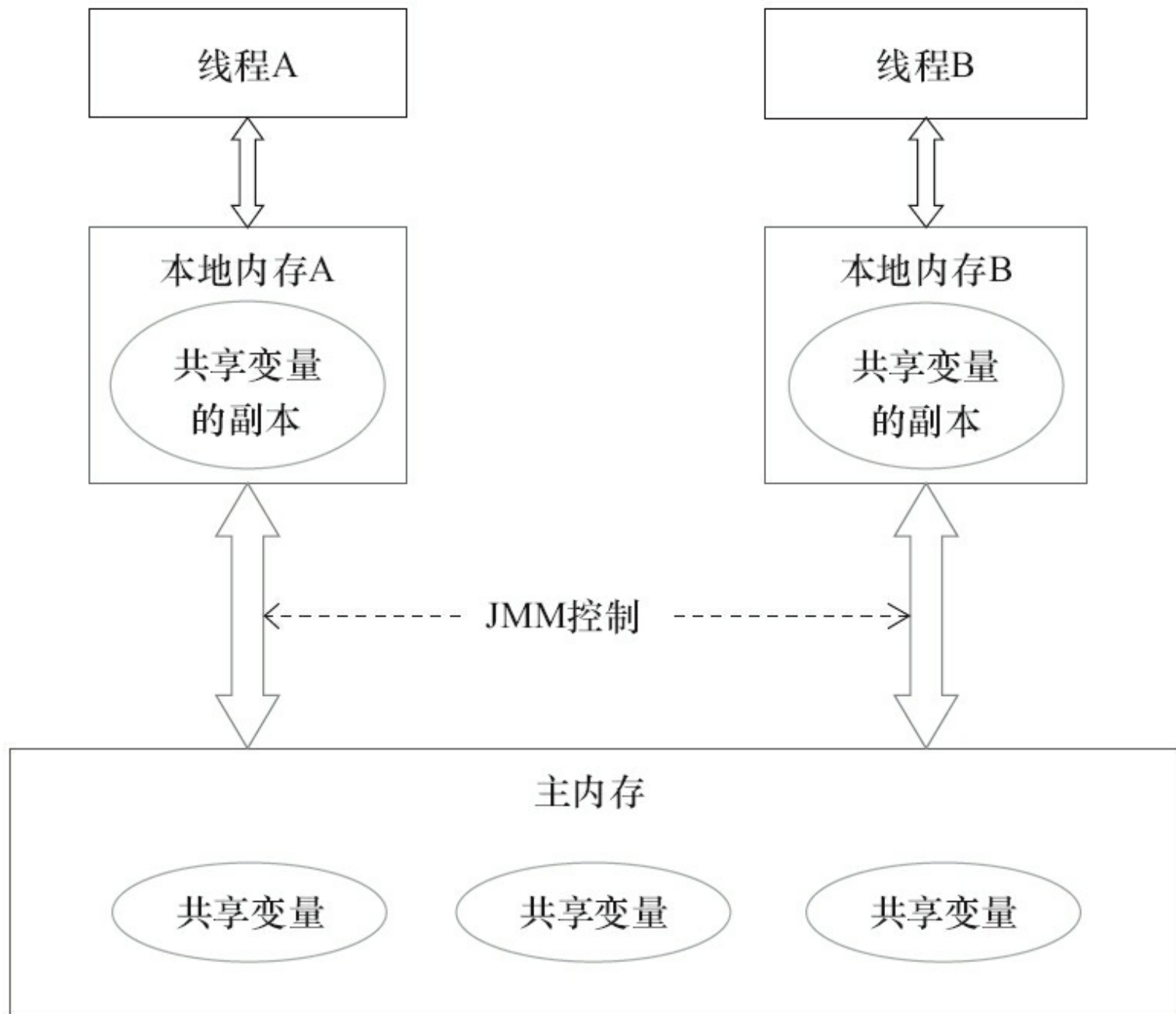


图3-1 Java内存模型的抽象结构示意图

从图3-1来看，如果线程A与线程B之间要通信的话，必须要经历下面2个步骤。

1)线程A把本地内存A中更新过的共享变量刷新到主内存中去。

2)线程B到主内存中去读取线程A之前已更新过的共享变量。

下面通过示意图(见图3-2)来说明这两个步骤。

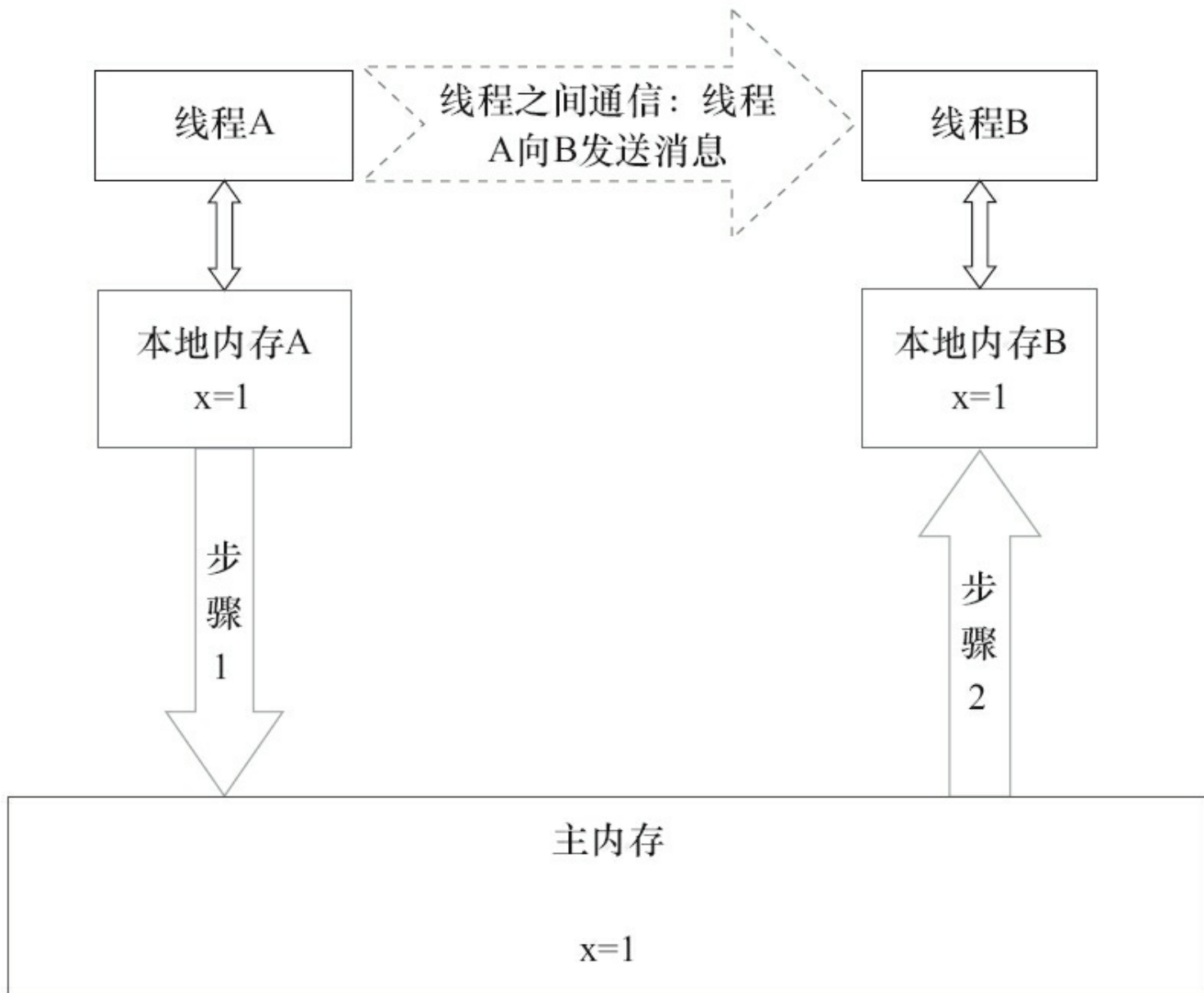


图3-2 线程之间的通信图

如图3-2所示，本地内存A和本地内存B由主内存中共享变量x的副本。假设初始时，这3个内存中的x值都为0。线程A在执行时，把更新后的x值(假设值为1)临时存放在自己的本地内存A中。当线程A和线程B需要通信时，线程A首先会把自己本地内存中修改后的x值刷新到主内存中，此时主内存中的x值变为了1。随后，线程B到主内存中去读取线程A更新后的x值，此时线程B的本地内存的x值也变为了1。

从整体来看，这两个步骤实质上是线程A在向线程B发送消息，而且这个通信过程必须要经过主内存。**JMM通过控制主内存与每个线程的本地内存之间的交互**，来为Java程序员提供内存可见性保证。

3.1.3 从源代码到指令序列的重排序

在执行程序时，为了提高性能，编译器和处理器常常会对指令做重排序。**重排序分3种类型。**

1) 编译器优化的重排序。编译器在不改变单线程程序语义的前提下，可以重新安排语句的执行顺序。

2) 指令级并行的重排序。现代处理器采用了指令级并行技术(Instruction-Level Parallelism, ILP)来将多条指令重叠执行。如果不存在数据依赖性，处理器可以改变语句对应机器指令的执行顺序。

3) 内存系统的重排序。由于处理器使用缓存和读/写缓冲区，这使得加载和存储操作看上去可能是在乱序执行。

从Java源代码到最终实际执行的指令序列，会分别经历下面3种重排序，如图3-3所示。

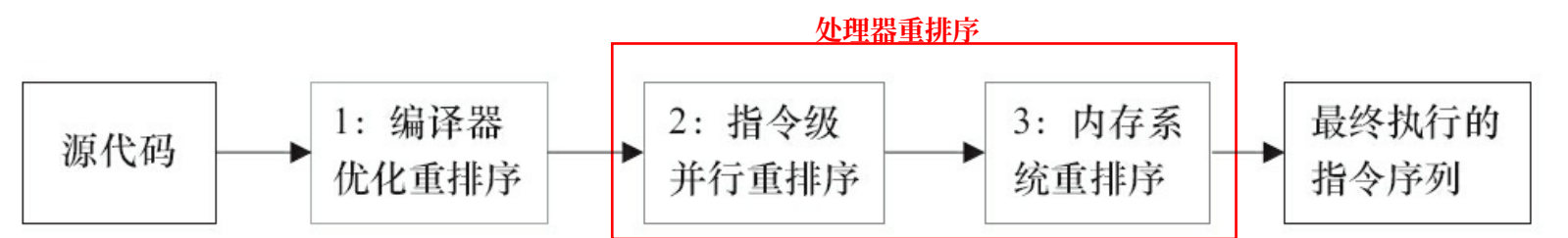


图3-3 **从源码到最终执行的指令序列的示意图**

上述的1属于编译器重排序，2和3属于处理器重排序。这些重排序可能会导致多线程程序出现内存可见性问题。对于编译器，JMM的编译器重排序规则会禁止特定类型的编译器重排序（不是所有的编译器重排序都要禁止）。对于处理器重排序，JMM的处理器重排序规则会要求Java编译器在生成指令序列时，插入特定类型的内存屏障(Memory Barriers, Intel称之为Memory Fence)指令，通过内存屏障指令来禁止特定类型的处理器重排序。

JMM属于语言级的内存模型，它确保在不同的编译器和不同的处理器平台之上，通过禁

止特定类型的编译器重排序和处理器重排序，为程序员提供一致的内存可见性保证。

3.1.4 并发编程模型的分类

现代的处理器使用写缓冲区临时保存向内存写入的数据。写缓冲区可以保证指令流水线持续运行，它可以避免由于处理器停顿下来等待向内存写入数据而产生的延迟。同时，通过以批处理的方式刷新写缓冲区，以及合并写缓冲区中对同一内存地址的多次写，减少对内存总线的占用。虽然写缓冲区有这么好处，但每个处理器上的写缓冲区，仅仅对它所在的处理器可见。这个特性会对内存操作的执行顺序产生重要的影响：处理器对内存的读/写操作的执行顺序，不一定与内存实际发生的读/写操作顺序一致！为了具体说明，请看下面的表3-1。

表3-1 处理器操作内存的执行结果

处理器	Processor A	Processor B
示例项		
代码	a = 1; //A1 x = b; //A2	b = 2; //B1 y = a; //B2
运行结果	初始状态：a = b = 0 处理器允许执行后得到结果：x = y = 0	

假设处理器A和处理器B按程序的顺序并行执行内存访问，最终可能得到x=y=0的结果。具体的原因如图3-4所示。

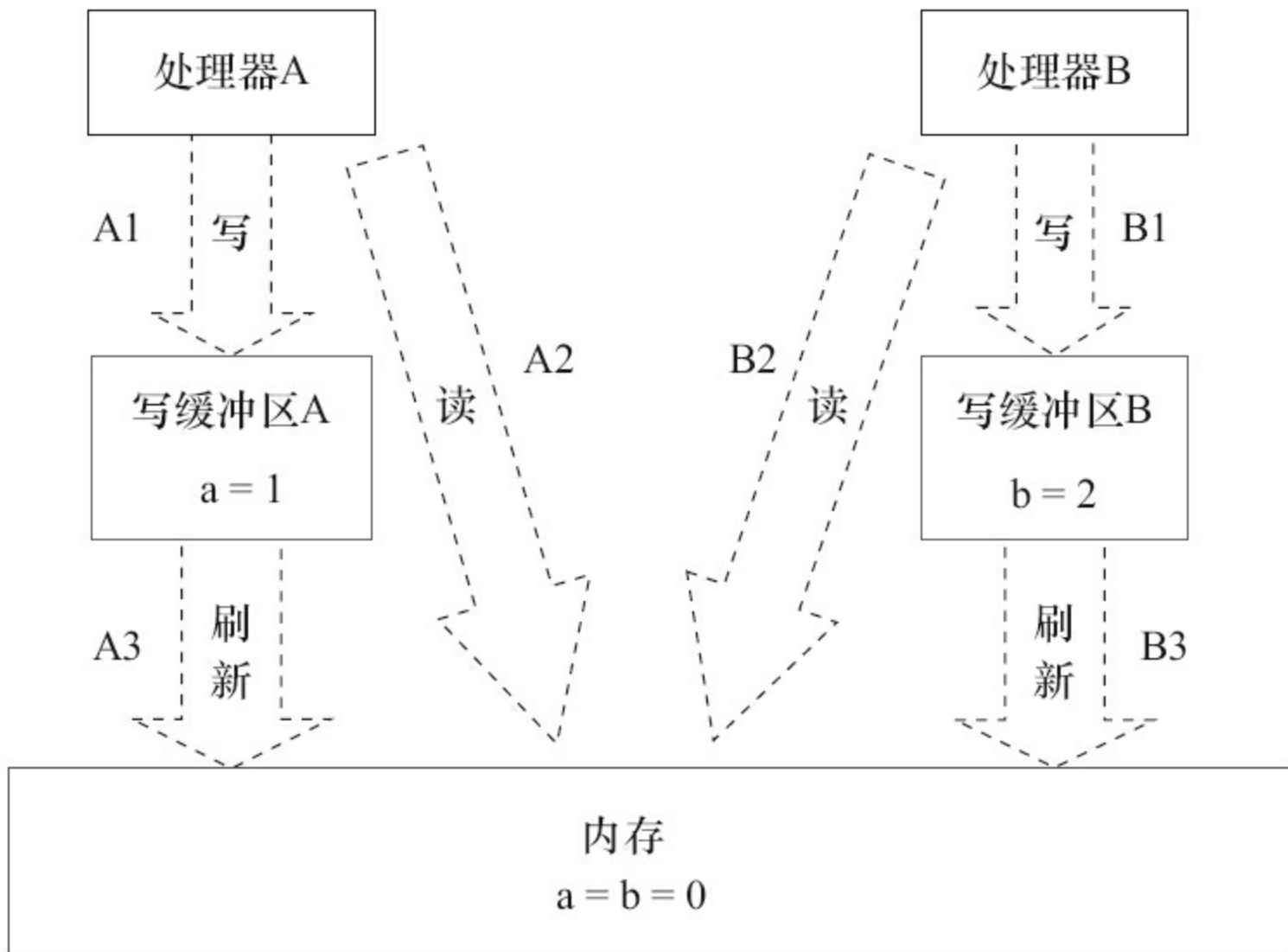


图3-4 处理器和内存的交互

这里处理器A和处理器B可以同时把共享变量写入自己的写缓冲区(A1, B1), 然后从内存中读取另一个共享变量(A2, B2), 最后才把自己写缓存区中保存的脏数据刷新到内存中(A3, B3)。当以这种时序执行时, 程序就可以得到 $x=y=0$ 的结果。

从内存操作实际发生的顺序来看, 直到处理器A执行A3来刷新自己的写缓存区, 写操作A1才算真正执行了。虽然处理器A执行内存操作的顺序为: $A1 \rightarrow A2$, 但内存操作实际发生的顺序却是 $A2 \rightarrow A1$ 。此时, 处理器A的内存操作顺序被重排序了(处理器B的情况和处理器A一样, 这里就不赘述了)。

这里的关键是, 由于写缓冲区仅对自己的处理器可见, 它会导致处理器执行内存操作的顺序可能会与内存实际的操作执行顺序不一致。由于现代的处理器的都会使用写缓冲区, 因此

现代的处理器都会允许对写-读操作进行重排序。

表3-2是常见处理器允许的重排序类型的列表。

表3-2 处理器的重排序规则

规则 处理器	Load-Load	Load-Store	Store-Store	Store-Load	数据依赖
SPARC-TSO	N	N	N	Y	N
x86	N	N	N	Y	N
IA64	Y	Y	Y	Y	N
PowerPC	Y	Y	Y	Y	N

注意，表3-2单元格中的“N”表示处理器不允许两个操作重排序，“Y”表示允许重排序。

从表3-2我们可以看出：常见的处理器都允许Store-Load重排序；常见的处理器都不允许对存在数据依赖的操作做重排序。sparc-TSO和X86拥有相对较强的处理器内存模型，它们仅允许对写-读操作做重排序（因为它们都使用了写缓冲区）。



注意

- sparc-TSO是指以TSO(Total Store Order)内存模型运行时sparc处理器的特性。
- 表3-2中的X86包括X64及AMD64。
- 由于ARM处理器的内存模型与PowerPC处理器的内存模型非常类似，本文将忽略它。
- 数据依赖性后文会专门说明。

为了保证内存可见性，Java编译器在生成指令序列的适当位置会插入内存屏障指令来禁止特定类型的处理器重排序。JMM把内存屏障指令分为4类，如表3-3所示。

表3-3 内存屏障类型表

屏障类型	指令示例	说 明
LoadLoad Barriers	Load1; LoadLoad; Load2	确保 Load1 数据的装载先于 Load2 及所有后续装载指令的装载
StoreStore Barriers	Store1; StoreStore; Store2	确保 Store1 数据对其他处理器可见（刷新到内存）先于 Store2 及所有后续存储指令的存储
LoadStore Barriers	Load1; LoadStore; Store2	确保 Load1 数据装载先于 Store2 及所有后续的存储指令刷新到内存
StoreLoad Barriers	Store1; StoreLoad; Load2	确保 Store1 数据对其他处理器变得可见（指刷新到内存）先于 Load2 及所有后续装载指令的装载。StoreLoad Barriers 会使该屏障之前的所有内存访问指令（存储和装载指令）完成之后，才执行该屏障之后的内存访问指令

StoreLoad Barriers是一个“全能型”的屏障，它同时具有其他3个屏障的效果。现代的多处理器大多支持该屏障（其他类型的屏障不一定被所有处理器支持）。执行该屏障开销会很昂贵，因为当前处理器通常要把写缓冲区中的数据全部刷新到内存中（Buffer Fully Flush）。

3.1.5 happens-before简介

从JDK 5开始, Java使用新的JSR-133内存模型(除非特别说明, 本文针对的都是JSR-133内存模型)。JSR-133使用happens-before的概念来阐述操作之间的内存可见性。**在JMM中, 如果一个操作执行的结果需要对另一个操作可见, 那么这两个操作之间必须要存在happens-before关系。**这里提到的两个操作既可以是在一个线程之内, 也可以是在不同线程之间。

与程序员密切相关的happens-before规则如下。

- 程序顺序规则: 一个线程中的每个操作, happens-before于该线程中的任意后续操作。
- 监视器锁规则: 对一个锁的解锁, happens-before于随后对这个锁的加锁。
- volatile变量规则: 对一个volatile域的写, happens-before于任意后续对这个volatile域的读。
- 传递性: 如果A happens-before B, 且B happens-before C, 那么A happens-before C。

 **注意** 两个操作之间具有happens-before关系, 并不意味着前一个操作必须要在后一个操作之前执行!**happens-before仅仅要求前一个操作(执行的结果)对后一个操作可见, 且前一个操作按顺序排在第二个操作之前(the first is visible to and ordered before the second)。**happens-before的定义很微妙, 后文会具体说明happens-before为什么要这么定义。

happens-before与JMM的关系如图3-5所示。

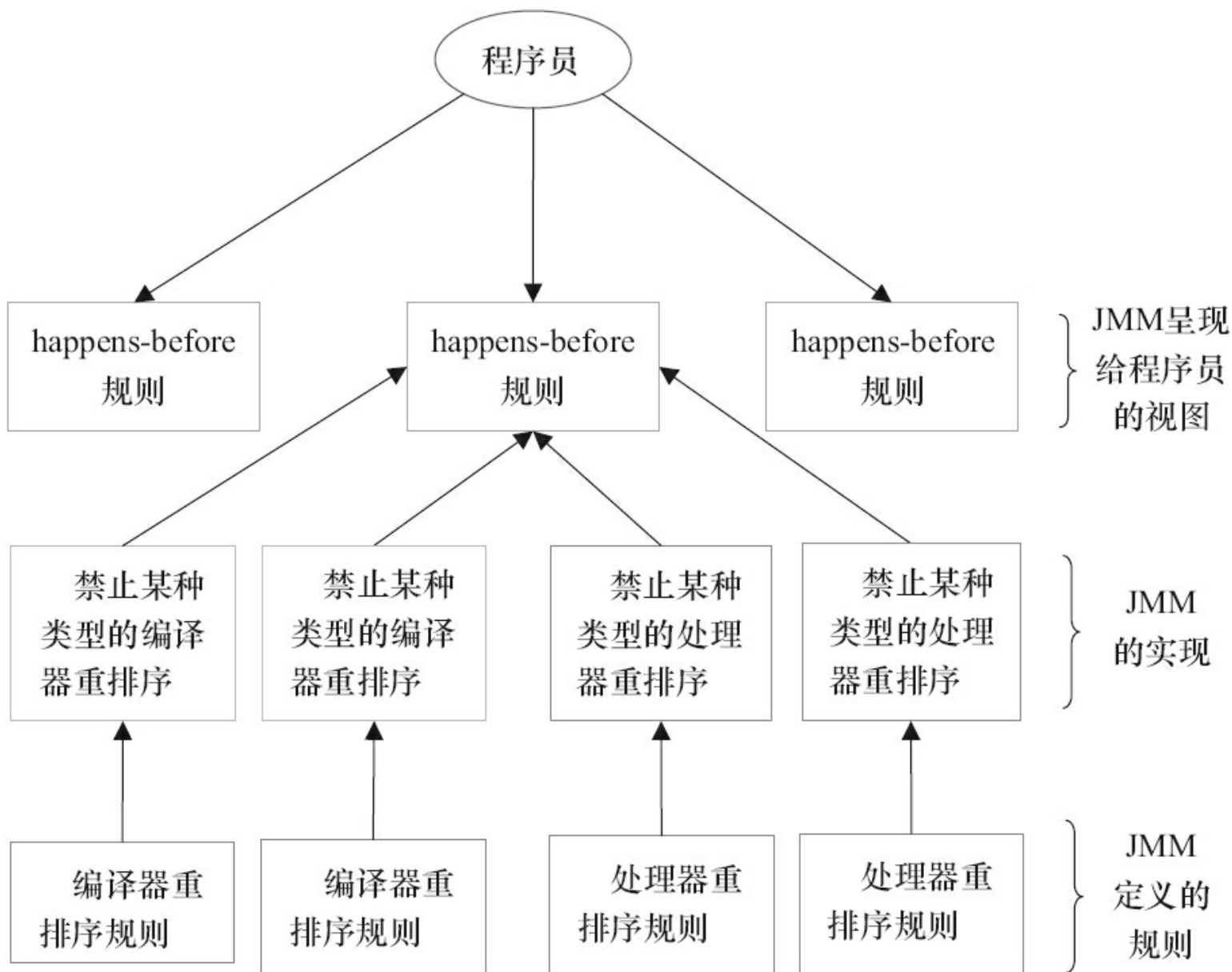


图3-5 happens-before与JMM的关系

如图3-5所示，一个happens-before规则对应于一个或多个编译器和处理器重排序规则。对于Java程序员来说，happens-before规则简单易懂，它避免Java程序员为了理解JMM提供的内存可见性保证而去学习复杂的重排序规则以及这些规则的具体实现方法。

3.2 重排序

重排序是指编译器和处理器为了优化程序性能而对指令序列进行重新排序的一种手段。

3.2.1 数据依赖性

如果两个操作访问同一个变量, 且这两个操作中有一个为写操作, 此时这两个操作之间就存在数据依赖性。数据依赖分为下列3种类型, 如表3-4所示。

表3-4 数据依赖类型表

名 称	代码示例	说 明
写后读	a = 1; b = a;	写一个变量之后, 再读这个位置
写后写	a = 1; a = 2;	写一个变量之后, 再写这个变量
读后写	a = b; b = 1;	读一个变量之后, 再写这个变量

上面3种情况, 只要重排序两个操作的执行顺序, 程序的执行结果就会被改变。

前面提到过, 编译器和处理器可能会对操作做重排序。编译器和处理器在重排序时, 会遵守数据依赖性, 编译器和处理器不会改变存在数据依赖关系的两个操作的执行顺序。

这里所说的数据依赖性仅针对单个处理器中执行的指令序列和单个线程中执行的操作, 不同处理器之间和不同线程之间的数据依赖性不被编译器和处理器考虑。

3.2.2 as-if-serial语义

as-if-serial语义的意思是：不管怎么重排序(编译器和处理器为了提高并行度)，(单线程)程序的执行结果不能被改变。**编译器、runtime和处理器都必须遵守as-if-serial语义。**

为了遵守as-if-serial语义，编译器和处理器不会对存在数据依赖关系的操作做重排序，因为这种重排序会改变执行结果。但是，如果操作之间不存在数据依赖关系，这些操作就可能被编译器和处理器重排序。为了具体说明，请看下面计算圆面积的代码示例。

```
double pi  = 3.14;           // A
double r   = 1.0;            // B
double area = pi * r * r;     // C
```

上面3个操作的数据依赖关系如图3-6所示。

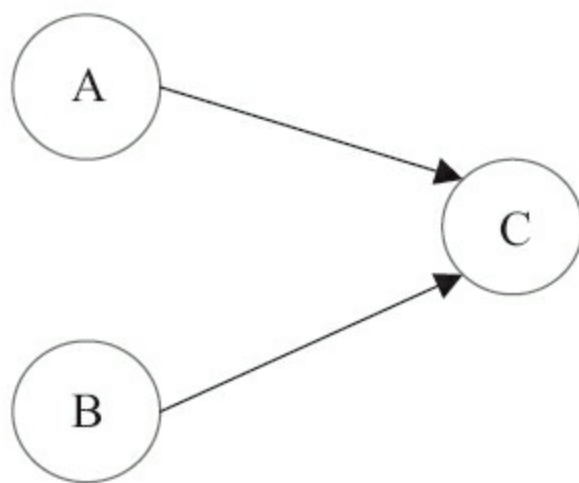
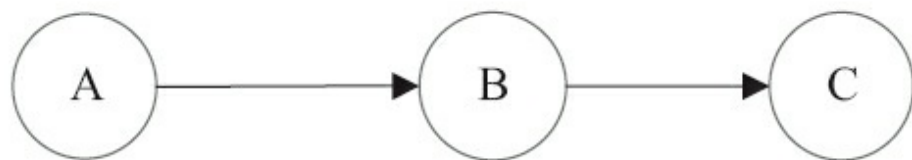
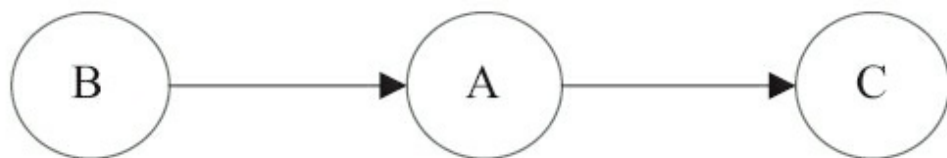


图3-6 3个操作之间的依赖关系

如图3-6所示，A和C之间存在数据依赖关系，同时B和C之间也存在数据依赖关系。因此在最终执行的指令序列中，C不能被重排序到A和B的前面(C排到A和B的前面，程序的结果将会被改变)。但A和B之间没有数据依赖关系，编译器和处理器可以重排序A和B之间的执行顺序。图3-7是该程序的两种执行顺序。



按程序顺序的执行结果：
area = 3.14



重排序后的执行结果：
area = 3.14

图3-7 程序的两种执行顺序

as-if-serial语义把单线程程序保护了起来, 遵守as-if-serial语义的编译器、runtime和处理器

共同为编写单线程程序的程序员创建了一个幻觉: 单线程程序是按程序的顺序来执行的。as-

if-serial语义使单线程程序员无需担心重排序会干扰他们, 也无需担心内存可见性问题。

3.2.3 程序顺序规则

根据happens-before的程序顺序规则, 上面计算圆的面积的示例代码存在3个happens-before关系。

1) A happens-before B。

2) B happens-before C。

3) A happens-before C。

这里的第3个happens-before关系, 是根据happens-before的传递性推导出来的。

这里A happens-before B, 但实际执行时B却可以排在A之前执行(看上面的重排序后的执行顺序)。如果A happens-before B, JMM并不要求A一定要在B之前执行。JMM仅仅要求前一个操作(执行的结果)对后一个操作可见, 且前一个操作按顺序排在第二个操作之前。这里操作A的执行结果不需要对操作B可见; 而且重排序操作A和操作B后的执行结果, 与操作A和操作B按happens-before顺序执行的结果一致。在这种情况下, JMM会认为这种重排序并不非法(not illegal), JMM允许这种重排序。

在计算机中, 软件技术和硬件技术有一个共同的目标: 在不改变程序执行结果的前提下, 尽可能提高并行度。编译器和处理器遵从这一目标, 从happens-before的定义我们可以看出, JMM同样遵从这一目标。

3.2.4 重排序对多线程的影响

现在让我们来看看，重排序是否会改变多线程程序的执行结果。请看下面的示例代码。

```
class ReorderExample {
    int a = 0;
    boolean flag = false;
    public void writer() {
        a = 1;                // 1
        flag = true;          // 2
    }
    public void reader() {
        if (flag) {           // 3
            int i = a * a;     // 4
            .....
        }
    }
}
```

flag变量是个标记，用来标识变量a是否已被写入。这里假设有两个线程A和B，A首先执行writer()方法，随后B线程接着执行reader()方法。线程B在执行操作4时，能否看到线程A在操作1对共享变量a的写入呢？

答案是：不一定能看到。

由于操作1和操作2没有数据依赖关系，编译器和处理器可以对这两个操作重排序；同样，操作3和操作4没有数据依赖关系，编译器和处理器也可以对这两个操作重排序。让我们先来看看，当操作1和操作2重排序时，可能会产生什么效果？请看下面的程序执行时序图，如图3-8所示。

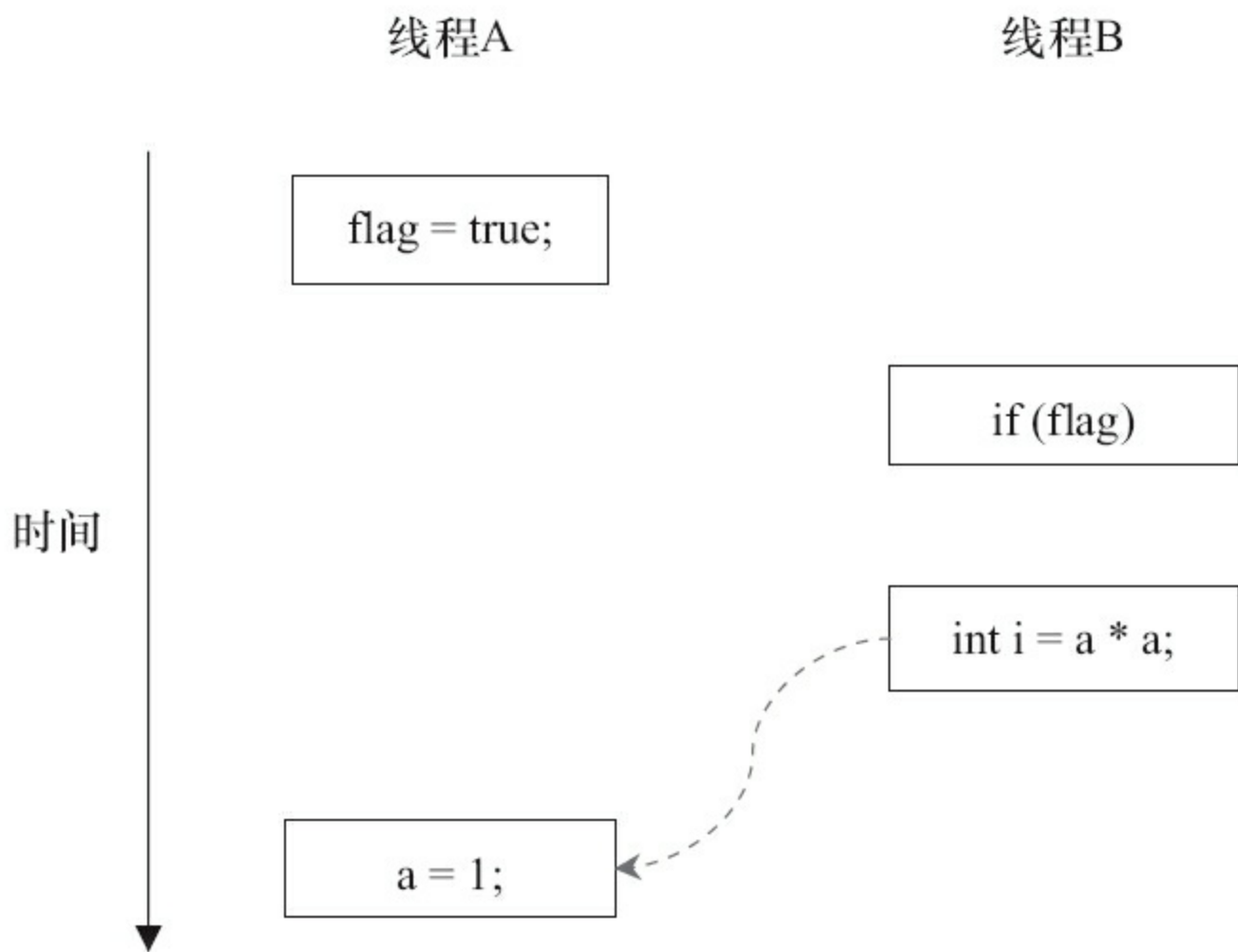


图3-8 程序执行时序图

如图3-8所示，操作1和操作2做了重排序。程序执行时，线程A首先写标记变量flag，随后线程B读这个变量。由于条件判断为真，线程B将读取变量a。此时，变量a还没有被线程A写入，在这里多线程程序的语义被重排序破坏了！

 **注意** 本文统一用虚箭线标识错误的读操作，用实箭线标识正确的读操作。

下面再让我们看看，当操作3和操作4重排序时会产生什么效果（借助这个重排序，可以顺便说明控制依赖性）。下面是操作3和操作4重排序后，程序执行的时序图，如图3-9所示。

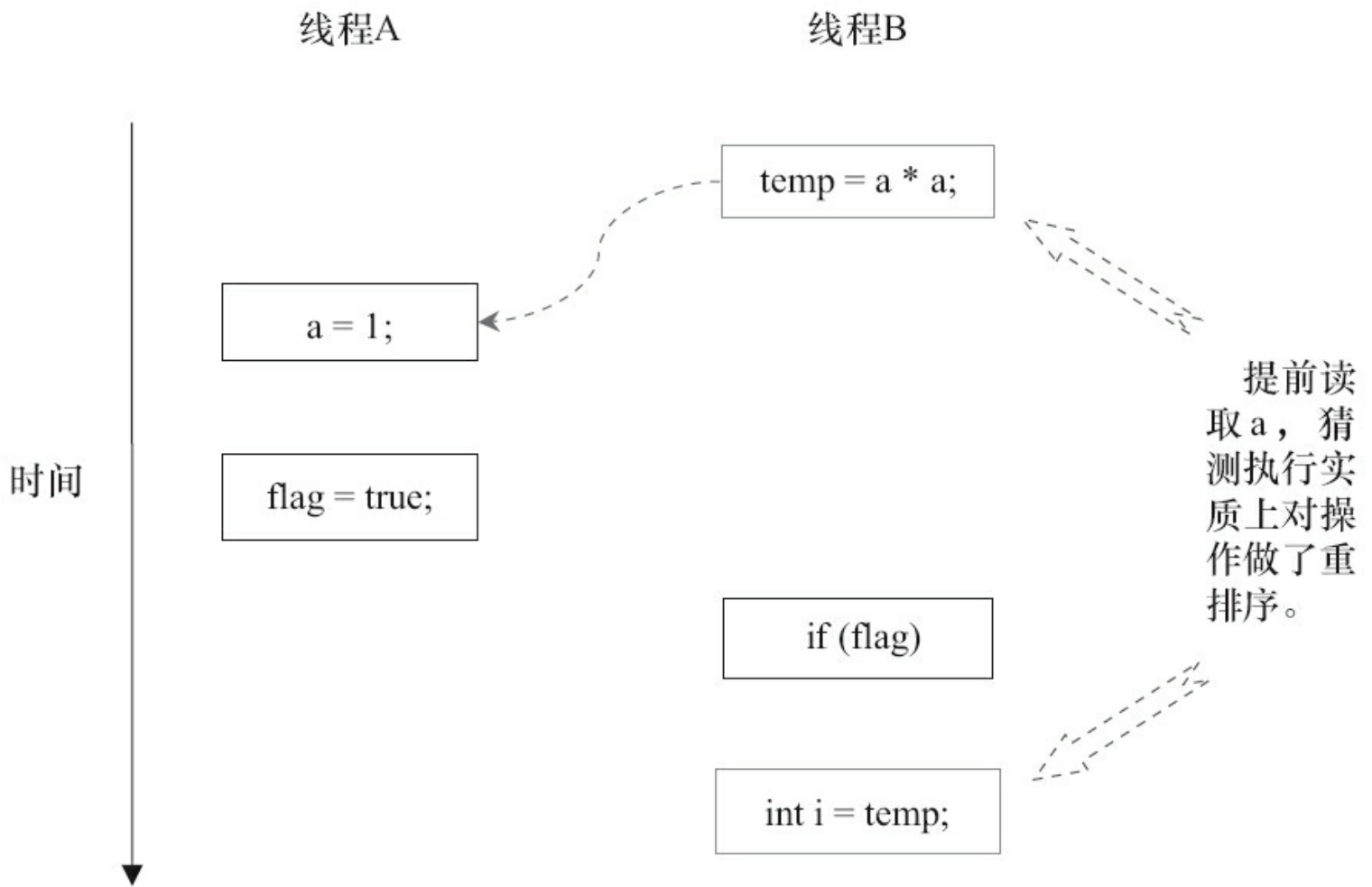


图3-9 程序的执行时序图

在程序中，操作3和操作4存在控制依赖关系。当代码中存在控制依赖性时，会影响指令序列执行的并行度。为此，编译器和处理器会采用猜测 (Speculation) 执行来克服控制相关性对并行度的影响。以处理器的猜测执行为例，执行线程B的处理器可以提前读取并计算 $a * a$ ，然后把计算结果临时保存到一个名为重排序缓冲 (Reorder Buffer, ROB) 的硬件缓存中。当操作3的条件判断为真时，就把该计算结果写入变量 `i` 中。

从图3-9中我们可以看出，猜测执行实质上对操作3和4做了重排序。重排序在这里破坏了多线程程序的语义！

在单线程程序中，对存在控制依赖的操作重排序，不会改变执行结果 (这也是 `as-if-serial` 语义允许对存在控制依赖的操作做重排序的原因)；但在多线程程序中，对存在控制依赖的操作重排序，可能会改变程序的执行结果。

3.3 顺序一致性

顺序一致性内存模型是一个理论参考模型，在设计的时候，处理器的内存模型和编程语言的内存模型都会以顺序一致性内存模型作为参照。

3.3.1 数据竞争与顺序一致性

当程序未正确同步时，就可能会存在数据竞争。**Java内存模型规范对数据竞争的定义如下。**

在一个线程中写一个变量，

在另一个线程读同一个变量，

而且写和读没有通过同步来排序。

当代码中包含数据竞争时，程序的执行往往产生违反直觉的结果（前一章的示例正是如此）。如果一个多线程程序能正确同步，这个程序将是一个没有数据竞争的程序。

JMM对正确同步的多线程程序的内存一致性做了如下保证。

如果程序是正确同步的，程序的执行将具有顺序一致性(Sequentially Consistent)——即程序的执行结果与该程序在顺序一致性内存模型中的执行结果相同。马上我们就会看到，这对于程序员来说是一个极强的保证。这里的同步是指广义上的同步，包括对常用同步原语(synchronized、volatile和final)的正确使用。

3.3.2 顺序一致性内存模型

顺序一致性内存模型是一个被计算机科学家理想化了的理论参考模型，它为程序员提供了极强的内存可见性保证。顺序一致性内存模型有两大特性。

1) 一个线程中的所有操作必须按照程序的顺序来执行。

2) (不管程序是否同步) 所有线程都只能看到一个单一的操作执行顺序。在顺序一致性内存模型中，每个操作都必须原子执行且立刻对所有线程可见。

顺序一致性内存模型为程序员提供的视图如图3-10所示。

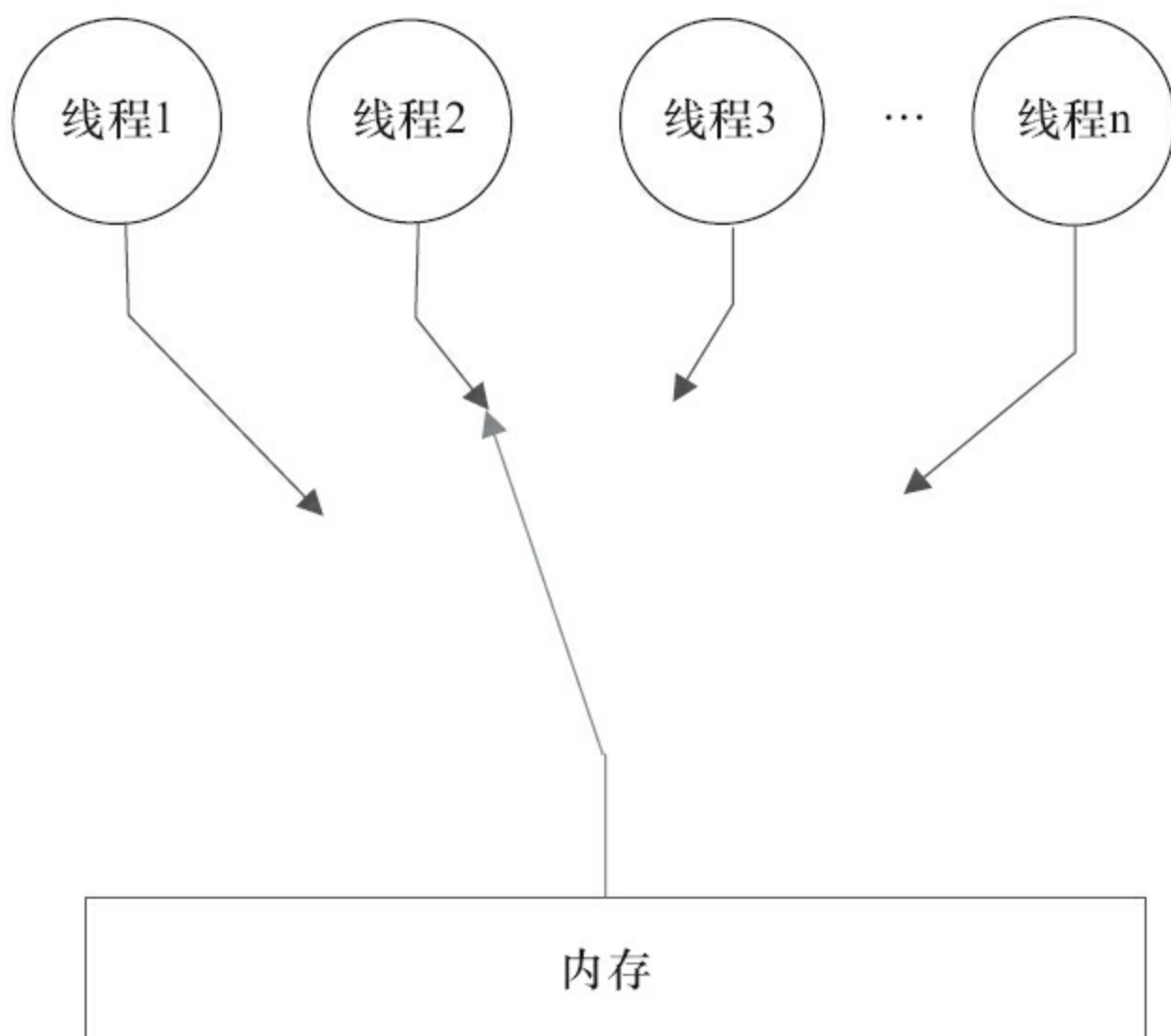


图3-10 顺序一致性内存模型的视图

在概念上，顺序一致性模型有一个单一的全局内存，这个内存通过一个左右摆动的开关可以连接到任意一个线程，同时每一个线程必须按照程序的顺序来执行内存读/写操作。从上面的示意图可以看出，在任意时间点最多只能有一个线程可以连接到内存。当多个线程并发执行时，图中的开关装置能把所有线程的所有内存读/写操作串行化(即在顺序一致性模型中，所有操作之间具有全序关系)。

为了更好进行理解，下面通过两个示意图来对顺序一致性模型的特性做进一步的说明。

假设有两个线程A和B并发执行。其中A线程有3个操作，它们在程序中的顺序是：
 $A1 \rightarrow A2 \rightarrow A3$ 。B线程也有3个操作，它们在程序中的顺序是： $B1 \rightarrow B2 \rightarrow B3$ 。

假设这两个线程使用监视器锁来正确同步：A线程的3个操作执行后释放监视器锁，随后B线程获取同一个监视器锁。那么程序在顺序一致性模型中的执行效果将如图3-11所示。

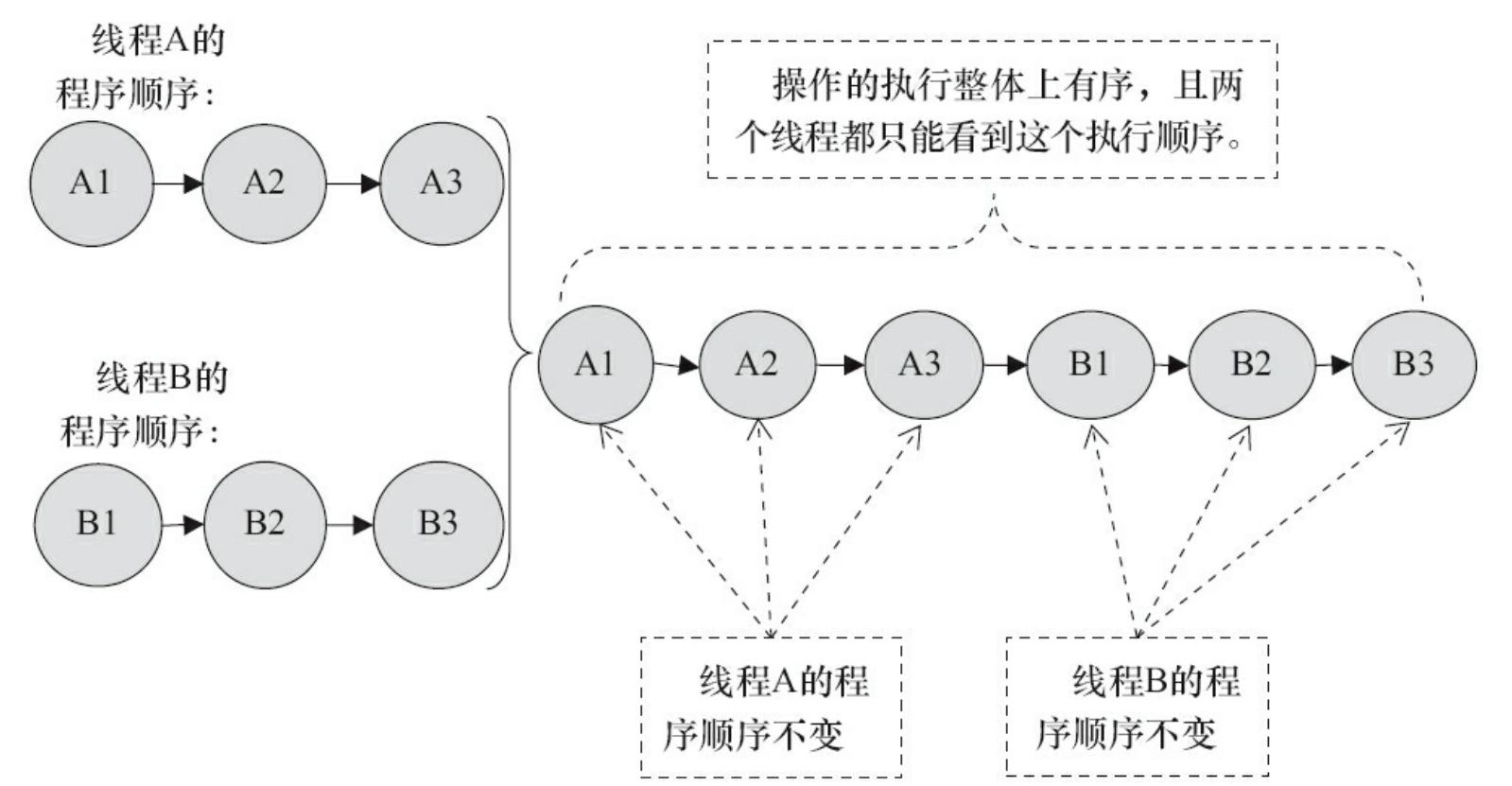


图3-11 顺序一致性模型的一种执行效果

现在再假设这两个线程没有做同步，下面是这个未同步程序在顺序一致性模型中的

执行示意图，如图3-12所示。

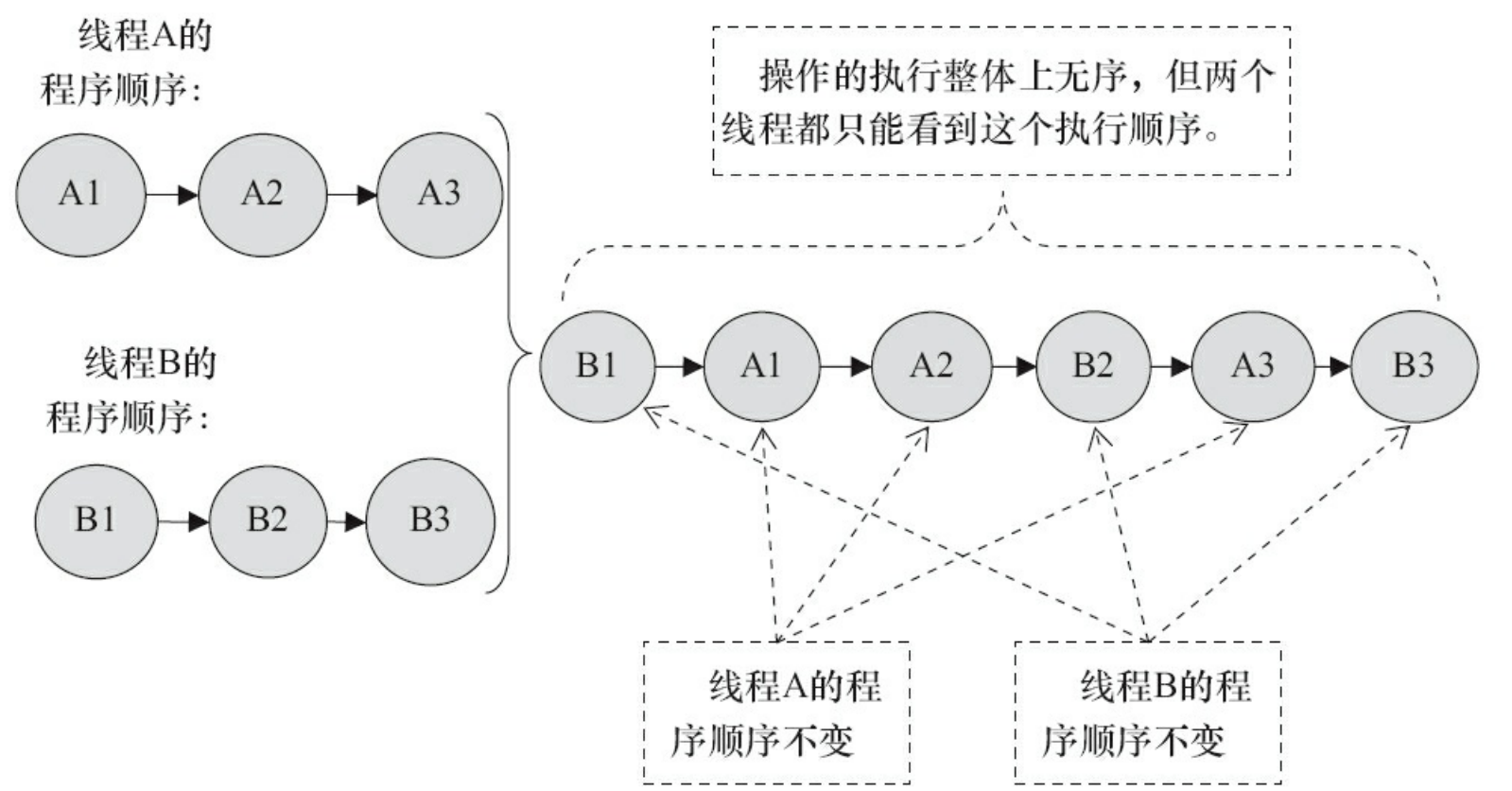


图3-12 顺序一致性模型中的另一种执行效果

未同步程序在顺序一致性模型中虽然整体执行顺序是无序的，但所有线程都只能看到一个一致的整体执行顺序。以上图为例，线程A和B看到的执行顺序都是：
B1→A1→A2→B2→A3→B3。之所以能得到这个保证是因为**顺序一致性内存模型中的每个操作必须立即对任意线程可见。**

但是，在JMM中就没有这个保证。未同步程序在JMM中不但整体的执行顺序是无序的，而且所有线程看到的操作执行顺序也可能不一致。比如，在当前线程把写过的数据缓存在本地内存中，在没有刷新到主内存之前，这个写操作仅对当前线程可见；从其他线程的角度来观察，会认为这个写操作根本没有被当前线程执行。只有当前线程把本地内存中写过的数据刷新到主内存之后，这个写操作才能对其他线程可见。在这种情况下，当前线程和其他线程看到的操作执行顺序将不一致。

3.3.3 同步程序的顺序一致性效果

下面, 对前面的示例程序ReorderExample用锁来同步, 看看正确同步的程序如何具有顺序一致性。

请看下面的示例代码。

```
class SynchronizedExample {
    int a = 0;
    boolean flag = false;
    public synchronized void writer() {        // 获取锁
        a = 1;
        flag = true;
    }                                           // 释放锁
    public synchronized void reader() {        // 获取锁
        if (flag) {
            int i = a;
            .....
        }                                     // 释放锁
    }
}
```

在上面示例代码中, 假设A线程执行writer()方法后, B线程执行reader()方法。这是一个正确同步的多线程程序。根据JMM规范, 该程序的执行结果将与该程序在顺序一致性模型中的执行结果相同。下面是该程序在两个内存模型中的执行时序对比图, 如图3-13所示。

顺序一致性模型中, 所有操作完全按程序的顺序串行执行。而在JMM中, 临界区内的代码可以重排序(但JMM不允许临界区内的代码“逸出”到临界区之外, 那样会破坏监视器的语义)。JMM会在退出临界区和进入临界区这两个关键时间点做一些特别处理, 使得线程在这两个时间点具有与顺序一致性模型相同的内存视图(具体细节后文会说明)。虽然线程A在临界区内做了重排序, 但由于监视器互斥执行的特性, 这里的线程B根本无法“观察”到线程A在临界区内的重排序。这种重排序既提高了执行效率, 又没有改变程序的执行结果。

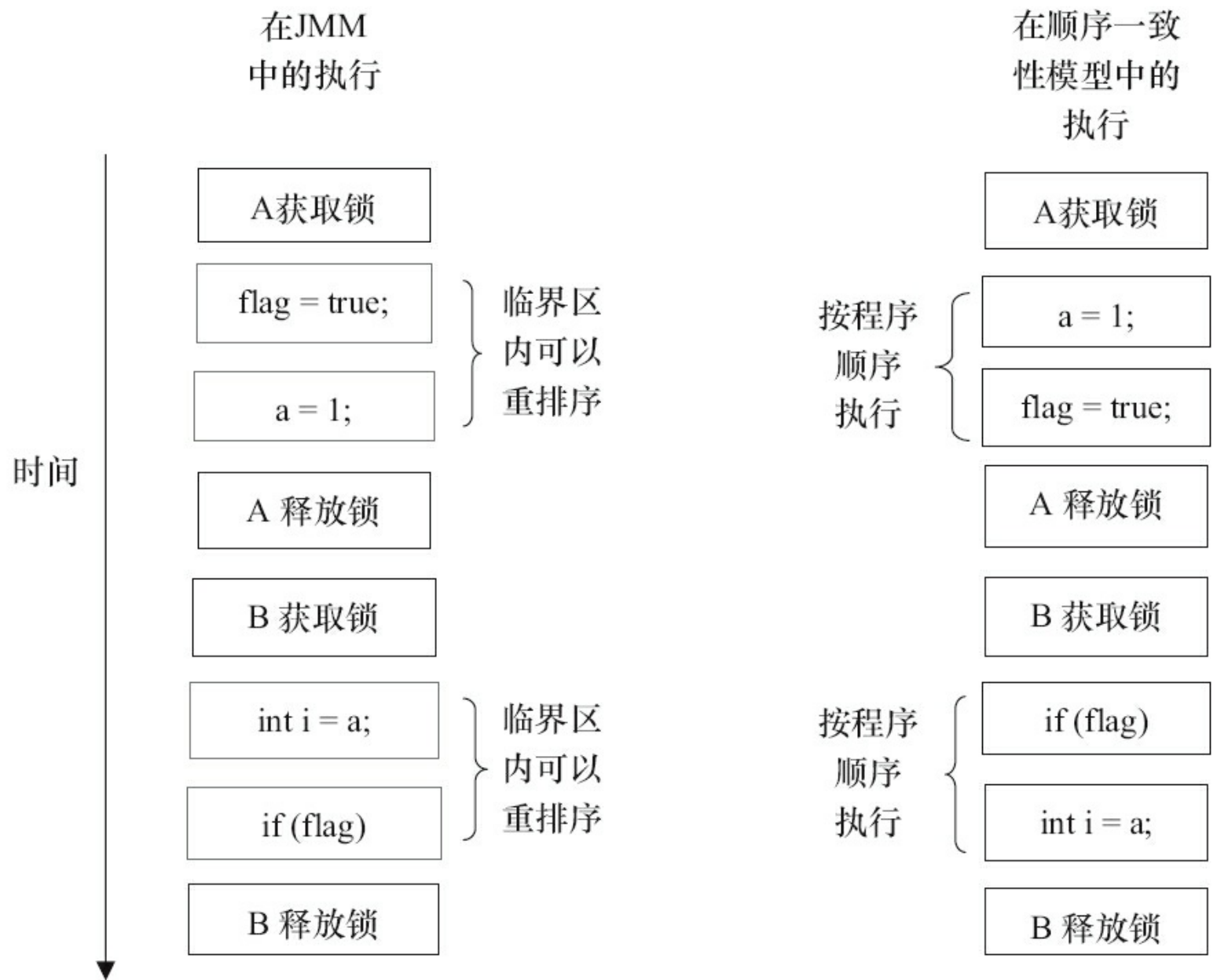


图3-13 两个内存模型中的执行时序对比图

从这里我们可以看到, JMM在具体实现上的基本方针为:在不改变(正确同步的)程序执行结果的前提下,尽可能地为编译器和处理器的优化打开方便之门。

3.3.4 未同步程序的执行特性

对于未同步或未正确同步的多线程程序, JMM只提供最小安全性:线程执行时读取到的值, 要么是之前某个线程写入的值, 要么是默认值(0, Null, False), JMM保证线程读操作读取到的值不会无中生有(Out Of Thin Air)的冒出来。为了实现最小安全性, JVM在堆上分配对象时, 首先会对内存空间进行清零, 然后才会在上面分配对象(JVM内部会同步这两个操作)。因此, 在已清零的内存空间(Pre-zeroed Memory)分配对象时, 域的默认初始化已经完成了。

JMM不保证未同步程序的执行结果与该程序在顺序一致性模型中的执行结果一致。因为如果想要保证执行结果一致, JMM需要禁止大量的处理器和编译器的优化, 这对程序的执行性能会产生很大的影响。而且未同步程序在顺序一致性模型中执行时, 整体是无序的, 其执行结果往往无法预知。而且, 保证未同步程序在这两个模型中的执行结果一致没什么意义。

未同步程序在JMM中的执行时, 整体上是无序的, 其执行结果无法预知。**未同步程序在两个模型中的执行特性有如下几个差异。**

1) 顺序一致性模型保证单线程内的操作会按程序的顺序执行, 而JMM不保证单线程内的操作会按程序的顺序执行(比如上面正确同步的多线程程序在临界区内的重排序)。这一点前面已经讲过了, 这里就不再赘述。

2) 顺序一致性模型保证所有线程只能看到一致的操作执行顺序, 而JMM不保证所有线程能看到一致的操作执行顺序。这一点前面也已经讲过, 这里就不再赘述。

3) JMM不保证对64位的long型和double型变量的写操作具有原子性, 而顺序一致性模型保证对所有的内存读/写操作都具有原子性。

第3个差异与处理器总线的工作机制密切相关。在计算机中, 数据通过总线在处理器和内存之间传递。每次处理器和内存之间的数据传递都是通过一系列步骤来完成的, 这一系列步

骤称之为总线事务(Bus Transaction)。总线事务包括读事务(Read Transaction)和写事务(Write Transaction)。读事务从内存传送数据到处理器，写事务从处理器传送数据到内存，每个事务会读/写内存中一个或多个物理上连续的字。这里的关键是，总线会同步试图并发使用总线的事务。在一个处理器执行总线事务期间，总线会禁止其他的处理器和I/O设备执行内存的读/写。下面，让我们通过一个示意图来说明总线的工作机制，如图3-14所示。

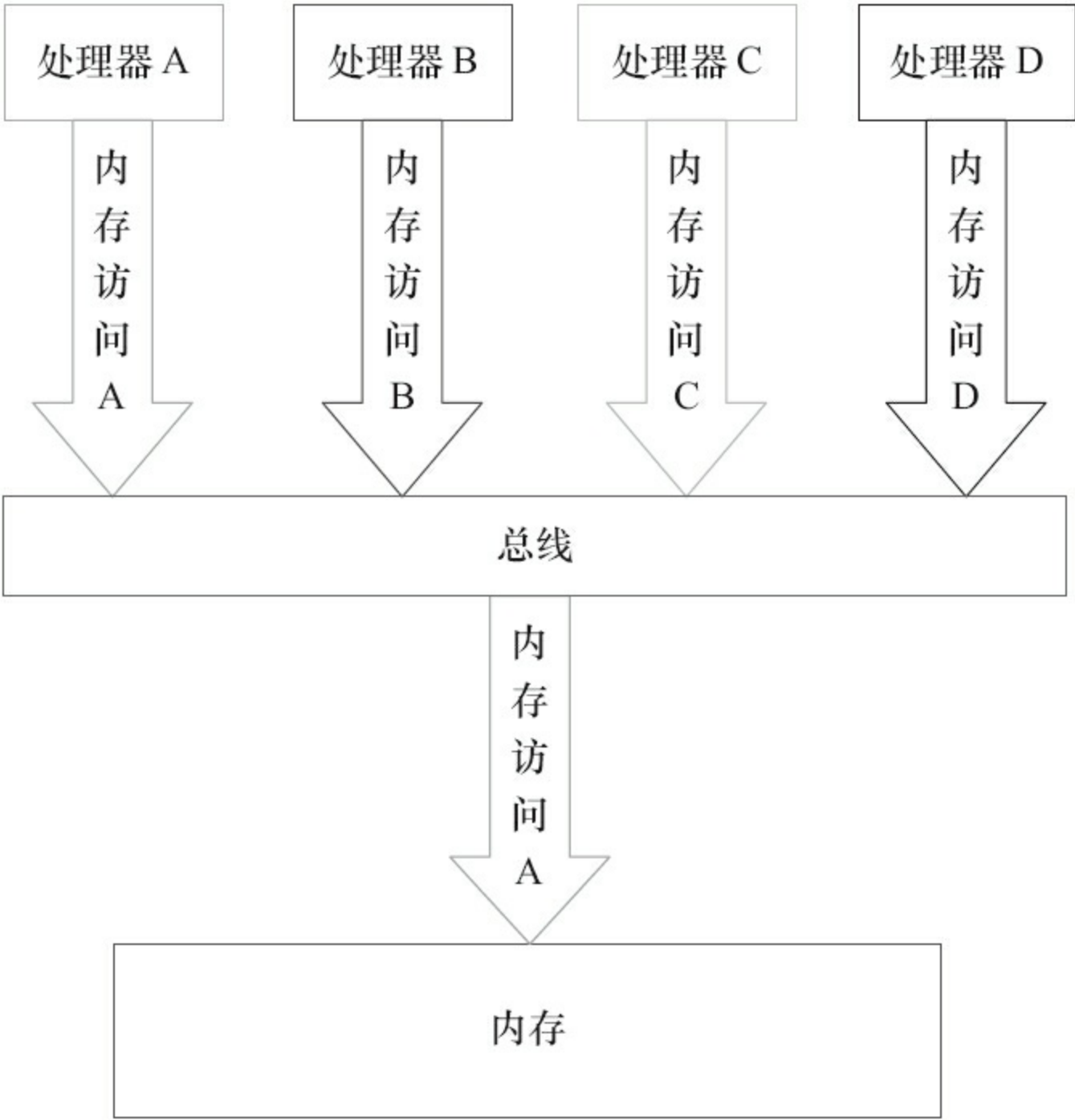


图3-14 总线的工作机制

由图可知，假设处理器A、B和C同时向总线发起总线事务，这时总线仲裁(Bus Arbitration)会对竞争做出裁决，这里假设总线在仲裁后判定处理器A在竞争中获胜(总线仲裁会确保所有处理器都能公平的访问内存)。此时处理器A继续它的总线事务，而其他两个处理器则要等待处理器A的总线事务完成后才能再次执行内存访问。假设在处理器A执行总线事务期间(不管这个总线事务是读事务还是写事务)，处理器D向总线发起了总线事务，此时处理器D的请求会被总线禁止。

总线的这些工作机制可以把所有处理器对内存的访问以串行化的方式来执行。在任意时间点，最多只能有一个处理器可以访问内存。这个特性确保了单个总线事务之中的内存读/写操作具有原子性。

在一些32位的处理器上，如果要求对64位数据的写操作具有原子性，会有比较大的开销。为了照顾这种处理器，Java语言规范鼓励但不强求JVM对64位的long型变量和double型变量的写操作具有原子性。当JVM在这种处理器上运行时，可能会把一个64位long/double型变量的写操作拆分为两个32位的写操作来执行。这两个32位的写操作可能会被分配到不同的总线事务中执行，此时对这个64位变量的写操作将不具有原子性。

当单个内存操作不具有原子性时，可能会产生意想不到的后果。请看示意图，如图3-15所示。

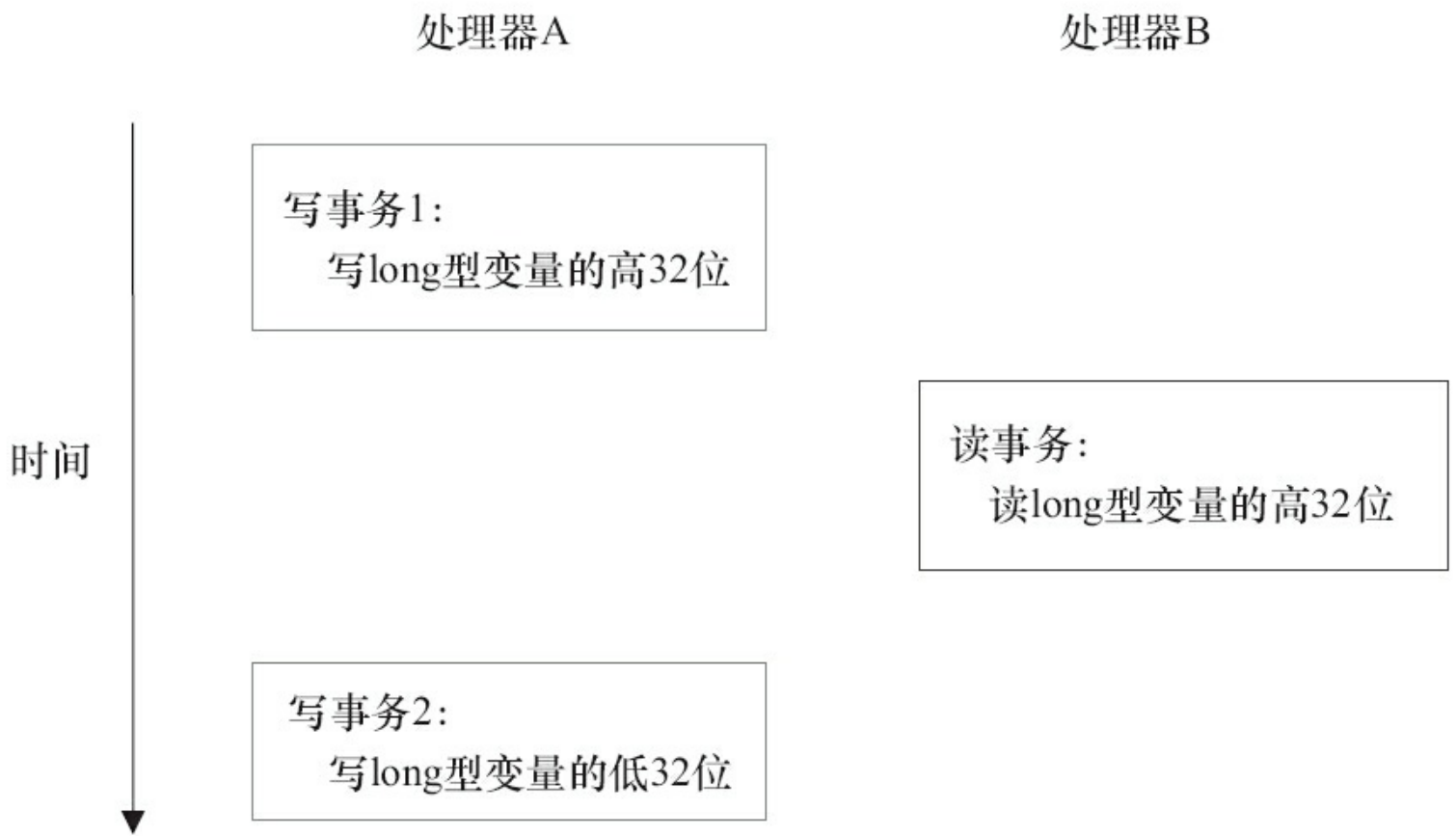


图3-15 总线事务执行的时序图

如上图所示, 假设处理器A写一个long型变量, 同时处理器B要读这个long型变量。处理器A中64位的写操作被拆分为两个32位的写操作, 且这两个32位的写操作被分配到不同的写事务中执行。同时, 处理器B中64位的读操作被分配到单个的读事务中执行。当处理器A和B按上图的时序来执行时, 处理器B将看到仅仅被处理器A“写了一半”的无效值。

注意, 在JSR-133之前的旧内存模型中, 一个64位long/double型变量的读/写操作可以被拆分为两个32位的读/写操作来执行。从JSR-133内存模型开始(即从JDK5开始), 仅仅只允许把一个64位long/double型变量的写操作拆分为两个32位的写操作来执行, 任意的读操作在JSR-133中都必须具有原子性(即任意读操作必须要在单个读事务中执行)。

3.4 volatile的内存语义

当声明共享变量为volatile后, 对这个变量的读/写将会很特别。为了揭开volatile的神秘面纱, 下面将介绍volatile的内存语义及volatile内存语义的实现。

3.4.1 volatile的特性

理解volatile特性的一个好方法是把对volatile变量的单个读/写,看成是使用同一个锁对这些单个读/写操作做了同步。下面通过具体的示例来说明,示例代码如下。

```
class VolatileFeaturesExample {
    volatile long vl = 0L;                // 使用volatile声明64位的long型变量
    public void set(long l) {              // 单个volatile变量的写
        vl = l;
    }
    public void getAndIncrement () {        // 复合(多个)volatile变量的读/写
        vl++;
    }
    public long get() {                    // 单个volatile变量的读
        return vl;
    }
}
```

假设有多个线程分别调用上面程序的3个方法,这个程序在语义上和下面程序等价。

```
class VolatileFeaturesExample {
    long vl = 0L;                          // 64位的long型普通变量
    public synchronized void set(long l) { // 对单个的普通变量的写用同一个锁同步
        vl = l;
    }
    public void getAndIncrement () {        // 普通方法调用
        long temp = get();                 // 调用已同步的读方法
        temp += 1L;                        // 普通写操作
        set(temp);                         // 调用已同步的写方法
    }
    public synchronized long get() {        // 对单个的普通变量的读用同一个锁同步
        return vl;
    }
}
```

如上面示例程序所示,一个volatile变量的单个读/写操作,与一个普通变量的读/写操作都是使用同一个锁来同步,它们之间的执行效果相同。

锁的happens-before规则保证释放锁和获取锁的两个线程之间的内存可见性,这意味着对一个volatile变量的读,总是能看到(任意线程)对这个volatile变量最后的写入。

锁的语义决定了临界区代码的执行具有原子性。这意味着，即使是64位的long型和double型变量，只要它是volatile变量，对该变量的读/写就具有原子性。如果是多个volatile操作或类似于volatile++这种复合操作，这些操作整体上不具有原子性。

简而言之，volatile变量自身具有下列特性。

- 可见性：对一个volatile变量的读，总是能看到(任意线程)对这个volatile变量最后的写入。
- 原子性：对任意单个volatile变量的读/写具有原子性，但类似于volatile++这种复合操作不具有原子性。

3.4.2 volatile写-读建立的happens-before关系

上面讲的是volatile变量自身的特性, 对程序员来说, volatile对线程的内存可见性的影响比volatile自身的特性更为重要, 也更需要我们去关注。

从JSR-133开始(即从JDK5开始), volatile变量的写-读可以实现线程之间的通信。

从内存语义的角度来说, volatile的写-读与锁的释放-获取有相同的内存效果:volatile写和锁的释放有相同的内存语义;volatile读与锁的获取有相同的内存语义。

请看下面使用volatile变量的示例代码。

```
class VolatileExample {
    int a = 0;
    volatile boolean flag = false;
    public void writer() {
        a = 1;           // 1
        flag = true;     // 2
    }
    public void reader() {
        if (flag) {      // 3
            int i = a;    // 4
            .....
        }
    }
}
```

假设线程A执行writer()方法之后, 线程B执行reader()方法。根据happens-before规则, 这个过程建立的happens-before关系可以分为3类:

- 1) 根据程序次序规则, 1 happens-before 2; 3 happens-before 4。
- 2) 根据volatile规则, 2 happens-before 3。
- 3) 根据happens-before的传递性规则, 1 happens-before 4。

上述happens-before关系的图形化表现形式如下。

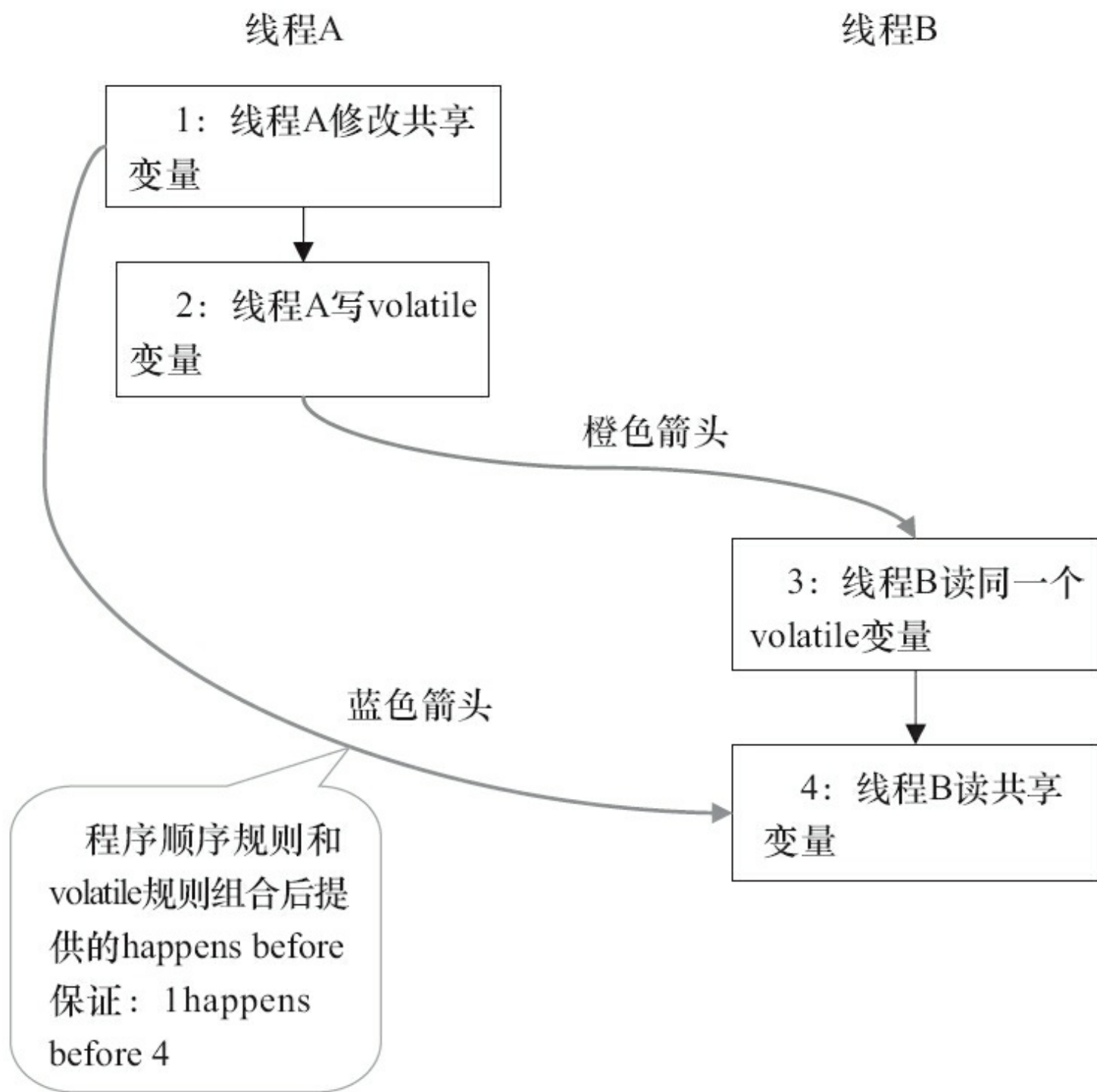


图3-16 happens-before关系

在上图中，每一个箭头链接的两个节点，代表了一个happens-before关系。黑色箭头表示程序顺序规则；橙色箭头表示volatile规则；蓝色箭头表示组合这些规则后提供的happens-before保证。

这里A线程写一个volatile变量后，B线程读同一个volatile变量。A线程在写volatile变量之前所有可见的共享变量，在B线程读同一个volatile变量后，将立即变得对B线程可见。



注意 本文统一用粗实线标识组合后产生的happens-before关系。

3.4.3 volatile写-读的内存语义

volatile写的内存语义如下。

lock命令的两个特性

当写一个volatile变量时, JMM会把该线程对应的本地内存中的共享变量值刷新到主内存。

以上面示例程序VolatileExample为例, 假设线程A首先执行writer()方法, 随后线程B执行reader()方法, 初始时两个线程的本地内存中的flag和a都是初始状态。图3-17是线程A执行volatile写后, 共享变量的状态示意图。

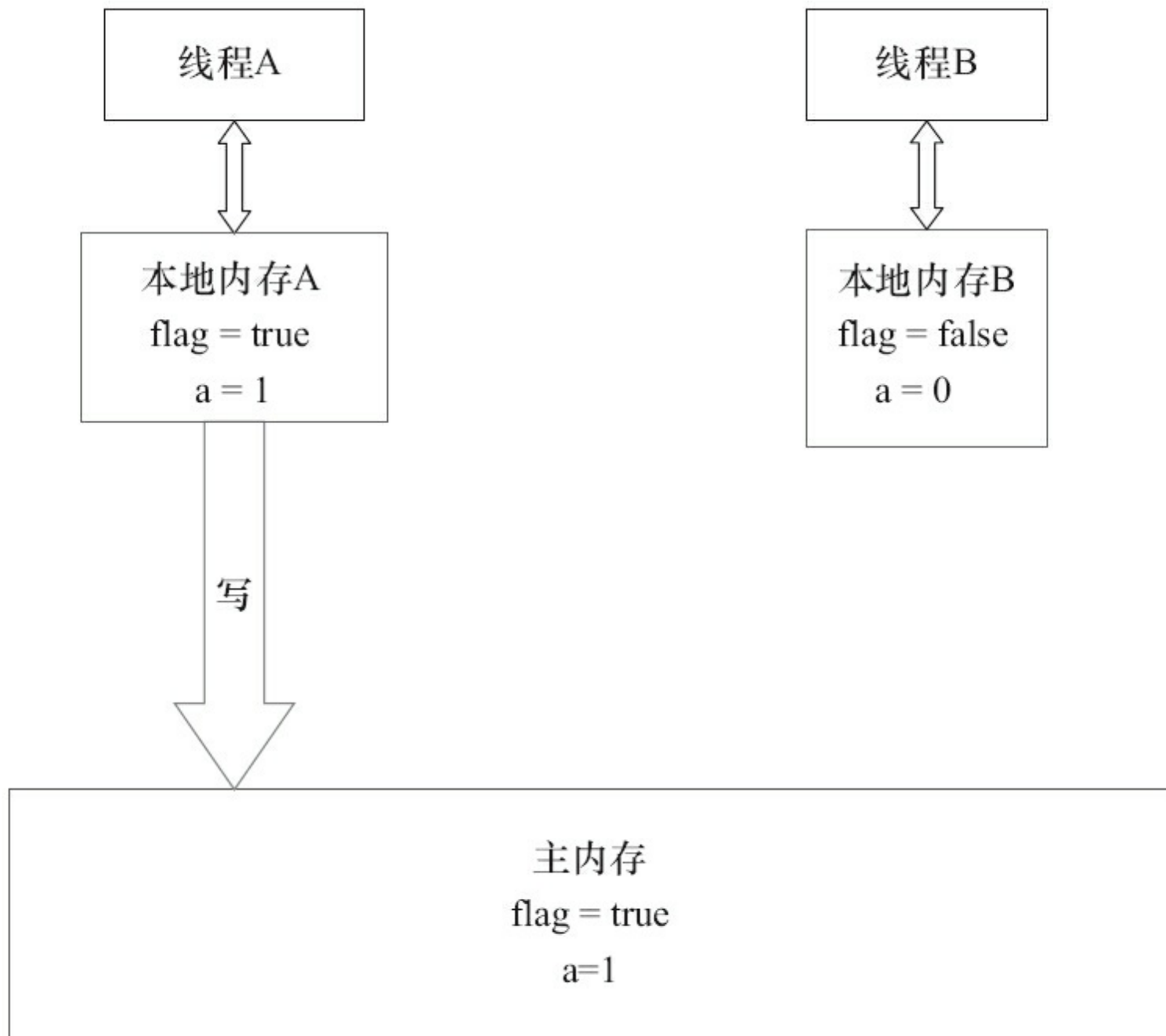


图3-17 共享变量的状态示意图

如图3-17所示，线程A在写flag变量后，本地内存A中被线程A更新过的两个共享变量的值被刷新到主内存中。此时，本地内存A和主内存中的共享变量的值是一致的。

volatile读的内存语义如下。

当读一个volatile变量时，JMM会把该线程对应的本地内存置为无效。线程接下来将从主内存中读取共享变量。

图3-18为线程B读同一个volatile变量后，共享变量的状态示意图。

如图所示，在读flag变量后，本地内存B包含的值已经被置为无效。此时，线程B必须从主内存中读取共享变量。线程B的读取操作将导致本地内存B与主内存中的共享变量的值变成一致。

如果我们把volatile写和volatile读两个步骤综合起来看的话，在读线程B读一个volatile变量后，写线程A在写这个volatile变量之前所有可见的共享变量的值都将立即变得对读线程B可见。

下面对volatile写和volatile读的内存语义做个总结。

- 线程A写一个volatile变量，实质上是线程A向接下来将要读这个volatile变量的某个线程发出了(其对共享变量所做修改的)消息。

- 线程B读一个volatile变量，实质上是线程B接收了之前某个线程发出的(在写这个volatile变量之前对共享变量所做修改的)消息。

线程A写一个volatile变量，随后线程B读这个volatile变量，这个过程实质上是线程A通过主内存向线程B发送消息。

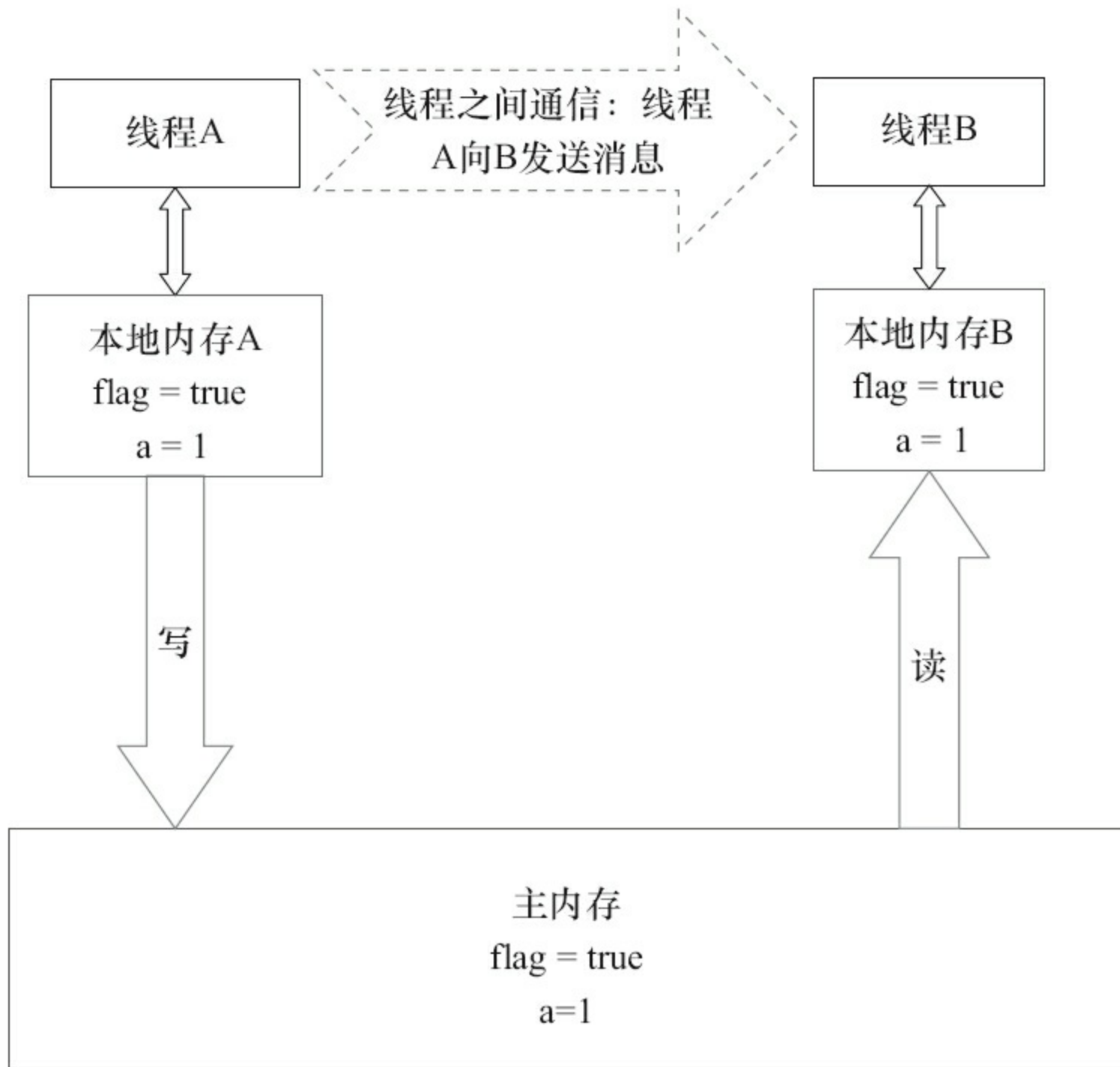


图3-18 共享变量的状态示意图

3.4.4 volatile内存语义的实现

下面来看看JMM如何实现volatile写/读的内存语义。

前文提到过重排序分为编译器重排序和处理器重排序。为了实现volatile内存语义, JMM会分别限制这两种类型的重排序类型。表3-5是JMM针对编译器制定的volatile重排序规则表。

表3-5 volatile重排序规则表

是否能重排序	第二个操作		
第一个操作	普通读 / 写	volatile 读	volatile 写
普通读 / 写			NO
volatile 读	NO	NO	NO
volatile 写		NO	NO

举例来说, 第三行最后一个单元格的意思是: 在程序中, 当第一个操作为普通变量的读或写时, 如果第二个操作为volatile写, 则编译器不能重排序这两个操作。

从表3-5我们可以看出。

- 当第二个操作是volatile写时, 不管第一个操作是什么, 都不能重排序。这个规则确保volatile写之前的操作不会被编译器重排序到volatile写之后。
- 当第一个操作是volatile读时, 不管第二个操作是什么, 都不能重排序。这个规则确保volatile读之后的操作不会被编译器重排序到volatile读之前。
- 当第一个操作是volatile写, 第二个操作是volatile读时, 不能重排序。

为了实现volatile的内存语义, 编译器在生成字节码时, 会在指令序列中插入内存屏障来禁止特定类型的处理器重排序。对于编译器来说, 发现一个最优布置来最小化插入屏障的总数几乎不可能。为此, JMM采取保守策略。下面是基于保守策略的JMM内存屏障插入策略。

- 在每个volatile写操作的前面插入一个StoreStore屏障。
- 在每个volatile写操作的后面插入一个StoreLoad屏障。
- 在每个volatile读操作的后面插入一个LoadLoad屏障。
- 在每个volatile读操作的后面插入一个LoadStore屏障。

上述内存屏障插入策略非常保守，但它可以保证在任意处理器平台，任意的程序中都能得到正确的volatile内存语义。

下面是保守策略下，volatile写插入内存屏障后生成的指令序列示意图，如图3-19所示。

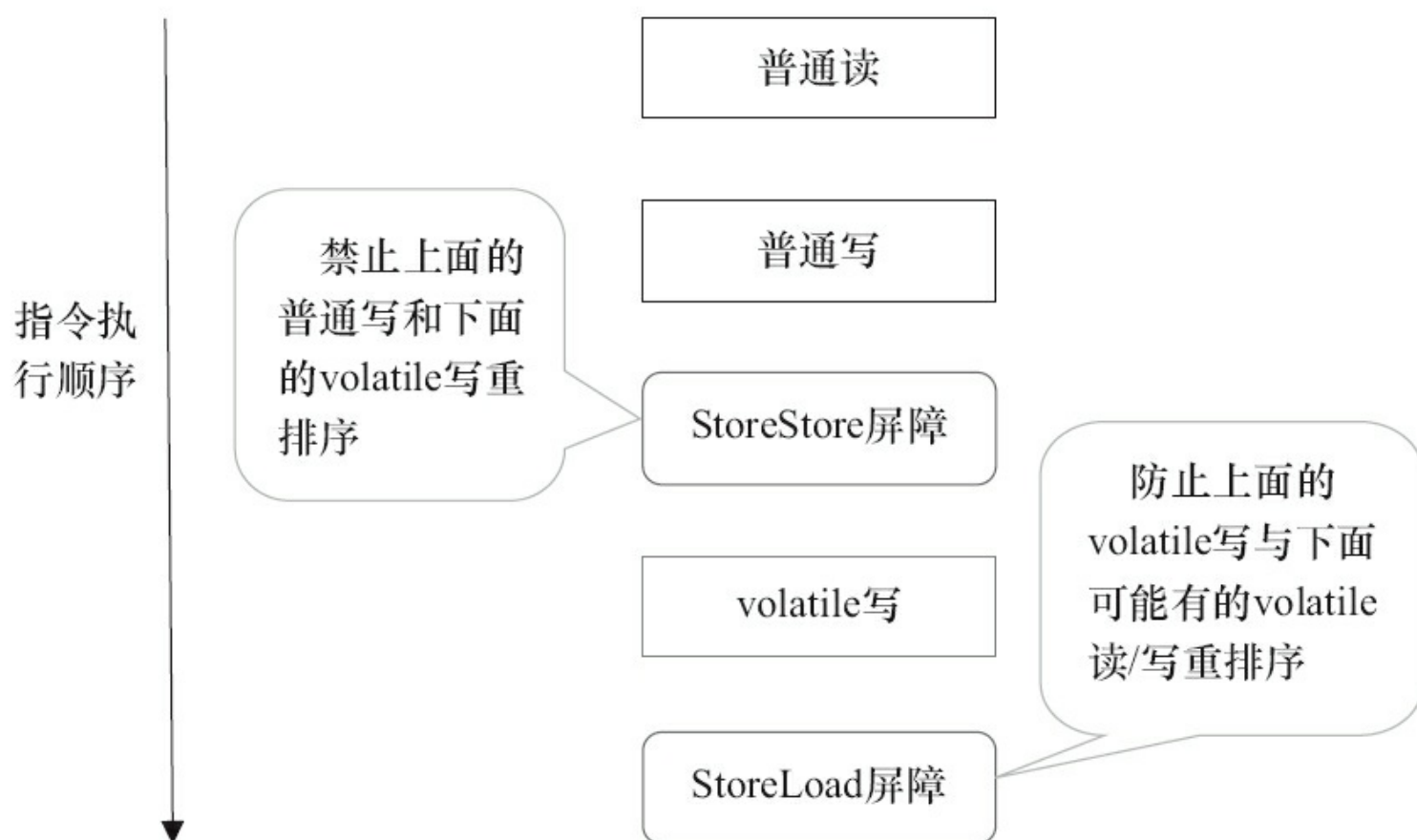


图3-19 指令序列示意图

图3-19中的StoreStore屏障可以保证在volatile写之前，其前面的所有普通写操作已经对任意处理器可见了。这是因为StoreStore屏障将保障上面所有的普通写在volatile写之前刷新到主

内存。

这里比较有意思的是，volatile写后面的StoreLoad屏障。此屏障的作用是避免volatile写与后面可能有的volatile读/写操作重排序。因为编译器常常无法准确判断在一个volatile写的后面是否需要插入一个StoreLoad屏障(比如，一个volatile写之后方法立即return)。为了保证能正确实现volatile的内存语义，JMM在采取了保守策略：在每个volatile写的后面，或者在每个volatile读的前面插入一个StoreLoad屏障。从整体执行效率的角度考虑，JMM最终选择了在每个volatile写的后面插入一个StoreLoad屏障。因为volatile写-读内存语义的常见使用模式是：一个写线程写volatile变量，多个读线程读同一个volatile变量。当读线程的数量大大超过写线程时，选择在volatile写之后插入StoreLoad屏障将带来可观的执行效率的提升。从这里可以看到JMM在实现上的一个特点：首先确保正确性，然后再去追求执行效率。

下面是在保守策略下，volatile读插入内存屏障后生成的指令序列示意图，如图3-20所示。

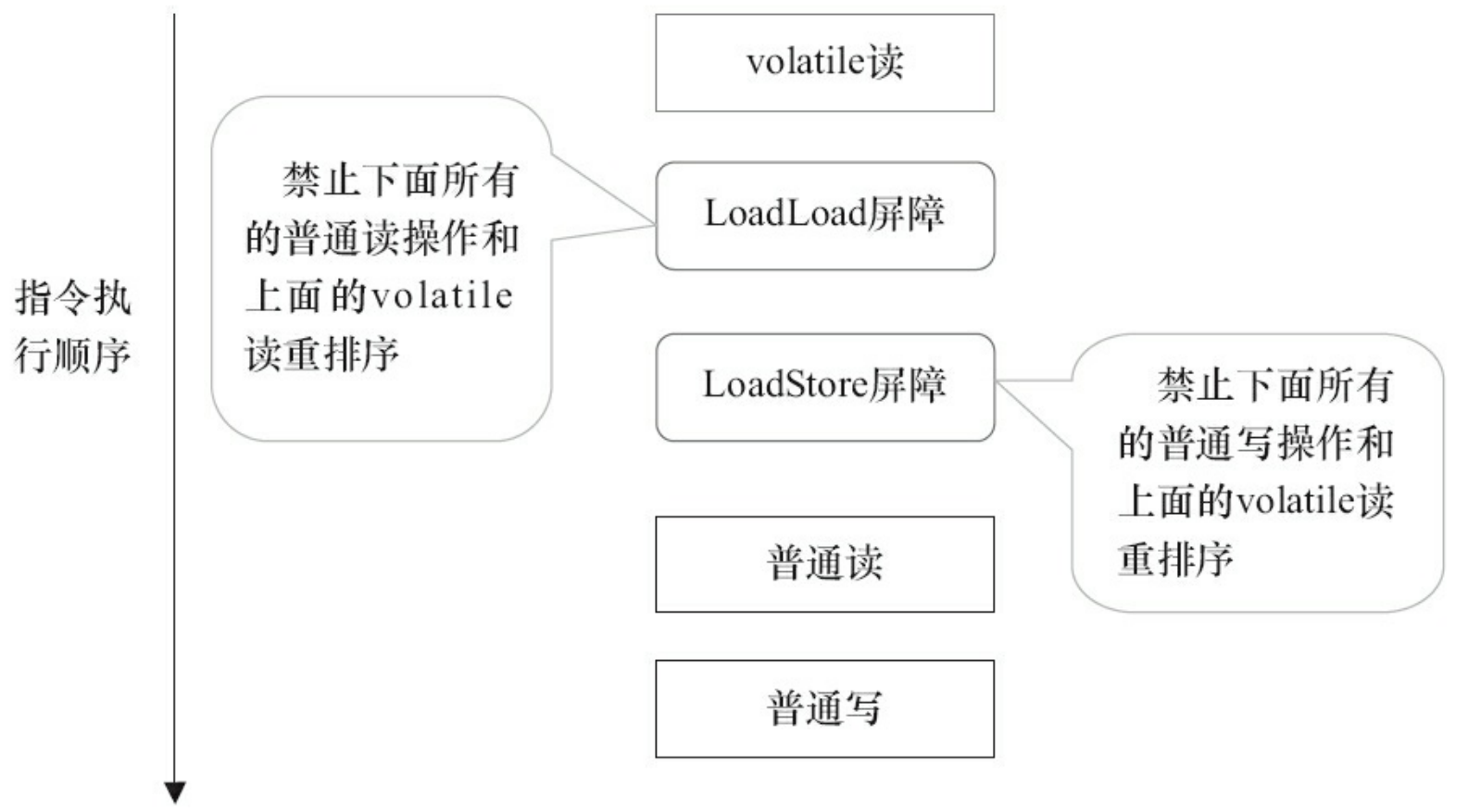


图3-20 指令序列示意图

图3-20中的LoadLoad屏障用来禁止处理器把上面的volatile读与下面的普通读重排序。

LoadStore屏障用来禁止处理器把上面的volatile读与下面的普通写重排序。

上述volatile写和volatile读的内存屏障插入策略非常保守。在实际执行时, 只要不改变volatile写-读的内存语义, 编译器可以根据具体情况省略不必要的屏障。下面通过具体的示例代码进行说明。

```
class VolatileBarrierExample {
    int a;
    volatile int v1 = 1;
    volatile int v2 = 2;
    void readAndWrite() {
        int i = v1;          // 第一个volatile读
        int j = v2;          // 第二个volatile读
        a = i + j;           // 普通写
        v1 = i + 1;          // 第一个volatile写
        v2 = j * 2;          // 第二个 volatile写
    }
    ...                     // 其他方法
}
```

针对readAndWrite()方法, 编译器在生成字节码时可以做如下的优化。

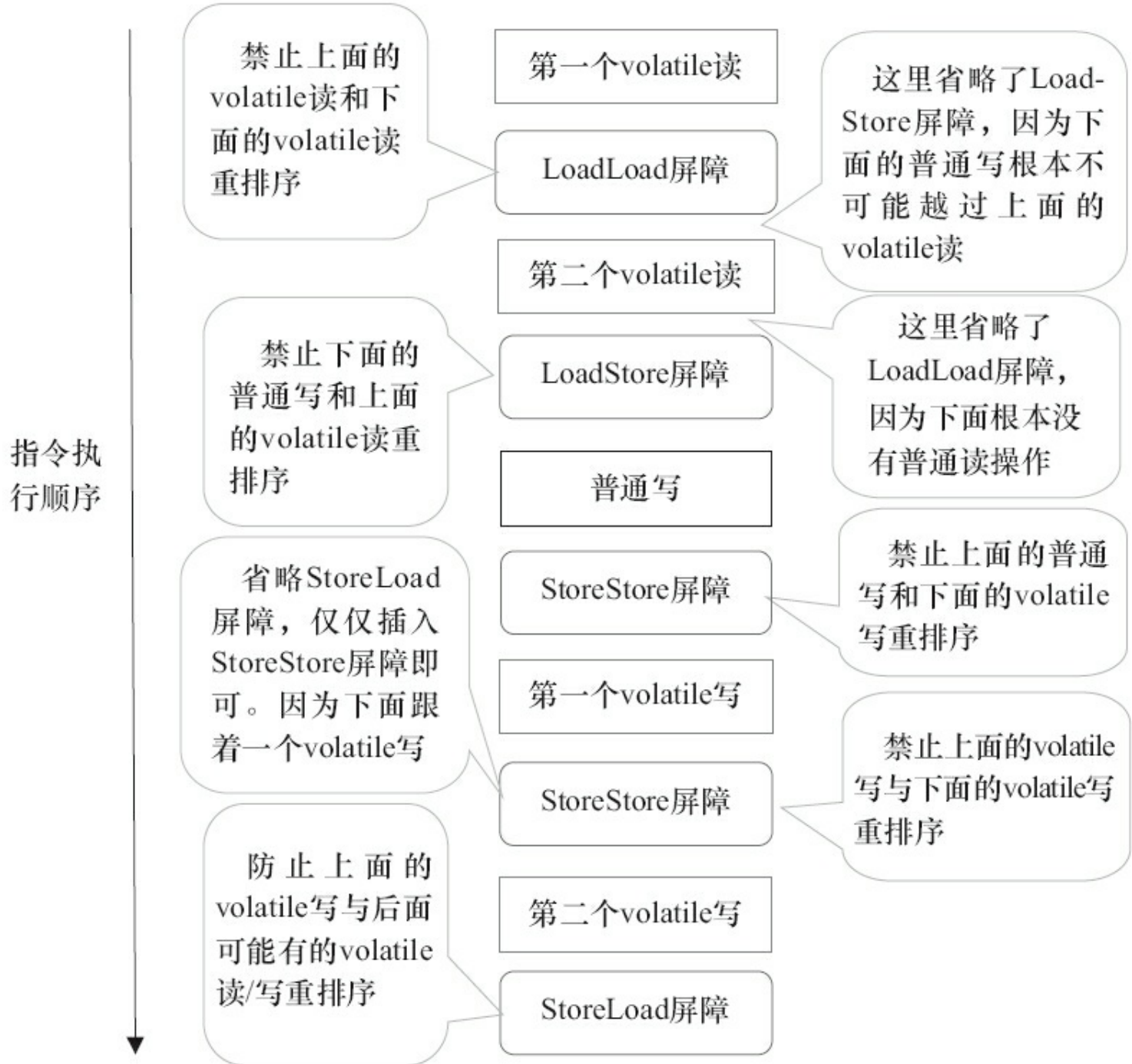


图3-21 指令序列示意图

注意，最后的StoreLoad屏障不能省略。因为第二个volatile写之后，方法立即return。此时编译器可能无法准确断定后面是否会有volatile读或写，为了安全起见，编译器通常会在这里插入一个StoreLoad屏障。

上面的优化针对任意处理器平台，由于不同的处理器有不同“松紧度”的处理器内存模型，内存屏障的插入还可以根据具体的处理器内存模型继续优化。以X86处理器为例，图3-21中除最后的StoreLoad屏障外，其他的屏障都会被省略。

前面保守策略下的volatile读和写，在X86处理器平台可以优化成如图3-22所示。

前文提到过，X86处理器仅会对写-读操作做重排序。X86不会对读-读、读-写和写-写操作做重排序，因此在X86处理器中会省略掉这3种操作类型对应的内存屏障。在X86中，JMM仅需在volatile写后面插入一个StoreLoad屏障即可正确实现volatile写-读的内存语义。这意味着在X86处理器中，volatile写的开销比volatile读的开销会大很多（因为执行StoreLoad屏障开销会比较大）。

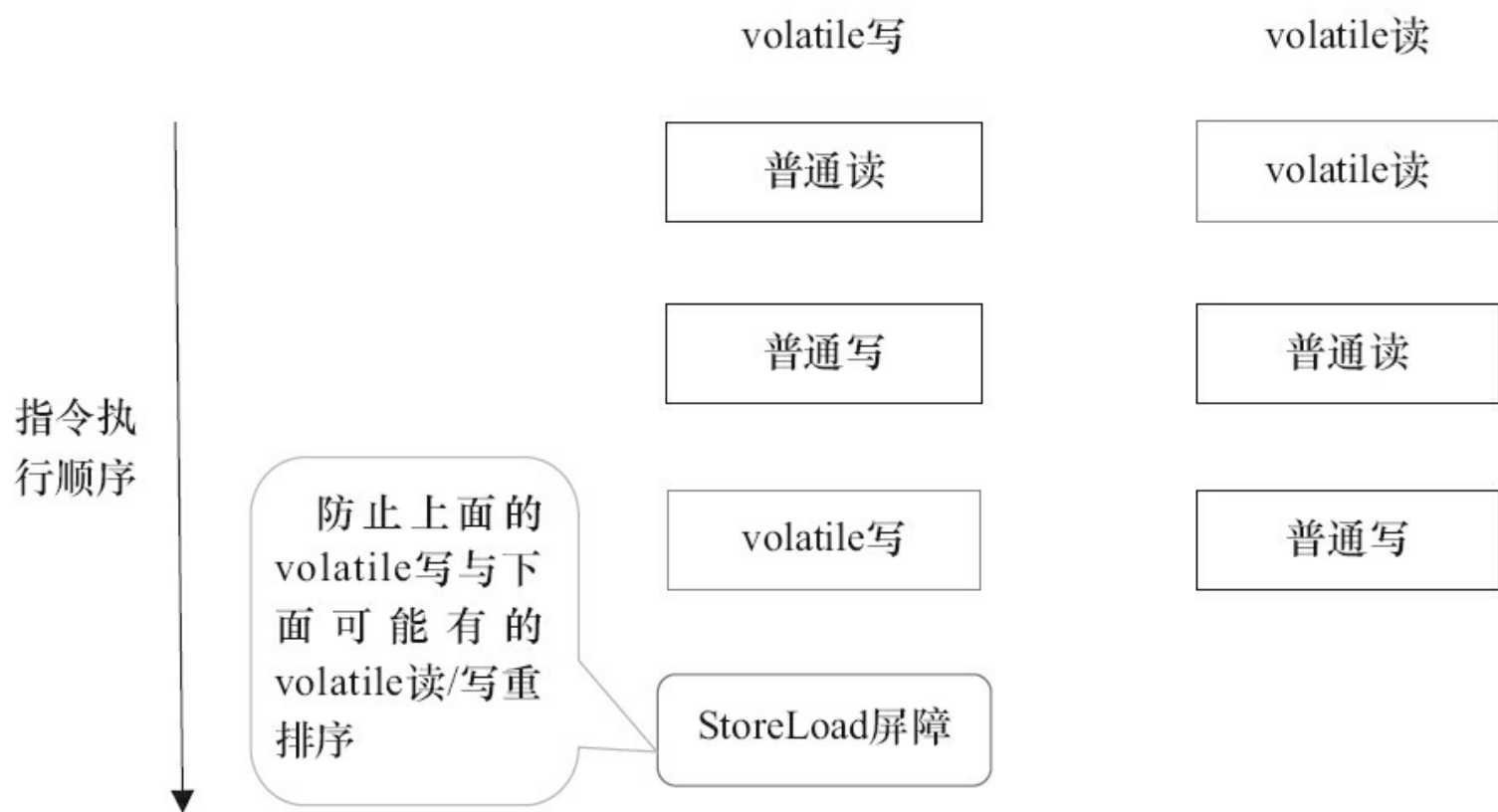


图3-22 指令序列示意图

3.4.5 JSR-133为什么要增强volatile的内存语义

在JSR-133之前的旧Java内存模型中，虽然不允许volatile变量之间重排序，但旧的Java内存模型允许volatile变量与普通变量重排序。在旧的内存模型中，VolatileExample示例程序可能被重排序成下列时序来执行，如图3-23所示。

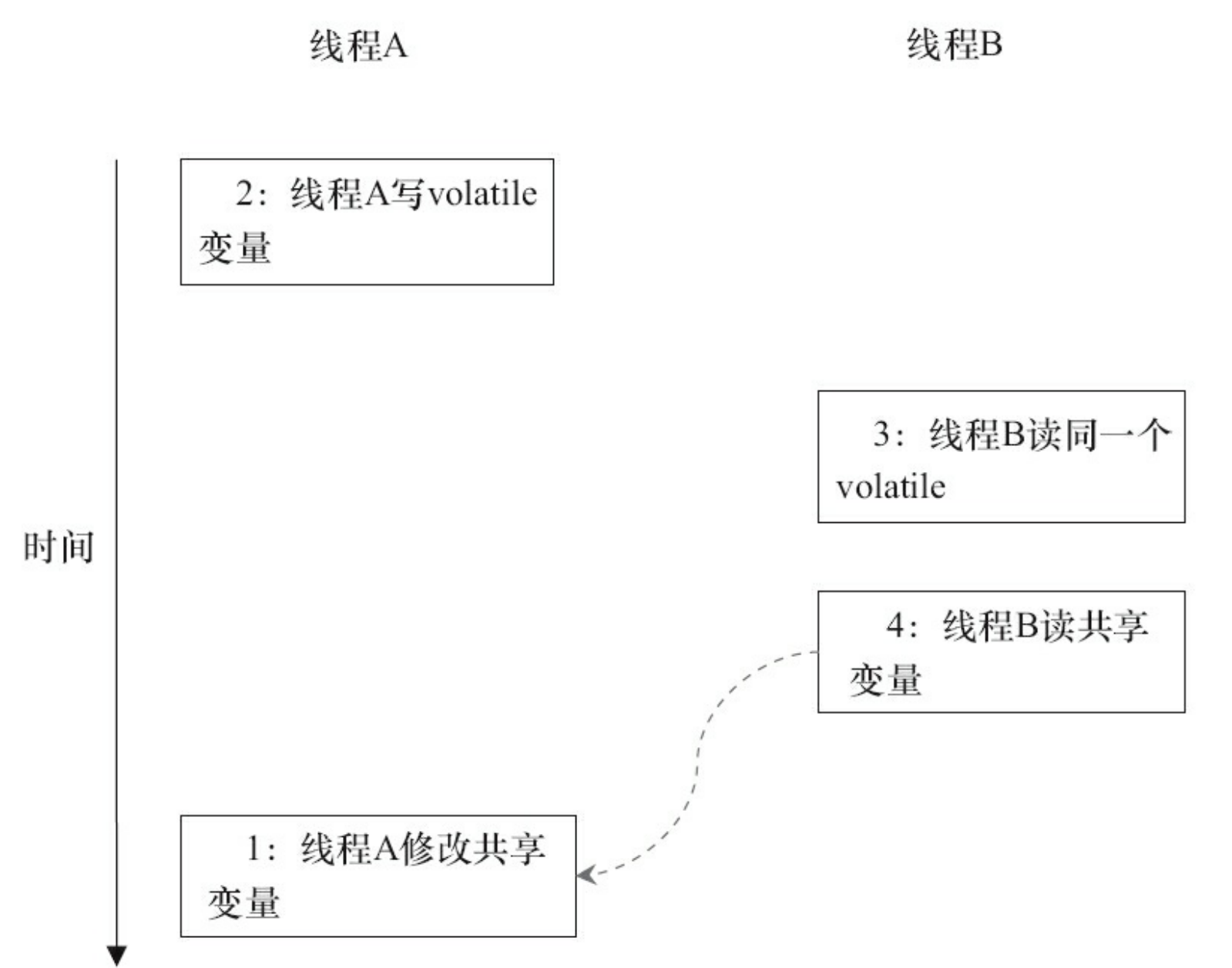


图3-23 线程执行时序图

在旧的内存模型中，当1和2之间没有数据依赖关系时，1和2之间就可能被重排序(3和4类似)。其结果就是：读线程B执行4时，不一定能看到写线程A在执行1时对共享变量的修改。

因此，在旧的内存模型中，volatile的写-读没有锁的释放-获所具有的内存语义。为了提供一种比锁更轻量级的线程之间通信的机制，JSR-133专家组决定增强volatile的内存语义：严格限制编译器和处理器对volatile变量与普通变量的重排序，确保volatile的写-读和锁的释放-获取具有相同的内存语义。从编译器重排序规则和处理器内存屏障插入策略来看，只要volatile变量与普通变量之间的重排序可能会破坏volatile的内存语义，这种重排序就会被编译器重排序规则和处理器内存屏障插入策略禁止。

由于volatile仅仅保证对单个volatile变量的读/写具有原子性，而锁的互斥执行的特性可以确保对整个临界区代码的执行具有原子性。在功能上，锁比volatile更强大；在可伸缩性和执行性能上，volatile更有优势。如果读者想在程序中用volatile代替锁，请一定谨慎，[具体详情请参阅Brian Goetz的文章《Java理论与实践：正确使用Volatile变量》。](#)

3.5 锁的内存语义

众所周知, 锁可以让临界区互斥执行。这里将介绍锁的另一个同样重要, 但常常被忽视的功能: 锁的内存语义。

3.5.1 锁的释放-获取建立的happens-before关系

锁是Java并发编程中最重要的同步机制。锁除了让临界区互斥执行外，还可以让释放锁的线程向获取同一个锁的线程发送消息。

下面是锁释放-获取的示例代码。

```
class MonitorExample {
    int a = 0;
    public synchronized void writer() {           // 1
        a++;                                       // 2
    }                                             // 3
    public synchronized void reader() {           // 4
        int i = a;                               // 5
        .....
    }                                             // 6
}
```

假设线程A执行writer()方法，随后线程B执行reader()方法。根据happens-before规则，这个过程包含的happens-before关系可以分为3类。

1) 根据程序次序规则，1 happens-before 2, 2 happens-before 3; 4 happens-before 5, 5 happens-before 6。

2) 根据监视器锁规则，3 happens-before 4。

3) 根据happens-before的传递性，2 happens-before 5。

上述happens-before关系的图形化表现形式如图3-24所示。

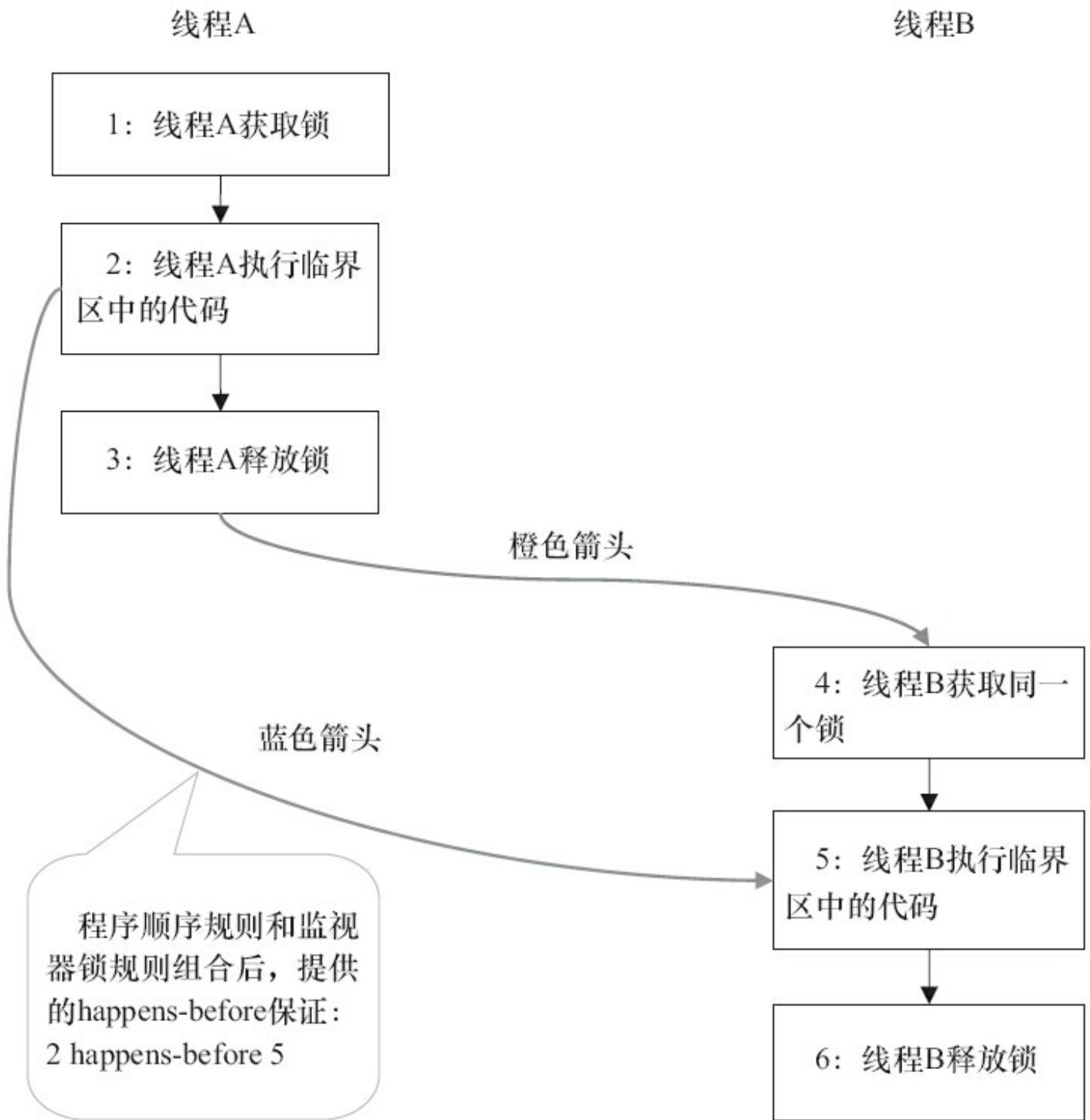


图3-24 happens-before关系图

在图3-24中，每一个箭头链接的两个节点，代表了一个happens-before关系。黑色箭头表示程序顺序规则；橙色箭头表示监视器锁规则；蓝色箭头表示组合这些规则后提供的happens-before保证。

图3-24表示在线程A释放了锁之后，随后线程B获取同一个锁。在上图中，2 happens-before 5。因此，线程A在释放锁之前所有可见的共享变量，在线程B获取同一个锁之后，将立刻变得

对B线程可见。

3.5.2 锁的释放和获取的内存语义

当线程释放锁时，JMM会把该线程对应的本地内存中的共享变量刷新到主内存中。上面的MonitorExample程序为例，A线程释放锁后，共享数据的状态示意图如图3-25所示。

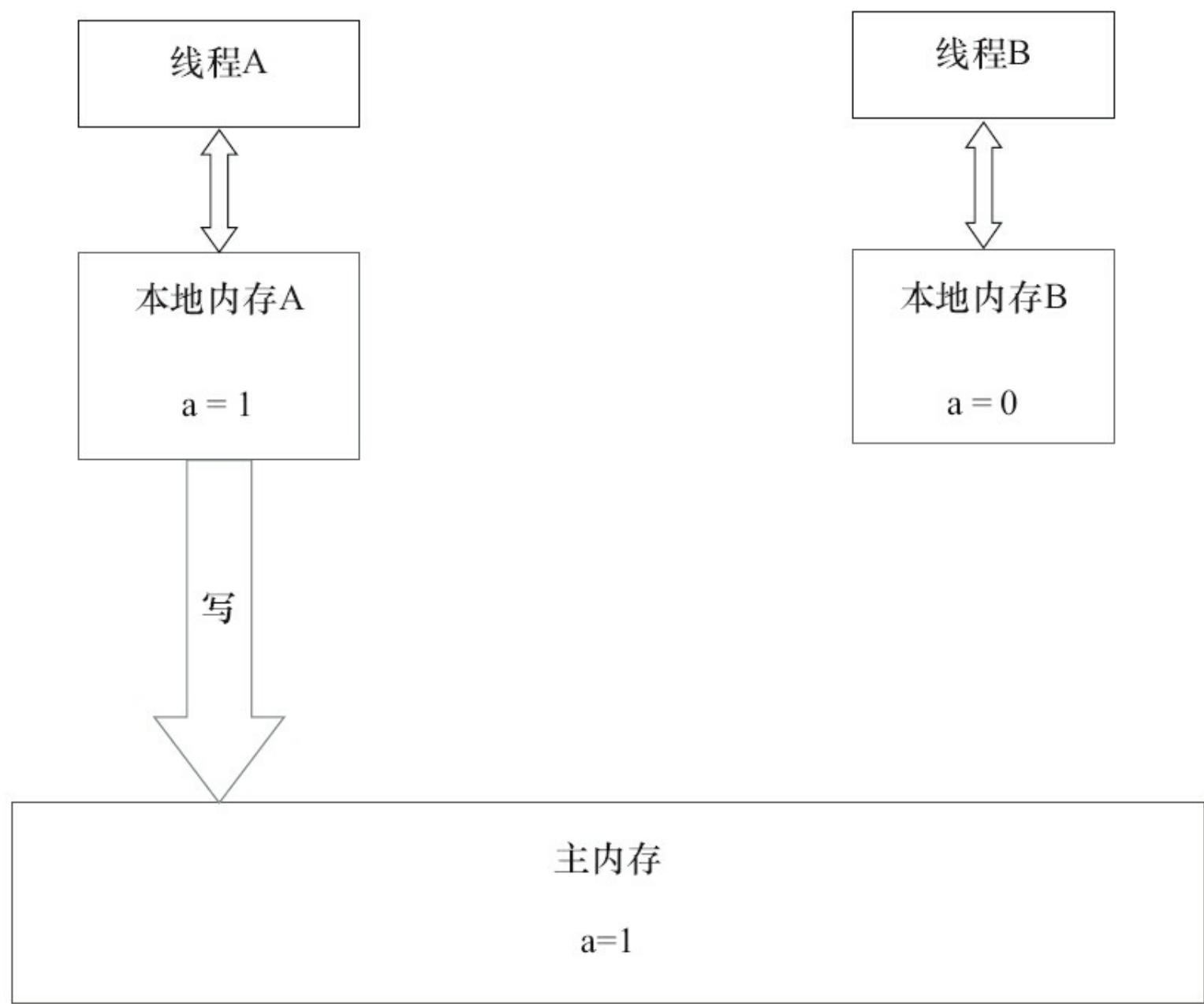


图3-25 共享数据的状态示意图

当线程获取锁时，JMM会把该线程对应的本地内存置为无效。从而使得被监视器保护的临界区代码必须从主内存中读取共享变量。图3-26是锁获取的状态示意图。

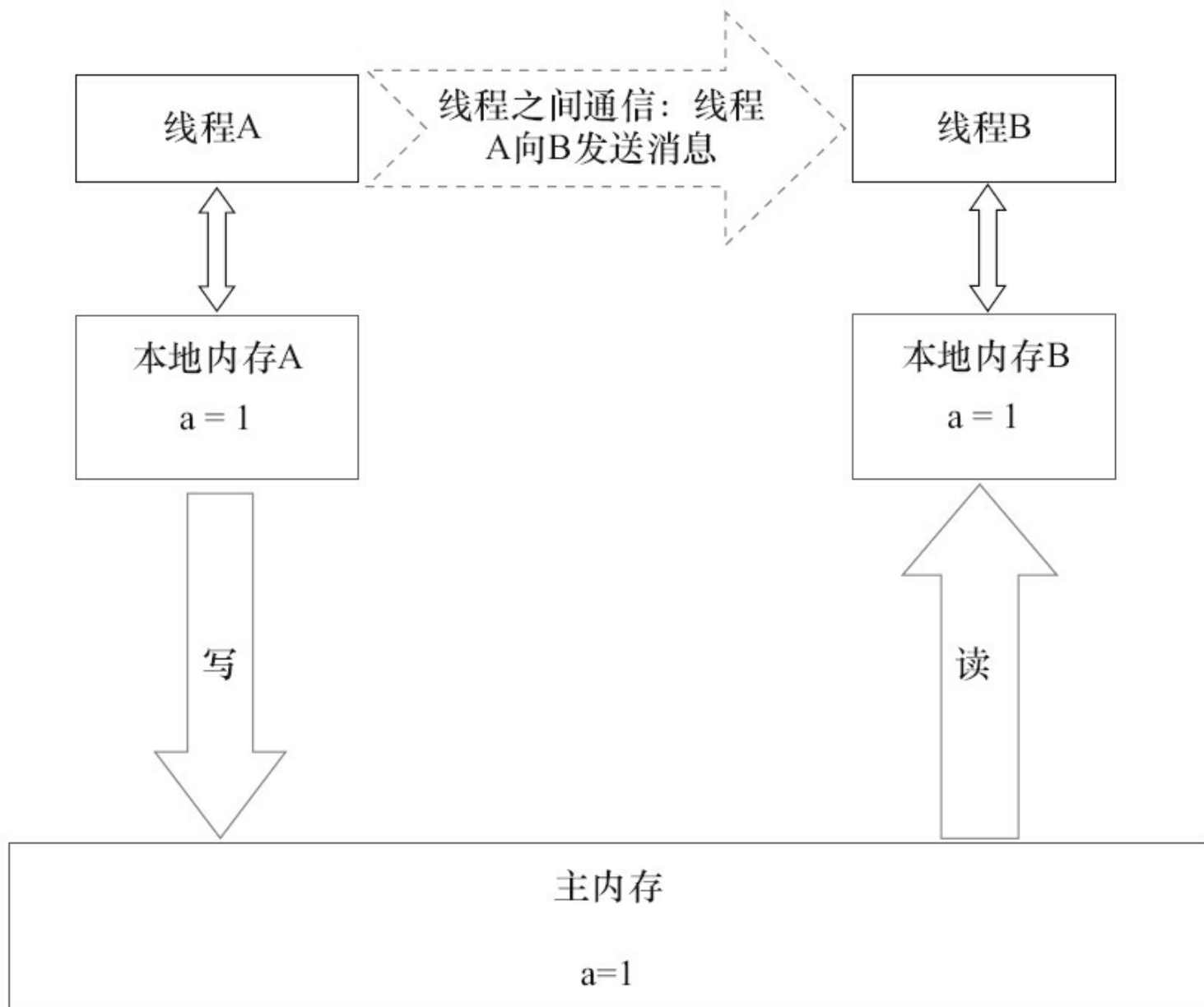


图3-26 锁获取的状态示意图

对比锁释放-获取的内存语义与volatile写-读的内存语义可以看出：锁释放与volatile写有相同的内存语义；锁获取与volatile读有相同的内存语义。

下面对锁释放和锁获取的内存语义做个总结。

·线程A释放一个锁，实质上是线程A向接下来将要获取这个锁的某个线程发出了(线程A对共享变量所做修改的)消息。

·线程B获取一个锁，实质上是线程B接收了之前某个线程发出的(在释放这个锁之前对共享变量所做修改的)消息。

·线程A释放锁, 随后线程B获取这个锁, 这个过程实质上是线程A通过主内存向线程B发送消息。

3.5.3 锁内存语义的实现

本文将借助ReentrantLock的源代码, 来分析锁内存语义的具体实现机制。

请看下面的示例代码。

```
class ReentrantLockExample {
    int a = 0;
    ReentrantLock lock = new ReentrantLock();
    public void writer() {
        lock.lock();          // 获取锁
        try {
            a++;
        } finally {
            lock.unlock();    // 释放锁
        }
    }
    public void reader () {
        lock.lock();          // 获取锁
        try {
            int i = a;
            .....
        } finally {
            lock.unlock();    // 释放锁
        }
    }
}
```

在ReentrantLock中, 调用lock()方法获取锁;调用unlock()方法释放锁。

ReentrantLock的实现依赖于Java同步器框架AbstractQueuedSynchronizer(本文简称之为AQS)。AQS使用一个整型的volatile变量(命名为state)来维护同步状态, 马上我们会看到, 这个volatile变量是ReentrantLock内存语义实现的关键。

图3-27是ReentrantLock的类图(仅画出与本文相关的部分)。

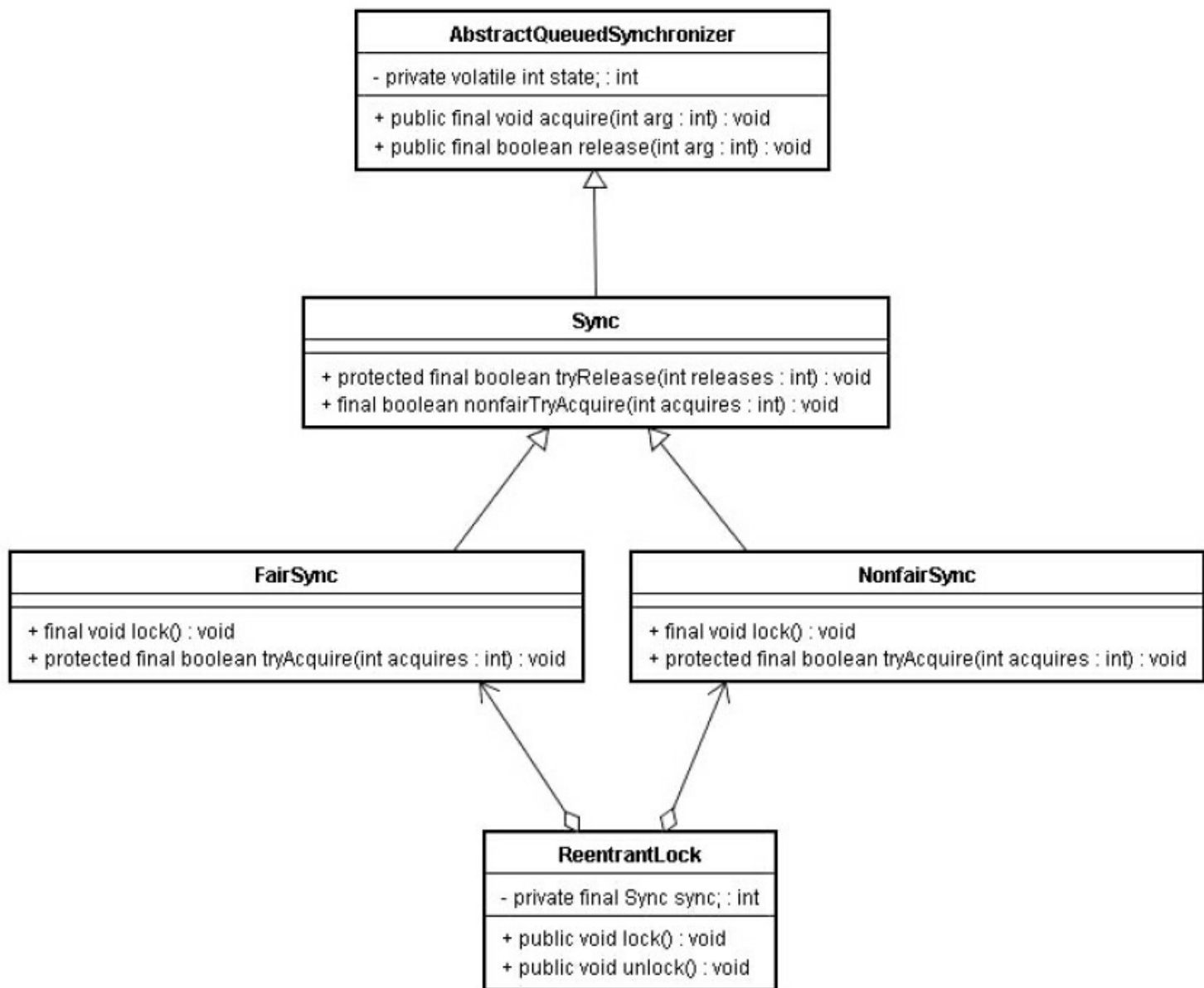


图3-27 ReentrantLock的类图

ReentrantLock分为公平锁和非公平锁，我们首先分析公平锁。

使用公平锁时，加锁方法lock()调用轨迹如下。

- 1) ReentrantLock:lock()。
- 2) FairSync:lock()。
- 3) AbstractQueuedSynchronizer:acquire(int arg)。

4) ReentrantLock:tryAcquire(int acquires)。

在第4步真正开始加锁，下面是该方法的源代码。

```
protected final boolean tryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();    // 获取锁的开始, 首先读volatile变量state
    if (c == 0) {
        if (isFirst(current) &&
            compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    }
    else if (current == getExclusiveOwnerThread()) {
        int nextc = c + acquires;
        if (nextc < 0)
            throw new Error("Maximum lock count exceeded");
        setState(nextc);
        return true;
    }
    return false;
}
```

从上面源代码中我们可以看出，加锁方法首先读volatile变量state。

在使用公平锁时，解锁方法unlock()调用轨迹如下。

1) ReentrantLock:unlock()。

2) AbstractQueuedSynchronizer:release(int arg)。

3) Sync:tryRelease(int releases)。

在第3步真正开始释放锁，下面是该方法的源代码。

```
protected final boolean tryRelease(int releases) {
    int c = getState() - releases;
    if (Thread.currentThread() != getExclusiveOwnerThread())
        throw new IllegalMonitorStateException();
    boolean free = false;
    if (c == 0) {
        free = true;
        setExclusiveOwnerThread(null);
    }
}
```

```
}
    setState(c);        // 释放锁的最后, 写volatile变量state
    return free;
}
```

从上面的源代码可以看出, 在释放锁的最后写volatile变量state。

公平锁在释放锁的最后写volatile变量state, 在获取锁时首先读这个volatile变量。根据volatile的happens-before规则, 释放锁的线程在写volatile变量之前可见的共享变量, 在获取锁的线程读取同一个volatile变量后将立即变得对获取锁的线程可见。

现在我们来分析非公平锁的内存语义的实现。非公平锁的释放和公平锁完全一样, 所以这里仅仅分析非公平锁的获取。使用非公平锁时, 加锁方法lock()调用轨迹如下。

- 1) ReentrantLock:lock()。
- 2) NonfairSync:lock()。
- 3) AbstractQueuedSynchronizer:compareAndSetState(int expect,int update)。

在第3步真正开始加锁, 下面是该方法的源代码。

```
protected final boolean compareAndSetState(int expect, int update) {
    return unsafe.compareAndSwapInt(this, stateOffset, expect, update);
}
```

该方法以原子操作的方式更新state变量, 本文把Java的compareAndSet()方法调用简称为CAS。JDK文档对该方法的说明如下: 如果当前状态值等于预期值, 则以原子方式将同步状态设置为给定的更新值。此操作具有volatile读和写的内存语义。

这里我们分别从编译器和处理器的角度来分析, CAS如何同时具有volatile读和volatile写的内存语义。

前文我们提到过, 编译器不会对volatile读与volatile读后面的任意内存操作重排序; 编译

器不会对volatile写与volatile写前面的任意内存操作重排序。组合这两个条件,意味着为了同时实现volatile读和volatile写的内存语义,编译器不能对CAS与CAS前面和后面的任意内存操作重排序。

下面我们来分析在常见的intel X86处理器中, CAS是如何同时具有volatile读和volatile写的内存语义的。

下面是sun.misc.Unsafe类的compareAndSwapInt()方法的源代码。

```
public final native boolean compareAndSwapInt(Object o, long offset,
```

i
i

可以看到, 这是一个本地方法调用。这个本地方法在openjdk中依次调用的c++代码为:

unsafe.cpp, atomic.cpp和atomic_windows_x86.inline.hpp。这个本地方法的最终实现在openjdk的如下位置: openjdk-7-fcs-src-b147-

27_jun_2011\openjdk\hotspot\src\os_cpu\windows_x86\vm\atomic_windows_x86.inline.hpp(对应于Windows操作系统, X86处理器)。下面是对应于intel X86处理器的源代码的片段。

```
inline jint    Atomic::cmpxchg    (jint    exchange_value, volatile jint*    dest,
    jint    compare_value) {
    // alternative for InterlockedCompareExchange
    int mp = os::is_MP();
    __asm {
        mov edx, dest
        mov ecx, exchange_value
        mov eax, compare_value
        LOCK_IF_MP(mp)
        cmpxchg dword ptr [edx], ecx
    }
}
```

JAVA CAS操作底层原理: lock指令加锁 总线锁 或 缓存行锁

如上面源代码所示, 程序会根据当前处理器的类型来决定是否为cmpxchg指令添加lock前缀。如果程序是在多处理器上运行, 就为cmpxchg指令加上lock前缀(Lock Cmpxchg)。反之, 如果程序是在单处理器上运行, 就省略lock前缀(单处理器自身会维护单处理器内的顺序一致

性，不需要lock前缀提供的内存屏障效果）。

intel的手册对lock前缀的说明如下。

1) 确保对内存的读-改-写操作原子执行。在Pentium及Pentium之前的处理器中，带有lock前缀的指令在执行期间会锁住总线，使得其他处理器暂时无法通过总线访问内存。很显然，这会带来昂贵的开销。从Pentium 4、Intel Xeon及P6处理器开始，Intel使用缓存锁定(Cache Locking)来保证指令执行的原子性。缓存锁定将大大降低lock前缀指令的执行开销。

2) 禁止该指令，与之前和之后的读和写指令重排序。

3) 把写缓冲区中的所有数据刷新到内存中。

上面的第2点和第3点所具有的内存屏障效果，足以同时实现volatile读和volatile写的内存语义。

经过上面的分析，现在我们终于能明白为什么JDK文档说CAS同时具有volatile读和volatile写的内存语义了。

现在对公平锁和非公平锁的内存语义做个总结。

· 公平锁和非公平锁释放时，最后都要写一个volatile变量state。

· 公平锁获取时，首先会去读volatile变量。

· 非公平锁获取时，首先会用CAS更新volatile变量，这个操作同时具有volatile读和volatile写的内存语义。

从本文对ReentrantLock的分析可以看出，锁释放-获取的内存语义的实现至少有下面两种方式。

1) 利用volatile变量的写-读所具有的内存语义。

2) 利用CAS所附带的volatile读和volatile写的内存语义。

3.5.4 concurrent包的实现

由于Java的CAS同时具有volatile读和volatile写的内存语义，因此Java线程之间的通信现在有了下面4种方式。

- 1) A线程写volatile变量，随后B线程读这个volatile变量。
- 2) A线程写volatile变量，随后B线程用CAS更新这个volatile变量。
- 3) A线程用CAS更新一个volatile变量，随后B线程用CAS更新这个volatile变量。
- 4) A线程用CAS更新一个volatile变量，随后B线程读这个volatile变量。

Java的CAS会使用现代处理器上提供的高效机器级别的原子指令，这些原子指令以原子方式对内存执行读-改-写操作，这是在多处理器中实现同步的关键(从本质上来说，能够支持原子性读-改-写指令的计算机，是顺序计算图灵机的异步等价机器，因此任何现代的多处理器都会去支持某种能对内存执行原子性读-改-写操作的原子指令)。同时，volatile变量的读/写和CAS可以实现线程之间的通信。把这些特性整合在一起，就形成了整个concurrent包得以实现的基石。如果我们仔细分析concurrent包的源代码实现，会发现一个通用化的实现模式。

首先，声明共享变量为volatile。

然后，使用CAS的原子条件更新来实现线程之间的同步。

同时，配合以volatile的读/写和CAS所具有的volatile读和写的内存语义来实现线程之间的通信。

AQS，非阻塞数据结构和原子变量类(java.util.concurrent.atomic包中的类)，这些concurrent包中的基础类都是使用这种模式来实现的，而concurrent包中的高层类又是依赖于这些基础类来实现的。从整体来看，concurrent包的实现示意图如3-28所示。

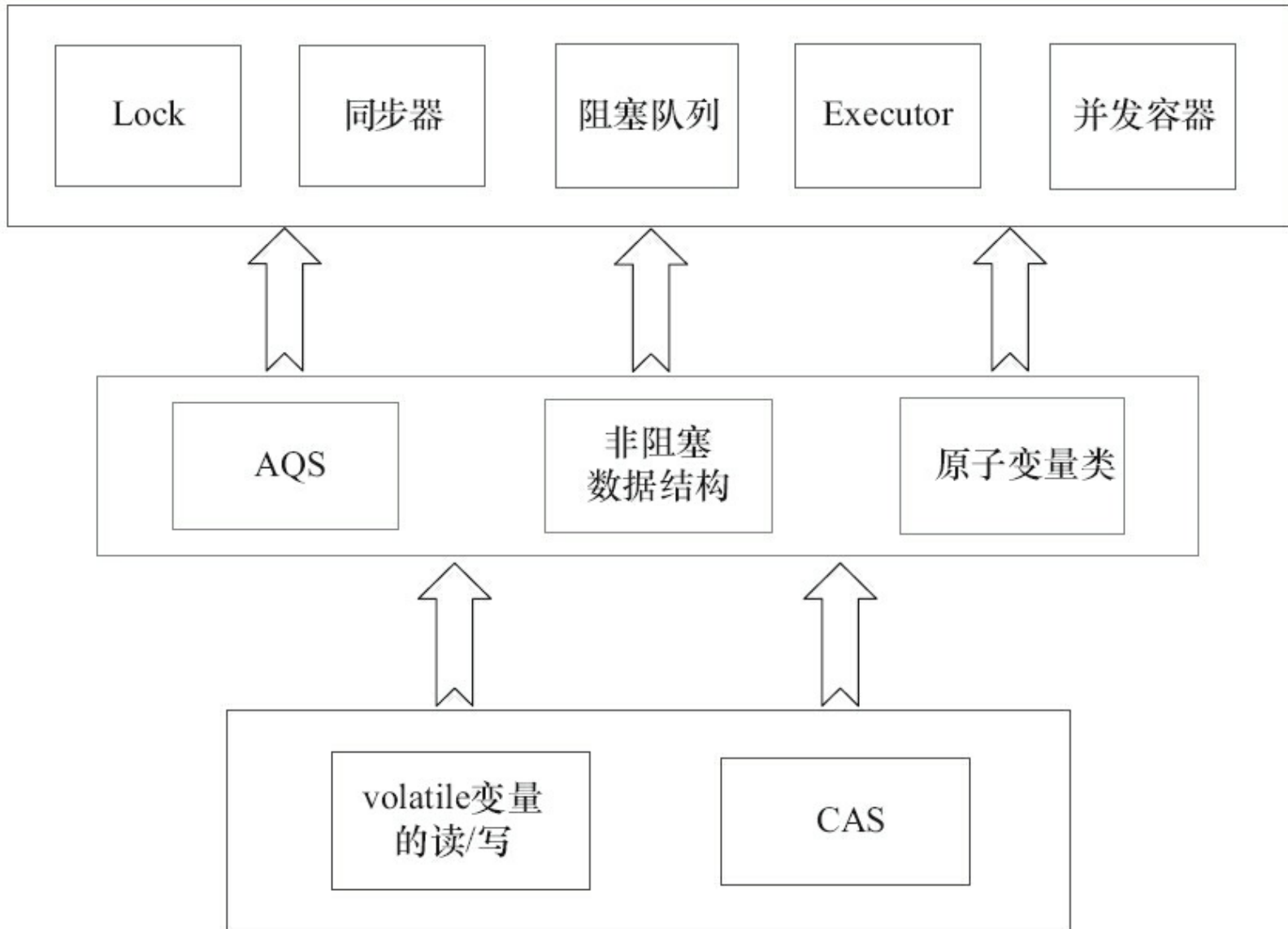


图3-28 concurrent包的实现示意图

3.6 final域的内存语义

与前面介绍的锁和volatile相比, 对final域的读和写更像是普通的变量访问。下面将介绍final域的内存语义。

3.6.1 final域的重排序规则

对于final域, 编译器和处理器要遵守两个重排序规则。

1) 在构造函数内对一个final域的写入, 与随后把这个被构造对象的引用赋值给一个引用变量, 这两个操作之间不能重排序。

2) 初次读一个包含final域的对象引用, 与随后初次读这个final域, 这两个操作之间不能重排序。

下面通过一些示例性的代码来分别说明这两个规则。

```
public class FinalExample {
    int i;                // 普通变量
    final int j;           // final变量
    static FinalExample obj;
    public FinalExample () {           // 构造函数
        i = 1;                        // 写普通域
        j = 2;                        // 写final域
    }
    public static void writer () {      // 写线程A执行
        obj = new FinalExample ();
    }
    public static void reader () {      // 读线程B执行
        FinalExample object = obj;     // 读对象引用
        int a = object.i;              // 读普通域
        int b = object.j;              // 读final域
    }
}
```

这里假设一个线程A执行writer()方法, 随后另一个线程B执行reader()方法。下面我们通过这两个线程的交互来说明这两个规则。

3.6.2 写final域的重排序规则

写final域的重排序规则禁止把final域的写重排序到构造函数之外。这个规则的实现包含下面2个方面。

1)JMM禁止编译器把final域的写重排序到构造函数之外。

2)编译器会在final域的写之后，构造函数return之前，插入一个StoreStore屏障。这个屏障禁止处理器把final域的写重排序到构造函数之外。

现在让我们分析writer()方法。writer()方法只包含一行代码：`finalExample=new FinalExample()`。这行代码包含两个步骤，如下。

1)构造一个FinalExample类型的对象。

2)把这个对象的引用赋值给引用变量obj。

假设线程B读对象引用与读对象的成员域之间没有重排序(马上会说明为什么需要这个假设)，图3-29是一种可能的执行时序。

在图3-29中，写普通域的操作被编译器重排序到了构造函数之外，读线程B错误地读取了普通变量i初始化之前的值。而写final域的操作，被写final域的重排序规则“限定”在了构造函数之内，读线程B正确地读取了final变量初始化之后的值。

写final域的重排序规则可以确保：在对象引用为任意线程可见之前，对象的final域已经被正确初始化过了，而普通域不具有这个保障。以上图为例，在读线程B“看到”对象引用obj时，很可能obj对象还没有构造完成(对普通域i的写操作被重排序到构造函数外，此时初始值1还没有写入普通域i)。

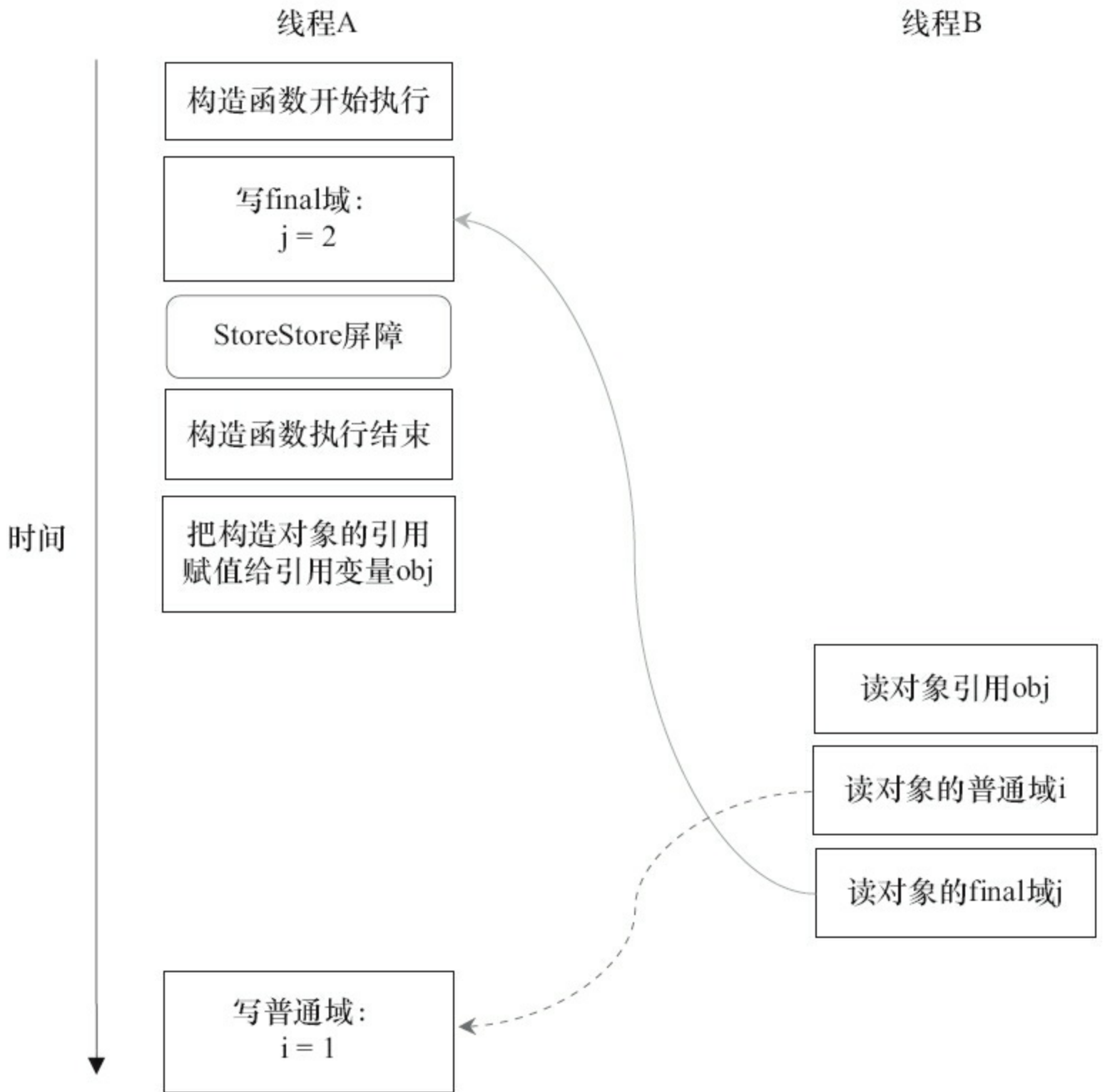


图3-29 线程执行时序图

3.6.3 读final域的重排序规则

读final域的重排序规则是，在一个线程中，初次读对象引用与初次读该对象包含的final域，JMM禁止处理器重排序这两个操作（注意，这个规则仅仅针对处理器）。编译器会在读final域操作的前面插入一个LoadLoad屏障。

初次读对象引用与初次读该对象包含的final域，这两个操作之间存在间接依赖关系。由于编译器遵守间接依赖关系，因此编译器不会重排序这两个操作。大多数处理器也会遵守间接依赖，也不会重排序这两个操作。但有少数处理器允许对存在间接依赖关系的操作做重排序（比如alpha处理器），这个规则就是专门用来针对这种处理器的。

reader()方法包含3个操作。

- 初次读引用变量obj。
- 初次读引用变量obj指向对象的普通域j。
- 初次读引用变量obj指向对象的final域i。

现在假设写线程A没有发生任何重排序，同时程序在不遵守间接依赖的处理器上执行，图3-30所示是一种可能的执行时序。

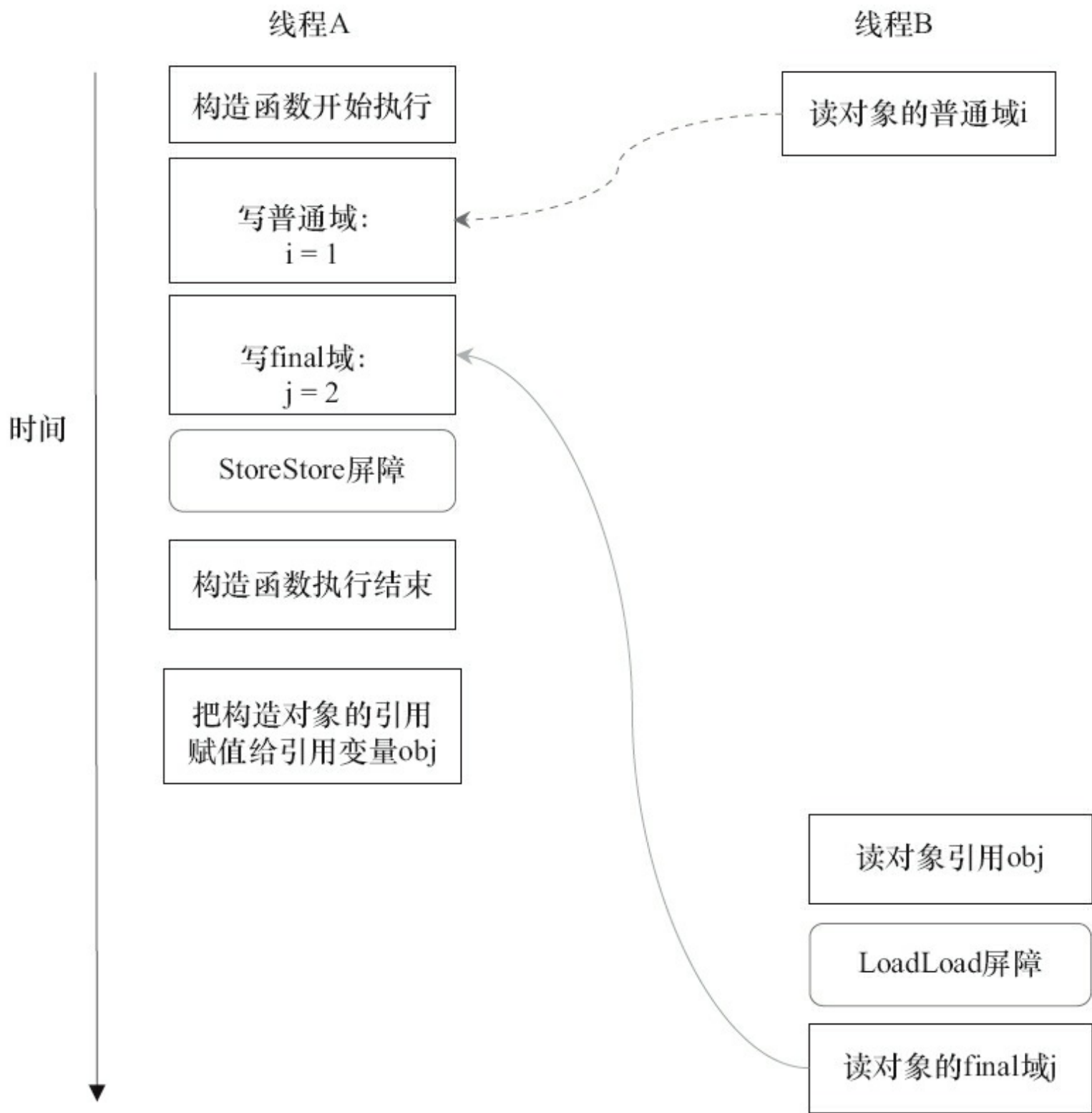


图3-30 线程执行时序图

在图3-30中，读对象的普通域的操作被处理器重排序到读对象引用之前。读普通域时，该域还没有被写线程A写入，这是一个错误的读取操作。而读final域的重排序规则会把读对象final域的操作“限定”在读对象引用之后，此时该final域已经被A线程初始化过了，这是一个正确的读取操作。

读final域的重排序规则可以确保:在读一个对象的final域之前,一定会先读包含这个final域的对象引用。在这个示例程序中,如果该引用不为null,那么引用对象的final域一定已经被A线程初始化过了。

3.6.4 final域为引用类型

上面我们看到的final域是基础数据类型，如果final域是引用类型，将会有什么效果？请看下列示例代码。

```
public class FinalReferenceExample {
    final int[] intArray;           // final是引用类型
    static FinalReferenceExample obj;
    public FinalReferenceExample () { // 构造函数
        intArray = new int[1];      // 1
        intArray[0] = 1;            // 2
    }
    public static void writerOne () { // 写线程A执行
        obj = new FinalReferenceExample (); // 3
    }
    public static void writerTwo () { // 写线程B执行
        obj.intArray[0] = 2;         // 4
    }
    public static void reader () {   // 读线程C执行
        if (obj != null) {           // 5
            int temp1 = obj.intArray[0]; // 6
        }
    }
}
```

本例final域为一个引用类型，它引用一个int型的数组对象。对于引用类型，写final域的重排序规则对编译器和处理器增加了如下约束：在构造函数内对一个final引用的对象的成员域的写入，与随后在构造函数外把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。

对上面的示例程序，假设首先线程A执行writerOne()方法，执行完后线程B执行writerTwo()方法，执行完后线程C执行reader()方法。图3-31是一种可能的线程执行时序。

在图3-31中，1是对final域的写入，2是对这个final域引用的对象的成员域的写入，3是把被构造的对象的引用赋值给某个引用变量。这里除了前面提到的1不能和3重排序外，2和3也不能重排序。

JMM可以确保读线程C至少能看到写线程A在构造函数中对final引用对象的成员域的写入。即C至少能看到数组下标0的值为1。而写线程B对数组元素的写入，读线程C可能看得到，也可能看不到。JMM不保证线程B的写入对读线程C可见，因为写线程B和读线程C之间存在数据竞争，此时的执行结果不可预知。

如果想要确保读线程C看到写线程B对数组元素的写入，写线程B和读线程C之间需要使用同步原语(lock或volatile)来确保内存可见性。

3.6.5 为什么final引用不能从构造函数内“溢出”

前面我们提到过，写final域的重排序规则可以确保：在引用变量为任意线程可见之前，该引用变量指向的对象的final域已经在构造函数中被正确初始化过了。其实，要得到这个效果，还需要一个保证：在构造函数内部，不能让这个被构造对象的引用为其他线程所见，也就是对象引用不能在构造函数中“溢出”。为了说明问题，让我们来看下面的示例代码。

```
public class FinalReferenceEscapeExample {
    final int i;
    static FinalReferenceEscapeExample obj;
    public FinalReferenceEscapeExample () {
        i = 1;                // 1写final域
        obj = this;           // 2 this引用在此"溢出"
    }
    public static void writer() {
        new FinalReferenceEscapeExample ();
    }
    public static void reader() {
        if (obj != null) {    // 3
            int temp = obj.i; // 4
        }
    }
}
```

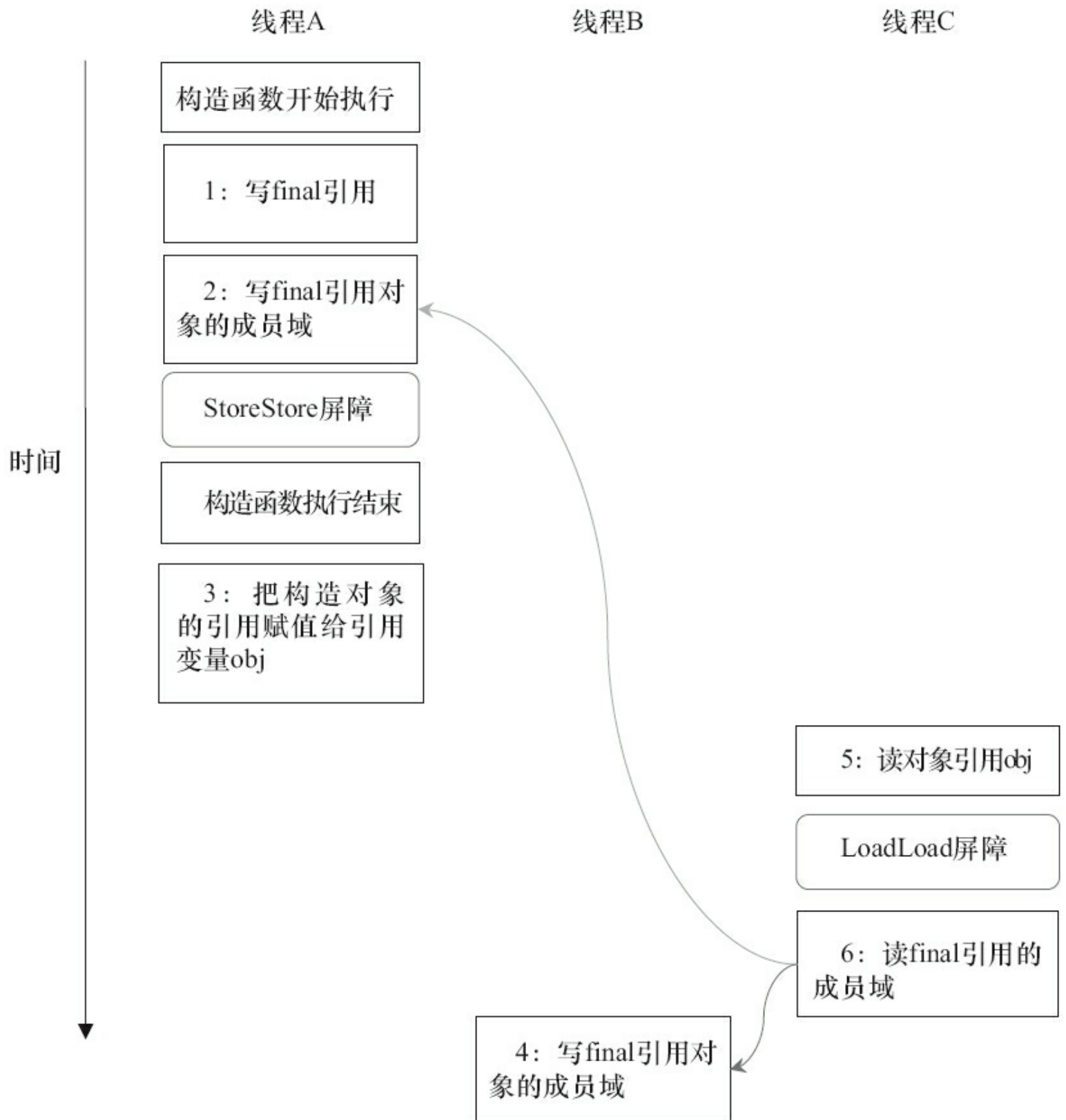


图3-31 引用型final的执行时序图

假设一个线程A执行writer()方法, 另一个线程B执行reader()方法。这里的操作2使得对象还未完成构造前就为线程B可见。即使这里的操作2是构造函数的最后一步, 且在程序中操作2排在操作1后面, 执行read()方法的线程仍然可能无法看到final域被初始化后的值, 因为这里的

操作1和操作2之间可能被重排序。实际的执行时序可能如图3-32所示。

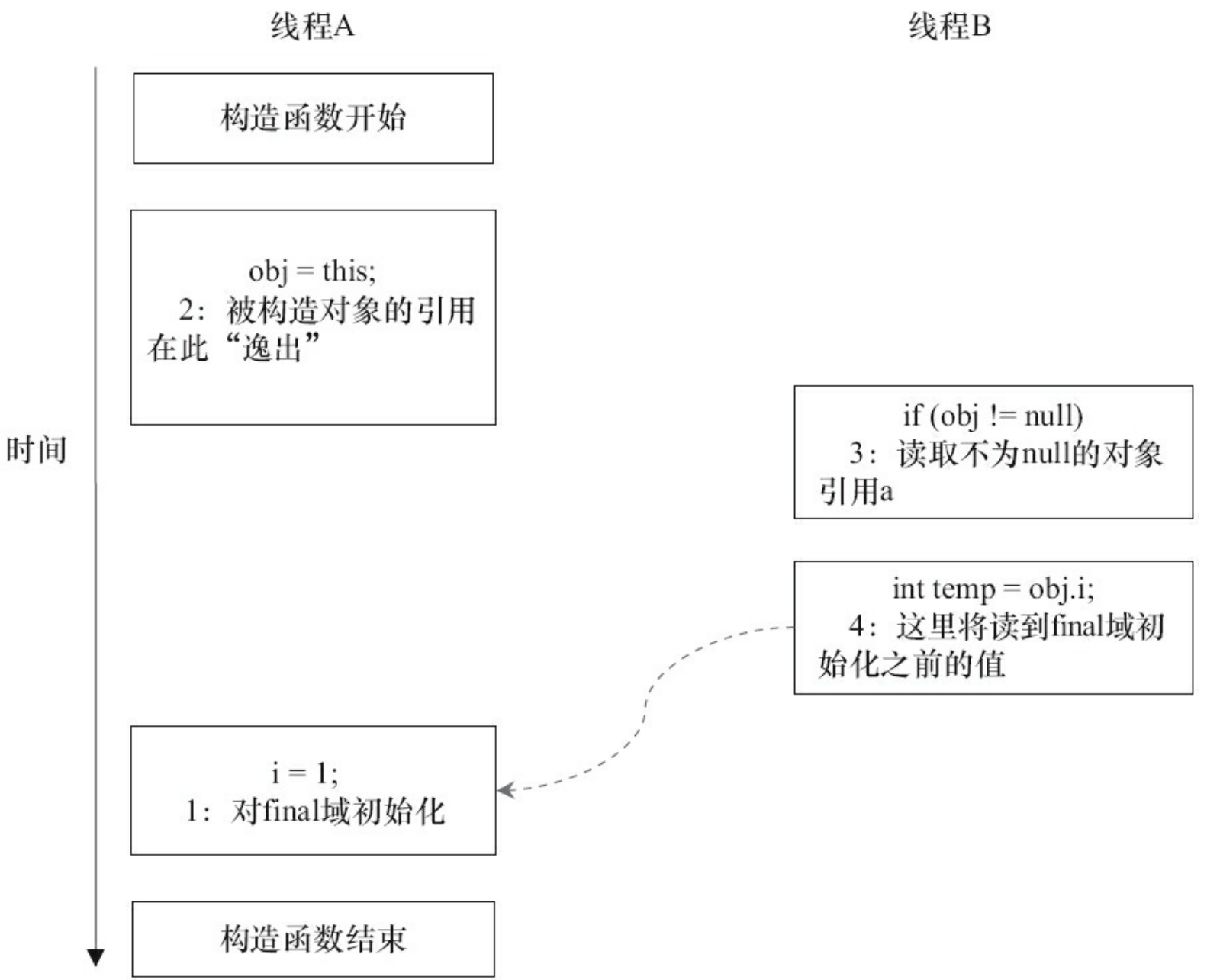


图3-32 多线程执行时序图

从图3-32可以看出:在构造函数返回前,被构造对象的引用不能为其他线程所见,因为此时的final域可能还没有被初始化。在构造函数返回后,任意线程都将保证能看到final域正确初始化之后的值。

3.6.6 final语义在处理器中的实现

现在我们以X86处理器为例，说明final语义在处理器中的具体实现。

上面我们提到，写final域的重排序规则会要求编译器在final域的写之后，构造函数return之前插入一个StoreStore屏障。读final域的重排序规则要求编译器在读final域的操作前面插入一个LoadLoad屏障。

由于X86处理器不会对写-写操作做重排序，所以在X86处理器中，写final域需要的StoreStore屏障会被省略掉。同样，由于X86处理器不会对存在间接依赖关系的操作做重排序，所以在X86处理器中，读final域需要的LoadLoad屏障也会被省略掉。也就是说，在X86处理器中，final域的读/写不会插入任何内存屏障！

3.6.7 JSR-133为什么要增强final的语义

在旧的Java内存模型中，一个最严重的缺陷就是线程可能看到final域的值会改变。比如，一个线程当前看到一个整型final域的值为0(还未初始化之前的默认值)，过一段时间之后这个线程再去读这个final域的值时，却发现值变为1(被某个线程初始化之后的值)。最常见的例子就是在旧的Java内存模型中，String的值可能会改变。

为了修补这个漏洞，JSR-133专家组增强了final的语义。通过为final域增加写和读重排序规则，可以为Java程序员提供初始化安全保证：只要对象是正确构造的(被构造对象的引用在构造函数中没有“逸出”)，那么不需要使用同步(指lock和volatile的使用)就可以保证任意线程都能看到这个final域在构造函数中被初始化之后的值。

3.7 happens-before

happens-before是JMM最核心的概念。对应Java程序员来说, 理解happens-before是理解JMM的关键。

3.7.1 JMM的设计

首先, 让我们来看JMM的设计意图。从JMM设计者的角度, 在设计JMM时, 需要考虑两个

关键因素。

- 程序员对内存模型的使用。程序员希望内存模型易于理解、易于编程。程序员希望基于一个强内存模型来编写代码。

- 编译器和处理器对内存模型的实现。编译器和处理器希望内存模型对它们的束缚越少越好, 这样它们就可以做尽可能多的优化来提高性能。编译器和处理器希望实现一个弱内存模型。

由于这两个因素互相矛盾, 所以JSR-133专家组在设计JMM时的核心目标就是找到一个好的平衡点: 一方面, 要为程序员提供足够强的内存可见性保证; 另一方面, 对编译器和处理器的限制要尽可能地放松。下面让我们来看JSR-133是如何实现这一目标的。

```
double pi    = 3.14;           // A
double r     = 1.0;           // B
double area  = pi * r * r;     // C
```

上面计算圆的面积的示例代码存在3个happens-before关系, 如下。

- A happens-before B。

- B happens-before C。

- A happens-before C。

在3个happens-before关系中, 2和3是必需的, 但1是不必要的。因此, JMM把happens-before

要求禁止的重排序分为了下面两类。

·会改变程序执行结果的重排序。

·不会改变程序执行结果的重排序。

JMM对这两种不同性质的重排序，采取了不同的策略，如下。

·对于会改变程序执行结果的重排序，JMM要求编译器和处理器必须禁止这种重排序。

·对于不会改变程序执行结果的重排序，JMM对编译器和处理器不做要求(JMM允许这种重排序)。

图3-33是JMM的设计示意图。

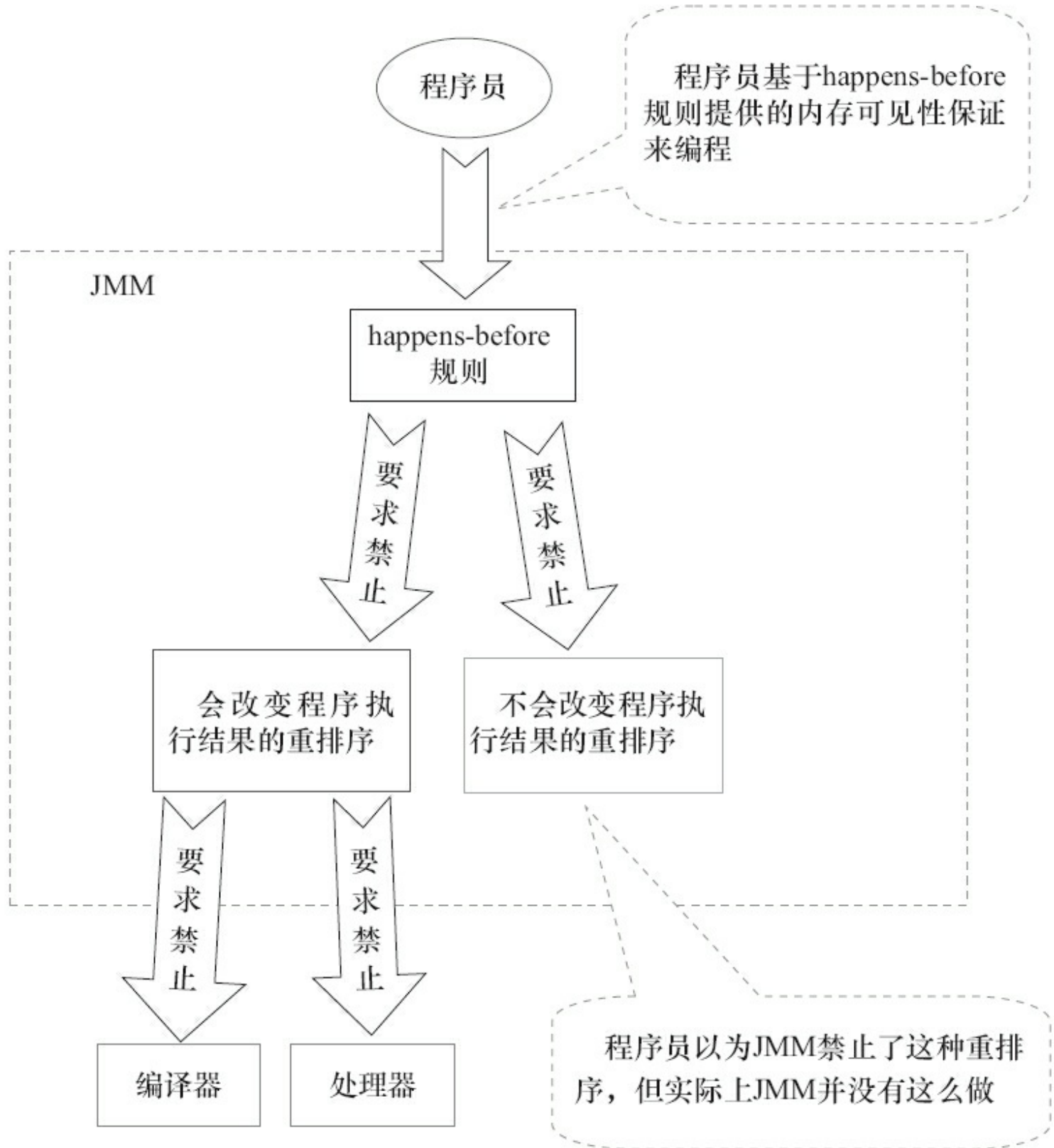


图3-33 JMM的设计示意图

从图3-33可以看出两点，如下。

·JMM向程序员提供的happens-before规则能满足程序员的需求。JMM的happens-before规则不但简单易懂，而且也向程序员提供了足够强的内存可见性保证(有些内存可见性保证其实

并不一定真实存在，比如上面的A happens-before B)。

·JMM对编译器和处理器的束缚已经尽可能少。从上面的分析可以看出，JMM其实是在遵循一个基本原则：只要不改变程序的执行结果(指的是单线程程序和正确同步的多线程程序)，编译器和处理器怎么优化都行。例如，如果编译器经过细致的分析后，认定一个锁只会被单个线程访问，那么这个锁可以被消除。再如，如果编译器经过细致的分析后，认定一个volatile变量只会被单个线程访问，那么编译器可以把这个volatile变量当作一个普通变量来对待。这些优化既不会改变程序的执行结果，又能提高程序的执行效率。

3.7.2 happens-before的定义

happens-before的概念最初由Leslie Lamport在其一篇影响深远的论文(《Time, Clocks and the Ordering of Events in a Distributed System》)中提出。Leslie Lamport使用happens-before来定义分布式系统中事件之间的偏序关系(partial ordering)。Leslie Lamport在这篇论文中给出了一个分布式算法,该算法可以将该偏序关系扩展为某种全序关系。

JSR-133使用happens-before的概念来指定两个操作之间的执行顺序。由于这两个操作可以在一个线程之内,也可以是在不同线程之间。因此,JMM可以通过happens-before关系向程序员提供跨线程的内存可见性保证(如果A线程的写操作a与B线程的读操作b之间存在happens-before关系,尽管a操作和b操作在不同的线程中执行,但JMM向程序员保证a操作将对b操作可见)。

《JSR-133:Java Memory Model and Thread Specification》对happens-before关系的定义如下。

1) 如果一个操作happens-before另一个操作,那么第一个操作的执行结果将对第二个操作可见,而且第一个操作的执行顺序排在第二个操作之前。

2) 两个操作之间存在happens-before关系,并不意味着Java平台的具体实现必须要按照happens-before关系指定的顺序来执行。如果重排序之后的执行结果,与按happens-before关系来执行的结果一致,那么这种重排序并不非法(也就是说,JMM允许这种重排序)。

上面的1)是JMM对程序员的承诺。从程序员的角度来说,可以这样理解happens-before关系:如果A happens-before B,那么Java内存模型将向程序员保证——A操作的结果将对B可见,且A的执行顺序排在B之前。注意,这只是Java内存模型向程序员做出的保证!

上面的2)是JMM对编译器和处理器重排序的约束原则。正如前面所言,JMM其实是在遵循一个基本原则:只要不改变程序的执行结果(指的是单线程程序和正确同步的多线程程序),

编译器和处理器怎么优化都行。JMM这么做的原因是：程序员对于这两个操作是否真的被重排序并不关心，程序员关心的是程序执行时的语义不能被改变（即执行结果不能被改变）。因此，happens-before关系本质上和as-if-serial语义是一回事。

·as-if-serial语义保证单线程内程序的执行结果不被改变，happens-before关系保证正确同步的多线程程序的执行结果不被改变。

·as-if-serial语义给编写单线程程序的程序员创造了一个幻境：单线程程序是按程序的顺序来执行的。happens-before关系给编写正确同步的多线程程序的程序员创造了一个幻境：正确同步的多线程程序是按happens-before指定的顺序来执行的。

as-if-serial语义和happens-before这么做的目的，都是为了在不改变程序执行结果的前提下，尽可能地提高程序执行的并行度。

3.7.3 happens-before规则

《JSR-133:Java Memory Model and Thread Specification》定义了如下happens-before规则。

1) **程序顺序规则**: 一个线程中的每个操作, happens-before于该线程中的任意后续操作。

2) **监视器锁规则**: 对一个锁的解锁, happens-before于随后对这个锁的加锁。

3) **volatile变量规则**: 对一个volatile域的写, happens-before于任意后续对这个volatile域的读。

4) **传递性**: 如果A happens-before B, 且B happens-before C, 那么A happens-before C。

5) **start()规则**: 如果线程A执行操作ThreadB.start() (启动线程B), 那么A线程的ThreadB.start()操作happens-before于线程B中的任意操作。

6) **join()规则**: 如果线程A执行操作ThreadB.join()并成功返回, 那么线程B中的任意操作happens-before于线程A从ThreadB.join()操作成功返回。

这里的规则1)、2)、3)和4)前面都讲到过, 这里再做个总结。由于2)和3)情况类似, 这里只以1)、3)和4)为例来说明。图3-34是volatile写-读建立的happens-before关系图。

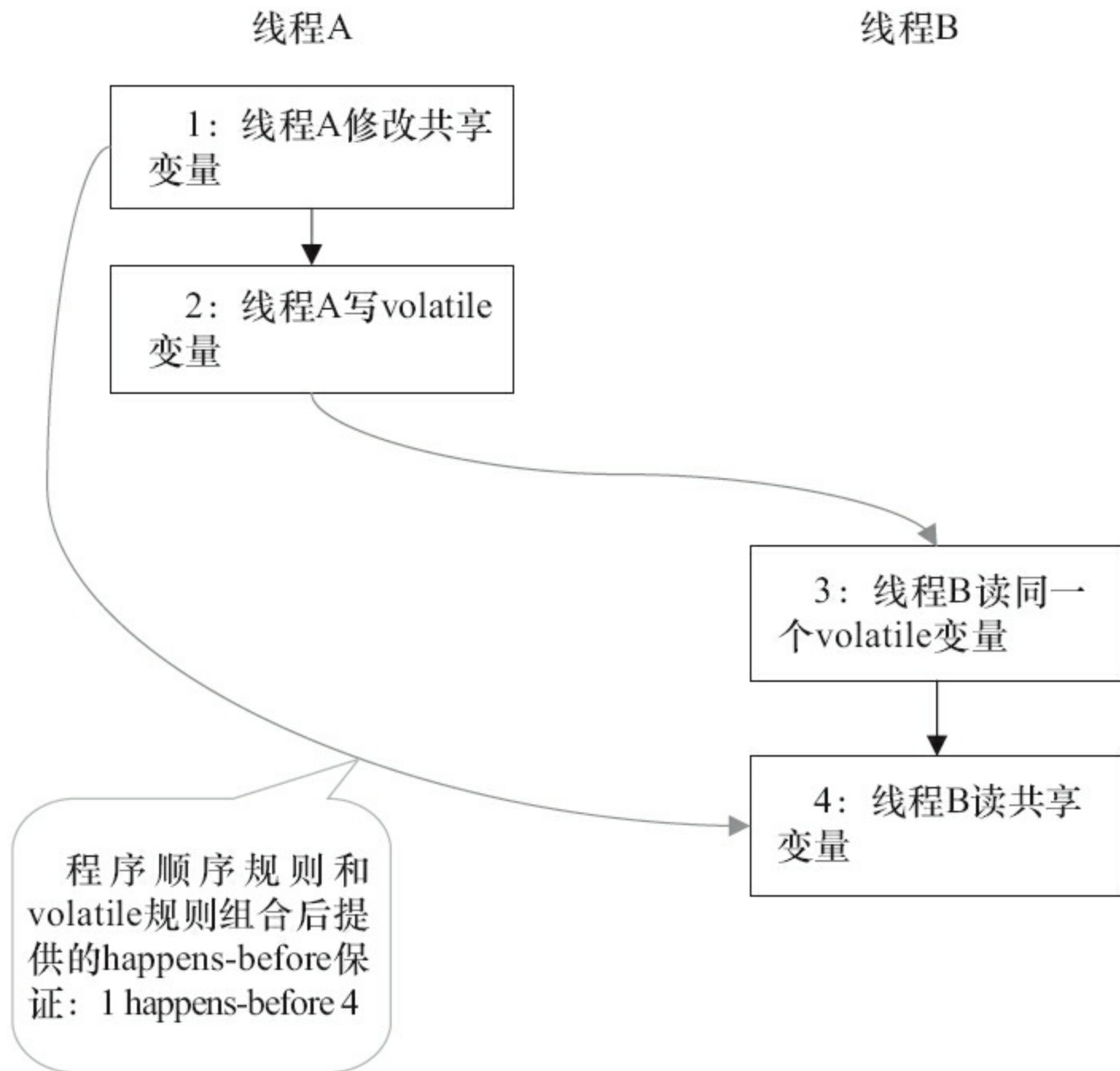


图3-34 happens-before关系的示意图

结合图3-34, 我们做以下分析。

· 1 happens-before 2和3 happens-before 4由程序顺序规则产生。由于编译器和处理器都要遵守as-if-serial语义, 也就是说, as-if-serial语义保证了程序顺序规则。因此, 可以把程序顺序规则看成是对as-if-serial语义的“封装”。

· 2 happens-before 3是由volatile规则产生。前面提到过, 对一个volatile变量的读, 总是能看

到(任意线程)之前对这个volatile变量最后的写入。因此, volatile的这个特性可以保证实现volatile规则。

· 1 happens-before 4是由传递性规则产生的。这里的传递性是由volatile的内存屏障插入策略和volatile的编译器重排序规则共同来保证的。

下面我们来看start()规则。假设线程A在执行的过程中, 通过执行ThreadB.start()来启动线程B;同时, 假设线程A在执行ThreadB.start()之前修改了一些共享变量, 线程B在开始执行后会读这些共享变量。图3-35是该程序对应的happens-before关系图。

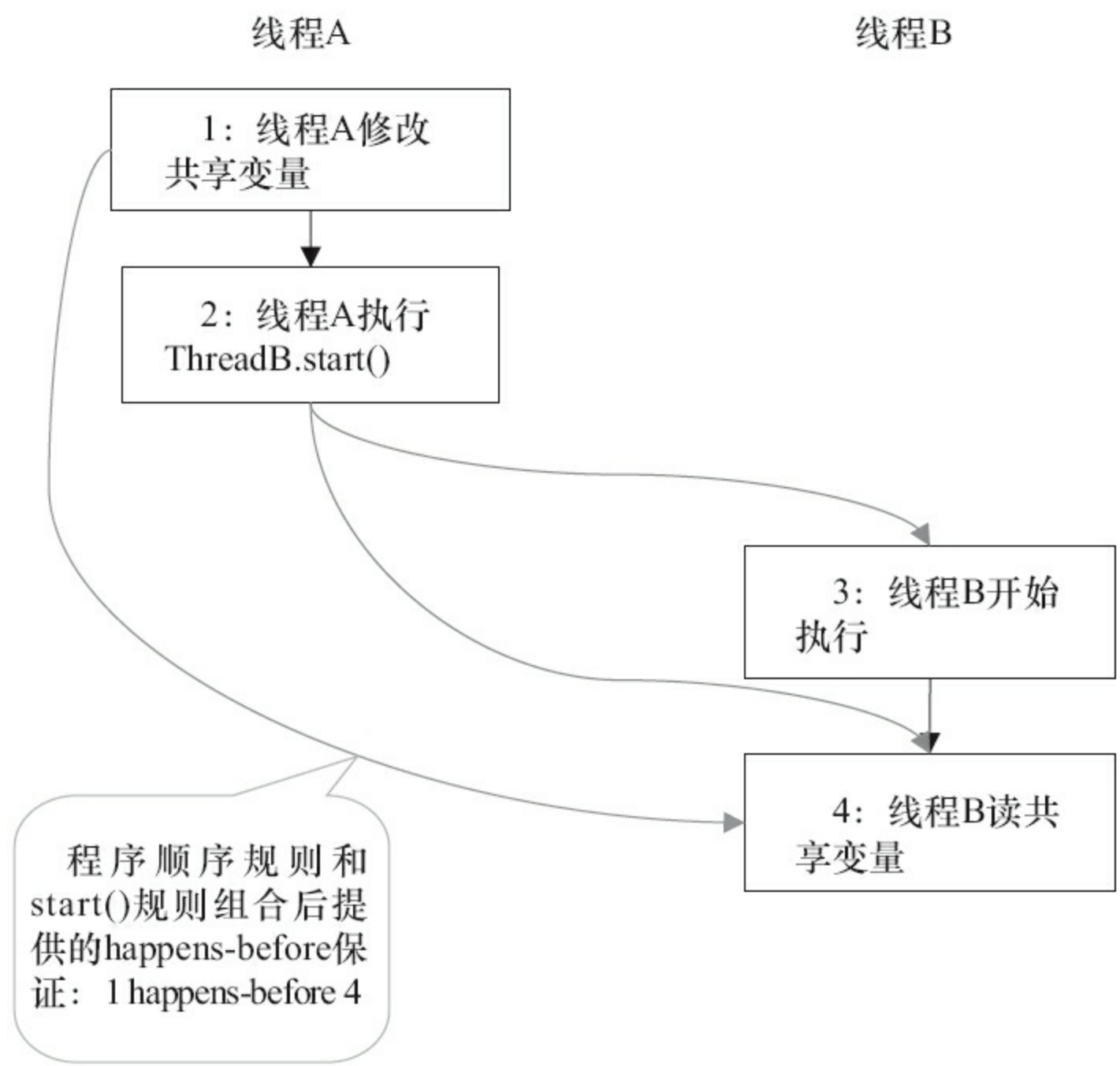


图3-35 happens-before关系的示意图

在图3-35中, 1 happens-before 2由程序顺序规则产生。2 happens-before 4由start()规则产生。根据传递性, 将有1 happens-before 4。这实意味着, 线程A在执行ThreadB.start()之前对共享变量所做的修改, 接下来在线程B开始执行后都将确保对线程B可见。

下面我们来看join()规则。假设线程A在执行的过程中, 通过执行ThreadB.join()来等待线程B终止;同时, 假设线程B在终止之前修改了一些共享变量, 线程A从ThreadB.join()返回后会读这些共享变量。图3-36是该程序对应的happens-before关系图。

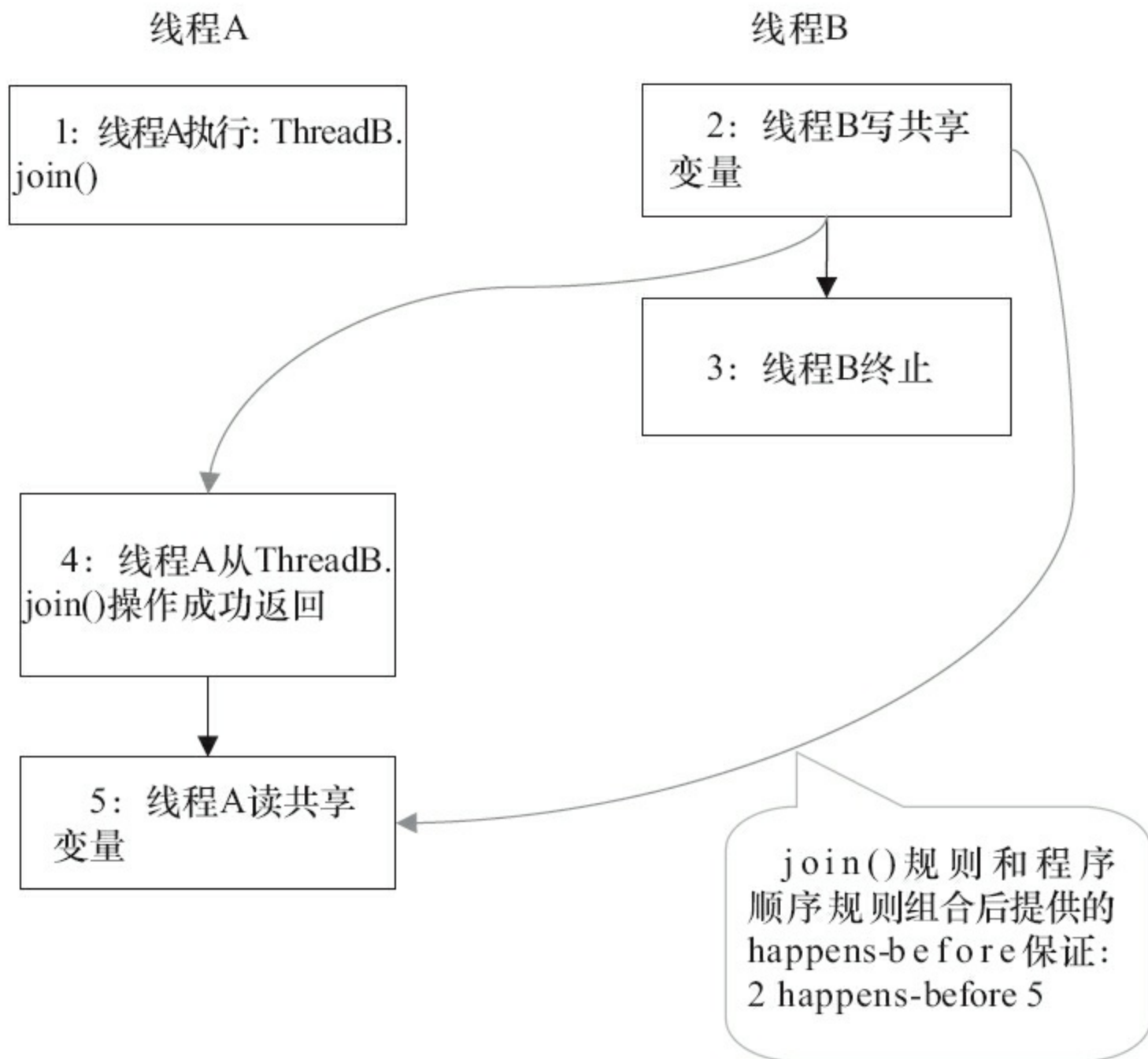


图3-36 happens-before关系的示意图

在图3-36中, 2 happens-before 4由join()规则产生; 4 happens-before 5由程序顺序规则产生。根据传递性规则, 将有2 happens-before 5。这意味着, 线程A执行操作ThreadB.join()并成功返回后, 线程B中的任意操作都将对线程A可见。

3.8 双重检查锁定与延迟初始化

在Java多线程程序中，有时候需要采用延迟初始化来降低初始化类和创建对象的开销。双重检查锁定是常见的延迟初始化技术，但它是一个错误的用法。本文将分析双重检查锁定的错误根源，以及两种线程安全的延迟初始化方案。

3.8.1 双重检查锁定的由来

在Java程序中，有时候可能需要推迟一些高开销的对象初始化操作，并且只有在使用这些对象时才进行初始化。此时，程序员可能会采用延迟初始化。但要正确实现线程安全的延迟初始化需要一些技巧，否则很容易出现问题。比如，下面是非线程安全的延迟初始化对象的示例代码。

```
public class UnsafeLazyInitialization {
    private static Instance instance;
    public static Instance getInstance() {
        if (instance == null)           // 1:A线程执行
            instance = new Instance();  // 2:B线程执行
        return instance;
    }
}
```

在UnsafeLazyInitialization类中，假设A线程执行代码1的同时，B线程执行代码2。此时，线程A可能会看到instance引用的对象还没有完成初始化（出现这种情况的原因见3.8.2节）。

对于UnsafeLazyInitialization类，我们可以对getInstance()方法做同步处理来实现线程安全的延迟初始化。示例代码如下。

```
public class SafeLazyInitialization {
    private static Instance instance;
    public synchronized static Instance getInstance() {
        if (instance == null)
            instance = new Instance();
        return instance;
    }
}
```

由于对getInstance()方法做了同步处理，synchronized将导致性能开销。如果getInstance()方法被多个线程频繁的调用，将会导致程序执行性能的下降。反之，如果getInstance()方法不会被多个线程频繁的调用，那么这个延迟初始化方案将能提供令人满意的性能。

在早期的JVM中，synchronized(甚至是无竞争的synchronized)存在巨大的性能开销。因此，人们想出了一个“聪明”的技巧：双重检查锁定(Double-Checked Locking)。人们想通过双重检查锁定来降低同步的开销。下面是使用双重检查锁定来实现延迟初始化的示例代码。

```
public class DoubleCheckedLocking {                                     // 1
    private static Instance instance;                                   // 2
    public static Instance getInstance() {                               // 3
        if (instance == null) {                                         // 4:第一次检查
            synchronized (DoubleCheckedLocking.class) {                 // 5:加锁
                if (instance == null)                                     // 6:第二次检查
                    instance = new Instance();                           // 7:问题的根源出在这里
            }                                                             // 8
        }                                                                 // 9
        return instance;                                                 // 10
    }                                                                     // 11
}
```

如上面代码所示，如果第一次检查instance不为null，那么就不需要执行下面的加锁和初始化操作。因此，可以大幅降低synchronized带来的性能开销。上面代码表面上看起来，似乎两全其美。

- 多个线程试图在同一时间创建对象时，会通过加锁来保证只有一个线程能创建对象。
- 在对象创建好之后，执行getInstance()方法将不需要获取锁，直接返回已创建好的对象。

双重检查锁定看起来似乎很完美，但这是一个错误的优化！在线程执行到第4行，代码读取到instance不为null时，instance引用的对象有可能还没有完成初始化。

3.8.2 问题的根源

前面的双重检查锁定示例代码的第7行(instance=new Singleton();)创建了一个对象。这一行代码可以分解为如下的3行伪代码。

```
memory = allocate();    // 1:分配对象的内存空间
ctorInstance(memory);    // 2:初始化对象
instance = memory;       // 3:设置instance指向刚分配的内存地址
```

上面3行伪代码中的2和3之间,可能会被重排序(在一些JIT编译器上,这种重排序是真实发生的,详情见参考文献1的“Out-of-order writes”部分)。2和3之间重排序之后的执行时序如下。

```
memory = allocate();    // 1:分配对象的内存空间
instance = memory;       // 3:设置instance指向刚分配的内存地址
                        // 注意,此时对象还没有被初始化!
ctorInstance(memory);    // 2:初始化对象
```

根据《The Java Language Specification,Java SE 7 Edition》(后文简称为Java语言规范),所有线程在执行Java程序时必须遵守intra-thread semantics。intra-thread semantics保证重排序不会改变单线程内的程序执行结果。换句话说, intra-thread semantics允许那些在单线程内,不会改变单线程程序执行结果的重排序。上面3行伪代码的2和3之间虽然被重排序了,但这个重排序并不会违反intra-thread semantics。这个重排序在没有改变单线程程序执行结果的前提下,可以提高程序的执行性能。

为了更好地理解intra-thread semantics,请看如图3-37所示的示意图(假设一个线程A在构造对象后,立即访问这个对象)。

如图3-37所示,只要保证2排在4的前面,即使2和3之间重排序了,也不会违反intra-thread semantics。

下面，再让我们查看多线程并发执行的情况。如图3-38所示。

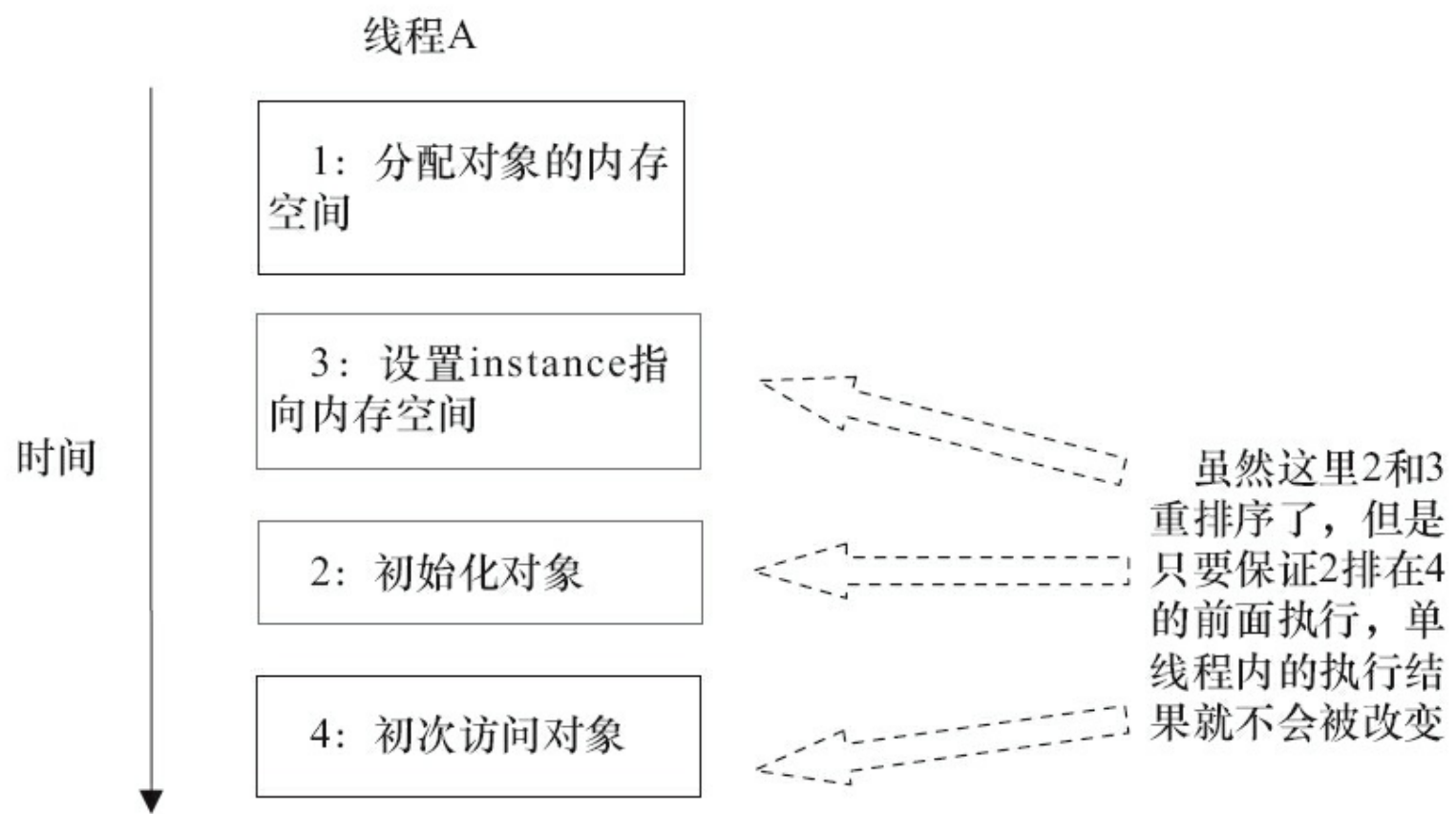


图3-37 线程执行时序图

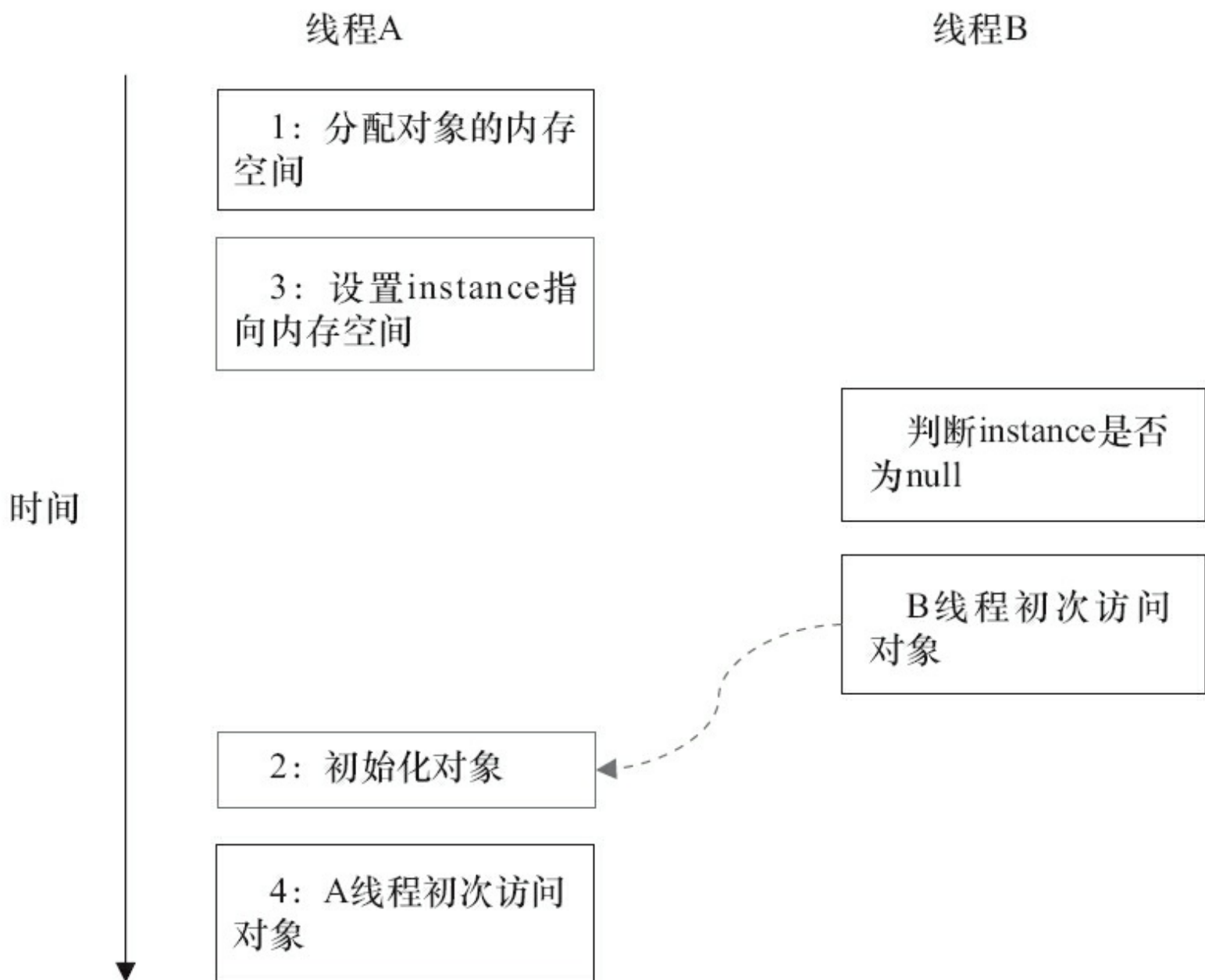


图3-38 多线程执行时序图

由于单线程内要遵守intra-thread semantics, 从而能保证A线程的执行结果不会被改变。但是, 当线程A和B按图3-38的时序执行时, B线程将看到一个还没有被初始化的对象。

回到本文的主题, DoubleCheckedLocking示例代码的第7行(instance=new Singleton();)如果发生重排序, 另一个并发执行的线程B就有可能在第4行判断instance不为null。线程B接下来将访问instance所引用的对象, 但此时这个对象可能还没有被A线程初始化! 表3-6是这个场景的具体执行时序。

表3-6 多线程执行时序表

时间	线程 A	线程 B
t1	A1：分配对象的内存空间	
t2	A3：设置 instance 指向内存空间	
t3		B1：判断 instance 是否为空
t4		B2：由于 instance 不为 null，线程 B 将访问 instance 引用的对象
t5	A2：初始化对象	
t6	A4：访问 instance 引用的对象	

这里A2和A3虽然重排序了，但Java内存模型的intra-thread semantics将确保A2一定会排在A4前面执行。因此，线程A的intra-thread semantics没有改变，但A2和A3的重排序，将导致线程B在B1处判断出instance不为空，线程B接下来将访问instance引用的对象。此时，线程B将会访问到一个还未初始化的对象。

在知晓了问题发生的根源之后，我们可以想出两个办法来实现线程安全的延迟初始化。

- 1)不允许2和3重排序。
- 2)允许2和3重排序，但不允许其他线程“看到”这个重排序。

后文介绍的两个解决方案，分别对应于上面这两点。

3.8.3 基于volatile的解决方案

对于前面的基于双重检查锁定来实现延迟初始化的方案(指DoubleCheckedLocking示例代码),只需要做一点小的修改(把instance声明为volatile型),就可以实现线程安全的延迟初始化。请看下面的示例代码。

```
public class SafeDoubleCheckedLocking {
    private volatile static Instance instance;
    public static Instance getInstance() {
        if (instance == null) {
            synchronized (SafeDoubleCheckedLocking.class) {
                if (instance == null)
                    instance = new Instance();           // instance为volatile, 现在没问题了
            }
        }
        return instance;
    }
}
```

 **注意** 这个解决方案需要JDK 5或更高版本(因为从JDK 5开始使用新的JSR-133内存模型规范, 这个规范增强了volatile的语义)。

当声明对象的引用为volatile后, 3.8.2节中的3行伪代码中的2和3之间的重排序, 在多线程环境中将会被禁止。上面示例代码将按如下的时序执行, 如图3-39所示。

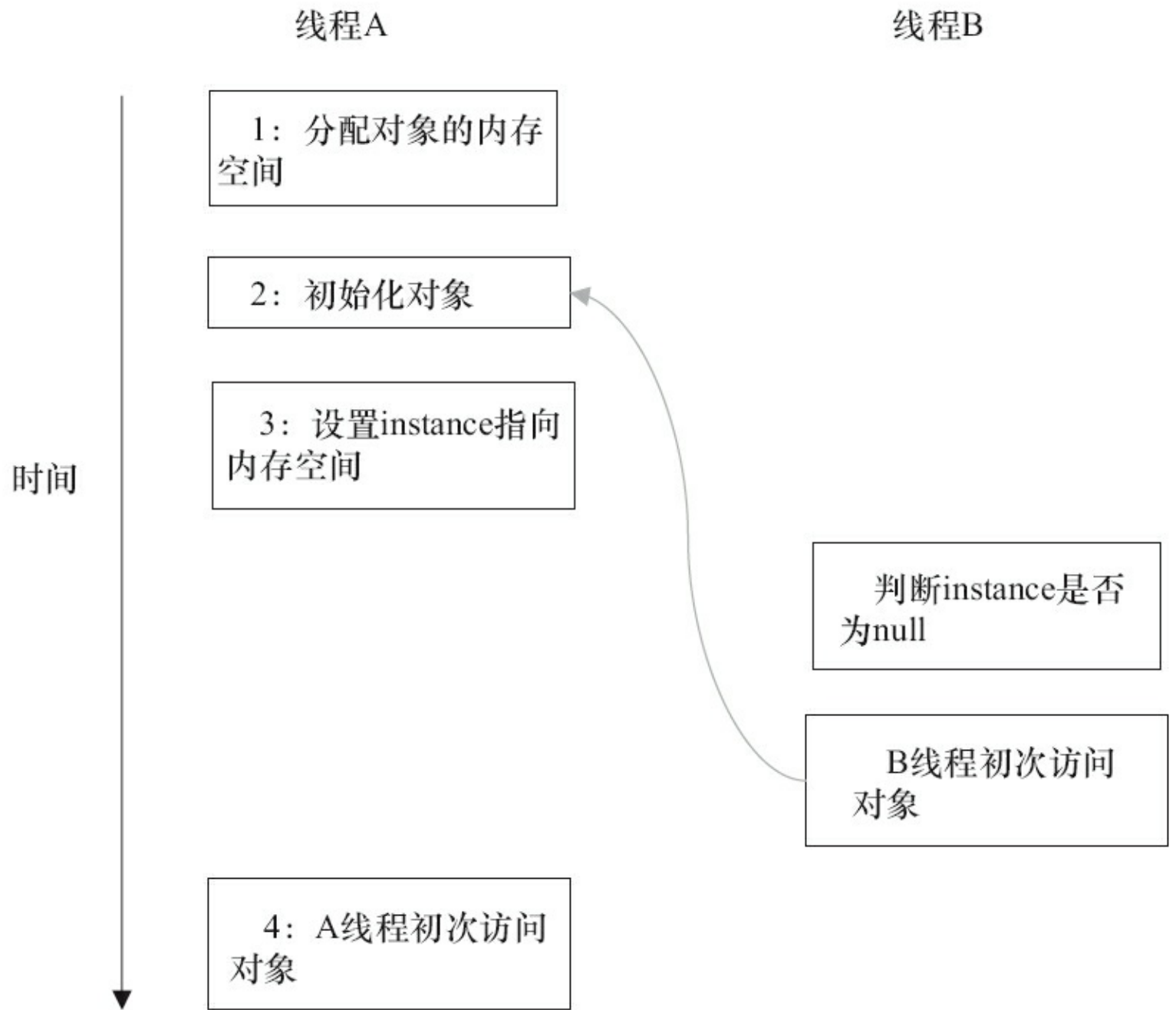


图3-39 多线程执行时序图

这个方案本质上是禁止图3-39中的2和3之间的重排序，来保证线程安全的延迟初始化。

3.8.4 基于类初始化的解决方案

JVM在类的初始化阶段(即在Class被加载后, 且被线程使用之前), 会执行类的初始化。在执行类的初始化期间, JVM会去获取一个锁。这个锁可以同步多个线程对同一个类的初始化。

基于这个特性, 可以实现另一种线程安全的延迟初始化方案(这个方案被称之为 Initialization On Demand Holder idiom)。

```
public class InstanceFactory {  
    private static class InstanceHolder {  
        public static Instance instance = new Instance();  
    }  
    public static Instance getInstance() {  
        return InstanceHolder.instance ;    // 这里将导致InstanceHolder类被初始化  
    }  
}
```

假设两个线程并发执行getInstance()方法, 下面是执行的示意图, 如图3-40所示。

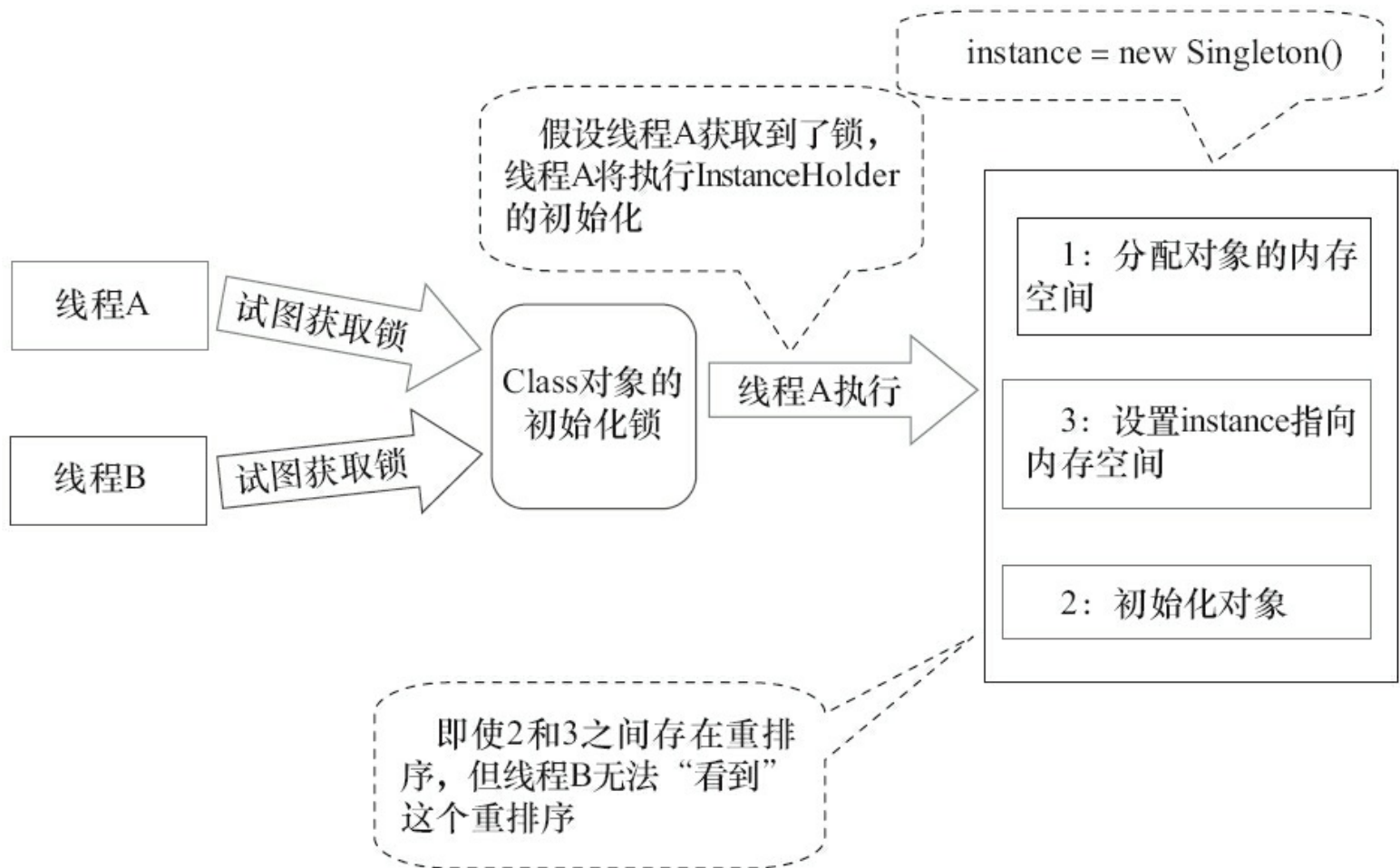


图3-40 两个线程并发执行的示意图

这个方案的实质是：允许3.8.2节中的3行伪代码中的2和3重排序，但不允许非构造线程（这里指线程B）“看到”这个重排序。

初始化一个类，包括执行这个类的静态初始化和初始化在这个类中声明的静态字段。根据Java语言规范，在首次发生下列任意一种情况时，一个类或接口类型T将被立即初始化。

- 1) T是一个类，而且一个T类型的实例被创建。
- 2) T是一个类，且T中声明的一个静态方法被调用。
- 3) T中声明的一个静态字段被赋值。
- 4) T中声明的一个静态字段被使用，而且这个字段不是一个常量字段。

5) T是一个顶级类(Top Level Class, 见Java语言规范的§7.6), 而且一个断言语句嵌套在T内部被执行。

在InstanceFactory示例代码中, 首次执行getInstance()方法的线程将导致InstanceHolder类被初始化(符合情况4)。

由于Java语言是多线程的, 多个线程可能在同一时间尝试去初始化同一个类或接口(比如这里多个线程可能在同一时刻调用getInstance()方法来初始化InstanceHolder类)。因此, 在Java中初始化一个类或者接口时, 需要做细致的同步处理。

Java语言规范规定, 对于每一个类或接口C, 都有一个唯一的初始化锁LC与之对应。从C到LC的映射, 由JVM的具体实现去自由实现。JVM在类初始化期间会获取这个初始化锁, 并且每个线程至少获取一次锁来确保这个类已经被初始化过了(事实上, Java语言规范允许JVM的具体实现在这里做一些优化, 见后文的说明)。

对于类或接口的初始化, Java语言规范制定了精巧而复杂的类初始化处理过程。Java初始化一个类或接口的处理过程如下(这里对类初始化处理过程的说明, 省略了与本文无关的部分; 同时为了更好的说明类初始化过程中的同步处理机制, 笔者人为的把类初始化的处理过程分为了5个阶段)。

第1阶段: 通过在Class对象上同步(即获取Class对象的初始化锁), 来控制类或接口的初始化。这个获取锁的线程会一直等待, 直到当前线程能够获取到这个初始化锁。

假设Class对象当前还没有被初始化(初始化状态state, 此时被标记为state=noInitialization), 且有两个线程A和B试图同时初始化这个Class对象。图3-41是对应的示意图。

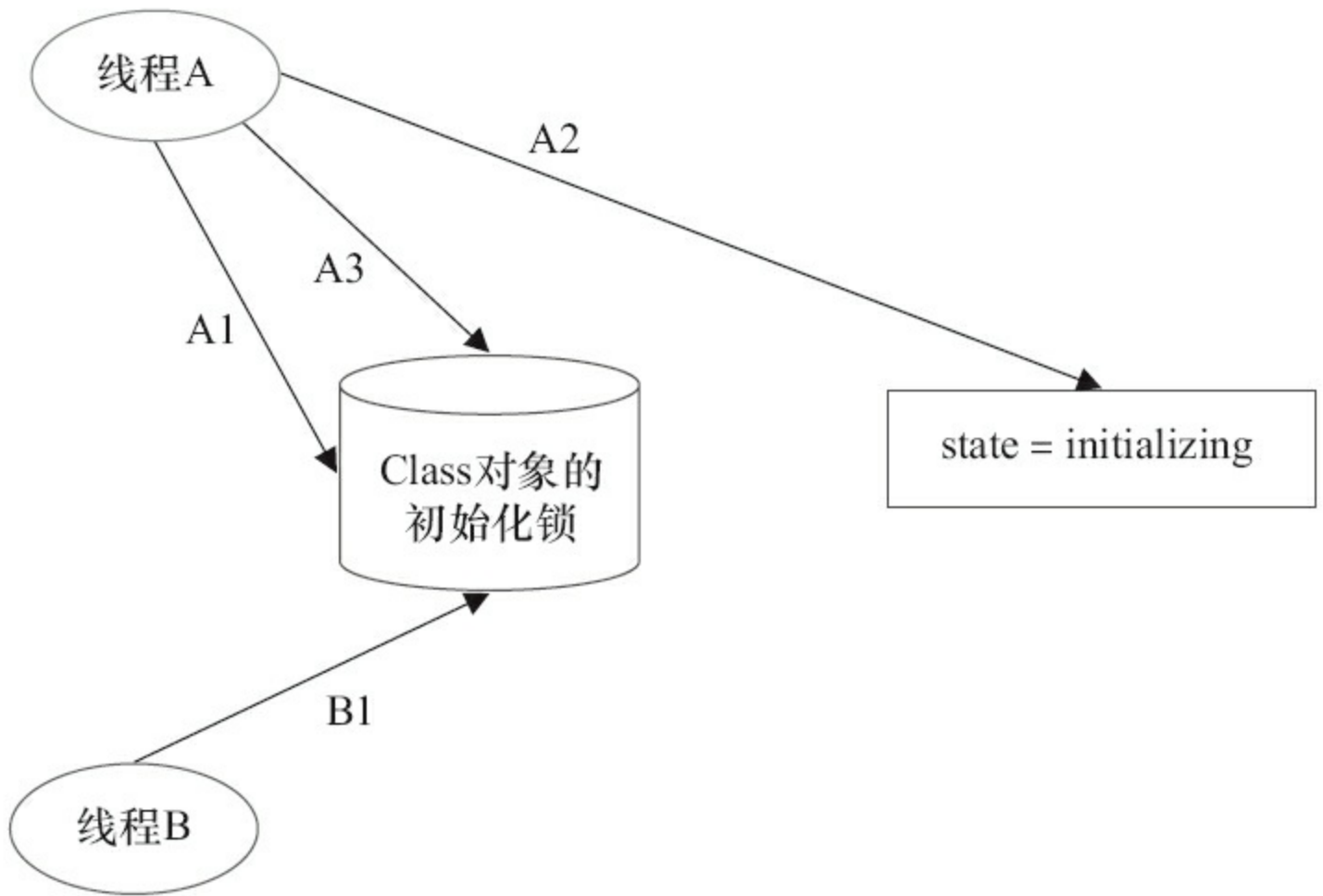


图3-41 类初始化——第1阶段

表3-7是这个示意图的说明。

表3-7 类初始化——第1阶段的执行时序表

时间	线程 A	线程 B
t1	A1: 尝试获取 Class 对象的初始化锁。这里假设线程 A 获取到了初始化锁	B1: 尝试获取 Class 对象的初始化锁，由于线程 A 获取到了锁，线程 B 将一直等待获取初始化锁
t2	A2 : 线程 A 看到线程还未被初始化（因为读取得到 state == noInitialization），线程设置 state = initializing	
t3	A3: 线程 A 释放初始化锁	

第2阶段:线程A执行类的初始化, 同时线程B在初始化锁对应的condition上等待。

表3-8是这个示意图的说明。

表3-8 类初始化——第2阶段的执行时序表

时间	线程 A	线程 B
t1	A1: 执行类的静态初始化和初始化类中声明的静态字段	B1: 获取到初始化锁
t2		B2: 读取到 state = initializing
t3		B3: 释放初始化锁
t4		B4: 在初始化锁的 condition 中等待

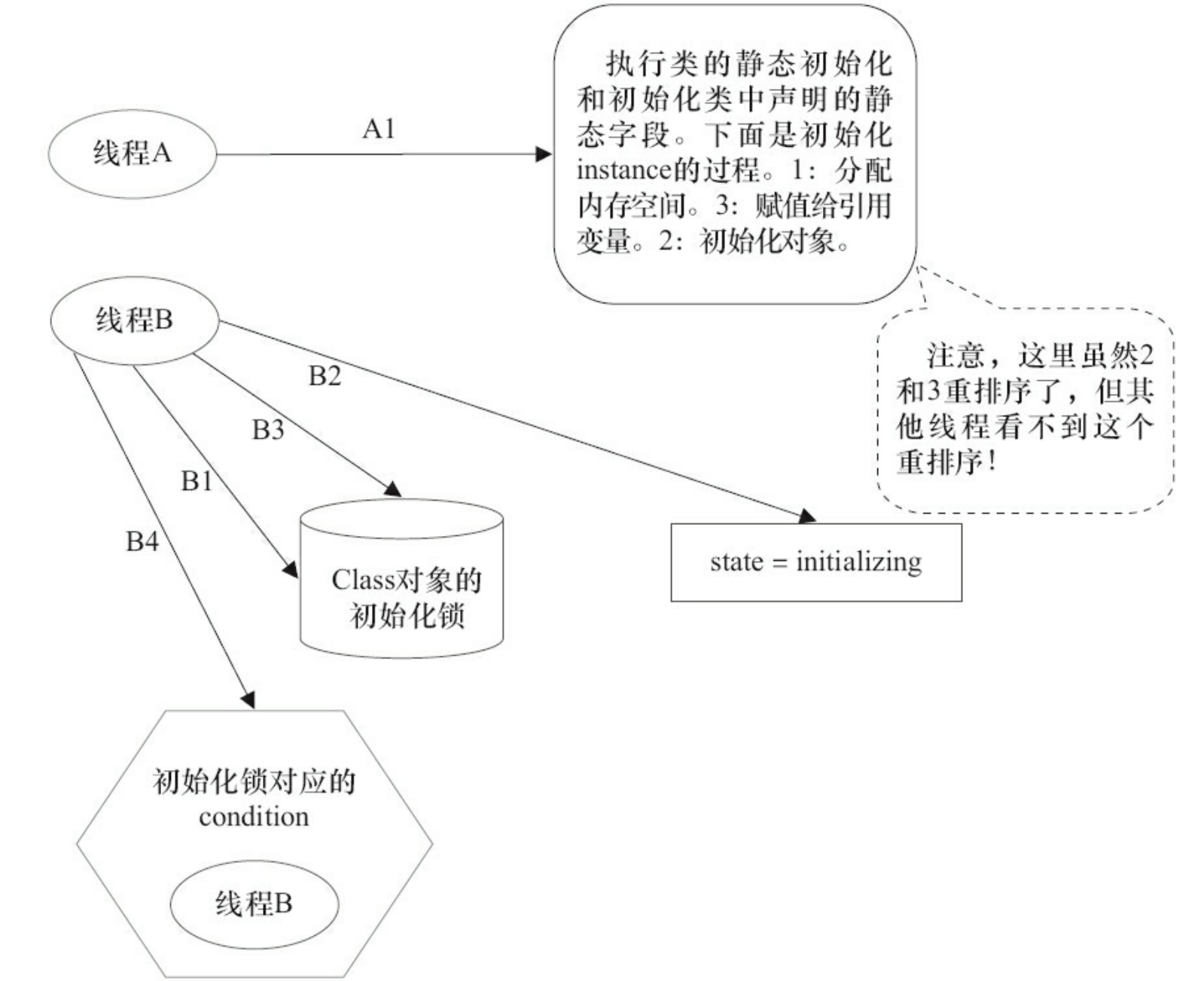


图3-42 类初始化——第2阶段

第3阶段: 线程A设置state=initialized, 然后唤醒在condition中等待的所有线程。

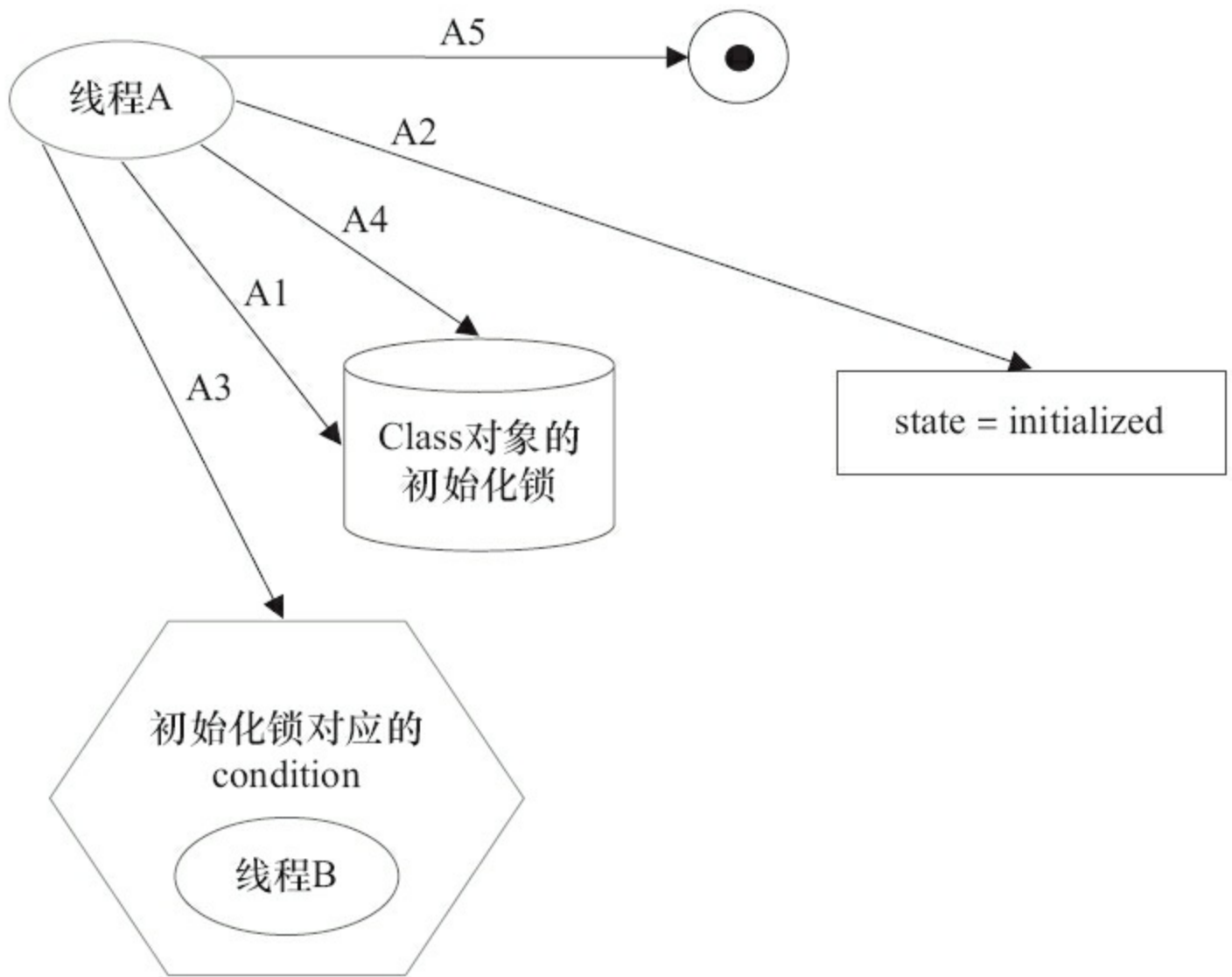


图3-43 类初始化——第3阶段

表3-9是这个示意图的说明。

表3-9 类初始化——第3阶段的执行时序表

时间	线程 A	线程 B
t1	A1：执行类的静态初始化和初始化类中声明的静态字段	B1：获取到初始化锁
t2		B2：读取到 state = initializing
t3		B3：释放初始化锁
t4		B4：在初始化锁的 condition 中等待

第4阶段:线程B结束类的初始化处理。

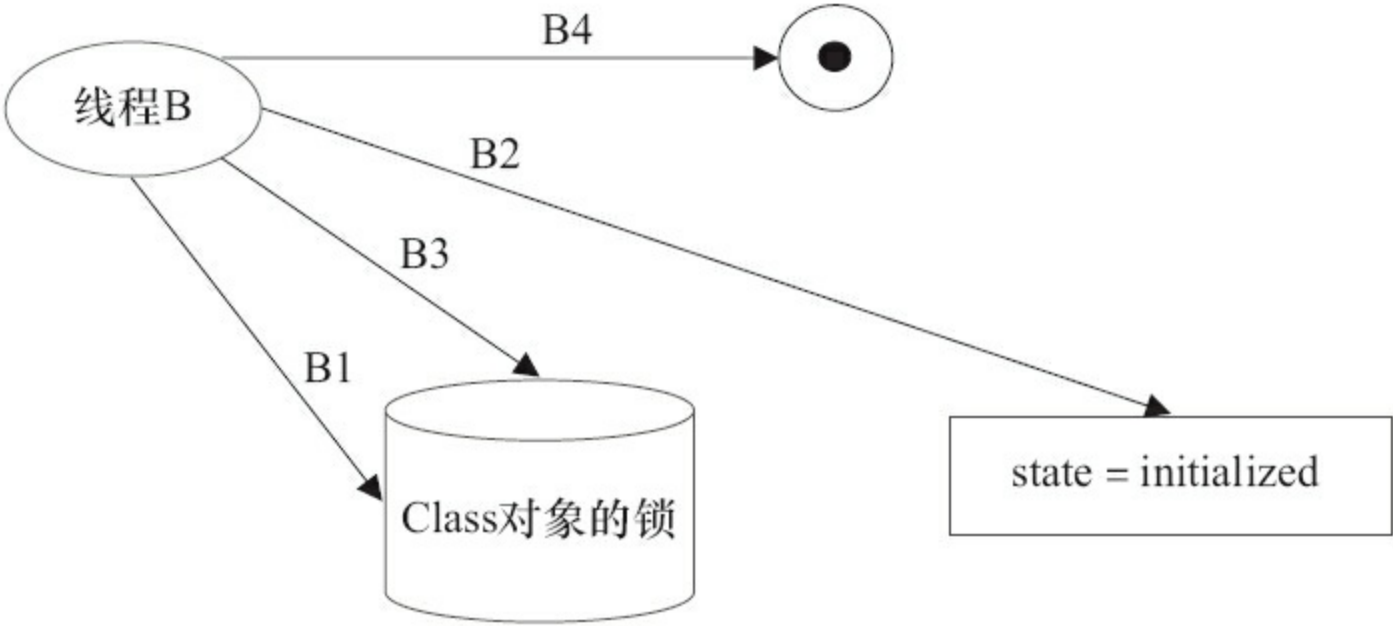


图3-44 类初始化——第4阶段

表3-10是这个示意图的说明。

表3-10 类初始化——第4阶段的执行时序表

时间	线程 B	时间	线程 B
t1	B1：获取初始化锁	t3	B3：释放初始化锁
t2	B2：读取到 state = initialized	t4	B4：线程 B 的类初始化处理过程完成

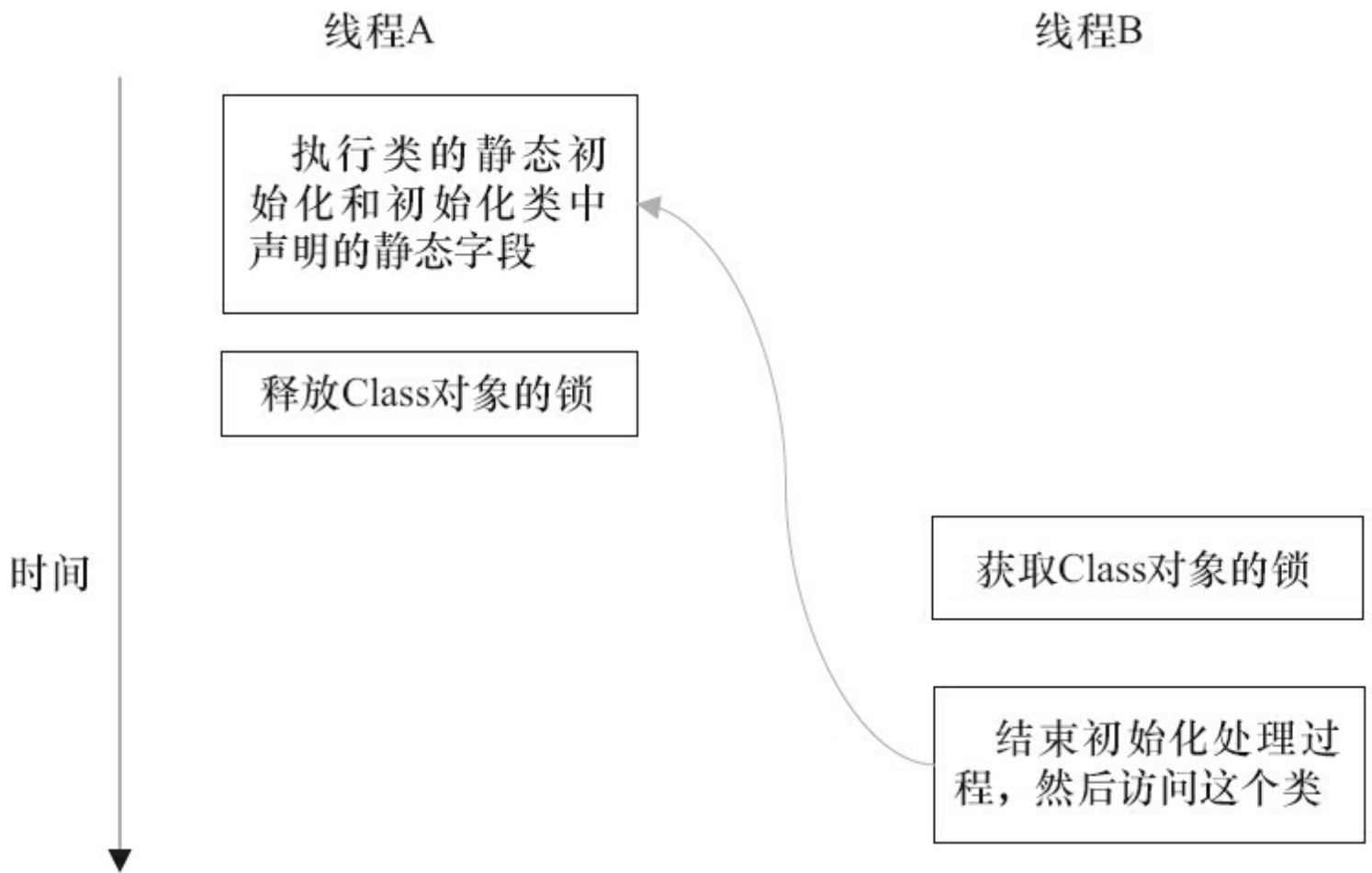


图3-45 多线程执行时序图

线程A在第2阶段的A1执行类的初始化，并在第3阶段的A4释放初始化锁；线程B在第4阶段的B1获取同一个初始化锁，并在第4阶段的B4之后才开始访问这个类。根据Java内存模型规范的锁规则，这里将存在如下的happens-before关系。

这个happens-before关系将保证：线程A执行类的初始化时的写入操作（执行类的静态初始化和初始化类中声明的静态字段），线程B一定能看到。

第5阶段：线程C执行类的初始化的处理。

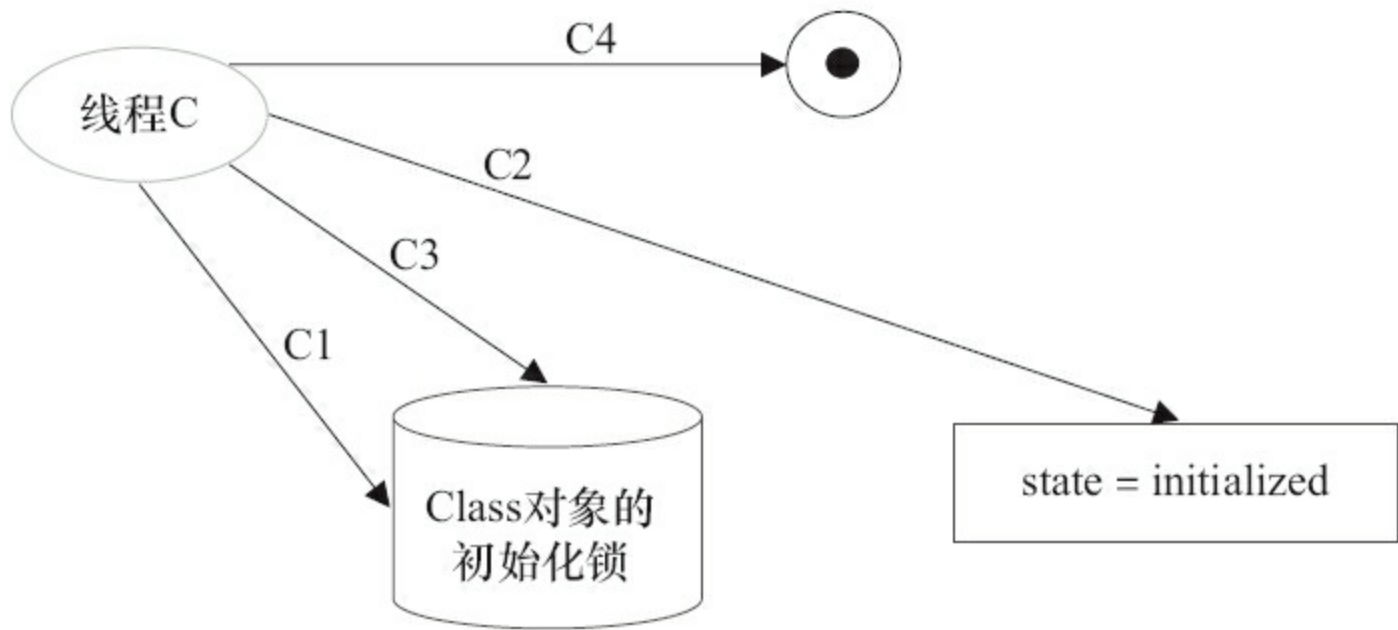


图3-46 类初始化——第5阶段

表3-11是这个示意图的说明。

表3-11 类初始化——第5阶段的执行时序表

时间	线程 B
t1	C1: 获取初始化锁
t2	C2: 读取到 state = initialized
t3	C3: 释放初始化锁
t4	C4: 线程 C 的类初始化处理过程完成

在第3阶段之后，类已经完成了初始化。因此线程C在第5阶段的类初始化处理过程相对简单一些（前面的线程A和B的类初始化处理过程都经历了两次锁获取-锁释放，而线程C的类初始化处理只需要经历一次锁获取-锁释放）。

线程A在第2阶段的A1执行类的初始化，并在第3阶段的A4释放锁；线程C在第5阶段的C1获取同一个锁，并在在第5阶段的C4之后才开始访问这个类。根据Java内存模型规范的锁规则，将存在如下的happens-before关系。

这个happens-before关系将保证：线程A执行类的初始化时的写入操作，线程C一定能看

到。

注意 这里的condition和state标记是本文虚构出来的。Java语言规范并没有硬性规定一定要使用condition和state标记。JVM的具体实现只要实现类似功能即可。

注意 Java语言规范允许Java的具体实现，优化类的初始化处理过程(对这里的第5阶段做优化)，具体细节参见Java语言规范的12.4.2节。

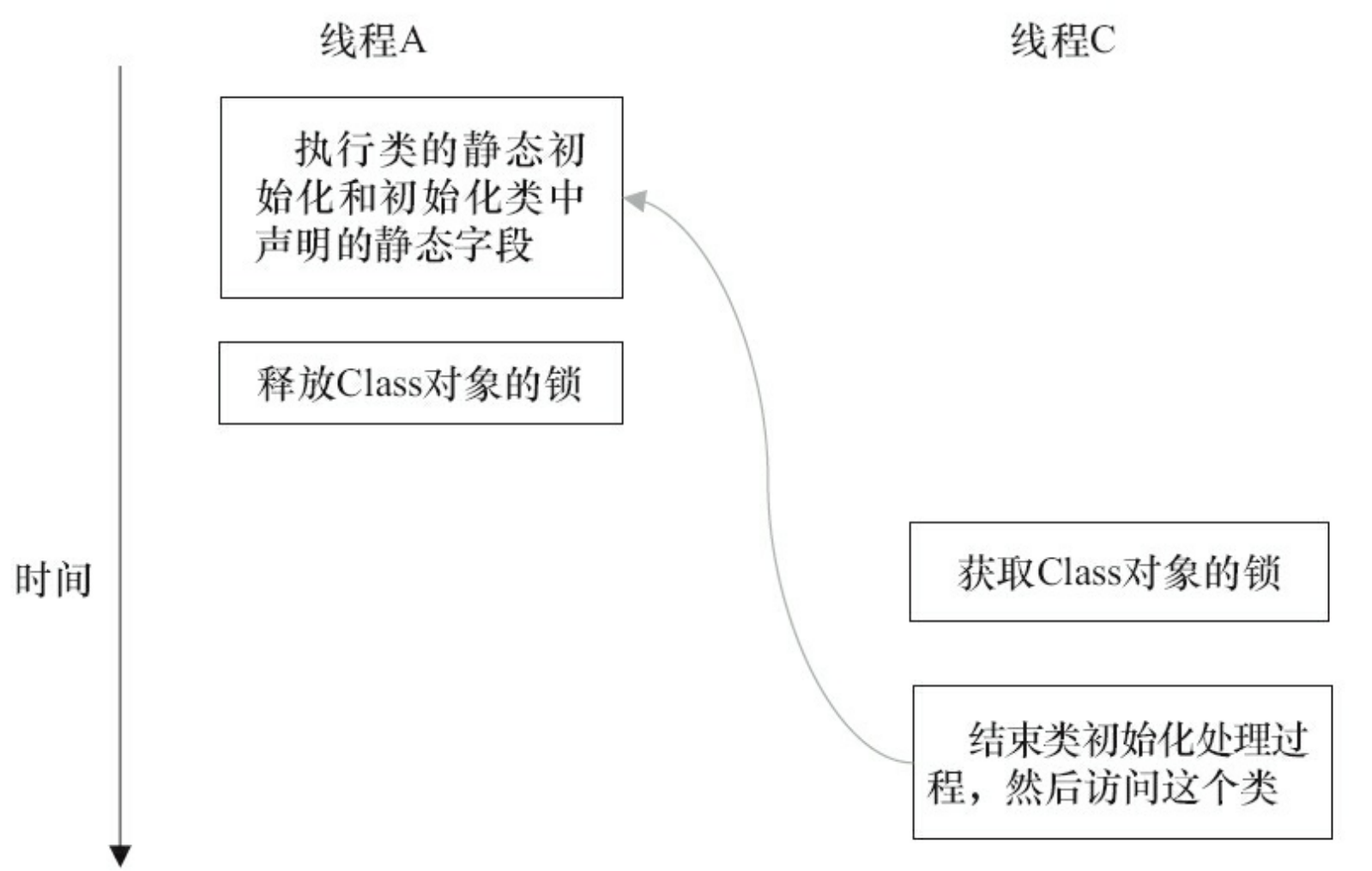


图3-47 多线程执行时序图

通过对比基于volatile的双重检查锁定的方案和基于类初始化的方案，我们会发现基于类初始化的方案的实现代码更简洁。但基于volatile的双重检查锁定的方案有一个额外的优势：除了可以对静态字段实现延迟初始化外，还可以对实例字段实现延迟初始化。

字段延迟初始化降低了初始化类或创建实例的开销，但增加了访问被延迟初始化的字段的开销。在大多数时候，正常的初始化要优于延迟初始化。如果确实需要对实例字段使用线程安全的延迟初始化，请使用上面介绍的基于volatile的延迟初始化的方案；如果确实需要对静态字段使用线程安全的延迟初始化，请使用上面介绍的基于类初始化的方案。

3.9 Java内存模型综述

前面对Java内存模型的基础知识和内存模型的具体实现进行了说明。下面对Java内存模型的相关知识做一个总结。

3.9.1 处理器的内存模型

顺序一致性内存模型是一个理论参考模型，JMM和处理器内存模型在设计时通常会以顺序一致性内存模型为参照。在设计时，JMM和处理器内存模型会对顺序一致性模型做一些放松，因为如果完全按照顺序一致性模型来实现处理器和JMM，那么很多的处理器和编译器优化都要被禁止，这对执行性能将会有很大的影响。

根据对不同类型的读/写操作组合的执行顺序的放松，可以把常见处理器的内存模型划分为如下几种类型。

- 放松程序中写-读操作的顺序，由此产生了Total Store Ordering内存模型(简称为TSO)。
- 在上面的基础上，继续放松程序中写-写操作的顺序，由此产生了Partial Store Order内存模型(简称为PSO)。
- 在前面两条的基础上，继续放松程序中读-写和读-读操作的顺序，由此产生了Relaxed Memory Order内存模型(简称为RMO)和PowerPC内存模型。

注意，这里处理器对读/写操作的放松，是以两个操作之间不存在数据依赖性为前提的(因为处理器要遵守as-if-serial语义，处理器不会对存在数据依赖性的两个内存操作做重排序)。

表3-12展示了常见处理器内存模型的细节特征如下。

表3-12 处理器内存模型的特征表

内存模型名称	对应的处理器	Store-Load重排序	Store-Store重排序	Load-Load 和 Load-Store重排序	可以更早读取到其他处理器的写	可以更早读取到当前处理器的写
TSO	sparc-TSO X64	Y				Y
PSO	sparc-PSO	Y	Y			Y
RMO	ia64	Y	Y	Y		Y
PowerPC	PowerPC	Y	Y	Y	Y	Y

从表3-12中可以看到, 所有处理器内存模型都允许写-读重排序, 原因在第1章已经说明过: 它们都使用了写缓存区。写缓存区可能导致写-读操作重排序。同时, 我们可以看到这些处理器内存模型都允许更早读到当前处理器的写, 原因同样是因为写缓存区。由于写缓存区仅对当前处理器可见, 这个特性导致当前处理器可以比其他处理器先看到临时保存在自己写缓存区中的写。

表3-12中的各种处理器内存模型, 从上到下, 模型由强变弱。越是追求性能的处理器, 内存模型设计得会越弱。因为这些处理器希望内存模型对它们的束缚越少越好, 这样它们就可以做尽可能多的优化来提高性能。

由于常见的处理器内存模型比JMM要弱, Java编译器在生成字节码时, 会在执行指令序列的适当位置插入内存屏障来限制处理器的重排序。同时, 由于各种处理器内存模型的强弱不同, 为了在不同的处理器平台向程序员展示一个一致的内存模型, JMM在不同的处理器中需要插入的内存屏障的数量和种类也不相同。图3-48展示了JMM在不同处理器内存模型中需要插入的内存屏障的示意图。

JMM屏蔽了不同处理器内存模型的差异, 它在不同的处理器平台之上为Java程序员呈现了一个一致的内存模型。

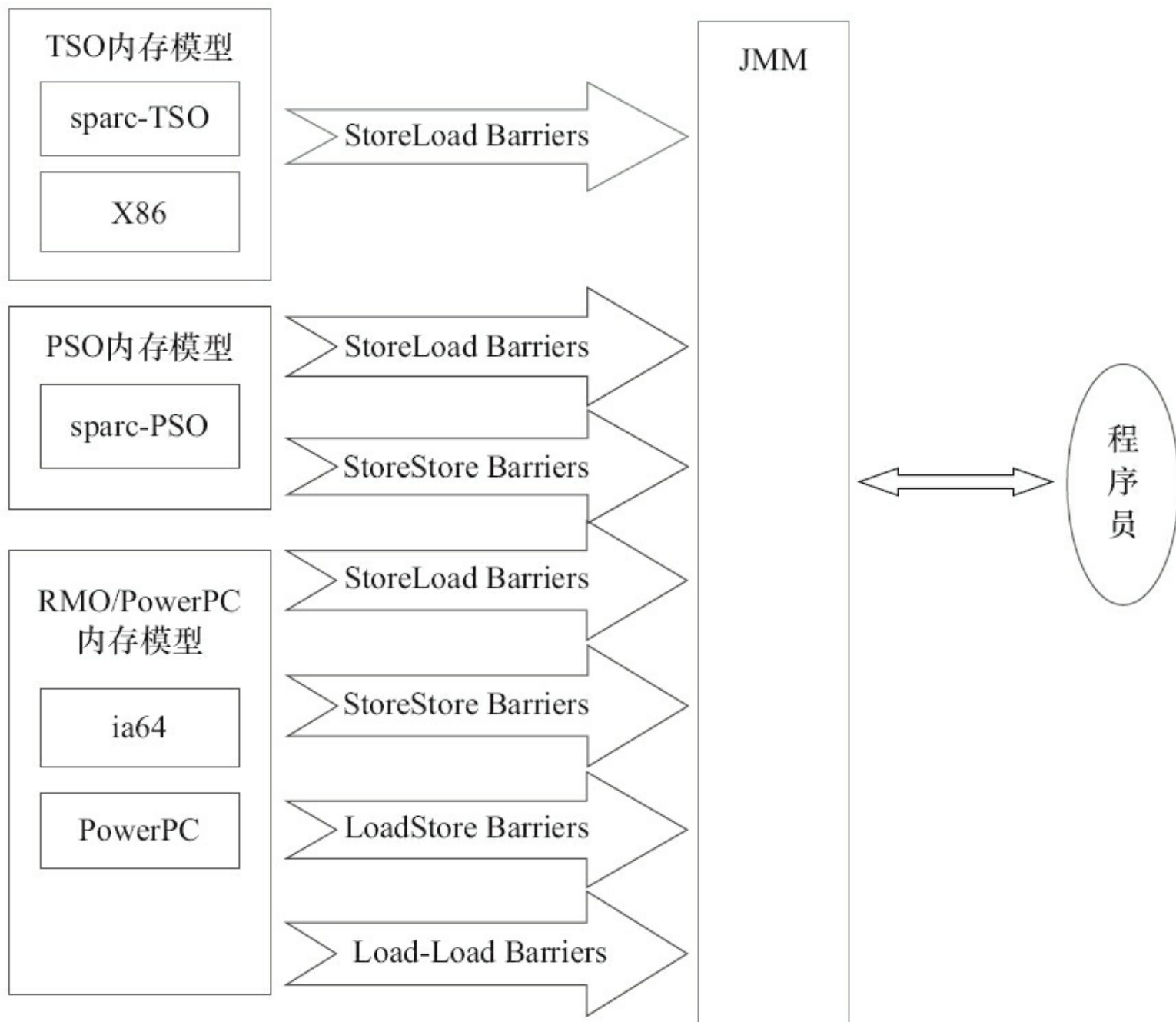


图3-48 JMM插入内存屏障的示意图

3.9.2 各种内存模型之间的关系

JMM是一个语言级的内存模型，处理器内存模型是硬件级的内存模型，顺序一致性内存模型是一个理论参考模型。下面是语言内存模型、处理器内存模型和顺序一致性内存模型的强弱对比示意图，如图3-49所示。

从图中可以看出：常见的4种处理器内存模型比常用的3中语言内存模型要弱，处理器内存模型和语言内存模型都比顺序一致性内存模型要弱。同处理器内存模型一样，越是追求执行性能的语言，内存模型设计得会越弱。

3.9.3 JMM的内存可见性保证

按程序类型, Java程序的内存可见性保证可以分为下列3类。

- 单线程程序。单线程程序不会出现内存可见性问题。编译器、runtime和处理器会共同确保单线程程序的执行结果与该程序在顺序一致性模型中的执行结果相同。
- 正确同步的多线程程序。正确同步的多线程程序的执行将具有顺序一致性(程序的执行结果与该程序在顺序一致性内存模型中的执行结果相同)。这是JMM关注的重点, JMM通过限制编译器和处理器的重排序来为程序员提供内存可见性保证。
- 未同步/未正确同步的多线程程序。JMM为它们提供了最小安全性保障:线程执行时读取到的值, 要么是之前某个线程写入的值, 要么是默认值(0、null、false)。

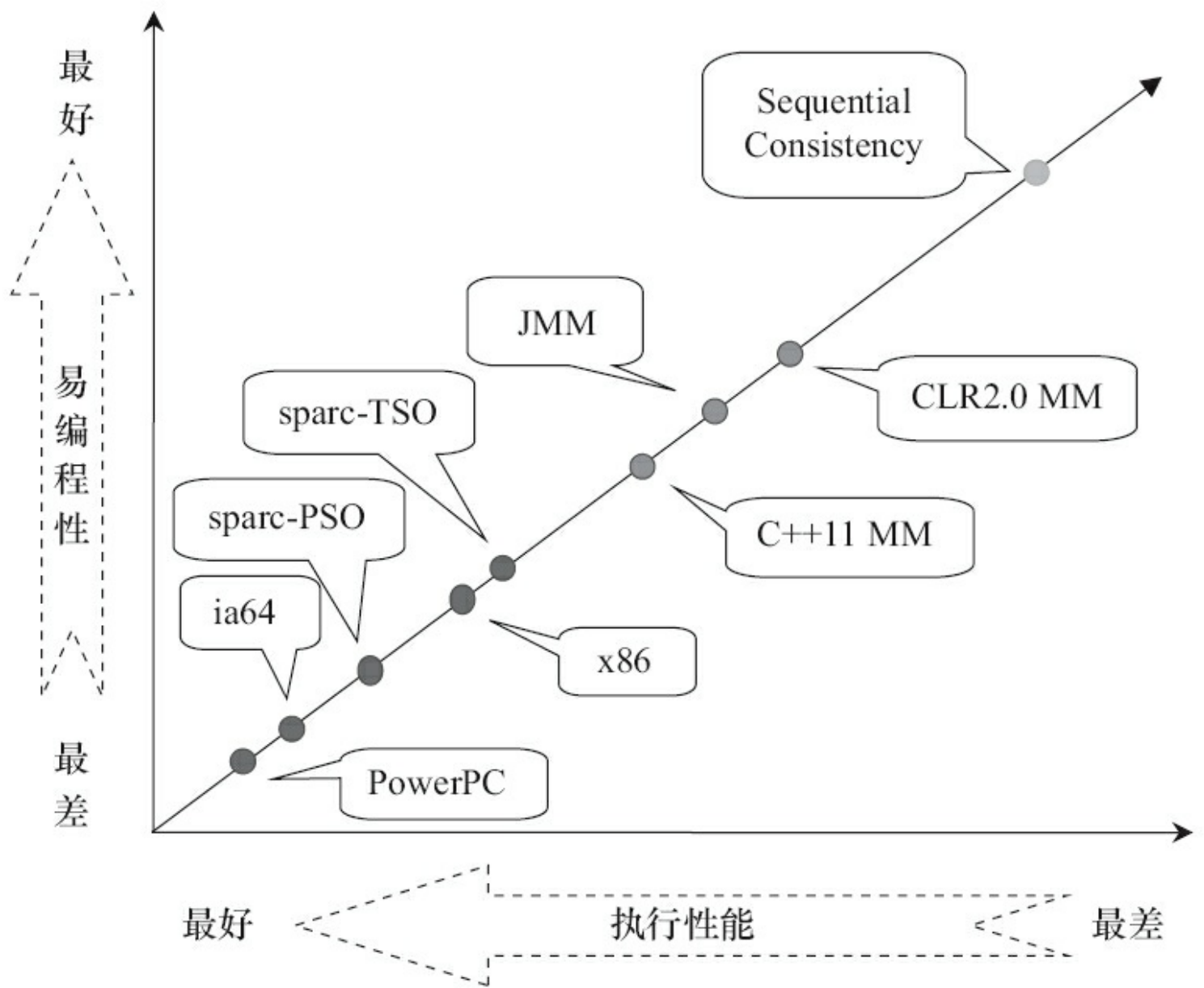


图3-49 各种CPU内存模型的强弱对比示意图

注意，最小安全性保障与64位数据的非原子性写并不矛盾。它们是两个不同的概念，它们“发生”的时间点也不同。最小安全性保证对象默认初始化之后（设置成员域为0、null或false），才会被任意线程使用。最小安全性“发生”在对象被任意线程使用之前。64位数据的非原子性写“发生”在对象被多个线程使用的过程中（写共享变量）。当发生问题时（处理器B看到仅仅被处理器A“写了一半”的无效值），这里虽然处理器B读取到一个被写了一半的无效值，但这个值仍然是处理器A写入的，只不过是处理器A还没有写完而已。最小安全性保证线程读取到的值，要么是之前某个线程写入的值，要么是默认值（0、null、false）。但最小安全性并不保证线程读取到的值，一定是某个线程写完后的值。最小安全性保证线程读取到的值不会无中生

有的冒出来,但并不保证线程读取到的值一定是正确的。

图3-50展示了这3类程序在JMM中与在顺序一致性内存模型中的执行结果的异同。

只要多线程程序是正确同步的, JMM保证该程序在任意的处理器平台上的执行结果, 与该程序在顺序一致性内存模型中的执行结果一致。

3.9.4 JSR-133对旧内存模型的修补

JSR-133对JDK 5之前的旧内存模型的修补主要有两个。

- 增强volatile的内存语义。旧内存模型允许volatile变量与普通变量重排序。JSR-133严格限制volatile变量与普通变量的重排序，使volatile的写-读和锁的释放-获取具有相同的内存语义。

- 增强final的内存语义。在旧内存模型中，多次读取同一个final变量的值可能会不相同。为此，JSR-133为final增加了两个重排序规则。在保证final引用不会从构造函数内逸出的情况下，final具有了初始化安全性。

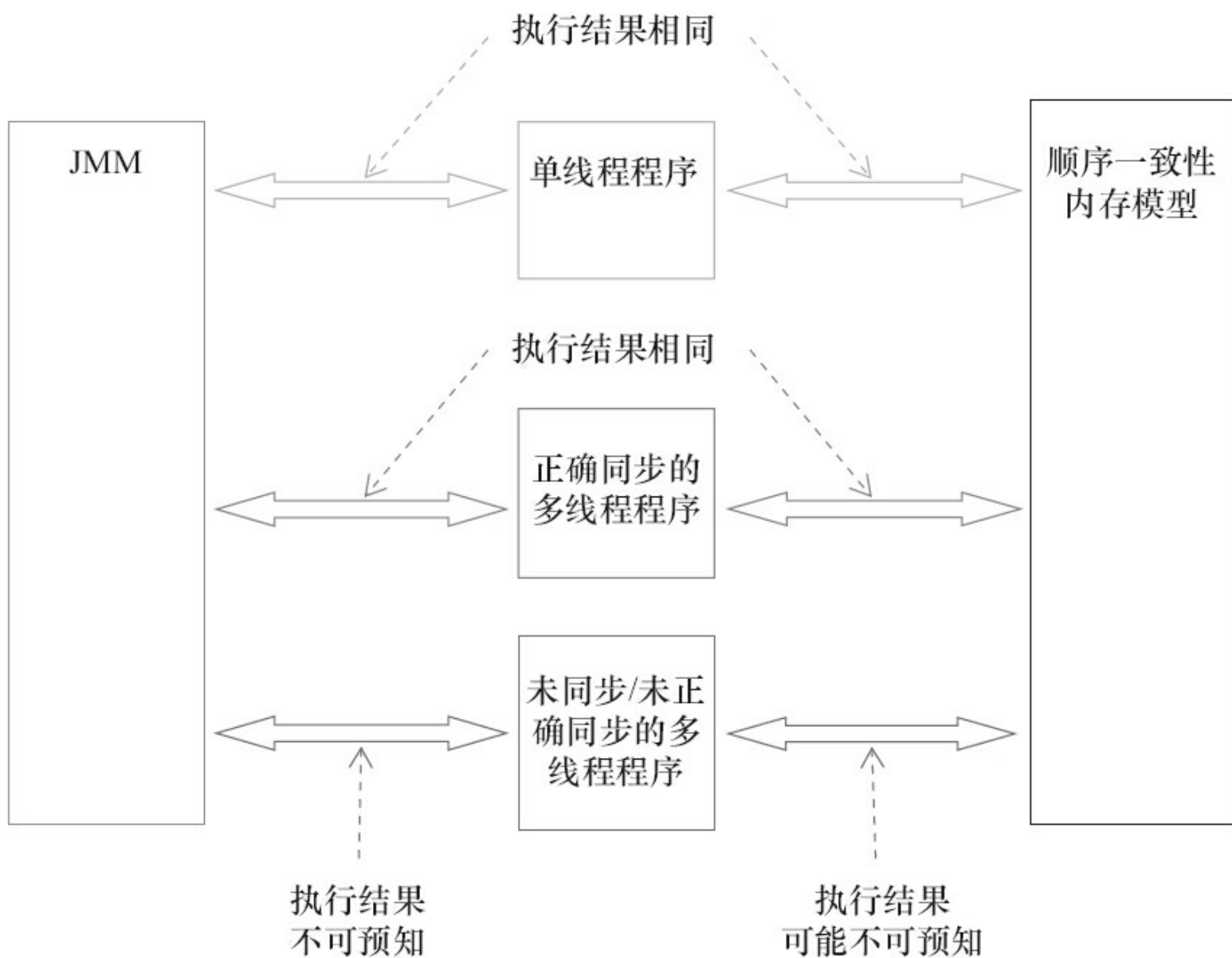


图3-50 3类程序的执行结果的对比图

3.10 本章小结

本章对Java内存模型做了比较全面的解读。希望读者阅读本章之后,对Java内存模型能够有一个比较深入的了解;同时,也希望本章可帮助读者解决在Java并发编程中经常遇到的各种内存可见性问题。

第4章 Java并发编程基础

Java从诞生开始就明智地选择了内置对多线程的支持,这使得Java语言相比同一时期的其他语言具有明显的优势。线程作为操作系统调度的最小单元,多个线程能够同时执行,这将显著提升程序性能,在多核环境中表现得更加明显。但是,过多地创建线程和对线程的不当管理也容易造成问题。本章将着重介绍Java并发编程的基础知识,从启动一个线程到线程间不同的通信方式,最后通过简单的线程池示例以及应用(简单的Web服务器)来串联本章所介绍的内容。

4.1 线程简介

4.1.1 什么是线程

现代操作系统在运行一个程序时，会为其创建一个进程。例如，启动一个Java程序，操作系统就会创建一个Java进程。现代操作系统调度的最小单元是线程，也叫轻量级进程(Light Weight Process)，在一个进程里可以创建多个线程，这些线程都拥有各自的计数器、堆栈和局部变量等属性，并且能够访问共享的内存变量。处理器在这些线程上高速切换，让使用者感觉到这些线程在同时执行。

一个Java程序从main()方法开始执行，然后按照既定的代码逻辑执行，看似没有其他线程参与，但实际上Java程序天生就是多线程程序，因为执行main()方法的是一个名称为main的线程。下面使用JMX来查看一个普通的Java程序包含哪些线程，如代码清单4-1所示。

代码清单4-1 MultiThread.java

```
public class MultiThread{
    public static void main(String[] args) {
        // 获取Java线程管理MXBean
        ThreadMXBean threadMXBean = ManagementFactory.getThreadMXBean();
        // 不需要获取同步的monitor和synchronizer信息, 仅获取线程和线程堆栈信息
        ThreadInfo[] threadInfos = threadMXBean.dumpAllThreads(false, false);
        // 遍历线程信息, 仅打印线程ID和线程名称信息
        for (ThreadInfo threadInfo : threadInfos) {
            System.out.println "[" + threadInfo.getThreadId() + " ] " + threadInfo.
                getThreadName();
        }
    }
}
```

输出如下所示(输出内容可能不同)。

```
[4] Signal Dispatcher // 分发处理发送给JVM信号的线程
[3] Finalizer          // 调用对象finalize方法的线程
[2] Reference Handler  // 清除Reference的线程
[1] main               // main线程, 用户程序入口
```

可以看到, 一个Java程序的运行不仅仅是main()方法的运行, 而是main线程和多个其他线程的同时运行。

4.1.2 为什么要使用多线程

执行一个简单的“Hello,World!”, 却启动了那么多的“无关”线程, 是不是把简单的问题复杂化了? 当然不是, 因为正确使用多线程, 总是能够给开发人员带来显著的好处, 而使用多线程的原因主要有以下几点。

(1)更多的处理器核心

随着处理器上的核心数量越来越多, 以及超线程技术的广泛运用, 现在大多数计算机都比以往更加擅长并行计算, 而处理器性能的提升方式, 也从更高的主频向更多的核心发展。如何利用好处理器上的多个核心也成了现在的主要问题。

线程是大多数操作系统调度的基本单元, 一个程序作为一个进程来运行, 程序运行过程中能够创建多个线程, 而一个线程在一个时刻只能运行在一个处理器核心上。试想一下, 一个单线程程序在运行时只能使用一个处理器核心, 那么再多的处理器核心加入也无法显著提升该程序的执行效率。相反, 如果该程序使用多线程技术, 将计算逻辑分配到多个处理器核心上, 就会显著减少程序的处理时间, 并且随着更多处理器核心的加入而变得更有效率。

(2)更快的响应时间

有时我们会编写一些较为复杂的代码(这里的复杂不是说复杂的算法, 而是复杂的业务逻辑), 例如, 一笔订单的创建, 它包括插入订单数据、生成订单快照、发送邮件通知卖家和记录货品销售数量等。用户从单击“订购”按钮开始, 就要等待这些操作全部完成才能看到订购成功的结果。但是这么多业务操作, 如何能够让其更快地完成呢?

在上面的场景中, 可以使用多线程技术, 即将数据一致性不强的操作派发给其他线程处理(也可以使用消息队列), 如生成订单快照、发送邮件等。这样做的好处是响应用户请求的线程能够尽可能快地处理完成, 缩短了响应时间, 提升了用户体验。

(3) 更好的编程模型

Java为多线程编程提供了良好、考究并且一致的编程模型，使开发人员能够更加专注于问题的解决，即为所遇到的问题建立合适的模型，而不是绞尽脑汁地考虑如何将其多线程化。一旦开发人员建立好了模型，稍做修改总是能够方便地映射到Java提供的多线程编程模型上。

4.1.3 线程优先级

现代操作系统基本采用时分的形式调度运行的线程，操作系统会分出一个个时间片，线程会分配到若干时间片，当线程的时间片用完了就会发生线程调度，并等待着下次分配。线程分配到的时间片多少也就决定了线程使用处理器资源的多少，而线程优先级就是决定线程需要多或者少分配一些处理器资源的线程属性。

在Java线程中，通过一个整型成员变量priority来控制优先级，优先级的范围从1~10，在线程构建的时候可以通过setPriority(int)方法来修改优先级，默认优先级是5，优先级高的线程分配时间片的数量要多于优先级低的线程。设置线程优先级时，针对频繁阻塞(休眠或者I/O操作)的线程需要设置较高优先级，而偏重计算(需要较多CPU时间或者偏运算)的线程则设置较低的优先级，确保处理器不会被独占。在不同的JVM以及操作系统上，线程规划会存在差异，有些操作系统甚至会忽略对线程优先级的设定，示例如代码清单4-2所示。

代码清单4-2 Priority.java

```
public class Priority {
    private static volatile boolean notStart = true;
    private static volatile boolean notEnd = true;
    public static void main(String[] args) throws Exception {
        List<Job> jobs = new ArrayList<Job>();
        for (int i = 0; i < 10; i++) {
            int priority = i < 5 ? Thread.MIN_PRIORITY : Thread.MAX_PRIORITY;
            Job job = new Job(priority);
            jobs.add(job);
            Thread thread = new Thread(job, "Thread:" + i);
            thread.setPriority(priority);
            thread.start();
        }
        notStart = false;
        TimeUnit.SECONDS.sleep(10);
        notEnd = false;
        for (Job job : jobs) {
            System.out.println("Job Priority : " + job.priority + ",
                               Count : " + job.jobCount);
        }
    }
    static class Job implements Runnable {
        private int priority;
        private long jobCount;
```

```
        public Job(int priority) {
            this.priority = priority;
        }
        public void run() {
            while (notStart) {
                Thread.yield();
            }
            while (notEnd) {
                Thread.yield();
                jobCount++;
            }
        }
    }
}
```

运行该示例, 在笔者机器上对应的输出如下。

```
Job Priority : 1, Count : 1259592
Job Priority : 1, Count : 1260717
Job Priority : 1, Count : 1264510
Job Priority : 1, Count : 1251897
Job Priority : 1, Count : 1264060
Job Priority : 10, Count : 1256938
Job Priority : 10, Count : 1267663
Job Priority : 10, Count : 1260637
Job Priority : 10, Count : 1261705
Job Priority : 10, Count : 1259967
```

从输出可以看到线程优先级没有生效, 优先级1和优先级10的Job计数的结果非常相近, 没有明显差距。这表示程序正确性不能依赖线程的优先级高低。

 **注意** 线程优先级不能作为程序正确性的依赖, 因为操作系统可以完全不用理会Java线程对于优先级的设定。笔者的环境为: Mac OS X 10.10, Java版本为1.7.0_71, 经过笔者验证该环境下所有Java线程优先级均为5(通过jstack查看), 对线程优先级的设置会被忽略。另外, 尝试在Ubuntu 14.04环境下运行该示例, 输出结果也表示该环境忽略了线程优先级的设置。

4.1.4 线程的状态

Java线程在运行的生命周期中可能处于表4-1所示的6种不同的状态，在给定的一个时刻，线程只能处于其中的一个状态。

表4-1 Java线程的状态

状态名称	说 明
NEW	初始状态，线程被构建，但是还没有调用 start() 方法
RUNNABLE	运行状态，Java 线程将操作系统中的就绪和运行两种状态笼统地称作“运行中”
BLOCKED	阻塞状态，表示线程阻塞于锁
WAITING	等待状态，表示线程进入等待状态，进入该状态表示当前线程需要等待其他线程做出一些特定动作（通知或中断）
TIME_WAITING	超时等待状态，该状态不同于 WAITING，它是可以在指定的时间自行返回的
TERMINATED	终止状态，表示当前线程已经执行完毕

下面我们使用jstack工具(可以选择打开终端，键入jstack或者到JDK安装目录的bin目录下执行命令)，尝试查看示例代码运行时的线程信息，更加深入地理解线程状态，示例如代码清单4-3所示。

代码清单4-3 ThreadState.java

```
public class ThreadState {
    public static void main(String[] args) {
        new Thread(new TimeWaiting (), "TimeWaitingThread").start();
        new Thread(new Waiting(), "WaitingThread").start();
        // 使用两个Blocked线程，一个获取锁成功，另一个被阻塞
        new Thread(new Blocked(), "BlockedThread-1").start();
        new Thread(new Blocked(), "BlockedThread-2").start();
    }
    // 该线程不断地进行睡眠
    static class TimeWaiting implements Runnable {
        @Override
        public void run() {
            while (true) {
                SleepUtils.second(100);
            }
        }
    }
    // 该线程在Waiting.class实例上等待
    static class Waiting implements Runnable {
```

```

@Override
public void run() {
    while (true) {
        synchronized (Waiting.class) {
            try {
                Waiting.class.wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

// 该线程在Blocked.class实例上加锁后, 不会释放该锁
static class Blocked implements Runnable {
    public void run() {
        synchronized (Blocked.class) {
            while (true) {
                SleepUtils.second(100);
            }
        }
    }
}
}

```

上述示例中使用的SleepUtils如代码清单4-4所示。

代码清单4-4 SleepUtils.java

```

public class SleepUtils {
    public static final void second(long seconds) {
        try {
            TimeUnit.SECONDS.sleep(seconds);
        } catch (InterruptedException e) {
        }
    }
}

```

运行该示例, 打开终端或者命令提示符, 键入“jps”, 输出如下。

```

611
935 Jps
929 ThreadState
270

```

可以看到运行示例对应的进程ID是929, 接着再键入“jstack 929”(这里的进程ID需要和读

者自己键入jps得出的ID一致), 部分输出如下所示。

```
// BlockedThread-2线程阻塞在获取Blocked.class示例的锁上
"BlockedThread-2" prio=5 tid=0x00007feacb05d000 nid=0x5d03 waiting for monitor
entry [0x0000000010fd58000]
    java.lang.Thread.State: BLOCKED (on object monitor)
// BlockedThread-1线程获取到了Blocked.class的锁
"BlockedThread-1" prio=5 tid=0x00007feacb05a000 nid=0x5b03 waiting on condition
[0x0000000010fc55000]
    java.lang.Thread.State: TIMED_WAITING (sleeping)
// WaitingThread线程在Waiting实例上等待
"WaitingThread" prio=5 tid=0x00007feacb059800 nid=0x5903 in Object.wait()
[0x0000000010fb52000]
    java.lang.Thread.State: WAITING (on object monitor)
// TimeWaitingThread线程处于超时等待
"TimeWaitingThread" prio=5 tid=0x00007feacb058800 nid=0x5703 waiting on condition
[0x0000000010fa4f000]
    java.lang.Thread.State: TIMED_WAITING (sleeping)
```

通过示例, 我们了解到Java程序运行中线程状态的具体含义。线程在自身的生命周期中, 并不是固定地处于某个状态, 而是随着代码的执行在不同的状态之间进行切换, Java线程状态变迁如图4-1示。

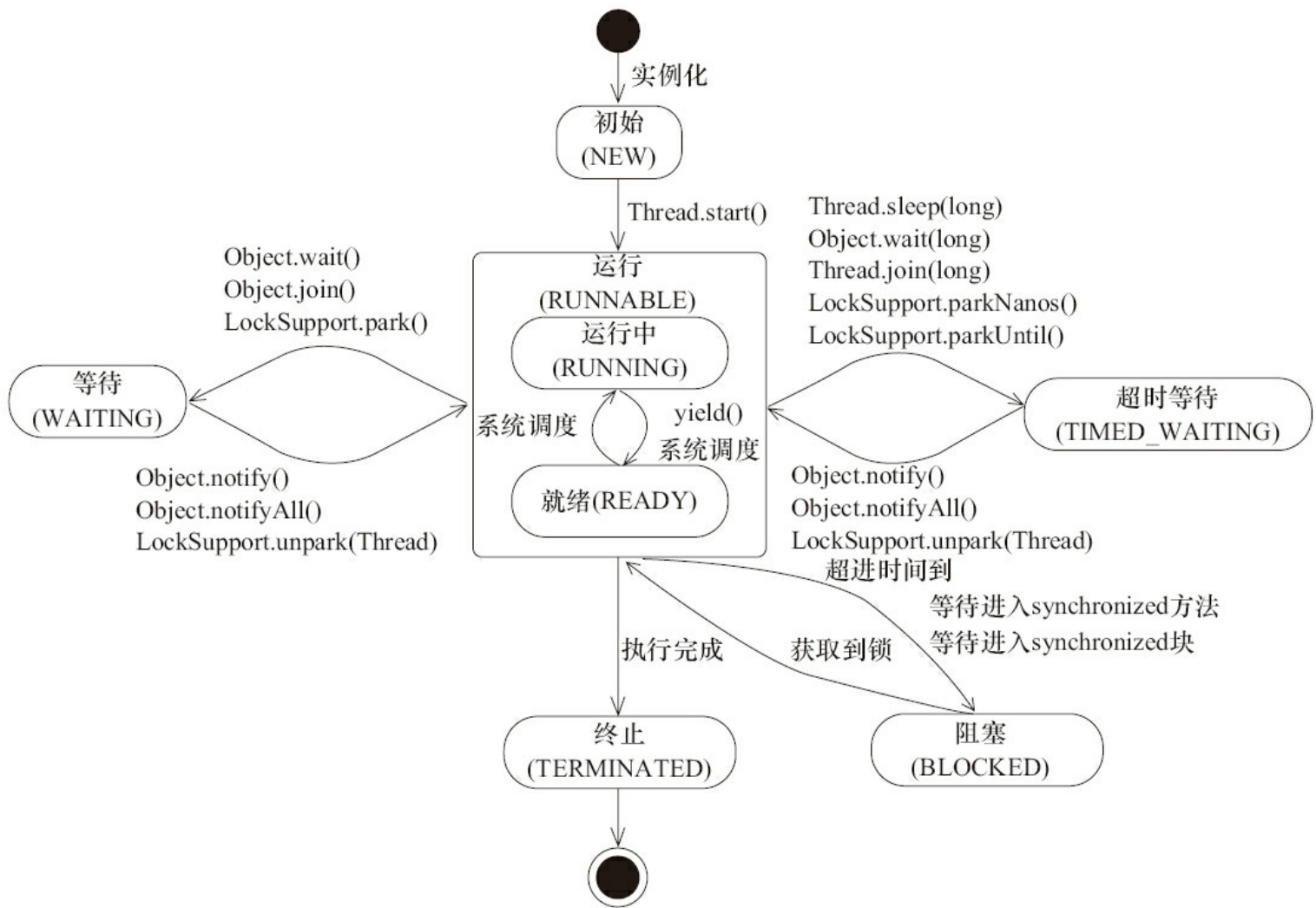


图4-1 Java线程状态变迁

由图4-1中可以看到，线程创建之后，调用start()方法开始运行。当线程执行wait()方法之后，线程进入等待状态。进入等待状态的线程需要依靠其他线程的通知才能够返回到运行状态，而超时等待状态相当于在等待状态的基础上增加了超时限制，也就是超时时间到达时将会返回到运行状态。当线程调用同步方法时，在没有获取到锁的情况下，线程将会进入到阻塞状态。线程在执行Runnable的run()方法之后将会进入到终止状态。



注意 Java将操作系统中的运行和就绪两个状态合并称为运行状态。阻塞状态是线程阻塞在进入synchronized关键字修饰的方法或代码块(获取锁)时的状态，但是阻塞在java.concurrent包中Lock接口的线程状态却是等待状态，因为java.concurrent包中Lock接口对于阻塞的实现均使用了LockSupport类中的相关方法。

4.1.5 Daemon线程

Daemon线程是一种支持型线程，因为它主要被用作程序中后台调度以及支持性工作。这意味着，**当一个Java虚拟机中不存在非Daemon线程的时候，Java虚拟机将会退出。**可以通过调用`Thread.setDaemon(true)`将线程设置为Daemon线程。



注意 Daemon属性需要在启动线程之前设置，不能在启动线程之后设置。

Daemon线程被用作完成支持性工作，但是在**Java虚拟机退出时Daemon线程中的finally块并不一定会执行**，示例如代码清单4-5所示。

代码清单4-5 Daemon.java

```
public class Daemon {
    public static void main(String[] args) {
        Thread thread = new Thread(new DaemonRunner(), "DaemonRunner");
        thread.setDaemon(true);
        thread.start();
    }

    static class DaemonRunner implements Runnable {
        @Override
        public void run() {
            try {
                SleepUtils.second(10);
            } finally {
                System.out.println("DaemonThread finally run.");
            }
        }
    }
}
```

运行Daemon程序，可以看到在终端或者命令提示符上没有任何输出。main线程（非Daemon线程）在启动了线程DaemonRunner之后随着main方法执行完毕而终止，而此时Java虚拟机中已经没有非Daemon线程，虚拟机需要退出。Java虚拟机中的所有Daemon线程都需要立即终止，因此DaemonRunner立即终止，但是DaemonRunner中的**finally块并没有执行。**



注意 在构建Daemon线程时，不能依靠finally块中的内容来确保执行关闭或清理资源

的逻辑。

4.2 启动和终止线程

在前面章节的示例中通过调用线程的start()方法进行启动, 随着run()方法的执行完毕, 线程也随之终止, 大家对此一定不会陌生, 下面将详细介绍线程的启动和终止。

4.2.1 构造线程

在运行线程之前首先要构造一个线程对象, 线程对象在构造的时候需要提供线程所需要的属性, 如线程所属的线程组、线程优先级、是否是Daemon线程等信息。代码清单4-6所示的代码摘自java.lang.Thread中对线程进行初始化的部分。

代码清单4-6 Thread.java

```
private void init(ThreadGroup g, Runnable target, String name, long stackSize,
AccessControlContext acc) {
    if (name == null) {
        throw new NullPointerException("name cannot be null");
    }
    // 当前线程就是该线程的父线程
    Thread parent = currentThread();
    this.group = g;
    // 将daemon、priority属性设置为父线程的对应属性
    this.daemon = parent.isDaemon();
    this.priority = parent.getPriority();
    this.name = name.toCharArray();
    this.target = target;
    setPriority(priority);
    // 将父线程的InheritableThreadLocal复制过来
    if (parent.inheritableThreadLocals != null)
        this.inheritableThreadLocals=ThreadLocal.createInheritedMap(parent.
inheritableThreadLocals);
    // 分配一个线程ID
    tid = nextThreadID();
}
```

在上述过程中, 一个新构造的线程对象是由其parent线程来进行空间分配的, 而child线程继承了parent是否为Daemon、优先级和加载资源的contextClassLoader以及可继承的ThreadLocal, 同时还会分配一个唯一的ID来标识这个child线程。至此, 一个能够运行的线程对象就初始化好了, 在堆内存中等待着运行。

4.2.2 启动线程

线程对象在初始化完成之后，调用start()方法就可以启动这个线程。**线程start()方法的含义**

是：当前线程（即parent线程）同步告知Java虚拟机，只要线程规划器空闲，应立即启动调用

start()方法的线程。



注意 启动一个线程前，最好为这个线程设置线程名称，因为这样在使用jstack分析程序或者进行问题排查时，就会给开发人员提供一些提示，自定义的线程最好能够起个名字。

4.2.3 理解中断

中断可以理解为线程的一个标识位属性，它表示一个运行中的线程是否被其他线程进行了中断操作。中断好比其他线程对该线程打了个招呼，其他线程通过调用该线程的interrupt()方法对其进行中断操作。

线程通过检查自身是否被中断来进行响应，线程通过方法isInterrupted()来进行判断是否被中断，也可以调用静态方法Thread.interrupted()对当前线程的中断标识位进行复位。如果该线程已经处于终结状态，即使该线程被中断过，在调用该线程对象的isInterrupted()时依旧会返回false。

- (1) interrupt()方法对其进行中断操作。
- (2) 静态方法Thread.interrupted()既得到中断值，又清除中断状态。
- (3) isInterrupted()获取中断状态，不会清除中断状态。
- (4) Thread.sleep()会清除中断状态。

从Java的API中可以看到，许多声明抛出InterruptedException的方法（例如Thread.sleep(long millis)方法）这些方法在抛出InterruptedException之前，Java虚拟机会先将该线程的中断标识位清除，然后抛出InterruptedException，此时调用isInterrupted()方法将会返回false。

在代码清单4-7所示的例子中，首先创建了两个线程，SleepThread和BusyThread，前者不停地睡眠，后者一直运行，然后对这两个线程分别进行中断操作，观察二者的中断标识位。

代码清单4-7 Interrupted.java

```
public class Interrupted {
    public static void main(String[] args) throws Exception {
        // sleepThread不停的尝试睡眠
        Thread sleepThread = new Thread(new SleepRunner(), "SleepThread");
        sleepThread.setDaemon(true);
        // busyThread不停的运行
        Thread busyThread = new Thread(new BusyRunner(), "BusyThread");
        busyThread.setDaemon(true);
        sleepThread.start();
        busyThread.start();
        // 休眠5秒，让sleepThread和busyThread充分运行
        TimeUnit.SECONDS.sleep(5);
        sleepThread.interrupt();
        busyThread.interrupt();
        System.out.println("SleepThread interrupted is " + sleepThread.isInterrupted());
        System.out.println("BusyThread interrupted is " + busyThread.isInterrupted());
        // 防止sleepThread和busyThread立刻退出
    }
}
```

```
        SleepUtils.second(2);
    }
    static class SleepRunner implements Runnable {
        @Override
        public void run() {
            while (true) {
                SleepUtils.second(10);
            }
        }
    }
    static class BusyRunner implements Runnable {
        @Override
        public void run() {
            while (true) {
            }
        }
    }
}
```

输出如下。

```
SleepThread interrupted is false
BusyThread interrupted is true
```

从结果可以看出，抛出InterruptedException的线程SleepThread，其中断标识位被清除了，而一直忙碌运作的线程BusyThread，中断标识位没有被清除。

4.2.4 过期的suspend()、resume()和stop()

大家对于CD机肯定不会陌生，如果把它播放音乐比作一个线程的运作，那么对音乐播放做出的暂停、恢复和停止操作对应在线程Thread的API就是suspend()、resume()和stop()。

在代码清单4-8所示的例子中，创建了一个线程PrintThread，它以1秒的频率进行打印，而主线程对其进行暂停、恢复和停止操作。

代码清单4-8 Deprecated.java

```
public class Deprecated {
    public static void main(String[] args) throws Exception {
        DateFormat format = new SimpleDateFormat("HH:mm:ss");
        Thread printThread = new Thread(new Runner(), "PrintThread");
        printThread.setDaemon(true);
        printThread.start();
        TimeUnit.SECONDS.sleep(3);
        // 将PrintThread进行暂停, 输出内容工作停止
        printThread.suspend();
        System.out.println("main suspend PrintThread at " + format.format(new Date()));
        TimeUnit.SECONDS.sleep(3);
        // 将PrintThread进行恢复, 输出内容继续
        printThread.resume();
        System.out.println("main resume PrintThread at " + format.format(new Date()));
        TimeUnit.SECONDS.sleep(3);
        // 将PrintThread进行终止, 输出内容停止
        printThread.stop();
        System.out.println("main stop PrintThread at " + format.format(new Date()));
        TimeUnit.SECONDS.sleep(3);
    }
    static class Runner implements Runnable {
        @Override
        public void run() {
            DateFormat format = new SimpleDateFormat("HH:mm:ss");
            while (true) {
                System.out.println(Thread.currentThread().getName() + " Run at " +
                    format.format(new Date()));
                SleepUtils.second(1);
            }
        }
    }
}
```

输出如下(输出内容中的时间与示例执行的具体时间相关)。

```
PrintThread Run at 17:34:36
PrintThread Run at 17:34:37
PrintThread Run at 17:34:38
main suspend PrintThread at 17:34:39
main resume PrintThread at 17:34:42
PrintThread Run at 17:34:42
PrintThread Run at 17:34:43
PrintThread Run at 17:34:44
main stop PrintThread at 17:34:45
```

在执行过程中，PrintThread运行了3秒，随后被暂停，3秒后恢复，最后经过3秒被终止。

通过示例的输出可以看到，suspend()、resume()和stop()方法完成了线程的暂停、恢复和终止工作，而且非常“人性化”。但是这些API是过期的，也就是不建议使用的。

不建议使用的原因主要有：以suspend()方法为例，在调用后，线程不会释放已经占有的资源（比如锁），而是占有着资源进入睡眠状态，这样容易引发死锁问题。同样，stop()方法在终结一个线程时不会保证线程的资源正常释放，通常是没有给予线程完成资源释放工作的机会，因此会导致程序可能工作在不确定状态下。

 **注意** 正因为suspend()、resume()和stop()方法带来的副作用，这些方法才被标注为不建议使用的过期方法，而暂停和恢复操作可以用后面提到的等待/通知机制来替代。

4.2.5 安全地终止线程

在4.2.3节中提到的中断状态是线程的一个标识位，而中断操作是一种简便的线程间交互方式，而这种交互方式最适合用来取消或停止任务。除了中断以外，还可以利用一个boolean变量来控制是否需要停止任务并终止该线程。

在代码清单4-9所示的例子中，创建了一个线程CountThread，它不断地进行变量累加，而主线程尝试对其进行中断操作和停止操作。

代码清单4-9 Shutdown.java

```
public class Shutdown {
    public static void main(String[] args) throws Exception {
        Runner one = new Runner();
        Thread countThread = new Thread(one, "CountThread");
        countThread.start();
        // 睡眠1秒, main线程对CountThread进行中断, 使CountThread能够感知中断而结束
        TimeUnit.SECONDS.sleep(1);
        countThread.interrupt();
        Runner two = new Runner();
        countThread = new Thread(two, "CountThread");
        countThread.start();
        // 睡眠1秒, main线程对Runner two进行取消, 使CountThread能够感知on为false而结束
        TimeUnit.SECONDS.sleep(1);
        two.cancel();
    }
    private static class Runner implements Runnable {
        private long i;
        private volatile boolean on = true;
        @Override
        public void run() {
            while (on && !Thread.currentThread().isInterrupted()) {
                i++;
            }
            System.out.println("Count i = " + i);
        }
        public void cancel() {
            on = false;
        }
    }
}
```

输出结果如下所示(输出内容可能不同)。


```
Count i = 543487324
Count i = 540898082
```

示例在执行过程中，main线程通过中断操作和cancel()方法均可使CountThread得以终止。这种通过标识位或者中断操作的方式能够使线程在终止时有机会去清理资源，而不是武断地将线程停止，因此这种终止线程的做法显得更加安全和优雅。

4.3 线程间通信

线程开始运行，拥有自己的栈空间，就如同一个脚本一样，按照既定的代码一步一步地执行，直到终止。但是，每个运行中的线程，如果仅仅是孤立地运行，那么没有一点儿价值，或者说价值很少，如果多个线程能够相互配合完成工作，这将会带来巨大的价值。

4.3.1 volatile和synchronized关键字

Java支持多个线程同时访问一个对象或者对象的成员变量, 由于每个线程可以拥有这个变量的拷贝(虽然对象以及成员变量分配的内存是在共享内存中的, 但是每个执行的线程还是可以拥有一份拷贝, 这样做的目的是加速程序的执行, 这是现代多核处理器的一个显著特性), 所以程序在执行过程中, 一个线程看到的变量并不一定是最新的。

关键字volatile可以用来修饰字段(成员变量), 就是告知程序任何对该变量的访问均需要从共享内存中获取, 而对它的改变必须同步刷新回共享内存, 它能保证所有线程对变量访问的可见性。

举个例子, 定义一个表示程序是否运行的成员变量boolean on=true, 那么另一个线程可能对它执行关闭动作(on=false), 这里涉及多个线程对变量的访问, 因此需要将其定义成为volatile boolean on=true, 这样其他线程对它进行改变时, 可以让所有线程感知到变化, 因为所有对on变量的访问和修改都需要以共享内存为准。但是, 过多地使用volatile是不必要的, 因为它会降低程序执行的效率。

关键字synchronized可以修饰方法或者以同步块的形式来进行使用, 它主要确保多个线程在同一个时刻, 只能有一个线程处于方法或者同步块中, 它保证了线程对变量访问的可见性和排他性。

在代码清单4-10所示的例子中, 使用了同步块和同步方法, 通过使用javap工具查看生成的class文件信息来分析synchronized关键字的实现细节, 示例如下。

代码清单4-10 Synchronized.java

```
public class Synchronized {  
    public static void main(String[] args) {  
        // 对Synchronized Class对象进行加锁  
        synchronized (Synchronized.class) {  
            }  
        }  
    }  
}
```

```
        // 静态同步方法, 对Synchronized Class对象进行加锁
        m();
    }
    public static synchronized void m() {
    }
}
```

在Synchronized.class同级目录执行javap-v Synchronized.class, 部分相关输出如下所示:

```
public static void main(java.lang.String[]);
  // 方法修饰符, 表示: public static flags: ACC_PUBLIC, ACC_STATIC
  Code:
    stack=2, locals=1, args_size=1
    0: ldc      #1    // class com/murdock/books/multithread/book/Synchronized
    2: dup
    3: monitorenter  // monitorenter:监视器进入, 获取锁
    4: monitorexit  // monitorexit:监视器退出, 释放锁
    5: invokestatic #16 // Method m:()V
    8: return
public static synchronized void m();
  // 方法修饰符, 表示: public static synchronized
  flags: ACC_PUBLIC, ACC_STATIC, ACC_SYNCHRONIZED
  Code:
    stack=0, locals=0, args_size=0
    0: return
```

上面class信息中, 对于同步块的实现使用了monitorenter和monitorexit指令, 而同步方法则

是依靠方法修饰符上的ACC_SYNCHRONIZED来完成的。无论采用哪种方式, 其本质是对一个对象的监视器(monitor)进行获取, 而这个获取过程是排他的, 也就是同一时刻只能有一个线程获取到由synchronized所保护对象的监视器。

任意一个对象都拥有自己的监视器, 当这个对象由同步块或者这个对象的同步方法调用时, 执行方法的线程必须先获取到该对象的监视器才能进入同步块或者同步方法, 而没有获取到监视器(执行该方法)的线程将会被阻塞在同步块和同步方法的入口处, 进入BLOCKED状态。

图4-2描述了对象、对象的监视器、同步队列和执行线程之间的关系。

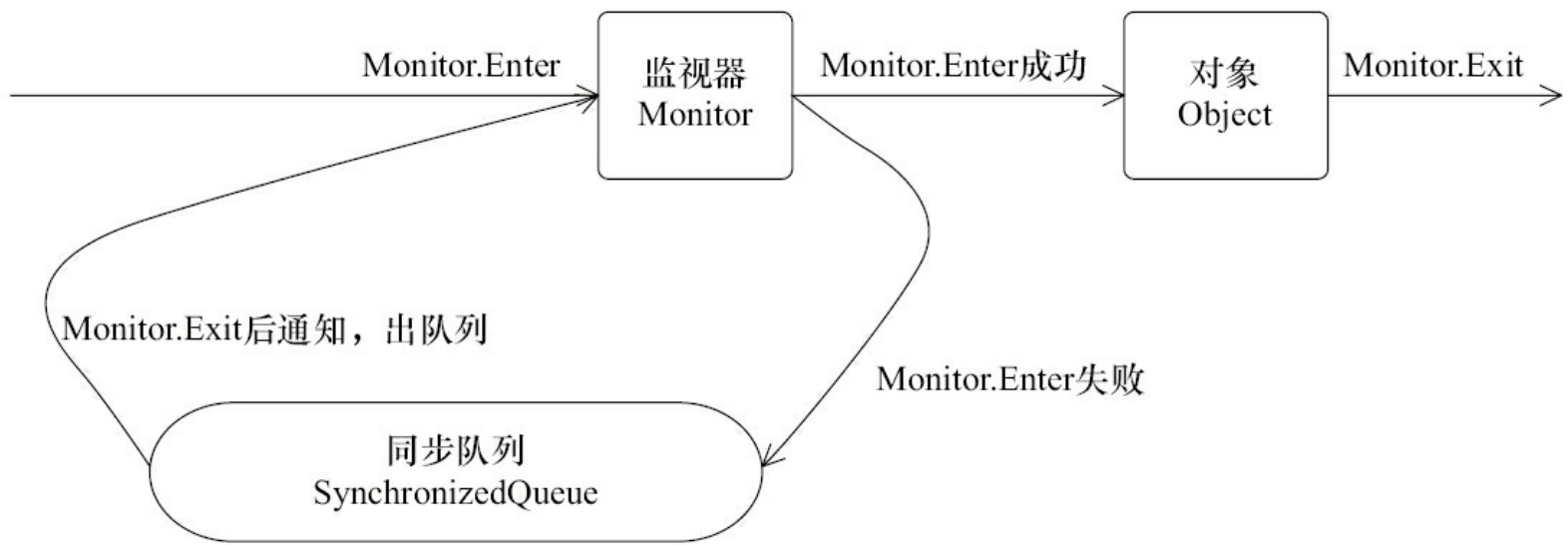


图4-2 对象、监视器、同步队列和执行线程之间的关系

从图4-2中可以看到, 任意线程对Object(Object由synchronized保护)的访问, 首先要获得Object的监视器。如果获取失败, 线程进入同步队列, 线程状态变为BLOCKED。当访问Object的前驱(获得了锁的线程)释放了锁, 则该释放操作唤醒阻塞在同步队列中的线程, 使其重新尝试对监视器的获取。

4.3.2 等待/通知机制

一个线程修改了一个对象的值，而另一个线程感知到了变化，然后进行相应的操作，整个过程开始于一个线程，而最终执行又是另一个线程。前者是生产者，后者就是消费者，这种模式隔离了“做什么”(what)和“怎么做”(How)，在功能层面上实现了解耦，体系结构上具备了良好的伸缩性，但是在Java语言中如何实现类似的功能呢？

简单的办法是让消费者线程不断地循环检查变量是否符合预期，如下面代码所示，在while循环中设置不满足的条件，如果条件满足则退出while循环，从而完成消费者的工作。

```
while (value != desire) {
    Thread.sleep(1000);
}
doSomething();
```

上面这段伪代码在条件不满足时就睡眠一段时间，这样做的目的是防止过快的“无效”尝试，这种方式看似能够解实现所需的功能，但是却存在如下问题。

- 1)难以确保及时性。在睡眠时，基本不消耗处理器资源，但是如果睡得过久，就不能及时发现条件已经变化，也就是及时性难以保证。
- 2)难以降低开销。如果降低睡眠的时间，比如休眠1毫秒，这样消费者能更加迅速地发现条件变化，但是却可能消耗更多的处理器资源，造成了无端的浪费。

以上两个问题，看似矛盾难以调和，但是Java通过内置的等待/通知机制能够很好地解决这个矛盾并实现所需的功能。

等待/通知的相关方法是任意Java对象都具备的，因为这些方法被定义在所有对象的超类java.lang.Object上，方法和描述如表4-2所示。

表4-2 等待/通知的相关方法

方法名称	描 述
notify()	通知一个在对象上等待的线程，使其从 wait() 方法返回，而返回的前提是该线程获取到了对象的锁
notifyAll()	通知所有等待在该对象上的线程
wait()	调用该方法的线程进入 WAITING 状态，只有等待另外线程的通知或被中断才会返回，需要注意，调用 wait() 方法后，会释放对象的锁
wait(long)	超时等待一段时间，这里的参数时间是毫秒，也就是等待长达 n 毫秒，如果没有通知就超时返回
wait(long, int)	对于超时时间更细粒度的控制，可以达到纳秒

等待/通知机制，是指一个线程A调用了对象O的wait()方法进入等待状态，而另一个线程B调用了对象O的notify()或者notifyAll()方法，线程A收到通知后从对象O的wait()方法返回，进而执行后续操作。上述两个线程通过对象O来完成交互，而对象上的wait()和notify/notifyAll()的关系就如同开关信号一样，用来完成等待方和通知方之间的交互工作。

在代码清单4-11所示的例子中，创建了两个线程——WaitThread和NotifyThread，前者检查flag值是否为false，如果符合要求，进行后续操作，否则在lock上等待，后者在睡眠了一段时间后对lock进行通知，示例如下所示。

代码清单4-11 WaitNotify.java

```
public class WaitNotify {
    static boolean flag = true;
    static Object lock = new Object();
    public static void main(String[] args) throws Exception {
        Thread waitThread = new Thread(new Wait(), "WaitThread");
        waitThread.start();
        TimeUnit.SECONDS.sleep(1);
        Thread notifyThread = new Thread(new Notify(), "NotifyThread");
        notifyThread.start();
    }
    static class Wait implements Runnable {
        public void run() {
            // 加锁, 拥有lock的Monitor
            synchronized (lock) {
                // 当条件不满足时, 继续wait, 同时释放了lock的锁
                while (flag) {
                    try {
                        System.out.println(Thread.currentThread() + " flag is true. wa
                        @ " + new SimpleDateFormat("HH:mm:ss").format(new Date()));
                        lock.wait();
                    } catch (InterruptedException e) {
                    }
                }
            }
            // 条件满足时, 完成工作
```

```

        System.out.println(Thread.currentThread() + " flag is false. running @ " + new SimpleDateFormat("HH:mm:ss").format(new Date()));
    }
}
}
static class Notify implements Runnable {
    public void run() {
        // 加锁, 拥有lock的Monitor
        synchronized (lock) {
            // 获取lock的锁, 然后进行通知, 通知时不会释放lock的锁,
            // 直到当前线程释放了lock后, WaitThread才能从wait方法中返回
            System.out.println(Thread.currentThread() + " hold lock. notify @ " +
                new SimpleDateFormat("HH:mm:ss").format(new Date()));
            lock.notifyAll();
            flag = false;
            SleepUtils.second(5);
        }
        // 再次加锁
        synchronized (lock) {
            System.out.println(Thread.currentThread() + " hold lock again. sleep @ " +
                new SimpleDateFormat("HH:mm:ss").format(new Date()));
            SleepUtils.second(5);
        }
    }
}
}
}

```

输出如下(输出内容可能不同, 主要区别在时间上)。

```

Thread[WaitThread,5,main] flag is true. wait @ 22:23:03
Thread[NotifyThread,5,main] hold lock. notify @ 22:23:04
Thread[NotifyThread,5,main] hold lock again. sleep @ 22:23:09
Thread[WaitThread,5,main] flag is false. running @ 22:23:14

```

上述第3行和第4行输出的顺序可能会互换, 而上述例子主要说明了调用wait()、notify()以及notifyAll()时需要注意的细节, 如下。

1) 使用wait()、notify()和notifyAll()时需要先对调用对象加锁。

2) 调用wait()方法后, 线程状态由RUNNING变为WAITING, 并将当前线程放置到对象的等待队列。

3) notify()或notifyAll()方法调用后, 等待线程依旧不会从wait()返回, 需要调用notify()或notifyAll()的线程释放锁之后, 等待线程才有机会从wait()返回。

4) notify()方法将等待队列中的一个等待线程从等待队列中移到同步队列中，而notifyAll()方法则是将等待队列中所有的线程全部移到同步队列，被移动的线程状态由WAITING变为BLOCKED。

5) 从wait()方法返回的前提是获得了调用对象的锁。

从上述细节中可以看到，等待/通知机制依托于同步机制，其目的就是确保等待线程从wait()方法返回时能够感知到通知线程对变量做出的修改。

图4-3描述了上述示例的过程。

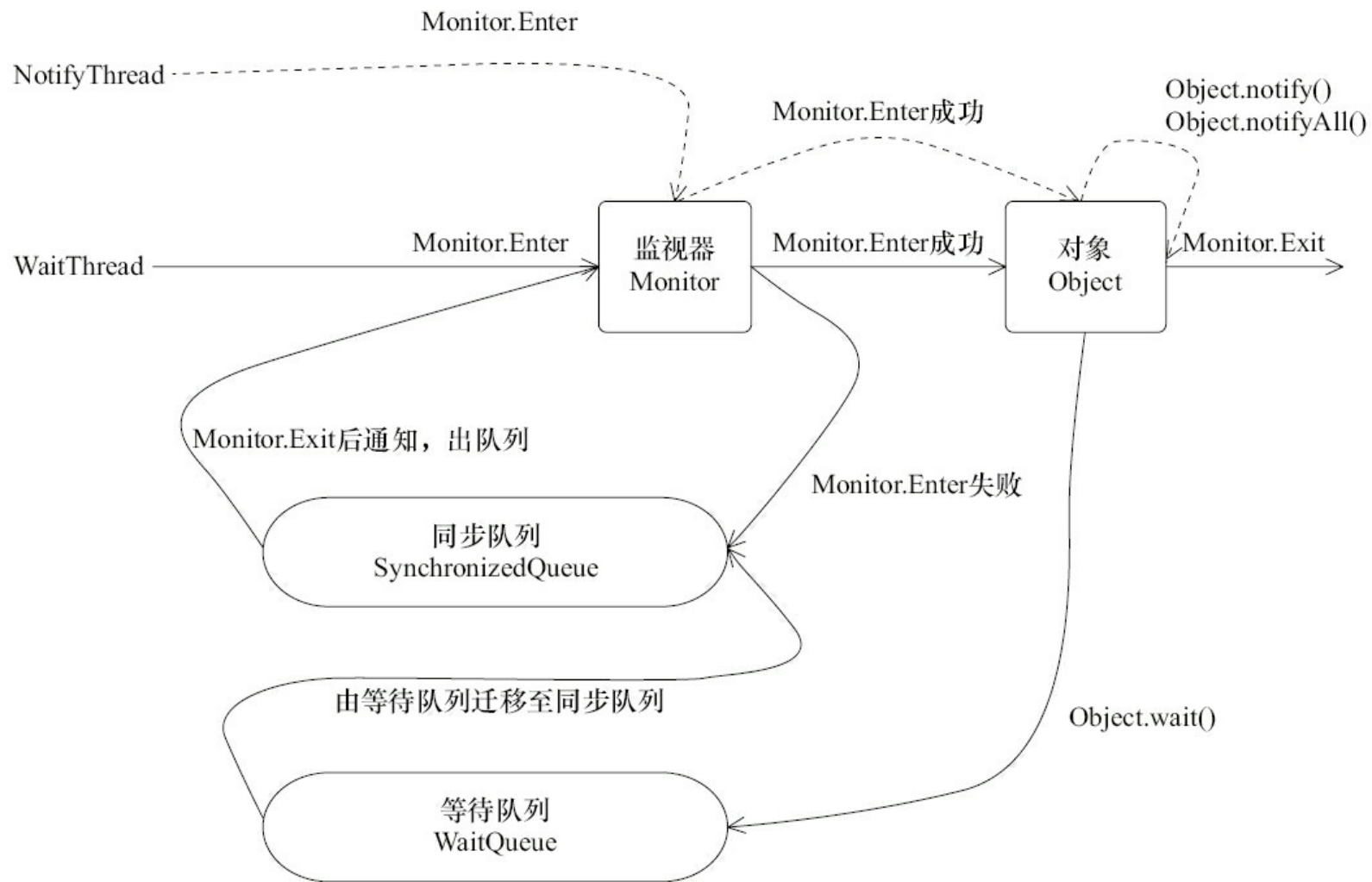


图4-3 WaitNotify.java运行过程

在图4-3中，WaitThread首先获取了对象的锁，然后调用对象的wait()方法，从而放弃了锁并进入了对象的等待队列WaitQueue中，进入等待状态。由于WaitThread释放了对象的锁，

NotifyThread随后获取了对象的锁，并调用对象的notify()方法，将WaitThread从WaitQueue移到SynchronizedQueue中，此时WaitThread的状态变为阻塞状态。NotifyThread释放了锁之后，WaitThread再次获取到锁并从wait()方法返回继续执行。

4.3.3 等待/通知的经典范式

从4.3.2节中的WaitNotify示例中可以提炼出等待/通知的经典范式，该范式分为两部分，分别针对等待方(消费者)和通知方(生产者)。

等待方遵循如下原则。

- 1) 获取对象的锁。
- 2) 如果条件不满足，那么调用对象的wait()方法，被通知后仍要检查条件。
- 3) 条件满足则执行对应的逻辑。

对应的伪代码如下。

```
synchronized(对象) {  
    while(条件不满足) {  
        对象.wait();  
    }  
    对应的处理逻辑  
}
```

通知方遵循如下原则。

- 1) 获得对象的锁。
- 2) 改变条件。
- 3) 通知所有等待在对象上的线程。

对应的伪代码如下。

```
synchronized(对象) {  
    改变条件  
    对象.notifyAll();  
}
```

4.3.4 管道输入/输出流

管道输入/输出流和普通文件输入/输出流或者网络输入/输出流不同之处在于，它主要用于线程之间的数据传输，而传输的媒介为内存。

管道输入/输出流主要包括了如下4种具体实现：PipedOutputStream、PipedInputStream、PipedReader和PipedWriter，前两种面向字节，而后两种面向字符。

在代码清单4-12所示的例子中，创建了printThread，它用来接受main线程的输入，任何main线程的输入均通过PipedWriter写入，而printThread在另一端通过PipedReader将内容读出并打印。

代码清单4-12 Piped.java

```
public class Piped {
    public static void main(String[] args) throws Exception {
        PipedWriter out = new PipedWriter();
        PipedReader in = new PipedReader();
        // 将输出流和输入流进行连接, 否则在使用时会抛出IOException
        out.connect(in);
        Thread printThread = new Thread(new Print(in), "PrintThread");
        printThread.start();
        int receive = 0;
        try {
            while ((receive = System.in.read()) != -1) {
                out.write(receive);
            }
        } finally {
            out.close();
        }
    }
    static class Print implements Runnable {
        private PipedReader in;
        public Print(PipedReader in) {
            this.in = in;
        }
        public void run() {
            int receive = 0;
            try {
                while ((receive = in.read()) != -1) {
                    System.out.print((char) receive);
                }
            } catch (IOException ex) {
```

```
}  
    }  
}  
}
```

运行该示例，输入一组字符串，可以看到被printThread进行了原样输出。

```
Repeat my words.  
Repeat my words.
```

对于Piped类型的流，必须先要进行绑定，也就是调用connect()方法，如果没有将输入/输出流绑定起来，对于该流的访问将会抛出异常。

4.3.5 Thread.join()的使用

join()方法内部实现:循环执行 判断线程是否存活 --> 存活就wait() -->判断线程是否存活.一直循环这个过程

如果一个线程A执行了thread.join()语句, 其含义是:当前线程A等待thread线程终止之后才

从thread.join()返回。线程Thread除了提供join()方法之外, 还提供了join(long millis)和join(long millis,int nanos)两个具备超时特性的方法。这两个超时方法表示, 如果线程thread在给定的超时时间里没有终止, 那么将会从该超时方法中返回。

在代码清单4-13所示的例子中, 创建了10个线程, 编号0~9, 每个线程调用前一个线程的join()方法, 也就是线程0结束了, 线程1才能从join()方法中返回, 而线程0需要等待main线程结束。

代码清单4-13 Join.java

```
public class Join {
    public static void main(String[] args) throws Exception {
        Thread previous = Thread.currentThread();
        for (int i = 0; i < 10; i++) {
            // 每个线程拥有前一个线程的引用, 需要等待前一个线程终止, 才能从等待中返回
            Thread thread = new Thread(new Domino(previous), String.valueOf(i));
            thread.start();
            previous = thread;
        }
        TimeUnit.SECONDS.sleep(5);
        System.out.println(Thread.currentThread().getName() + " terminate.");
    }
    static class Domino implements Runnable {
        private Thread thread;
        public Domino(Thread thread) {
            this.thread = thread;
        }
        public void run() {
            try {
                thread.join();
            } catch (InterruptedException e) {
            }
            System.out.println(Thread.currentThread().getName() + " terminate.");
        }
    }
}
```

输出如下。

```
main terminate.  
0 terminate.  
1 terminate.  
2 terminate.  
3 terminate.  
4 terminate.  
5 terminate.  
6 terminate.  
7 terminate.  
8 terminate.  
9 terminate.
```

从上述输出可以看到，每个线程终止的前提是前驱线程的终止，每个线程等待前驱线程终止后，才从join()方法返回，这里涉及了等待/通知机制（等待前驱线程结束，接收前驱线程结束通知）。

代码清单4-14是JDK中Thread.join()方法的源码（进行了部分调整）。

代码清单4-14 Thread.java

```
// 加锁当前线程对象  
public final synchronized void join() throws InterruptedException {  
    // 条件不满足，继续等待  
    while (isAlive()) {  
        wait(0);  
    }  
    // 条件符合，方法返回  
}
```

当线程终止时，会调用线程自身的notifyAll()方法，会通知所有等待在该线程对象上的线程。可以看到join()方法的逻辑结构与4.3.3节中描述的等待/通知经典范式一致，即加锁、循环和处理逻辑3个步骤。

4.3.6 ThreadLocal的使用

Thread内部有个成员变量ThreadLocalMap,这个map的key就是threadLocal,value就是值.所用threadLocal设置的值都是个线程自己独有的.

ThreadLocal, 即线程变量, 是一个以ThreadLocal对象为键、任意对象为值的存储结构。这

个结构被附带在线程上, 也就是说一个线程可以根据一个ThreadLocal对象查询到绑定在这个线程上的一个值。

可以通过set(T)方法来设置一个值, 在当前线程下再通过get()方法获取到原先设置的值。

在代码清单4-15所示的例子中, 构建了一个常用的Profiler类, 它具有begin()和end()两个方法, 而end()方法返回从begin()方法调用开始到end()方法被调用时的时间差, 单位是毫秒。

代码清单4-15 Profiler.java

```
public class Profiler {
    // 第一次get()方法调用时会进行初始化(如果set方法没有调用), 每个线程会调用一次
    private static final ThreadLocal<Long> TIME_THREADLOCAL = new ThreadLocal<Long>() {
        protected Long initialValue() {
            return System.currentTimeMillis();
        }
    };
    public static final void begin() {
        TIME_THREADLOCAL.set(System.currentTimeMillis());
    }
    public static final long end() {
        return System.currentTimeMillis() - TIME_THREADLOCAL.get();
    }
    public static void main(String[] args) throws Exception {
        Profiler.begin();
        TimeUnit.SECONDS.sleep(1);
        System.out.println("Cost: " + Profiler.end() + " mills");
    }
}
```

输出结果如下所示。

```
Cost: 1001 mills
```

Profiler可以被复用在方法调用耗时统计的功能上, 在方法的入口前执行begin()方法, 在

方法调用后执行end()方法，好处是两个方法的调用不用在一个方法或者类中，比如在AOP(面向方面编程)中，可以在方法调用前的切入点执行begin()方法，而在方法调用后的切入点执行end()方法，这样依旧可以获得方法的执行耗时。

4.4 线程应用实例

4.4.1 等待超时模式

开发人员经常会遇到这样的方法调用场景：调用一个方法时等待一段时间（一般来说是给定的一个时间段），如果该方法能够在给定的时间段之内得到结果，那么将结果立刻返回，反之，超时返回默认结果。

前面的章节介绍了等待/通知的经典范式，即加锁、条件循环和处理逻辑3个步骤，而这种范式无法做到超时等待。而超时等待的加入，只需要对经典范式做出非常小的改动，改动内容如下所示。

假设超时时间段是 T ，那么可以推断出在当前时间 $\text{now}+T$ 之后就会超时。

定义如下变量。

·等待持续时间： $\text{REMAINING}=T$ 。

·超时时间： $\text{FUTURE}=\text{now}+T$ 。

这时仅需要 $\text{wait}(\text{REMAINING})$ 即可，在 $\text{wait}(\text{REMAINING})$ 返回之后会将执行：

$\text{REMAINING}=\text{FUTURE}-\text{now}$ 。如果 REMAINING 小于等于0，表示已经超时，直接退出，否则将继续执行 $\text{wait}(\text{REMAINING})$ 。

上述描述等待超时模式的伪代码如下。

```
// 对当前对象加锁
public synchronized Object get(long mills) throws InterruptedException {
    long future = System.currentTimeMillis() + mills;
    long remaining = mills;
    // 当超时大于0并且result返回值不满足要求
    while ((result == null) && remaining > 0) {
        wait(remaining);
    }
}
```

```
        remaining = future - System.currentTimeMillis();  
    }  
    return result;  
}
```

可以看出，等待超时模式就是在等待/通知范式基础上增加了超时控制，这使得该模式相比原有范式更具有灵活性，因为即使方法执行时间过长，也不会“永久”阻塞调用者，而是会按照调用者的要求“按时”返回。

4.4.2 一个简单的数据库连接池示例

我们使用等待超时模式来构造一个简单的数据库连接池，在示例中模拟从连接池中获取、使用和释放连接的过程，而客户端获取连接的过程被设定为等待超时的模式，也就是在1000毫秒内如果无法获取到可用连接，将会返回给客户端一个null。设定连接池的大小为10个，然后通过调节客户端的线程数来模拟无法获取连接的场景。

首先看一下连接池的定义。它通过构造函数初始化连接的最大上限，通过一个双向队列来维护连接，调用方需要先调用fetchConnection(long)方法来指定在多少毫秒内超时获取连接，当连接使用完成后，需要调用releaseConnection(Connection)方法将连接放回线程池，示例如代码清单4-16所示。

代码清单4-16 ConnectionPool.java

```
public class ConnectionPool {
    private LinkedList<Connection> pool = new LinkedList<Connection>();
    public ConnectionPool(int initialSize) {
        if (initialSize > 0) {
            for (int i = 0; i < initialSize; i++) {
                pool.addLast(ConnectionDriver.createConnection());
            }
        }
    }
    public void releaseConnection(Connection connection) {
        if (connection != null) {
            synchronized (pool) {
                // 连接释放后需要进行通知, 这样其他消费者能够感知到连接池中已经归还了一个连接
                pool.addLast(connection);
                pool.notifyAll();
            }
        }
    }
    // 在mills内无法获取到连接, 将会返回null
    public Connection fetchConnection(long mills) throws InterruptedException {
        synchronized (pool) {
            // 完全超时
            if (mills <= 0) {
                while (pool.isEmpty()) {
                    pool.wait();
                }
                return pool.removeFirst();
            } else {

```

```

        long future = System.currentTimeMillis() + mills;
        long remaining = mills;
        while (pool.isEmpty() && remaining > 0) {
            pool.wait(remaining);
            remaining = future - System.currentTimeMillis();
        }
        Connection result = null;
        if (!pool.isEmpty()) {
            result = pool.removeFirst();
        }
        return result;
    }
}
}
}

```

由于java.sql.Connection是一个接口，最终的实现是由数据库驱动提供方来实现的，考虑到只是个示例，我们通过动态代理构造了一个Connection，该Connection的代理实现仅仅是在commit()方法调用时休眠100毫秒，示例如代码清单4-17所示。

代码清单4-17 ConnectionDriver.java

```

public class ConnectionDriver {
    static class ConnectionHandler implements InvocationHandler {
        public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
            if (method.getName().equals("commit")) {
                TimeUnit.MILLISECONDS.sleep(100);
            }
            return null;
        }
    }
    // 创建一个Connection的代理, 在commit时休眠100毫秒
    public static final Connection createConnection() {
        return (Connection) Proxy.newProxyInstance(ConnectionDriver.class.getClassLoader(),
            new Class<>[] { Connection.class }, new ConnectionHandler());
    }
}

```

下面通过一个示例来测试简易数据库连接池的工作情况，模拟客户端ConnectionRunner获取、使用、最后释放连接的过程，当它使用时连接将会增加获取到连接的数量，反之，将会增加未获取到连接的数量，示例如代码清单4-18所示。

代码清单4-18 ConnectionPoolTest.java

```

public class ConnectionPoolTest {
    static ConnectionPool pool = new ConnectionPool(10);
    // 保证所有ConnectionRunner能够同时开始
    static CountdownLatch start = new CountdownLatch(1);
    // main线程将会等待所有ConnectionRunner结束后才能继续执行
    static CountdownLatch end;
    public static void main(String[] args) throws Exception {
        // 线程数量, 可以修改线程数量进行观察
        int threadCount = 10;
        end = new CountdownLatch(threadCount);
        int count = 20;
        AtomicInteger got = new AtomicInteger();
        AtomicInteger notGot = new AtomicInteger();
        for (int i = 0; i < threadCount; i++) {
            Thread thread = new Thread(new ConnetionRunner(count, got, notGot),
                "ConnectionRunnerThread");
            thread.start();
        }
        start.countDown();
        end.await();
        System.out.println("total invoke: " + (threadCount * count));
        System.out.println("got connection: " + got);
        System.out.println("not got connection " + notGot);
    }
    static class ConnetionRunner implements Runnable {
        int count;
        AtomicInteger got;
        AtomicInteger notGot;
        public ConnetionRunner(int count, AtomicInteger got, AtomicInteger notGot) {
            this.count = count;
            this.got = got;
            this.notGot = notGot;
        }
        public void run() {
            try {
                start.await();
            } catch (Exception ex) {
            }
            while (count > 0) {
                try {
                    // 从线程池中获取连接, 如果1000ms内无法获取到, 将会返回null
                    // 分别统计连接获取的数量got和未获取到的数量notGot
                    Connection connection = pool.fetchConnection(1000);
                    if (connection != null) {
                        try {
                            connection.createStatement();
                            connection.commit();
                        } finally {
                            pool.releaseConnection(connection);
                            got.incrementAndGet();
                        }
                    } else {
                        notGot.incrementAndGet();
                    }
                } catch (Exception ex) {
                } finally {
                    count--;
                }
            }
        }
    }
}

```

```
    }  
    end.countDown();  
}  
}  
}
```

上述示例中使用了CountDownLatch来确保ConnectionRunnerThread能够同时开始执行，并且在全部结束之后，才使main线程从等待状态中返回。当前设定的场景是10个线程同时运行获取连接池（10个连接）中的连接，通过调节线程数量来观察未获取到连接的情况。线程数、总获取次数、获取到的数量、未获取到的数量以及未获取到的比率，如表4-3所示（笔者机器CPU: i7-3635QM, 内存为8GB, 实际输出可能与此表不同）。

表4-3 线程数量与连接获取的关系

线程数量	总获取次数	获取到次数	未获取到次数	未获取到比率
10	200	200	0	0%
20	400	387	13	3.25%
30	600	542	58	9.67%
40	800	700	100	12.5%
50	1 000	828	172	17.2%

从表中的数据统计可以看出，在资源一定的情况下（连接池中的10个连接），随着客户端线程的逐步增加，客户端出现超时无法获取连接的比率不断升高。虽然客户端线程在这种超时获取的模式下会出现连接无法获取的情况，但是它能够保证客户端线程不会一直挂在连接获取的操作上，而是“按时”返回，并告知客户端连接获取出现问题，是系统的一种自我保护机制。数据库连接池的设计也可以复用到其他的资源获取的场景，针对昂贵资源（比如数据库连接）的获取都应该加以超时限制。

4.4.3 线程池技术及其示例

对于服务端的程序，经常面对的是客户端传入的短小（执行时间短、工作内容较为单一）任务，需要服务端快速处理并返回结果。如果服务端每次接受到一个任务，创建一个线程，然后进行执行，这在原型阶段是个不错的选择，但是面对成千上万的任务递交进服务器时，如果还是采用一个任务一个线程的方式，那么将会创建数以万记的线程，这不是一个好的选择。因为这会使操作系统频繁的进行线程上下文切换，无故增加系统的负载，而线程的创建和消亡都是需要耗费系统资源的，也无疑浪费了系统资源。

线程池技术能够很好地解决这个问题，它预先创建了若干数量的线程，并且不能由用户直接对线程的创建进行控制，在这个前提下重复使用固定或较为固定数目的线程来完成任务的执行。这样做的好处是，一方面，消除了频繁创建和消亡线程的系统资源开销，另一方面，面对过量任务的提交能够平缓的劣化。

下面先看一个简单的线程池接口定义，示例如代码清单4-19所示。

代码清单4-19 ThreadPool.java

```
public interface ThreadPool<Job extends Runnable> {  
    // 执行一个Job, 这个Job需要实现Runnable  
    void execute(Job job);  
    // 关闭线程池  
    void shutdown();  
    // 增加工作者线程  
    void addWorkers(int num);  
    // 减少工作者线程  
    void removeWorker(int num);  
    // 得到正在等待执行的任务数量  
    int getJobSize();  
}
```

客户端可以通过execute(Job)方法将Job提交入线程池执行，而客户端自身不用等待Job的执行完成。除了execute(Job)方法以外，线程池接口提供了增大/减少工作者线程以及关闭线程池的方法。这里工作者线程代表着一个重复执行Job的线程，而每个由客户端提交的Job都将进

入到一个工作队列中等待工作者线程的处理。

接下来是线程池接口的默认实现，示例如代码清单4-20所示。

代码清单4-20 DefaultThreadPool.java

```
public class DefaultThreadPool<Job> extends Runnable> implements ThreadPool<Job> {
    // 线程池最大限制数
    private static final int      MAX_WORKER_NUMBERS      = 10;
    // 线程池默认的数量
    private static final int      DEFAULT_WORKER_NUMBERS = 5;
    // 线程池最小的数量
    private static final int      MIN_WORKER_NUMBERS      = 1;
    // 这是一个工作列表, 将会向里面插入工作
    private final LinkedList<Job>  jobs = new LinkedList<Job>();
    // 工作者列表
    private final List<Worker>     workers      = Collections.synchronizedList(new
    ArrayList<Worker>());
    // 工作者线程的数量
    private int      workerNum      = DEFAULT_WORKER_NUMBERS;
    // 线程编号生成
    private AtomicLong      threadNum      = new AtomicLong();
    public DefaultThreadPool() {
        initializeWokers(DEFAULT_WORKER_NUMBERS);
    }
    public DefaultThreadPool(int num) {
        workerNum = num > MAX_WORKER_NUMBERS ? MAX_WORKER_NUMBERS : num < MIN_WORKER_
        NUMBERS ? MIN_WORKER_NUMBERS : num;
        initializeWokers(workerNum);
    }
    public void execute(Job job) {
        if (job != null) {
            // 添加一个工作, 然后进行通知
            synchronized (jobs) {
                jobs.addLast(job);
                jobs.notify();
            }
        }
    }
    public void shutdown() {
        for (Worker worker : workers) {
            worker.shutdown();
        }
    }
    public void addWorkers(int num) {
        synchronized (jobs) {
            // 限制新增的Worker数量不能超过最大值
            if (num + this.workerNum > MAX_WORKER_NUMBERS) {
                num = MAX_WORKER_NUMBERS - this.workerNum;
            }
            initializeWokers(num);
            this.workerNum += num;
        }
    }
}
```

```

    }
}

public void removeWorker(int num) {
    synchronized (jobs) {
        if (num >= this.workerNum) {
            throw new IllegalArgumentException("beyond workNum");
        }
        // 按照给定的数量停止Worker
        int count = 0;
        while (count < num) {
            Worker worker = workers.get(count)
            if (workers.remove(worker)) {
                worker.shutdown();
                count++;
            }
        }
        this.workerNum -= count;
    }
}

public int getJobSize() {
    return jobs.size();
}

// 初始化线程工作者
private void initializeWokers(int num) {
    for (int i = 0; i < num; i++) {
        Worker worker = new Worker();
        workers.add(worker);
        Thread thread = new Thread(worker, "ThreadPool-Worker-" + threadNum.
            incrementAndGet());
        thread.start();
    }
}

// 工作者, 负责消费任务
class Worker implements Runnable {
    // 是否工作
    private volatile boolean    running    = true;
    public void run() {
        while (running) {
            Job job = null;
            synchronized (jobs) {
                // 如果工作者列表是空的, 那么就wait
                while (jobs.isEmpty()) {
                    try {
                        jobs.wait();
                    } catch (InterruptedException ex) {
                        // 感知到外部对WorkerThread的中断操作, 返回
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
                // 取出一个Job
                job = jobs.removeFirst();
            }
            if (job != null) {
                try {
                    job.run();
                } catch (Exception ex) {
                    // 忽略Job执行中的Exception
                }
            }
        }
    }
}

```

```
        }  
    }  
}  
}  
public void shutdown() {  
    running = false;  
}  
}
```

从线程池的实现可以看到，当客户端调用execute(Job)方法时，会不断地向任务列表jobs中添加Job，而每个工作者线程会不断地从jobs上取出一个Job进行执行，当jobs为空时，工作者线程进入等待状态。

添加一个Job后，对工作队列jobs调用了其notify()方法，而不是notifyAll()方法，因为能够确定有工作者线程被唤醒，这时使用notify()方法将会比notifyAll()方法获得更小的开销（避免将等待队列中的线程全部移动到阻塞队列中）。

可以看到，线程池的本质就是使用了一个线程安全的工作队列连接工作者线程和客户端线程，客户端线程将任务放入工作队列后便返回，而工作者线程则不断地从工作队列上取出工作并执行。当工作队列为空时，所有的工作者线程均等待在工作队列上，当有客户端提交了一个任务之后会通知任意一个工作者线程，随着大量的任务被提交，更多的工作者线程会被唤醒。

4.4.4 一个基于线程池技术的简单Web服务器

目前的浏览器都支持多线程访问，比如说在请求一个HTML页面的时候，页面中包含的图片资源、样式资源会被浏览器发起并发的获取，这样用户就不会遇到一直等到一个图片完全下载完成才能继续查看文字内容的尴尬情况。

如果Web服务器是单线程的，多线程的浏览器也没有用武之地，因为服务端还是一个请求一个请求的顺序处理。因此，大部分Web服务器都是支持并发访问的。常用的Java Web服务器，如Tomcat、Jetty，在其处理请求的过程中都使用到了线程池技术。

下面通过使用前一节中的线程池来构造一个简单的Web服务器，这个Web服务器用来处理HTTP请求，目前只能处理简单的文本和JPG图片内容。这个Web服务器使用main线程不断地接受客户端Socket的连接，将连接以及请求提交给线程池处理，这样使得Web服务器能够同时处理多个客户端请求，示例如代码清单4-21所示。

代码清单4-21 SimpleHttpServer.java

```
public class SimpleHttpServer {
    // 处理HttpRequest的线程池
    static ThreadPool<HttpRequestHandler> threadPool = new DefaultThreadPool
        <HttpRequestHandler>(1);
    // SimpleHttpServer的根路径
    static String basePath;
    static ServerSocket serverSocket;
    // 服务监听端口
    static int port = 8080;
    public static void setPort(int port) {
        if (port > 0) {
            SimpleHttpServer.port = port;
        }
    }
    public static void setBasePath(String basePath) {
        if (basePath != null && new File(basePath).exists() && new File(basePath).
            isDirectory()) {
            SimpleHttpServer.basePath = basePath;
        }
    }
    // 启动SimpleHttpServer
    public static void start() throws Exception {
        serverSocket = new ServerSocket(port);
```

```

Socket socket = null;
while ((socket = serverSocket.accept()) != null) {
    // 接收一个客户端Socket, 生成一个HttpRequestHandler, 放入线程池执行
    threadPool.execute(new HttpRequestHandler(socket));
}
serverSocket.close();
}

static class HttpRequestHandler implements Runnable {
    private Socket socket;
    public HttpRequestHandler(Socket socket) {
        this.socket = socket;
    }
    @Override
    public void run() {
        String line = null;
        BufferedReader br = null;
        BufferedReader reader = null;
        PrintWriter out = null;
        InputStream in = null;
        try {
            reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String header = reader.readLine();
            // 由相对路径计算出绝对路径
            String filePath = basePath + header.split(" ")[1];
            out = new PrintWriter(socket.getOutputStream());
            // 如果请求资源的后缀为jpg或者ico, 则读取资源并输出
            if (filePath.endsWith(".jpg") || filePath.endsWith(".ico")) {
                in = new FileInputStream(filePath);
                ByteArrayOutputStream baos = new ByteArrayOutputStream();
                int i = 0;
                while ((i = in.read()) != -1) {
                    baos.write(i);
                }
                byte[] array = baos.toByteArray();
                out.println("HTTP/1.1 200 OK");
                out.println("Server: Molly");
                out.println("Content-Type: image/jpeg");
                out.println("Content-Length: " + array.length);
                out.println("");
                socket.getOutputStream().write(array, 0, array.length);
            } else {
                br = new BufferedReader(new InputStreamReader(new
                    FileInputStream(filePath)));
                out = new PrintWriter(socket.getOutputStream());
                out.println("HTTP/1.1 200 OK");
                out.println("Server: Molly");
                out.println("Content-Type: text/html; charset=UTF-8");
                out.println("");
                while ((line = br.readLine()) != null) {
                    out.println(line);
                }
            }
            out.flush();
        } catch (Exception ex) {
            out.println("HTTP/1.1 500");
            out.println("");
            out.flush();
        } finally {

```

```

        close(br, in, reader, out, socket);
    }
}
// 关闭流或者Socket
private static void close(Closeable... closeables) {
    if (closeables != null) {
        for (Closeable closeable : closeables) {
            try {
                closeable.close();
            } catch (Exception ex) {
            }
        }
    }
}
}
}

```

该Web服务器处理用户请求的时序图如，图44所示。

在图4-4中，SimpleHttpServer在建立了与客户端的连接之后，并不会处理客户端的请求，而是将其包装成HttpRequestHandler并交由线程池处理。在线程池中的Worker处理客户端请求的同时，SimpleHttpServer能够继续完成后续客户端连接的建立，不会阻塞后续客户端的请求。

接下来，通过一个测试对比来认识线程池技术带来服务器吞吐量的提高。我们准备了一个简单的HTML页面，内容如代码清单4-22所示。

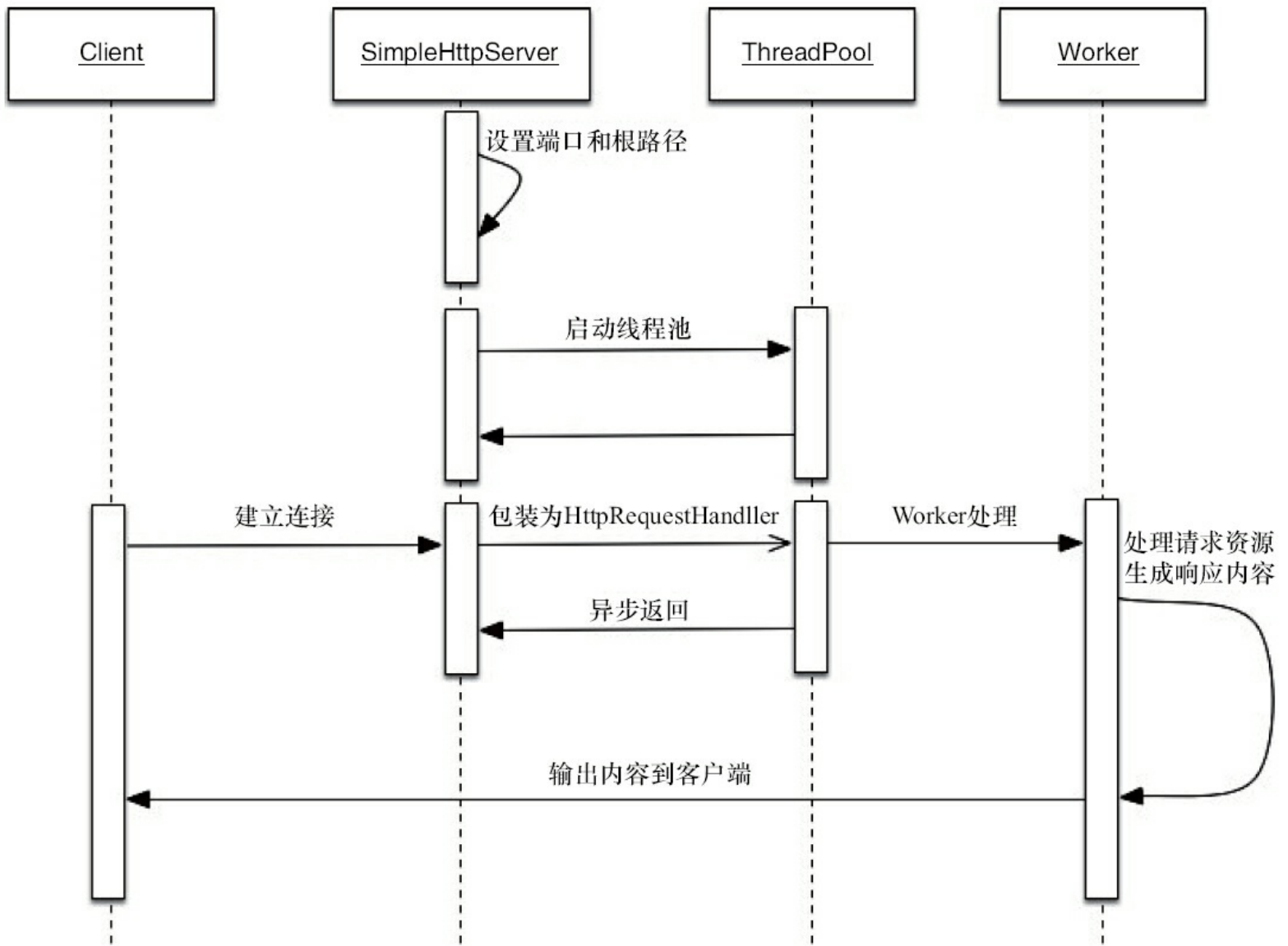


图4-4 SimpleHttpServer时序图

代码清单4-22 Index.html

```
<html>
  <head>
    <title>测试页面</title>
  </head>
  <body >
    <h1>第一张图片</h1>
    
    <h1>第二张图片</h1>
    
    <h1>第三张图片</h1>
    
  </body>
</html>
```


将SimpleHttpServer的根目录设定到该HTML页面所在目录, 并启动SimpleHttpServer, 通过Apache HTTP server benchmarking tool (版本2.3) 来测试不同线程数下, SimpleHttpServer的吞吐量表现。

测试场景是5000次请求, 分10个线程并发执行, 测试内容主要考察响应时间(越小越好)和每秒查询的数量(越高越好), 测试结果如表4-4所示(笔者机器CPU:i7-3635QM, 内存为8GB, 实际输出可能与此表不同)。

表4-4 测试结果

线程池线程数量	1	5	10
响应时间 (ms)	0.352	0.246	0.163
每秒查询的数量	3 076	4 065	6 123
测试完成时间 (s)	1.625	1.230	0.816

可以看到, 随着线程池中线程数量的增加, SimpleHttpServer的吞吐量不断增大, 响应时间不断变小, 线程池的作用非常明显。

但是, 线程池中线程数量并不是越多越好, 具体的数量需要评估每个任务的处理时间, 以及当前计算机的处理器能力和数量。使用的线程过少, 无法发挥处理器的性能;使用的线程过多, 将会增加系统的无故开销, 起到相反的作用。

4.5 本章小结

本章从介绍多线程技术带来的好处开始，讲述了如何启动和终止线程以及线程的状态，详细阐述了多线程之间进行通信的基本方式和等待/通知经典范式。在线程应用示例中，使用了等待超时、数据库连接池以及简单线程池3个不同的示例巩固本章前面章节所介绍的Java多线程基础知识。最后通过一个简单的Web服务器将上述知识点串联起来，加深我们对这些知识点的理解。

第5章 Java中的锁

本章将介绍Java并发包中与锁相关的API和组件，以及这些API和组件的使用方式和实现细节。内容主要围绕两个方面：**使用**，通过示例演示这些组件的使用方法以及详细介绍与锁相关的API；**实现**，通过分析源码来剖析实现细节，因为理解实现的细节方能更加得心应手且正确地使用这些组件。希望通过以上两个方面的讲解使开发者对锁的使用和实现两个层面有一定的了解。

5.1 Lock接口

锁是用来控制多个线程访问共享资源的方式，一般来说，一个锁能够防止多个线程同时访问共享资源（但是有些锁可以允许多个线程并发的访问共享资源，比如读写锁）。在Lock接口出现之前，Java程序是靠synchronized关键字实现锁功能的，而Java SE 5之后，并发包中新增了Lock接口（以及相关实现类）用来实现锁功能，它提供了与synchronized关键字类似的同步功能，只是在使用时需要显式地获取和释放锁。虽然它缺少了（通过synchronized块或者方法所提供的）隐式获取释放锁的便捷性，但是却拥有了锁获取与释放的可操作性、可中断的获取锁以及超时获取锁等多种synchronized关键字所不具备的同步特性。

使用synchronized关键字将会隐式地获取锁，但是它将锁的获取和释放固化了，也就是先获取再释放。当然，这种方式简化了同步的管理，可是扩展性没有显示的锁获取和释放来的好。例如，针对一个场景，手把手进行锁获取和释放，先获得锁A，然后再获取锁B，当锁B获得后，释放锁A同时获取锁C，当锁C获得后，再释放B同时获取锁D，以此类推。这种场景下，synchronized关键字就不那么容易实现了，而使用Lock却容易许多。

Lock的使用也很简单，代码清单5-1是Lock的使用的方式。

代码清单5-1 LockUseCase.java

```
Lock lock = new ReentrantLock();
lock.lock();
try {
} finally {
    lock.unlock();
}
```

在finally块中释放锁，目的是保证在获取到锁之后，最终能够被释放。

不要将获取锁的过程写在try块中，因为如果在获取锁（自定义锁的实现）时发生了异常，异常抛出的同时，也会导致锁无故释放。

Lock接口提供的synchronized关键字所不具备的主要特性如表5-1所示。

表5-1 Lock接口提供的synchronized关键字不具备的主要特性

特 性	描 述
尝试非阻塞地获取锁	当前线程尝试获取锁，如果这一时刻锁没有被其他线程获取到，则成功获取并持有锁
能被中断地获取锁	与 synchronized 不同，获取到锁的线程能够响应中断，当获取到锁的线程被中断时，中断异常将会被抛出，同时锁会被释放
超时获取锁	在指定的截止时间之前获取锁，如果截止时间到了仍旧无法获取锁，则返回

Lock是一个接口，它定义了锁获取和释放的基本操作，Lock的API如表5-2所示。

表5-2 Lock的API

方法名称	描 述
void lock()	获取锁，调用该方法当前线程将会获取锁，当锁获得后，从该方法返回
void lockInterruptibly() throws InterruptedException	可中断地获取锁，和 lock() 方法的不同之处在于该方法会响应中断，即在锁的获取中可以中断当前线程
boolean tryLock()	尝试非阻塞的获取锁，调用该方法后立刻返回，如果能够获取则返回 true，否则返回 false
boolean tryLock(long time, TimeUnit unit) throws InterruptedException	超时的获取锁，当前线程在以下 3 种情况下会返回： ①当前线程在超时时间内获得了锁 ②当前线程在超时时间内被中断 ③超时时间结束，返回 false
void unlock()	释放锁
Condition newCondition()	获取等待通知组件，该组件和当前的锁绑定，当前线程只有获得了锁，才能调用该组件的 wait() 方法，而调用后，当前线程将释放锁

这里先简单介绍一下Lock接口的API, 随后的章节会详细介绍同步器

AbstractQueuedSynchronizer以及常用Lock接口的实现ReentrantLock。Lock接口的实现基本都是通过聚合了一个同步器的子类来完成线程访问控制的。

5.2 队列同步器

队列同步器AbstractQueuedSynchronizer(以下简称同步器),是用来构建锁或者其他同步组件的基础框架,它使用了一个int成员变量表示同步状态,通过内置的FIFO队列来完成资源获取线程的排队工作,并发包的作者(Doug Lea)期望它能够成为实现大部分同步需求的基础。

同步器的主要使用方式是继承,子类通过继承同步器并实现它的抽象方法来管理同步状态,在抽象方法的实现过程中免不了要对同步状态进行更改,这时就需要使用同步器提供的3个方法(getState()、setState(int newState)和compareAndSetState(int expect,int update))来进行操作,因为它们能够保证状态的改变是安全的。子类推荐被定义为自定义同步组件的静态内部类,同步器自身没有实现任何同步接口,它仅仅是定义了若干同步状态获取和释放的方法来供自定义同步组件使用,同步器既可以支持独占式地获取同步状态,也可以支持共享式地获取同步状态,这样就可以方便实现不同类型的同步组件(ReentrantLock、ReentrantReadWriteLock和CountDownLatch等)。

同步器是实现锁(也可以是任意同步组件)的关键,在锁的实现中聚合同步器,利用同步器实现锁的语义。可以这样理解二者之间的关系:锁是面向使用者的,它定义了使用者与锁交互的接口(比如可以允许两个线程并行访问),隐藏了实现细节;同步器面向的是锁的实现者,它简化了锁的实现方式,屏蔽了同步状态管理、线程的排队、等待与唤醒等底层操作。锁和同步器很好地隔离了使用者和实现者所需关注的领域。

5.2.1 队列同步器的接口与示例

同步器的设计是基于模板方法模式的，也就是说，使用者需要继承同步器并重写指定的方法，随后将同步器组合在自定义同步组件的实现中，并调用同步器提供的模板方法，而这些模板方法将会调用使用者重写的方法。

重写同步器指定的方法时，需要使用同步器提供的如下3个方法来访问或修改同步状态。

- getState(): 获取当前同步状态。
- setState(int newState): 设置当前同步状态。
- compareAndSetState(int expect,int update): 使用CAS设置当前状态，该方法能够保证状态设置的原子性。

同步器可重写的方法与描述如表5-3所示。

表5-3 同步器可重写的方法

方法名称	描 述
protected boolean tryAcquire(int arg)	独占式获取同步状态，实现该方法需要查询当前状态并判断同步状态是否符合预期，然后再进行 CAS 设置同步状态
protected boolean tryRelease(int arg)	独占式释放同步状态，等待获取同步状态的线程将有机会获取同步状态
(续)	
方法名称	描 述
protected int tryAcquireShared(int arg)	共享式获取同步状态，返回大于等于 0 的值，表示获取成功，反之，获取失败
protected boolean tryReleaseShared(int arg)	共享式释放同步状态
protected boolean isHeldExclusively()	当前同步器是否在独占模式下被线程占用，一般该方法表示是否被当前线程所独占

实现自定义同步组件时，将会调用同步器提供的模板方法，这些(部分)模板方法与描述

如表5-4所示。

表5-4 同步器提供的模板方法

方法名称	描 述
void acquire(int arg)	独占式获取同步状态，如果当前线程获取同步状态成功，则由该方法返回，否则，将会进入同步队列等待，该方法将会调用重写的 tryAcquire(int arg) 方法
void acquireInterruptibly(int arg)	与 acquire(int arg) 相同，但是该方法响应中断，当前线程未获取到同步状态而进入同步队列中，如果当前线程被中断，则该方法会抛出 InterruptedException 并返回
boolean tryAcquireNanos(int arg, long nanos)	在 acquireInterruptibly(int arg) 基础上增加了超时限制，如果当前线程在超时时间内没有获取到同步状态，那么将会返回 false，如果获取到了返回 true
void acquireShared(int arg)	共享式的获取同步状态，如果当前线程未获取到同步状态，将会进入同步队列等待，与独占式获取的主要区别是在同一时刻可以有多个线程获取到同步状态
void acquireSharedInterruptibly(int arg)	与 acquireShared(int arg) 相同，该方法响应中断
boolean tryAcquireSharedNanos(int arg, long nanos)	在 acquireSharedInterruptibly(int arg) 基础上增加了超时限制
boolean release(int arg)	独占式的释放同步状态，该方法会在释放同步状态之后，将同步队列中第一个节点包含的线程唤醒
boolean releaseShared(int arg)	共享式的释放同步状态
Collection<Thread> getQueuedThreads()	获取等待在同步队列上的线程集合

同步器提供的模板方法基本上分为3类:独占式获取与释放同步状态、共享式获取与释放同步状态和查询同步队列中的等待线程情况。自定义同步组件将使用同步器提供的模板方法来实现自己的同步语义。

只有掌握了同步器的工作原理才能更加深入地理解并发包中其他的并发组件，所以下面通过一个独占锁的示例来深入了解一下同步器的工作原理。

顾名思义，独占锁就是在同一时刻只能有一个线程获取到锁，而其他获取锁的线程只能处于同步队列中等待，只有获取锁的线程释放了锁，后继的线程才能够获取锁，如代码清单5-2所示。


```
class Mutex implements Lock {
    // 静态内部类, 自定义同步器
    private static class Sync extends AbstractQueuedSynchronizer {
        // 是否处于占用状态
        protected boolean isHeldExclusively() {
            return getState() == 1;
        }
        // 当状态为0的时候获取锁
        public boolean tryAcquire(int acquires) {
            if (compareAndSetState(0, 1)) {
                setExclusiveOwnerThread(Thread.currentThread());
                return true;
            }
            return false;
        }
        // 释放锁, 将状态设置为0
        protected boolean tryRelease(int releases) {
            if (getState() == 0) throw new
                IllegalMonitorStateException();
            setExclusiveOwnerThread(null);
            setState(0);
            return true;
        }
        // 返回一个Condition, 每个condition都包含了一个condition队列
        Condition newCondition() { return new ConditionObject(); }
    }
    // 仅需要将操作代理到Sync上即可
    private final Sync sync = new Sync();
    public void lock() { sync.acquire(1); }
    public boolean tryLock() { return sync.tryAcquire(1); }
    public void unlock() { sync.release(1); }
    public Condition newCondition() { return sync.newCondition(); }
    public boolean isLocked() { return sync.isHeldExclusively(); }
    public boolean hasQueuedThreads() { return sync.hasQueuedThreads(); }
    public void lockInterruptibly() throws InterruptedException {
        sync.acquireInterruptibly(1);
    }
    public boolean tryLock(long timeout, TimeUnit unit) throws InterruptedException {
        return sync.tryAcquireNanos(1, unit.toNanos(timeout));
    }
}
```

上述示例中, 独占锁Mutex是一个自定义同步组件, 它在同一时刻只允许一个线程占有锁。Mutex中定义了一个静态内部类, 该内部类继承了同步器并实现了独占式获取和释放同步状态。在tryAcquire(int acquires)方法中, 如果经过CAS设置成功(同步状态设置为1), 则代表获取了同步状态, 而在tryRelease(int releases)方法中只是将同步状态重置为0。用户使用Mutex时并不会直接和内部同步器的实现打交道, 而是调用Mutex提供的方法, 在Mutex的实现中, 以获取锁的lock()方法为例, 只需要在方法实现中调用同步器的模板方法acquire(int args)即可, 当

前线程调用该方法获取同步状态失败后会被加入到同步队列中等待，这样就大大降低了实现一个可靠自定义同步组件的门槛。

5.2.2 队列同步器的实现分析

接下来将从实现角度分析同步器是如何完成线程同步的, 主要包括: 同步队列、独占式同步状态获取与释放、共享式同步状态获取与释放以及超时获取同步状态等同步器的核心数据结构与模板方法。

1.同步队列

同步器依赖内部的同步队列(一个FIFO双向队列)来完成同步状态的管理, 当前线程获取同步状态失败时, 同步器会将当前线程以及等待状态等信息构造成为一个节点(Node)并将其加入同步队列, 同时会阻塞当前线程, 当同步状态释放时, 会把首节点中的线程唤醒, 使其再次尝试获取同步状态。

同步队列中的节点(Node)用来保存获取同步状态失败的线程引用、等待状态以及前驱和后继节点, 节点的属性类型与名称以及描述如表5-5所示。

表5-5 节点的属性类型与名称以及描述

属性类型与名称	描 述
int waitStatus	等待状态。 包含如下状态。 ① CANCELLED，值为 1，由于在同步队列中等待的线程等待超时或者被中断，需要从同步队列中取消等待，节点进入该状态将不会变化 ② SIGNAL，值为 -1，后继节点的线程处于等待状态，而当前节点的线程如果释放了同步状态或者被取消，将会通知后继节点，使后继节点的线程得以运行 ③ CONDITION，值为 -2，节点在等待队列中，节点线程等待在 Condition 上，当其他线程对 Condition 调用了 signal() 方法后，该节点将会从等待队列中转移到同步队列中，加入到对同步状态的获取中 ④ PROPAGATE，值为 -3，表示下一次共享式同步状态获取将会无条件地被传播下去 ⑤ INITIAL，值为 0，初始状态
Node prev	前驱节点，当节点加入同步队列时被设置（尾部添加）
Node next	后继节点
Node nextWaiter	等待队列中的后继节点。如果当前节点是共享的，那么这个字段将是一个 SHARED 常量，也就是说节点类型（独占和共享）和等待队列中的后继节点共用同一个字段
Thread thread	获取同步状态的线程

节点是构成同步队列(等待队列, 在5.6节中将会介绍)的基础, 同步器拥有首节点(head)和尾节点(tail), 没有成功获取同步状态的线程将会成为节点加入该队列的尾部, 同步队列的基本结构如图5-1所示。

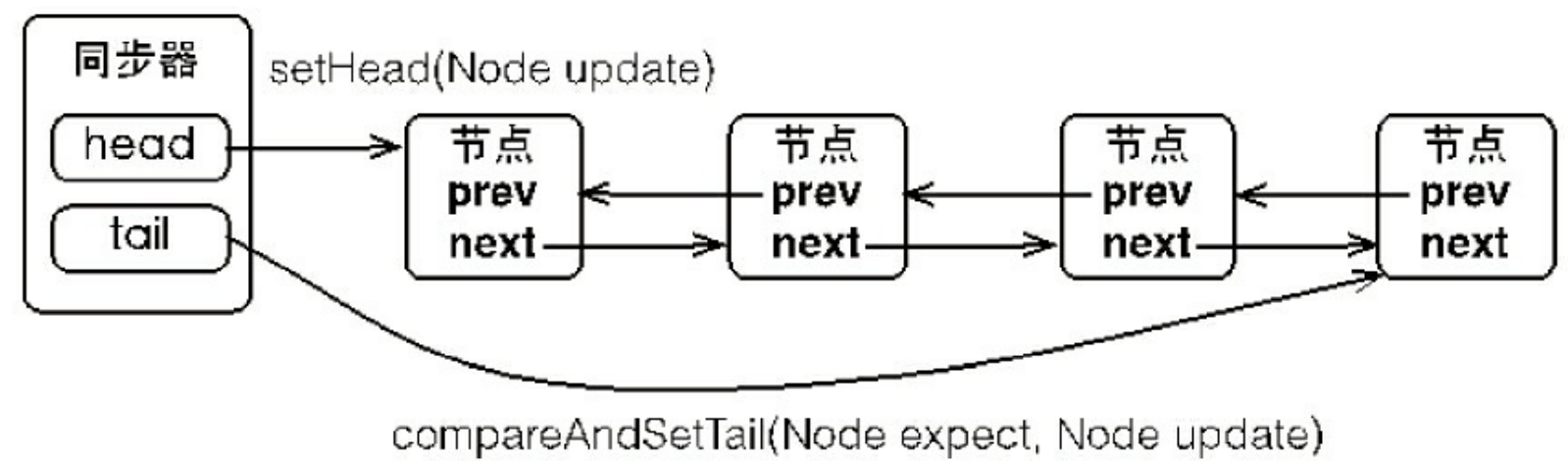


图5-1 同步队列的基本结构

在图5-1中, 同步器包含了两个节点类型的引用, 一个指向头节点, 而另一个指向尾节点。
试想一下, 当一个线程成功地获取了同步状态(或者锁), 其他线程将无法获取到同步状态, 转而被构造成为节点并加入到同步队列中, 而这个加入队列的过程必须要保证线程安全, 因此

同步器提供了一个基于CAS的设置尾节点的方法:compareAndSetTail(Node expect,Node update), 它需要传递当前线程“认为”的尾节点和当前节点, 只有设置成功后, 当前节点才正式与之前的尾节点建立关联。

同步器将节点加入到同步队列的过程如图5-2所示。

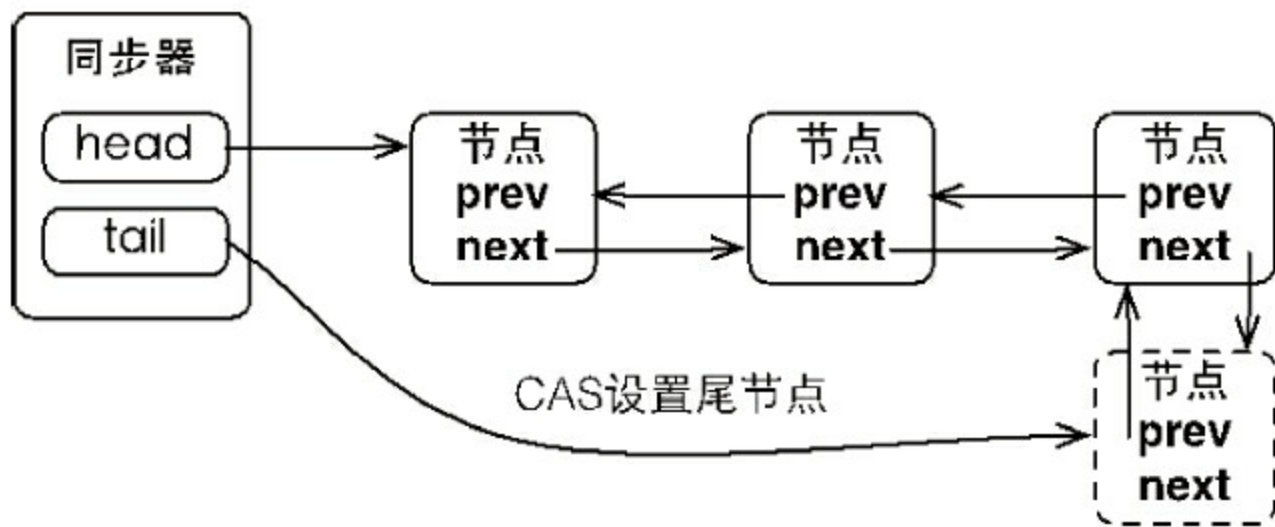


图5-2 节点加入到同步队列

同步队列遵循FIFO, 首节点是获取同步状态成功的节点, 首节点的线程在释放同步状态时, 将会唤醒后继节点, 而后继节点将会在获取同步状态成功时将自己设置为首节点, 该过程如图5-3所示。

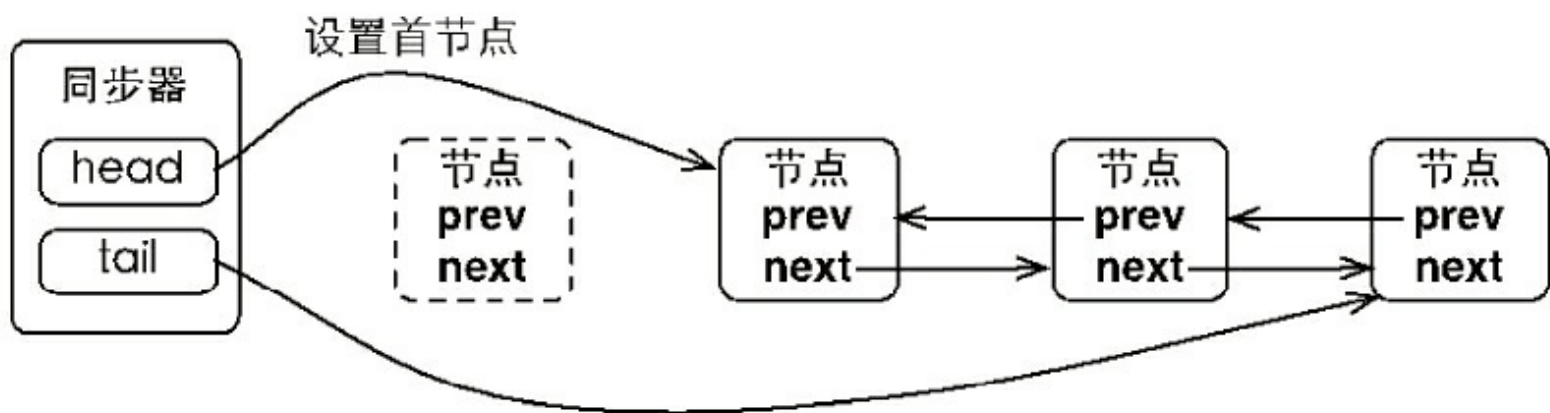


图5-3 首节点的设置

在图5-3中, 设置首节点是通过获取同步状态成功的线程来完成的, 由于只有一个线程能

够成功获取到同步状态，因此设置头节点的方法并不需要使用CAS来保证，它只需要将首节点设置成为原首节点的后继节点并断开原首节点的next引用即可。

2.独占式同步状态获取与释放

通过调用同步器的acquire(int arg)方法可以获取同步状态，该方法对中断不敏感，也就是由于线程获取同步状态失败后进入同步队列中，后续对线程进行中断操作时，线程不会从同步队列中移出，该方法代码如代码清单5-3所示。

代码清单5-3 同步器的acquire方法

```
public final void acquire(int arg) {
    if (!tryAcquire(arg) &&
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))
        selfInterrupt();
}
```

上述代码主要完成了同步状态获取、节点构造、加入同步队列以及在同步队列中自旋等待的相关工作，其主要逻辑是：首先调用自定义同步器实现的tryAcquire(int arg)方法，该方法保证线程安全的获取同步状态，如果同步状态获取失败，则构造同步节点(独占式Node.EXCLUSIVE，同一时刻只能有一个线程成功获取同步状态)并通过addWaiter(Node node)方法将该节点加入到同步队列的尾部，最后调用acquireQueued(Node node,int arg)方法，使得该节点以“死循环”的方式获取同步状态。如果获取不到则阻塞节点中的线程，而被阻塞线程的唤醒主要依靠前驱节点的出队或阻塞线程被中断来实现。

下面分析一下相关工作。首先是节点的构造以及加入同步队列，如代码清单5-4所示。

代码清单5-4 同步器的addWaiter和enq方法

```
private Node addWaiter(Node mode) {
    Node node = new Node(Thread.currentThread(), mode);
    // 快速尝试在尾部添加
    Node pred = tail;
```

```

        if (pred != null) {
            node.prev = pred;
            if (compareAndSetTail(pred, node)) {
                pred.next = node;
                return node;
            }
        }
        enq(node);
        return node;
    }
    private Node enq(final Node node) {
        for (;;) {
            Node t = tail;
            if (t == null) { // Must initialize
                if (compareAndSetHead(new Node()))
                    tail = head;
            } else {
                node.prev = t;
                if (compareAndSetTail(t, node)) {
                    t.next = node;
                    return t;
                }
            }
        }
    }
}

```

上述代码通过使用compareAndSetTail(Node expect,Node update)方法来确保节点能够被线程安全添加。试想一下：如果使用一个普通的LinkedList来维护节点之间的关系，那么当一个线程获取了同步状态，而其他多个线程由于调用tryAcquire(int arg)方法获取同步状态失败而并发地被添加到LinkedList时，LinkedList将难以保证Node的正确添加，最终的结果可能是节点的数量有偏差，而且顺序也是混乱的。

在enq(final Node node)方法中，同步器通过“死循环”来保证节点的正确添加，在“死循环”中只有通过CAS将节点设置成为尾节点之后，当前线程才能从该方法返回，否则，当前线程不断地尝试设置。可以看出，enq(final Node node)方法将并发添加节点的请求通过CAS变得“串行化”了。

节点进入同步队列之后，就进入了一个自旋的过程，每个节点(或者说每个线程)都在自省地观察，当条件满足，获取到了同步状态，就可以从这个自旋过程中退出，否则依旧留在这个自旋过程中(并会阻塞节点的线程)，如代码清单5-5所示。


```
final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        for (;;) {
            final Node p = node.predecessor();
            if (p == head && tryAcquire(arg)) {
                setHead(node);
                p.next = null; // help GC
                failed = false;
                return interrupted;
            }
            if (shouldParkAfterFailedAcquire(p, node) &&
                parkAndCheckInterrupt())
                interrupted = true;
        }
    } finally {
        if (failed)
            cancelAcquire(node);
    }
}
```

在acquireQueued(final Node node,int arg)方法中，当前线程在“死循环”中尝试获取同步状态，而只有前驱节点是头节点才能够尝试获取同步状态，这是为什么？原因有两个，如下。

第一，头节点是成功获取到同步状态的节点，而头节点的线程释放了同步状态之后，将会唤醒其后继节点，后继节点的线程被唤醒后需要检查自己的前驱节点是否是头节点。

第二，维护同步队列的FIFO原则。该方法中，节点自旋获取同步状态的行为如图5-4所示。

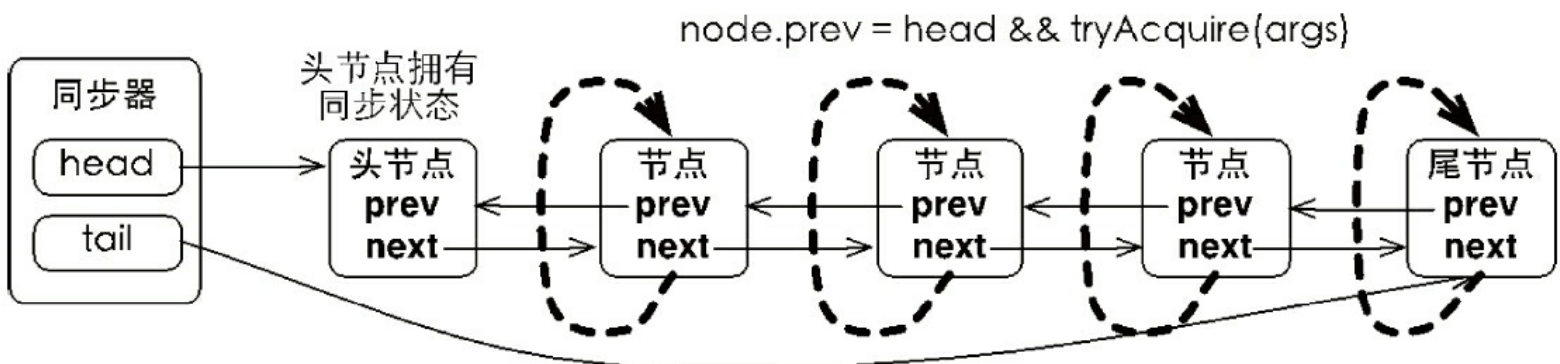


图5-4 节点自旋获取同步状态

在图5-4中，由于非首节点线程前驱节点出队或者被中断而从等待状态返回，随后检查自己的前驱是否是头节点，如果是则尝试获取同步状态。可以看到节点和节点之间在循环检查的过程中基本不相互通信，而是简单地判断自己的前驱是否为头节点，这样就使得节点的释放规则符合FIFO，并且也便于对过早通知的处理(过早通知是指前驱节点不是头节点的线程由于中断而被唤醒)。

独占式同步状态获取流程，也就是acquire(int arg)方法调用流程，如图5-5所示。

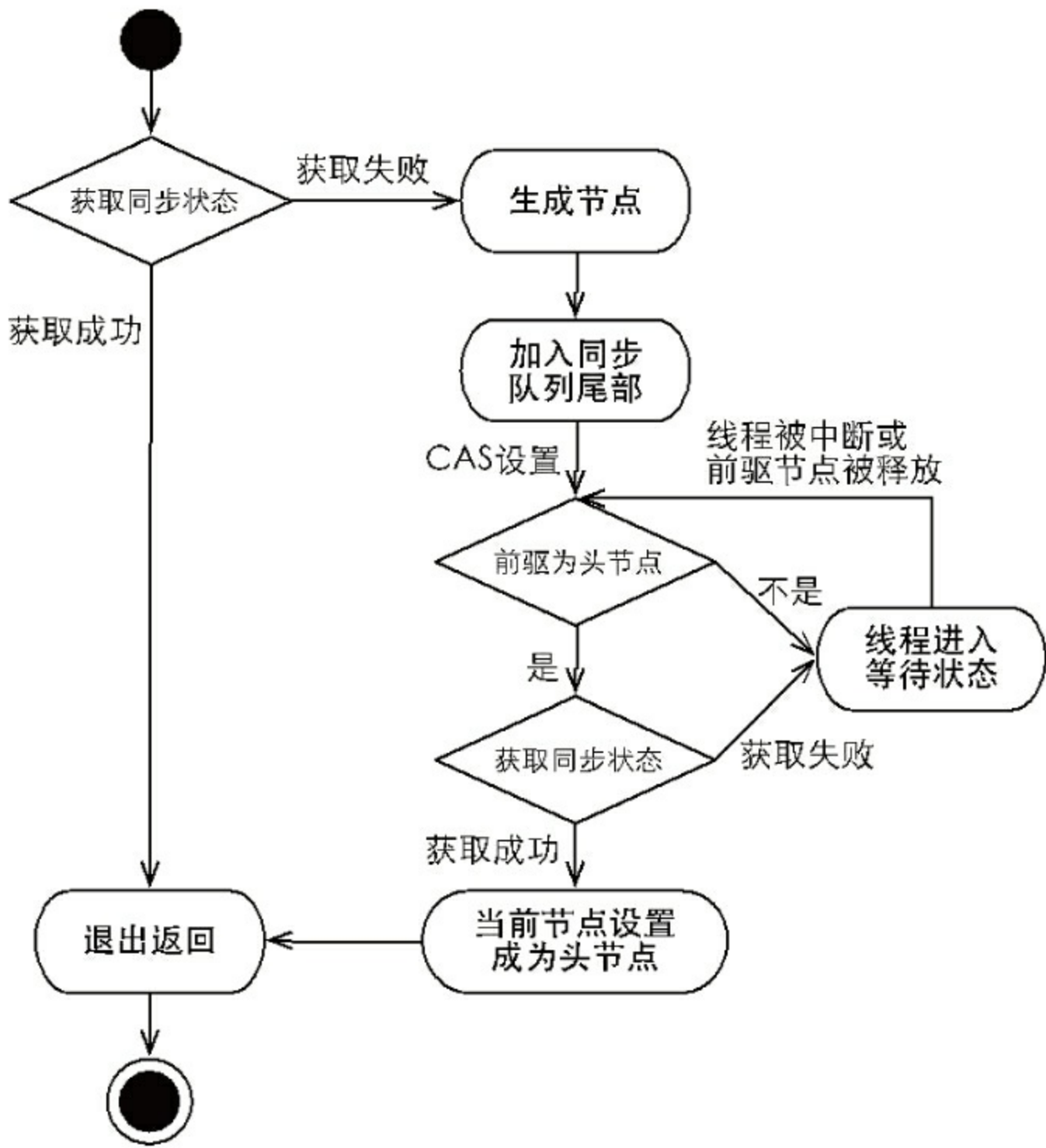


图5-5 独占式同步状态获取流程

在图5-5中，前驱节点为头节点且能够获取同步状态的判断条件和线程进入等待状态是获取同步状态的自旋过程。当同步状态获取成功之后，当前线程从acquire(int arg)方法返回，如果对于锁这种并发组件而言，代表着当前线程获取了锁。

当前线程获取同步状态并执行了相应逻辑之后，就需要释放同步状态，使得后续节点能够继续获取同步状态。通过调用同步器的release(int arg)方法可以释放同步状态，该方法在释放了同步状态之后，会唤醒其后继节点(进而使后继节点重新尝试获取同步状态)。该方法代码如代码清单5-6所示。

代码清单5-6 同步器的release方法

```
public final boolean release(int arg) {
    if (tryRelease(arg)) {
        Node h = head;
        if (h != null && h.waitStatus != 0)
            unparkSuccessor(h);
        return true;
    }
    return false;
}
```

该方法执行时，会唤醒头节点的后继节点线程，unparkSuccessor(Node node)方法使用LockSupport(在后面的章节会专门介绍)来唤醒处于等待状态的线程。

分析了独占式同步状态获取和释放过程后，适当做个总结：在获取同步状态时，同步器维护一个同步队列，获取状态失败的线程都会被加入到队列中并在队列中进行自旋；移出队列(或停止自旋)的条件是前驱节点为头节点且成功获取了同步状态。在释放同步状态时，同步器调用tryRelease(int arg)方法释放同步状态，然后唤醒头节点的后继节点。

3.共享式同步状态获取与释放

共享式获取与独占式获取最主要的区别在于同一时刻能否有多个线程同时获取到同步状态。以文件的读写为例，如果一个程序在对文件进行读操作，那么这一时刻对于该文件的写操作均被阻塞，而读操作能够同时进行。写操作要求对资源的独占式访问，而读操作可以是共享式访问，两种不同的访问模式在同一时刻对文件或资源的访问情况，如图5-6所示。

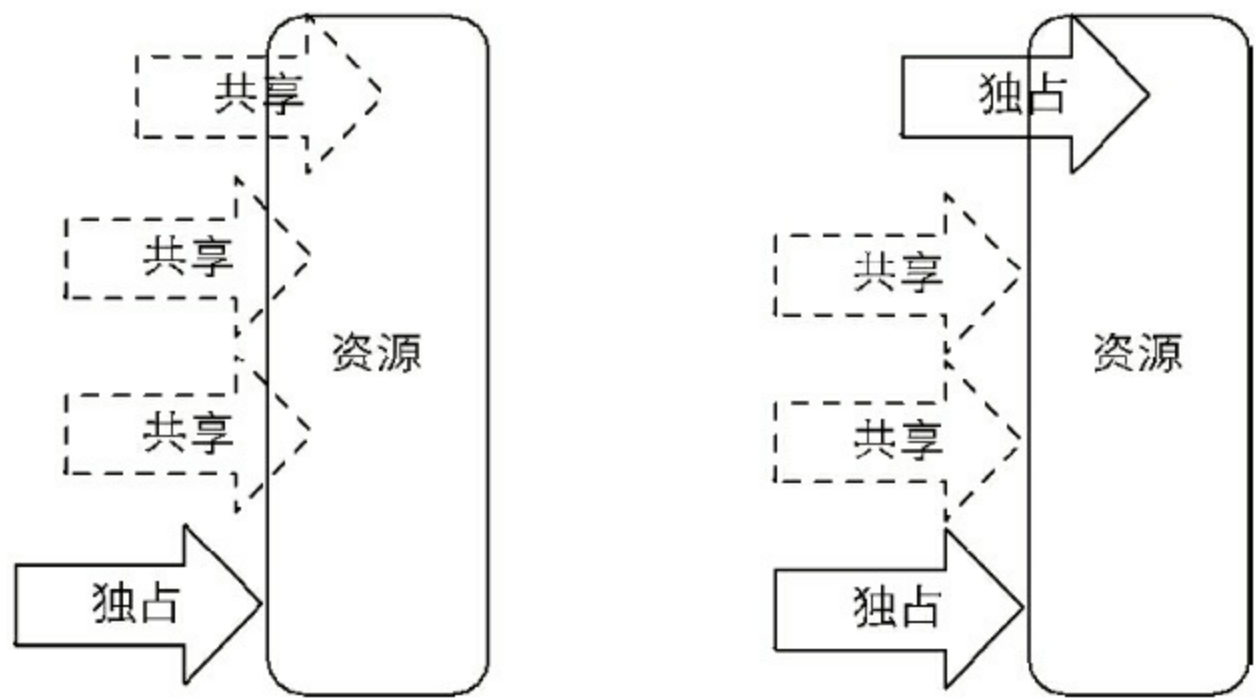


图5-6 共享式与独占式访问资源的对比

在图5-6中，左半部分，共享式访问资源时，其他共享式的访问均被允许，而独占式访问被阻塞，右半部分是独占式访问资源时，同一时刻其他访问均被阻塞。

通过调用同步器的acquireShared(int arg)方法可以共享式地获取同步状态，该方法代码如代码清单5-7所示。

代码清单5-7 同步器的acquireShared和doAcquireShared方法

```
public final void acquireShared(int arg) {
    if (tryAcquireShared(arg) < 0)
        doAcquireShared(arg);
}
private void doAcquireShared(int arg) {
    final Node node = addWaiter(Node.SHARED);
    boolean failed = true;
```

```

try {
    boolean interrupted = false;
    for (;;) {
        final Node p = node.predecessor();
        if (p == head) {
            int r = tryAcquireShared(arg);
            if (r >= 0) {
                setHeadAndPropagate(node, r);
                p.next = null;
                if (interrupted)
                    selfInterrupt();
                failed = false;
                return;
            }
        }
        if (shouldParkAfterFailedAcquire(p, node) &&
            parkAndCheckInterrupt())
            interrupted = true;
    }
} finally {
    if (failed)
        cancelAcquire(node);
}
}

```

在acquireShared(int arg)方法中，同步器调用tryAcquireShared(int arg)方法尝试获取同步状态，tryAcquireShared(int arg)方法返回值为int类型，当返回值大于等于0时，表示能够获取到同步状态。因此，在共享式获取的自旋过程中，成功获取到同步状态并退出自旋的条件就是tryAcquireShared(int arg)方法返回值大于等于0。可以看到，在doAcquireShared(int arg)方法的自旋过程中，如果当前节点的前驱为头节点时，尝试获取同步状态，如果返回值大于等于0，表示该次获取同步状态成功并从自旋过程中退出。

与独占式一样，共享式获取也需要释放同步状态，通过调用releaseShared(int arg)方法可以释放同步状态，该方法代码如代码清单5-8所示。

代码清单5-8 同步器的releaseShared方法

```

public final boolean releaseShared(int arg) {
    if (tryReleaseShared(arg)) {
        doReleaseShared();
        return true;
    }
    return false;
}

```

该方法在释放同步状态之后，将会唤醒后续处于等待状态的节点。对于能够支持多个线程同时访问的并发组件(比如Semaphore)，它和独占式主要区别在于tryReleaseShared(int arg)方法必须确保同步状态(或者资源数)线程安全释放，一般是通过循环和CAS来保证的，因为释放同步状态的操作会同时来自多个线程。

4.独占式超时获取同步状态

通过调用同步器的doAcquireNanos(int arg,long nanosTimeout)方法可以超时获取同步状态，即在指定的时间段内获取同步状态，如果获取到同步状态则返回true，否则，返回false。该方法提供了传统Java同步操作(比如synchronized关键字)所不具备的特性。

在分析该方法的实现前，先介绍一下响应中断的同步状态获取过程。在Java 5之前，当一个线程获取不到锁而被阻塞在synchronized之外时，对该线程进行中断操作，此时该线程的中断标志位会被修改，但线程依旧会阻塞在synchronized上，等待着获取锁。在Java 5中，同步器提供了acquireInterruptibly(int arg)方法，这个方法在等待获取同步状态时，如果当前线程被中断，会立刻返回，并抛出InterruptedException。

超时获取同步状态过程可以被视作响应中断获取同步状态过程的“增强版”，doAcquireNanos(int arg,long nanosTimeout)方法在支持响应中断的基础上，增加了超时获取的特性。针对超时获取，主要需要计算出需要睡眠的时间间隔nanosTimeout，为了防止过早通知，nanosTimeout计算公式为： $\text{nanosTimeout} = \text{now} - \text{lastTime}$ ，其中now为当前唤醒时间，lastTime为上次唤醒时间，如果nanosTimeout大于0则表示超时时间未到，需要继续睡眠nanosTimeout纳秒，反之，表示已经超时，该方法代码如代码清单5-9所示。

代码清单5-9 同步器的doAcquireNanos方法

```
private boolean doAcquireNanos(int arg, long nanosTimeout)
throws InterruptedException {
    long lastTime = System.nanoTime();
```

```

final Node node = addWaiter(Node.EXCLUSIVE);
boolean failed = true;
try {
    for (;;) {
        final Node p = node.predecessor();
        if (p == head && tryAcquire(arg)) {
            setHead(node);
            p.next = null; // help GC
            failed = false;
            return true;
        }
        if (nanosTimeout <= 0)
            return false;
        if (shouldParkAfterFailedAcquire(p, node)
            && nanosTimeout > spinForTimeoutThreshold)
            LockSupport.parkNanos(this, nanosTimeout);
        long now = System.nanoTime();
        //计算时间, 当前时间now减去睡眠之前的时间lastTime得到已经睡眠
        //的时间delta, 然后被原有超时时间nanosTimeout减去, 得到了
        //还应该睡眠的时间
        nanosTimeout -= now - lastTime;
        lastTime = now;
        if (Thread.interrupted())
            throw new InterruptedException();
    }
} finally {
    if (failed)
        cancelAcquire(node);
}
}

```

该方法在自旋过程中，当节点的前驱节点为头节点时尝试获取同步状态，如果获取成功则从该方法返回，这个过程和独占式同步获取的过程类似，但是在同步状态获取失败的处理上有所不同。如果当前线程获取同步状态失败，则判断是否超时（nanosTimeout小于等于0表示已经超时），如果没有超时，重新计算超时时间间隔nanosTimeout，然后使当前线程等待nanosTimeout纳秒（当已到设置的超时时间，该线程会从LockSupport.parkNanos(Object blocker, long nanos)方法返回）。

如果nanosTimeout小于等于spinForTimeoutThreshold(1000纳秒)时，将不会使该线程进行超时等待，而是进入快速的自旋过程。原因在于，非常短的超时等待无法做到十分精确，如果这时再进行超时等待，相反会让nanosTimeout的超时从整体上表现得反而不精确。因此，在超时非常短的场景下，同步器会进入无条件的快速自旋。

独占式超时获取同步态的流程如图5-7所示。

从图5-7中可以看出，独占式超时获取同步状态doAcquireNanos(int arg,long nanosTimeout)和独占式获取同步状态acquire(int args)在流程上非常相似，其主要区别在于未获取到同步状态时的处理逻辑。acquire(int args)在未获取到同步状态时，将会使当前线程一直处于等待状态，而doAcquireNanos(int arg,long nanosTimeout)会使当前线程等待nanosTimeout纳秒，如果当前线程在nanosTimeout纳秒内没有获取到同步状态，将会从等待逻辑中自动返回。

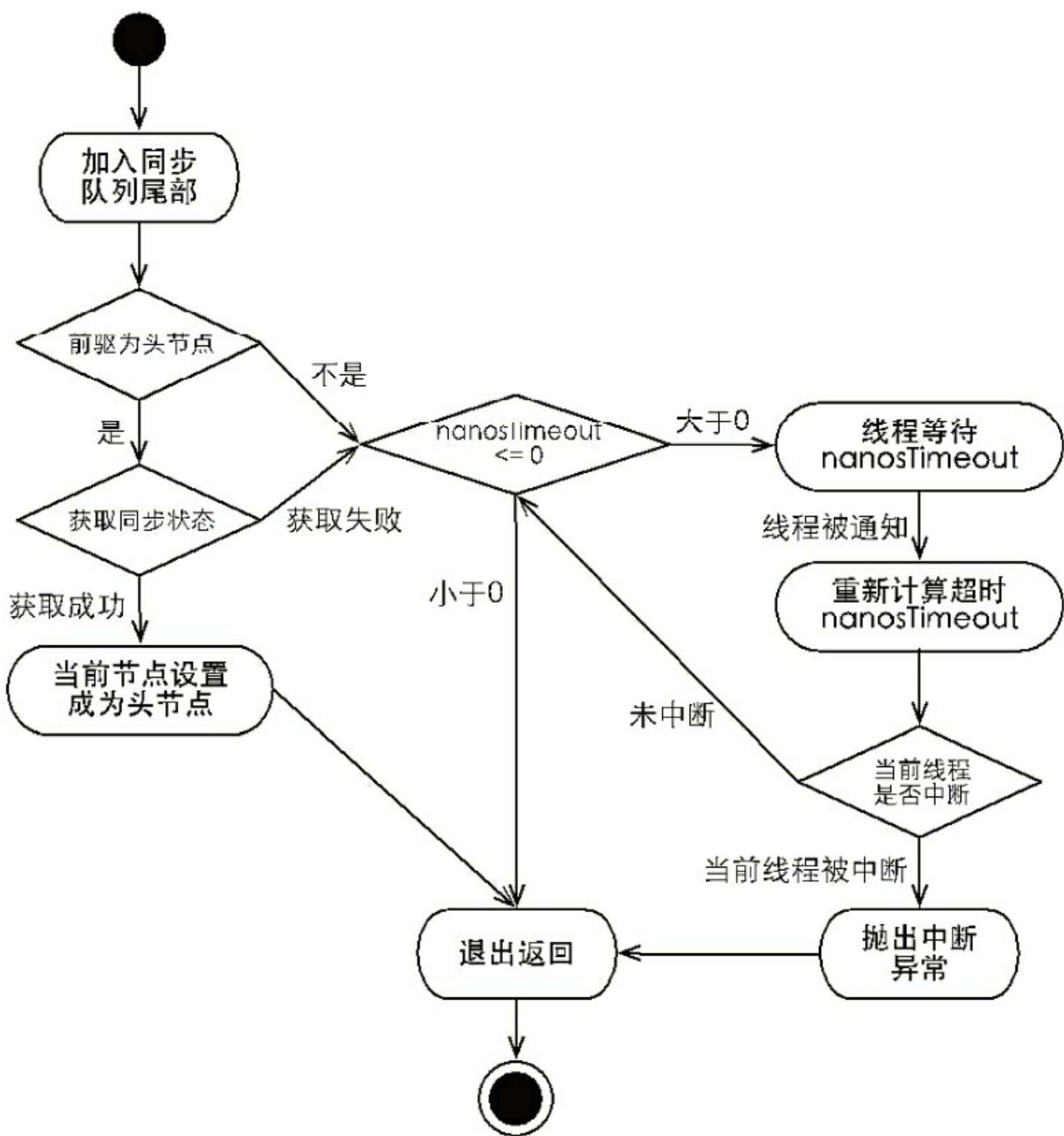


图5-7 独占式超时获取同步状态的流程

5.自定义同步组件——TwinsLock

在前面的章节中，对同步器AbstractQueuedSynchronizer进行了实现层面的分析，本节通过

编写一个自定义同步组件来加深对同步器的理解。

设计一个同步工具:该工具在同一时刻,只允许至多两个线程同时访问,超过两个线程的访问将被阻塞,我们将这个同步工具命名为TwinsLock。

首先,确定访问模式。TwinsLock能够在同一时刻支持多个线程的访问,这显然是共享式访问,因此,需要使用同步器提供的acquireShared(int args)方法等和Shared相关的方法,这就要求TwinsLock必须重写tryAcquireShared(int args)方法和tryReleaseShared(int args)方法,这样才能保证同步器的共享式同步状态的获取与释放方法得以执行。

其次,定义资源数。TwinsLock在同一时刻允许至多两个线程的同时访问,表明同步资源数为2,这样可以设置初始状态status为2,当一个线程进行获取, status减1,该线程释放,则status加1,状态的合法范围为0、1和2,其中0表示当前已经有两个线程获取了同步资源,此时再有其他线程对同步状态进行获取,该线程只能被阻塞。在同步状态变更时,需要使用compareAndSet(int expect,int update)方法做原子性保障。

最后,组合自定义同步器。前面的章节提到,自定义同步组件通过组合自定义同步器来完成同步功能,一般情况下自定义同步器会被定义为自定义同步组件的内部类。

TwinsLock(部分)代码如代码清单5-10所示。

代码清单5-10 TwinsLock.java

```
public class TwinsLock implements Lock {
    private final Sync sync = new Sync(2);
    private static final class Sync extends AbstractQueuedSynchronizer {
        Sync(int count) {
            if (count <= 0) {
                throw new IllegalArgumentException("count must large than zero.");
            }
            setState(count);
        }
        public int tryAcquireShared(int reduceCount) {
            for (;;) {
                int current = getState();
```

```

        int newCount = current - reduceCount;
        if (newCount < 0 || compareAndSetState(current,
            newCount)) {
            return newCount;
        }
    }
}

public boolean tryReleaseShared(int returnCount) {
    for (;;) {
        int current = getState();
        int newCount = current + returnCount;
        if (compareAndSetState(current, newCount)) {
            return true;
        }
    }
}

public void lock() {
    sync.acquireShared(1);
}

public void unlock() {
    sync.releaseShared(1);
}

// 其他接口方法略
}

```

在上述示例中，TwinsLock实现了Lock接口，提供了面向使用者的接口，使用者调用lock()方法获取锁，随后调用unlock()方法释放锁，而同一时刻只能有两个线程同时获取到锁。TwinsLock同时包含了一个自定义同步器Sync，而该同步器面向线程访问和同步状态控制。以共享式获取同步状态为例：同步器会先计算出获取后的同步状态，然后通过CAS确保状态的正确设置，当tryAcquireShared(int reduceCount)方法返回值大于等于0时，当前线程才获取同步状态，对于上层的TwinsLock而言，则表示当前线程获得了锁。

同步器作为一个桥梁，连接线程访问以及同步状态控制等底层技术与不同并发组件（比如Lock、CountDownLatch等）的接口语义。

下面编写一个测试来验证TwinsLock是否能按照预期工作。在测试用例中，定义了工作者线程Worker，该线程在执行过程中获取锁，当获取锁之后使当前线程睡眠1秒（并不释放锁），随后打印当前线程名称，最后再次睡眠1秒并释放锁，测试用例如代码清单5-11所示。

```
public class TwinsLockTest {
    @Test
    public void test() {
        final Lock lock = new TwinsLock();
        class Worker extends Thread {
            public void run() {
                while (true) {
                    lock.lock();
                    try {
                        SleepUtils.second(1);
                        System.out.println(Thread.currentThread().getName());
                        SleepUtils.second(1);
                    } finally {
                        lock.unlock();
                    }
                }
            }
        }
        // 启动10个线程
        for (int i = 0; i < 10; i++) {
            Worker w = new Worker();
            w.setDaemon(true);
            w.start();
        }
        // 每隔1秒换行
        for (int i = 0; i < 10; i++) {
            SleepUtils.second(1);
            System.out.println();
        }
    }
}
```

运行该测试用例，可以看到线程名称成对输出，也就是在同一时刻只有两个线程能够获取到锁，这表明TwinsLock可以按照预期正确工作。

5.3 重入锁

重入锁ReentrantLock, 顾名思义, 就是支持重进入的锁, 它表示该锁能够支持一个线程对资源的重复加锁。除此之外, 该锁的还支持获取锁时的公平和非公平性选择。

回忆在同步器一节中的示例(Mutex), 同时考虑如下场景: 当一个线程调用Mutex的lock()方法获取锁之后, 如果再次调用lock()方法, 则该线程将会被自己所阻塞, 原因是Mutex在实现tryAcquire(int acquires)方法时没有考虑占有锁的线程再次获取锁的场景, 而在调用tryAcquire(int acquires)方法时返回了false, 导致该线程被阻塞。简单地说, Mutex是一个不支持重进入的锁。而synchronized关键字隐式的支持重进入, 比如一个synchronized修饰的递归方法, 在方法执行时, 执行线程在获取了锁之后仍能连续多次地获得该锁, 而不像Mutex由于获取了锁, 而在下一次获取锁时出现阻塞自己的情况。

ReentrantLock虽然没能像synchronized关键字一样支持隐式的重进入, 但是在调用lock()方法时, 已经获取到锁的线程, 能够再次调用lock()方法获取锁而不被阻塞。

这里提到一个锁获取的公平性问题, 如果在绝对时间上, 先对锁进行获取的请求一定先被满足, 那么这个锁是公平的, 反之, 是不公平的。公平的获取锁, 也就是等待时间最长的线程最优先获取锁, 也可以说锁获取是顺序的。ReentrantLock提供了一个构造函数, 能够控制锁是否是公平的。

事实上, 公平的锁机制往往没有不公平的效率高, 但是, 并不是任何场景都是以TPS作为唯一的指标, 公平锁能够减少“饥饿”发生的概率, 等待越久的请求越是能够得到优先满足。

下面将着重分析ReentrantLock是如何实现重进入和公平性获取锁的特性, 并通过测试来验证公平性获取锁对性能的影响。

1.实现重进入

重进入是指任意线程在获取到锁之后能够再次获取该锁而不会被锁所阻塞，该特性的实现需要解决以下两个问题。

1) **线程再次获取锁**。锁需要去识别获取锁的线程是否为当前占据锁的线程，如果是，则再次成功获取。

2) **锁的最终释放**。线程重复n次获取了锁，随后在第n次释放该锁后，其他线程能够获取到该锁。锁的最终释放要求锁对于获取进行计数自增，计数表示当前锁被重复获取的次数，而锁被释放时，计数自减，当计数等于0时表示锁已经成功释放。

ReentrantLock是通过组合自定义同步器来实现锁的获取与释放，以非公平性(默认的)实现为例，获取同步状态的代码如代码清单5-12所示。

代码清单5-12 ReentrantLock的nonfairTryAcquire方法

```
final boolean nonfairTryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        if (compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    } else if (current == getExclusiveOwnerThread()) {
        int nextc = c + acquires;
        if (nextc < 0)
            throw new Error("Maximum lock count exceeded");
        setState(nextc);
        return true;
    }
    return false;
}
```

该方法增加了再次获取同步状态的处理逻辑：通过判断当前线程是否为获取锁的线程来决定获取操作是否成功，如果是获取锁的线程再次请求，则将同步状态值进行增加并返回true，表示获取同步状态成功。

成功获取锁的线程再次获取锁，只是增加了同步状态值，这也就要求ReentrantLock在释放同步状态时减少同步状态值，该方法的代码如代码清单5-13所示。

代码清单5-13 ReentrantLock的tryRelease方法

```
protected final boolean tryRelease(int releases) {
    int c = getState() - releases;
    if (Thread.currentThread() != getExclusiveOwnerThread())
        throw new IllegalMonitorStateException();
    boolean free = false;
    if (c == 0) {
        free = true;
        setExclusiveOwnerThread(null);
    }
    setState(c);
    return free;
}
```

如果该锁被获取了n次，那么前(n-1)次tryRelease(int releases)方法必须返回false，而只有同步状态完全释放了，才能返回true。可以看到，该方法将同步状态是否为0作为最终释放的条件，当同步状态为0时，将占有线程设置为null，并返回true，表示释放成功。

2.公平与非公平获取锁的区别

公平性与否是针对获取锁而言的，如果一个锁是公平的，那么锁的获取顺序就应该符合请求的绝对时间顺序，也就是FIFO。

回顾上一小节中介绍的nonfairTryAcquire(int acquires)方法，对于非公平锁，只要CAS设置同步状态成功，则表示当前线程获取了锁，而公平锁则不同，如代码清单5-14所示。

代码清单5-14 ReentrantLock的tryAcquire方法

```
protected final boolean tryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        if (!hasQueuedPredecessors() && compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
        }
    }
}
```

```

        return true;
    }
} else if (current == getExclusiveOwnerThread()) {
    int nexttc = c + acquires;
    if (nexttc < 0)
        throw new Error("Maximum lock count exceeded");
    setState(nexttc);
    return true;
}
return false;
}
}

```

该方法与`nonfairTryAcquire(int acquires)`比较, 唯一不同的位置为判断条件多了

`hasQueuedPredecessors()`方法, 即加入了同步队列中当前节点是否有前驱节点的判断, 如果该方法返回`true`, 则表示有线程比当前线程更早地请求获取锁, 因此需要等待前驱线程获取并释放锁之后才能继续获取锁。

下面编写一个测试来观察公平和非公平锁在获取锁时的区别, 在测试用例中定义了内部类`ReentrantLock2`, 该类主要公开了`getQueuedThreads()`方法, 该方法返回正在等待获取锁的线程列表, 由于列表是逆序输出, 为了方便观察结果, 将其进行反转, 测试用例(部分)如代码清单5-15所示。

代码清单5-15 FairAndUnfairTest.java

```

public class FairAndUnfairTest {
    private static Lock    fairLock        = new ReentrantLock2(true);
    private static Lock    unfairLock      = new ReentrantLock2(false);
    @Test
    public void fair() {
        testLock(fairLock);
    }
    @Test
    public void unfair() {
        testLock(unfairLock);
    }
    private void testLock(Lock lock) {
        // 启动5个Job(略)
    }
    private static class Job extends Thread {
        private Lock    lock;
        public Job(Lock lock) {
            this.lock = lock;
        }
        public void run() {

```

```
        // 连续2次打印当前的Thread和等待队列中的Thread(略)
    }
}
private static class ReentrantLock2 extends ReentrantLock {
    public ReentrantLock2(boolean fair) {
        super(fair);
    }
    public Collection<Thread> getQueuedThreads() {
        List<Thread> arrayList = new ArrayList<Thread>(super.
            getQueuedThreads());
        Collections.reverse(arrayList);
        return arrayList;
    }
}
}
```

分别运行fair()和unfair()两个测试方法，输出结果如表5-6所示。

表5-6 fair()和unfair()两个测试方法的输出结果

Fair (公平性锁)	Unfair (非公平性锁)
Lock by [4], Waiting by [0]	Lock by [4], Waiting by [0, 1, 2, 3]
Lock by [0], Waiting by [1, 2, 3, 4]	Lock by [4], Waiting by [0, 1, 2, 3]
Lock by [1], Waiting by [2, 3, 4, 0]	Lock by [0], Waiting by [1, 2, 3]
Lock by [2], Waiting by [3, 4, 0, 1]	Lock by [0], Waiting by [1, 2, 3]
Lock by [3], Waiting by [4, 0, 1, 2]	Lock by [1], Waiting by [2, 3]
Lock by [4], Waiting by [0, 1, 2, 3]	Lock by [1], Waiting by [2, 3]
Lock by [0], Waiting by [1, 2, 3]	Lock by [2], Waiting by [3]
Lock by [1], Waiting by [2, 3]	Lock by [2], Waiting by [3]
Lock by [2], Waiting by [3]	Lock by [3], Waiting by []
Lock by [3], Waiting by []	Lock by [3], Waiting by []

观察表5-6所示的结果(其中每个数字代表一个线程)，公平性锁每次都是从同步队列中的第一个节点获取到锁，而非公平性锁出现了一个线程连续获取锁的情况。

为什么会出现线程连续获取锁的情况呢？回顾nonfairTryAcquire(int acquires)方法，当一个线程请求锁时，只要获取了同步状态即成功获取锁。在这个前提下，刚释放锁的线程再次获取同步状态的几率会非常大，使得其他线程只能在同步队列中等待。

非公平性锁可能使线程“饥饿”，为什么它又被设定成默认的实现呢？再次观察上表的结

果，如果把每次不同线程获取到锁定义为1次切换，公平性锁在测试中进行了10次切换，而非公平性锁只有5次切换，这说明非公平性锁的开销更小。下面运行测试用例(测试环境:ubuntu server 14.04 i5-34708GB, 测试场景:10个线程, 每个线程获取100000次锁), 通过vmstat统计测试运行时系统线程上下文切换的次数, 运行结果如表5-7所示。

表5-7 公平性和非公平性在系统线程上下文切换方面的对比

对比项	Fair (公平性锁)	Unfair (非公平性锁)
切换次数 (每秒间隔)	187	159
	40 163	330
	350 577	14 390
	348 637	159
	349 682	
	349 994	
	354 223	
	211 737	
	183	
总共耗时 (单位: 毫秒)	5 754	61

在测试中公平性锁与非公平性锁相比，总耗时是其94.3倍，总切换次数是其133倍。可以看出，公平性锁保证了锁的获取按照FIFO原则，而代价是进行大量的线程切换。非公平性锁虽然可能造成线程“饥饿”，但极少的线程切换，保证了其更大的吞吐量。

5.4 读写锁

之前提到锁(如Mutex和ReentrantLock)基本都是排他锁,这些锁在同一时刻只允许一个线程进行访问,而读写锁在同一时刻可以允许多个读线程访问,但是在写线程访问时,所有的读线程和其他写线程均被阻塞。读写锁维护了一对锁,一个读锁和一个写锁,通过分离读锁和写锁,使得并发性相比一般的排他锁有了很大提升。

除了保证写操作对读操作的可见性以及并发性的提升之外,读写锁能够简化读写交互场景的编程方式。假设在程序中定义一个共享的用作缓存数据结构,它大部分时间提供读服务(例如查询和搜索),而写操作占有的时间很少,但是写操作完成之后的更新需要对后续的读服务可见。

在没有读写锁支持的(Java 5之前)时候,如果需要完成上述工作就要使用Java的等待通知机制,就是当写操作开始时,所有晚于写操作的读操作均会进入等待状态,只有写操作完成并进行通知之后,所有等待的读操作才能继续执行(写操作之间依靠synchronized关键进行同步),这样做的目的是使读操作能读取到正确的数据,不会出现脏读。改用读写锁实现上述功能,只需要在读操作时获取读锁,写操作时获取写锁即可。当写锁被获取到时,后续(非当前写操作线程)的读写操作都会被阻塞,写锁释放之后,所有操作继续执行,编程方式相对于使用等待通知机制的实现方式而言,变得简单明了。

一般情况下,读写锁的性能都会比排它锁好,因为大多数场景读是多于写的。在读多于写的情况下,读写锁能够提供比排它锁更好的并发性和吞吐量。Java并发包提供读写锁的实现是ReentrantReadWriteLock,它提供的特性如表5-8所示。

表5-8 ReentrantReadWriteLock的特性

特 性	说 明
公平性选择	支持非公平（默认）和公平的锁获取方式，吞吐量还是非公平优于公平
重进入	该锁支持重进入，以读写线程为例：读线程在获取了读锁之后，能够再次获取读锁。而写线程在获取了写锁之后能够再次获取写锁，同时也可以获取读锁
锁降级	遵循获取写锁、获取读锁再释放写锁的次序，写锁能够降级成为读锁

5.4.1 读写锁的接口与示例

ReadWriteLock仅定义了获取读锁和写锁的两个方法，即readLock()方法和writeLock()方法，而其实现——ReentrantReadWriteLock，除了接口方法之外，还提供了一些便于外界监控其内部工作状态的方法，这些方法以及描述如表5-9所示。

表5-9 ReentrantReadWriteLock展示内部工作状态的方法

方法名称	描 述
int getReadLockCount()	返回当前读锁被获取的次数。该次数不等于获取读锁的线程数，例如，仅一个线程，它连续获取（重进入）了 n 次读锁，那么占据读锁的线程数是 1，但该方法返回 n
int getReadHoldCount()	返回当前线程获取读锁的次数。该方法在 Java 6 中加入到 ReentrantReadWriteLock 中，使用 ThreadLocal 保存当前线程获取的次数，这也使得 Java 6 的实现变得更加复杂
boolean isWriteLocked()	判断写锁是否被获取
int getWriteHoldCount()	返回当前写锁被获取的次数

接下来，通过一个缓存示例说明读写锁的使用方式，示例代码如代码清单5-16所示。

代码清单5-16 Cache.java

```
public class Cache {
    static Map<String, Object> map = new HashMap<String, Object>();
    static ReentrantReadWriteLock rwl = new ReentrantReadWriteLock();
    static Lock r = rwl.readLock();
    static Lock w = rwl.writeLock();
    // 获取一个key对应的value
    public static final Object get(String key) {
        r.lock();
        try {
            return map.get(key);
        } finally {
            r.unlock();
        }
    }
    // 设置key对应的value, 并返回旧的value
    public static final Object put(String key, Object value) {
        w.lock();
        try {
            return map.put(key, value);
        } finally {
            w.unlock();
        }
    }
}
```

```
// 清空所有的内容
public static final void clear() {
    w.lock();
    try {
        map.clear();
    } finally {
        w.unlock();
    }
}
```

上述示例中，Cache组合一个非线程安全的HashMap作为缓存的实现，同时使用读写锁的读锁和写锁来保证Cache是线程安全的。在读操作get(String key)方法中，需要获取读锁，这使得并发访问该方法时不会被阻塞。写操作put(String key,Object value)方法和clear()方法，在更新HashMap时必须提前获取写锁，当获取写锁后，其他线程对于读锁和写锁的获取均被阻塞，而只有写锁被释放之后，其他读写操作才能继续。Cache使用读写锁提升读操作的并发性，也保证每次写操作对所有的读写操作的可见性，同时简化了编程方式。

5.4.2 读写锁的实现分析

接下来分析ReentrantReadWriteLock的实现，主要包括：读写状态的设计、写锁的获取与释放、读锁的获取与释放以及锁降级(以下没有特别说明读写锁均可认为是ReentrantReadWriteLock)。

1.读写状态的设计

读写锁同样依赖自定义同步器来实现同步功能，而读写状态就是其同步器的同步状态。回想ReentrantLock中自定义同步器的实现，同步状态表示锁被一个线程重复获取的次数，而读写锁的自定义同步器需要在同步状态(一个整型变量)上维护多个读线程和一个写线程的状态，使得该状态的设计成为读写锁实现的关键。

如果在一个整型变量上维护多种状态，就一定需要“按位切割使用”这个变量，读写锁将变量切分成了两个部分，高16位表示读，低16位表示写，划分方式如图5-8所示。

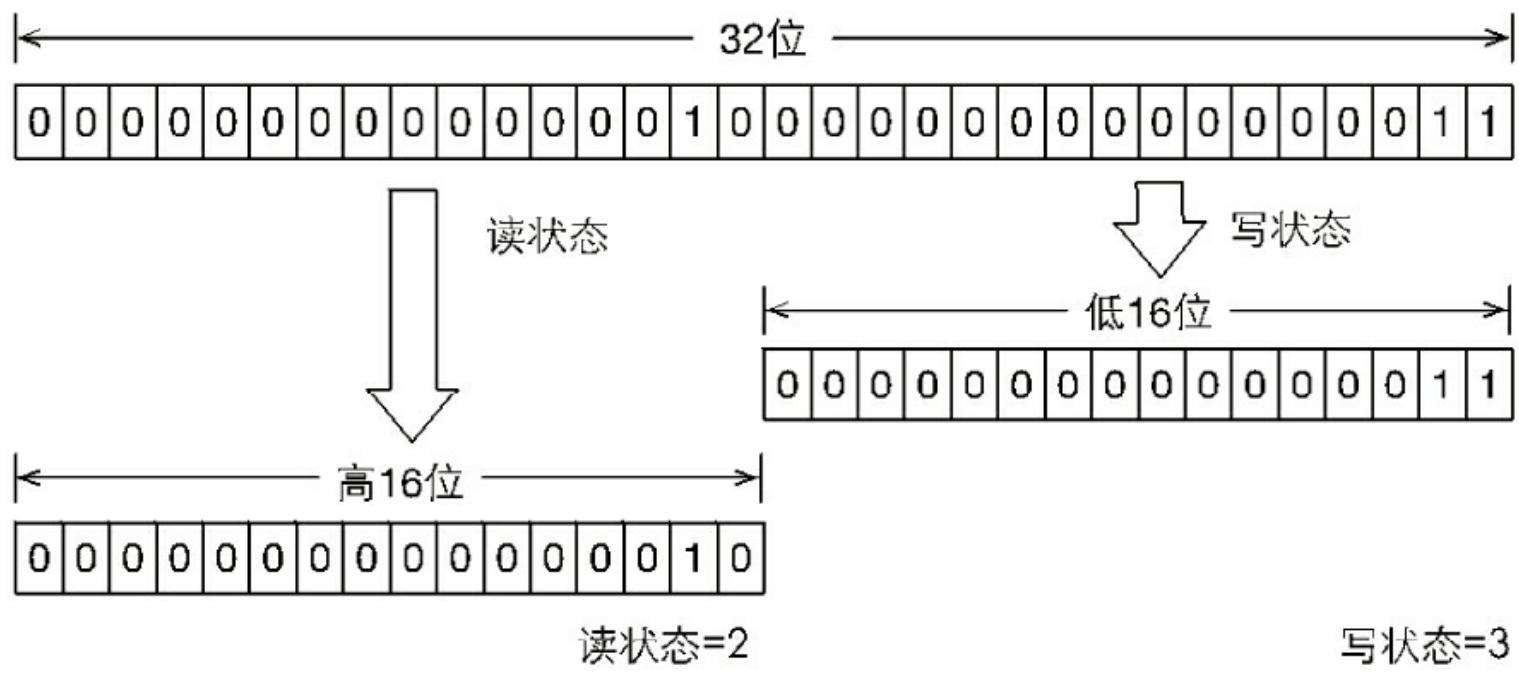


图5-8 读写锁状态的划分方式

当前同步状态表示一个线程已经获取了写锁，且重进入了两次，同时也连续获取了两次

读锁。读写锁是如何迅速确定读和写各自的状态呢？答案是通过位运算。假设当前同步状态值为S，写状态等于 $S \& 0x0000FFFF$ (将高16位全部抹去)，读状态等于 $S \gg 16$ (无符号补0右移16位)。当写状态增加1时，等于 $S+1$ ，当读状态增加1时，等于 $S+(1 \ll 16)$ ，也就是 $S+0x00010000$ 。

1. 锁状态state高16位表示读锁，低16位表示写锁。
2. 读锁被获取一次就将读状态加1操作，即state高16位加1操作。
3. 写锁被获取一次就将写状态加1操作，即state低16位加1操作。

根据状态的划分能得出一个推论：S不等于0时，当写状态($S \& 0x0000FFFF$)等于0时，则读状态($S \gg 16$)大于0，即读锁已被获取。

2.写锁的获取与释放

写锁是一个支持重进入的排它锁。如果当前线程已经获取了写锁，则增加写状态。如果当

前线程在获取写锁时，读锁已经被获取(读状态不为0)或者该线程不是已经获取写锁的线程，

则当前线程进入等待状态，获取写锁的代码如代码清单5-17所示。

代码清单5-17 ReentrantReadWriteLock的tryAcquire方法

```
protected final boolean tryAcquire(int acquires) {
    Thread current = Thread.currentThread();
    int c = getState();
    int w = exclusiveCount(c);
    if (c != 0) {
        // 存在读锁或者当前获取线程不是已经获取写锁的线程
        if (w == 0 || current != getExclusiveOwnerThread())
            return false;
        if (w + exclusiveCount(acquires) > MAX_COUNT)
            throw new Error("Maximum lock count exceeded");
        setState(c + acquires);
        return true;
    }
    if (writerShouldBlock() || !compareAndSetState(c, c + acquires)) {
        return false;
    }
    setExclusiveOwnerThread(current);
    return true;
}
```

该方法除了重入条件(当前线程为获取了写锁的线程)之外，增加了一个读锁是否存在的

判断。如果存在读锁，则写锁不能被获取，原因在于：读写锁要确保写锁的操作对读锁可见，如果允许读锁在已被获取的情况下对写锁的获取，那么正在运行的其他读线程就无法感知到当前写线程的操作。因此，只有等待其他读线程都释放了读锁，写锁才能被当前线程获取，而写锁一旦被获取，则其他读写线程的后续访问均被阻塞。

写锁的释放与ReentrantLock的释放过程基本类似，每次释放均减少写状态，当写状态为0时表示写锁已被释放，从而等待的读写线程能够继续访问读写锁，同时前次写线程的修改对后续读写线程可见。

3.读锁的获取与释放

读锁是一个支持重进入的共享锁，它能够被多个线程同时获取，在没有其他写线程访问（或者写状态为0）时，读锁总会被成功地获取，而所做的也只是（线程安全的）增加读状态。如果当前线程已经获取了读锁，则增加读状态。如果当前线程在获取读锁时，写锁已被其他线程获取，则进入等待状态。获取读锁的实现从Java 5到Java 6变得复杂许多，主要原因是新增了一些功能，例如getReadHoldCount()方法，作用是返回当前线程获取读锁的次数。读状态是所有线程获取读锁次数的总和，而每个线程各自获取读锁的次数只能选择保存在ThreadLocal中，由线程自身维护，这使获取读锁的实现变得复杂。因此，这里将获取读锁的代码做了删减，保留必要的部分，如代码清单5-18所示。

代码清单5-18 ReentrantReadWriteLock的tryAcquireShared方法

```
protected final int tryAcquireShared(int unused) {
    for (;;) {
        int c = getState();
        int nextc = c + (1 << 16);
        if (nextc < c)
            throw new Error("Maximum lock count exceeded");
        if (exclusiveCount(c) != 0 && owner != Thread.currentThread())
            return -1;
        if (compareAndSetState(c, nextc))
            return 1;
    }
}
```


在tryAcquireShared(int unused)方法中, 如果其他线程已经获取了写锁, 则当前线程获取读锁失败, 进入等待状态。如果当前线程获取了写锁或者写锁未被获取, 则当前线程(线程安全, 依靠CAS保证)增加读状态, 成功获取读锁。

读锁的每次释放(线程安全的, 可能有多个读线程同时释放读锁)均减少读状态, 减少的值是(1<<16)。

4. 锁降级

锁降级指的是写锁降级成为读锁。如果当前线程拥有写锁, 然后将其释放, 最后再获取读锁, 这种分段完成的过程不能称之为锁降级。锁降级是指把持住(当前拥有的)写锁, 再获取到读锁, 随后释放(先前拥有的)写锁的过程。从现象上看好像是从写锁降级成为读锁, 但实际上读、写锁的状态记录是相互独立的, 不存在所谓的降级转换。

接下来看一个锁降级的示例。因为数据不常变化, 所以多个线程可以并发地进行数据处理, 当数据变更后, 如果当前线程感知到数据变化, 则进行数据的准备工作, 同时其他处理线程被阻塞, 直到当前线程完成数据的准备工作, 如代码清单5-19所示。

代码清单5-19 processData方法

```
public void processData() {
    readLock.lock();
    if (!update) {
        // 必须先释放读锁
        readLock.unlock();
        // 锁降级从写锁获取到开始
        writeLock.lock();
        try {
            if (!update) {
                // 准备数据的流程(略)
                update = true;
            }
            readLock.lock();
        } finally {
            writeLock.unlock();
        }
        // 锁降级完成, 写锁降级为读锁
    }
    try {
```

```
        // 使用数据的流程(略)
    } finally {
        readLock.unlock();
    }
}
```

上述示例中, 当数据发生变更后, update变量(布尔类型且volatile修饰)被设置为false, 此时所有访问processData()方法的线程都能够感知到变化, 但只有一个线程能够获取到写锁, 其他线程会被阻塞在读锁和写锁的lock()方法上。当前线程获取写锁完成数据准备之后, 再获取读锁, 随后释放写锁, 完成锁降级。

锁降级中读锁的获取是否必要呢? 答案是必要的。主要是为了保证数据的可见性, 如果当前线程不获取读锁而是直接释放写锁, 假设此刻另一个线程(记作线程T)获取了写锁并修改了数据, 那么当前线程无法感知线程T的数据更新。如果当前线程获取读锁, 即遵循锁降级的步骤, 则线程T将会被阻塞, 直到当前线程使用数据并释放读锁之后, 线程T才能获取写锁进行数据更新。

ReentrantReadWriteLock不支持锁升级(把持读锁、获取写锁, 最后释放读锁的过程)。目的也是保证数据可见性, 如果读锁已被多个线程获取, 其中任意线程成功获取了写锁并更新了数据, 则其更新对其他获取到读锁的线程是不可见的。

5.5 LockSupport工具

回顾5.2节，当需要阻塞或唤醒一个线程的时候，都会使用LockSupport工具类来完成相应工作。LockSupport定义了一组的公共静态方法，这些方法提供了最基本的线程阻塞和唤醒功能，而LockSupport也成为构建同步组件的基础工具。**park(Thread thread)调用后线程处于WAITING状态，不是BLOCKED**

LockSupport定义了一组以park开头的方法用来阻塞当前线程，以及unpark(Thread thread)方法来唤醒一个被阻塞的线程。Park有停车的意思，假设线程为车辆，那么park方法代表着停车，而unpark方法则是指车辆启动离开，这些方法以及描述如表5-10所示。

表5-10 LockSupport提供的阻塞和唤醒方法

方法名称	描 述
void park()	阻塞当前线程，如果调用 unpark(Thread thread) 方法或者当前线程被中断，才能从 park() 方法返回
void parkNanos(long nanos)	阻塞当前线程，最长不超过 nanos 纳秒，返回条件在 park() 的基础上增加了超时返回
void parkUntil(long deadline)	阻塞当前线程，直到 deadline 时间（从 1970 年开始到 deadline 时间的毫秒数）
void unpark(Thread thread)	唤醒处于阻塞状态的线程 thread

在Java 6中，LockSupport增加了park(Object blocker)、parkNanos(Object blocker,long nanos)和parkUntil(Object blocker,long deadline)3个方法，用于实现阻塞当前线程的功能，其中参数blocker是用来标识当前线程在等待的对象（以下称为阻塞对象），该对象主要用于问题排查和系统监控。

现将blocker保起来，提供dump信息

下面的示例中，将对比parkNanos(long nanos)方法和parkNanos(Object blocker,long nanos)方法来展示阻塞对象blocker的用处，代码片段和线程dump（部分）如表5-11所示。

从表5-11的线程dump结果可以看出，代码片段的内容都是阻塞当前线程10秒，但从线程dump结果可以看出，有阻塞对象的parkNanos方法能够传递给开发人员更多的现场信息。这是由于在Java 5之前，当线程阻塞（使用synchronized关键字）在一个对象上时，通过线程dump能够

查看到该线程的阻塞对象, 方便问题定位, 而Java 5推出的Lock等并发工具时却遗漏了这一点, 致使在线程dump时无法提供阻塞对象的信息。因此, 在Java 6中, LockSupport新增了上述3个含有阻塞对象的park方法, 用以替代原有的park方法。

表5-11 Blocker在线程dump中的作用

方法 对比项	parkNanos(long nanos)	parkNanos(Object blocker, long nanos)
代码片段	LockSupport.parkNanos(TimeUnit.SECONDS.toNanos(10));	LockSupport.parkNanos(this, TimeUnit.SECONDS.toNanos(10));
线程 dump 结果	"main" prio=5 tid= 0x00007fe773000800 nid= 0x1303 waiting on condition [0x000000010bb85000] java.lang.Thread.State: TIMED_WAITING (parking) at sun.misc.Unsafe. park(Native Method) at java.util.concurrent. locks.LockSupport. parkNanos(LockSupport.java:349)	"main" prio=5 tid=0x00007fd248805800 nid=0x1303 waiting on condition [0x000000010d75f000] java.lang.Thread.State: TIMED_WAITING (parking) at sun.misc.Unsafe.park(Native Method) - parking to wait for <0x00000007d593ec98> (a com.murdock.books.multithread.book. LockSupportTest) at java.util.concurrent.locks.LockSupport. parkNanos(LockSupport.java:226)

5.6 Condition接口

任意一个Java对象, 都拥有一组监视器方法(定义在java.lang.Object上), 主要包括wait()、wait(long timeout)、notify()以及notifyAll()方法, 这些方法与synchronized同步关键字配合, 可以实现等待/通知模式。Condition接口也提供了类似Object的监视器方法, 与Lock配合可以实现等待/通知模式, 但是这两者在使用方式以及功能特性上还是有差别的。

通过对比Object的监视器方法和Condition接口, 可以更详细地了解Condition的特性, 对比项与结果如表5-12所示。

表5-12 Object的监视器方法与Condition接口的对比

对比项	Object Monitor Methods	Condition
前置条件	获取对象的锁	调用 Lock.lock() 获取锁 调用 Lock.newCondition() 获取 Condition 对象
调用方式	直接调用 如: object.wait()	直接调用 如: condition.await()
等待队列个数	一个	多个
当前线程释放锁并进入等待状态	支持	支持
当前线程释放锁并进入等待状态, 在等待状态中不响应中断	不支持	支持
当前线程释放锁并进入超时等待状态	支持	支持
当前线程释放锁并进入等待状态到将来的某个时间	不支持	支持
唤醒等待队列中的一个线程	支持	支持
唤醒等待队列中的全部线程	支持	支持

5.6.1 Condition接口与示例

Condition定义了等待/通知两种类型的方法，当前线程调用这些方法时，需要提前获取到Condition对象关联的锁。Condition对象是由Lock对象(调用Lock对象的newCondition()方法)创建出来的，换句话说，Condition是依赖Lock对象的。

Condition的使用方式比较简单，需要注意在调用方法前获取锁，使用方式如代码清单5-20所示。

代码清单5-20 ConditionUseCase.java

```
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition();
public void conditionWait() throws InterruptedException {
    lock.lock();
    try {
        condition.await();
    } finally {
        lock.unlock();
    }
}
public void conditionSignal() throws InterruptedException {
    lock.lock();
    try {
        condition.signal();
    } finally {
        lock.unlock();
    }
}
```

如示例所示，一般都会将Condition对象作为成员变量。当调用await()方法后，当前线程会释放锁并在此等待，而其他线程调用Condition对象的signal()方法，通知当前线程后，当前线程才从await()方法返回，并且在返回前已经获取了锁。

Condition定义的(部分)方法以及描述如表5-13所示。

表5-13 Condition的(部分)方法以及描述

方法名称	描 述
void await() throws InterruptedException	当前线程进入等待状态直到被通知（signal）或中断，当前线程将进入运行状态且从 await() 方法返回的情况，包括： 其他线程调用该 Condition 的 signal() 或 signalAll() 方法，而当前线程被选中唤醒 ❑ 其他线程（调用 interrupt() 方法）中断当前线程 ❑ 如果当前等待线程从 await() 方法返回，那么表明该线程已经获取了 Condition 对象所对应的锁

(续)

方法名称	描 述
void awaitUninterruptibly()	当前线程进入等待状态直到被通知，从方法名称上可以看出该方法对中断不敏感
long awaitNanos(long nanosTimeout) throws InterruptedException	当前线程进入等待状态直到被通知、中断或者超时。返回值表示剩余的时间，如果在 nanosTimeout 纳秒之前被唤醒，那么返回值就是 (nanosTimeout- 实际耗时)。如果返回值是 0 或者负数，那么可以认定已经超时了
boolean awaitUntil(Date deadline) throws InterruptedException	当前线程进入等待状态直到被通知、中断或者到某个时间。如果没有到指定时间就被通知，方法返回 true，否则，表示到了指定时间，方法返回 false
void signal()	唤醒一个等待在 Condition 上的线程，该线程从等待方法返回前必须获得与 Condition 相关联的锁
void signalAll()	唤醒所有等待在 Condition 上的线程，能够从等待方法返回的线程必须获得与 Condition 相关联的锁

获取一个Condition必须通过Lock的newCondition()方法。下面通过一个有界队列的示例来深入了解Condition的使用方式。有界队列是一种特殊的队列，当队列为空时，队列的获取操作将会阻塞获取线程，直到队列中有新增元素，当队列已满时，队列的插入操作将会阻塞插入线程，直到队列出现“空位”，如代码清单5-21所示。

代码清单5-21 BoundedQueue.java

```

public class BoundedQueue<T> {
    private Object[] items;
    // 添加的下标, 删除的下标和数组当前数量
    private int addIndex, removeIndex, count;
    private Lock lock = new ReentrantLock();
    private Condition notEmpty = lock.newCondition();
    private Condition notFull = lock.newCondition();
    public BoundedQueue(int size) {
        items = new Object[size];
    }
    // 添加一个元素, 如果数组满, 则添加线程进入等待状态, 直到有"空位"
    public void add(T t) throws InterruptedException {
        lock.lock();

```

```

        try {
            while (count == items.length)
                notFull.await();
            items[addIndex] = t;
            if (++addIndex == items.length)
                addIndex = 0;
            ++count;
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    // 由头部删除一个元素, 如果数组空, 则删除线程进入等待状态, 直到有新添加元素
    @SuppressWarnings("unchecked")
    public T remove() throws InterruptedException {
        lock.lock();
        try {
            while (count == 0)
                notEmpty.await();
            Object x = items[removeIndex];
            if (++removeIndex == items.length)
                removeIndex = 0;
            --count;
            notFull.signal();
            return (T) x;
        } finally {
            lock.unlock();
        }
    }
}

```

上述示例中, BoundedQueue通过add(T t)方法添加一个元素, 通过remove()方法移出一个元素。以添加方法为例。

首先需要获得锁, 目的是确保数组修改的可见性和排他性。当数组数量等于数组长度时, 表示数组已满, 则调用notFull.await(), 当前线程随之释放锁并进入等待状态。如果数组数量不等于数组长度, 表示数组未满, 则添加元素到数组中, 同时通知等待在notEmpty上的线程, 数组中已经有新元素可以获取。

在添加和删除方法中使用while循环而非if判断, 目的是防止过早或意外的通知, 只有条件符合才能够退出循环。回想之前提到的等待/通知的经典范式, 二者是非常类似的。

5.6.2 Condition的实现分析

ConditionObject是同步器AbstractQueuedSynchronizer的内部类，因为Condition的操作需要获取相关联的锁，所以作为同步器的内部类也较为合理。每个Condition对象都包含着一个队列(以下称为等待队列)，该队列是Condition对象实现等待/通知功能的关键。

下面将分析Condition的实现，主要包括：等待队列、等待和通知，下面提到的Condition如果不加说明均指的是ConditionObject。

1.等待队列

等待队列是一个FIFO的队列，在队列中的每个节点都包含了一个线程引用，该线程就是在Condition对象上等待的线程，如果一个线程调用了Condition.await()方法，那么该线程将会释放锁、构造成节点加入等待队列并进入等待状态。事实上，节点的定义复用了同步器中节点的定义，也就是说，同步队列和等待队列中节点类型都是同步器的静态内部类AbstractQueuedSynchronizer.Node。

一个Condition包含一个等待队列，Condition拥有首节点(firstWaiter)和尾节点(lastWaiter)。当前线程调用Condition.await()方法，将会以当前线程构造节点，并将节点从尾部加入等待队列，等待队列的基本结构如图5-9所示。

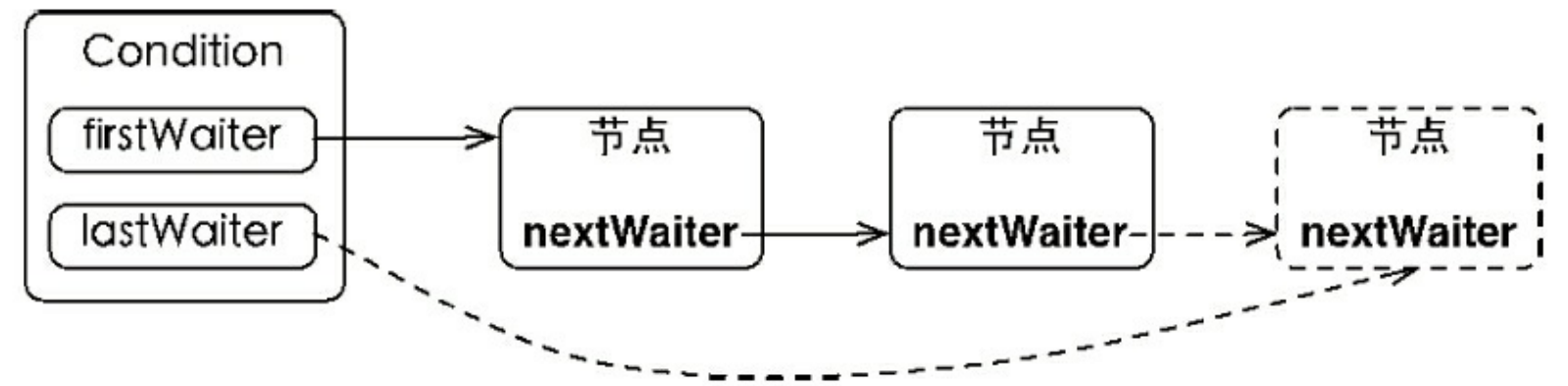


图5-9 等待队列的基本结构

如图所示，Condition拥有首尾节点的引用，而新增节点只需要将原有的尾节点nextWaiter指向它，并且更新尾节点即可。上述节点引用更新的过程并没有使用CAS保证，原因在于调用await()方法的线程必定是获取了锁的线程，也就是说该过程是由锁来保证线程安全的。

在Object的监视器模型上，一个对象拥有一个同步队列和等待队列，而并发包中的Lock(更确切地说是同步器)拥有一个同步队列和多个等待队列，其对应关系如图5-10所示。

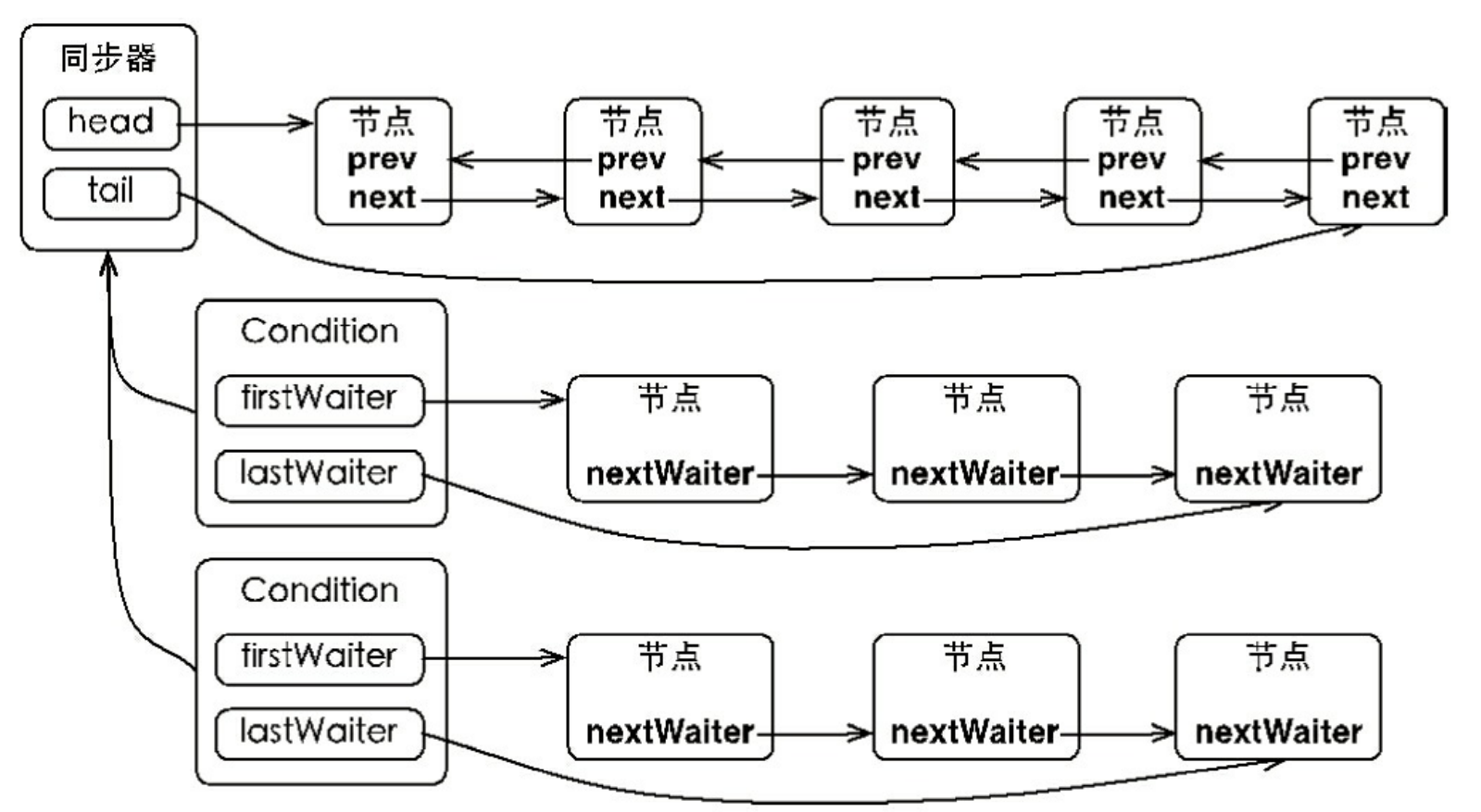


图5-10 同步队列与等待队列

如图所示，Condition的实现是同步器的内部类，因此每个Condition实例都能够访问同步器提供的方法，相当于每个Condition都拥有所属同步器的引用。

2.等待

调用Condition的await()方法(或者以await开头的方法)，会使当前线程进入等待队列并释放锁，同时线程状态变为等待状态。当从await()方法返回时，当前线程一定获取了Condition相关联的锁。

如果从队列(同步队列和等待队列)的角度看await()方法, 当调用await()方法时, 相当于同步队列的首节点(获取了锁的节点)移动到Condition的等待队列中。

Condition的await()方法, 如代码清单5-22所示。

代码清单5-22 ConditionObject的await方法

```
public final void await() throws InterruptedException {
    if (Thread.interrupted())
        throw new InterruptedException();
    // 当前线程加入等待队列
    Node node = addConditionWaiter();
    // 释放同步状态, 也就是释放锁
    int savedState = fullyRelease(node);
    int interruptMode = 0;
    while (!isOnSyncQueue(node)) {
        LockSupport.park(this);
        if ((interruptMode = checkInterruptWhileWaiting(node)) != 0)
            break;
    }
    if (acquireQueued(node, savedState) && interruptMode != THROW_IE)
        interruptMode = REINTERRUPT;
    if (node.nextWaiter != null)
        unlinkCancelledWaiters();
    if (interruptMode != 0)
        reportInterruptAfterWait(interruptMode);
}
```

调用该方法的线程成功获取了锁的线程, 也就是同步队列中的首节点, 该方法会将当前线程构造成节点并加入等待队列中, 然后释放同步状态, 唤醒同步队列中的后继节点, 然后当前线程会进入等待状态。

当等待队列中的节点被唤醒, 则唤醒节点的线程开始尝试获取同步状态。如果不是通过其他线程调用Condition.signal()方法唤醒, 而是对等待线程进行中断, 则会抛出InterruptedException。

如果从队列的角度去看, 当前线程加入Condition的等待队列, 该过程如图5-11示。

如图所示, 同步队列的首节点并不会直接加入等待队列, 而是通过addConditionWaiter()方

法把当前线程构造成为一个新的节点并将其加入等待队列中。

3.通知

调用Condition的signal()方法，将会唤醒在等待队列中等待时间最长的节点(首节点)，在唤醒节点之前，会将节点移到同步队列中。

Condition的signal()方法，如代码清单5-23所示。

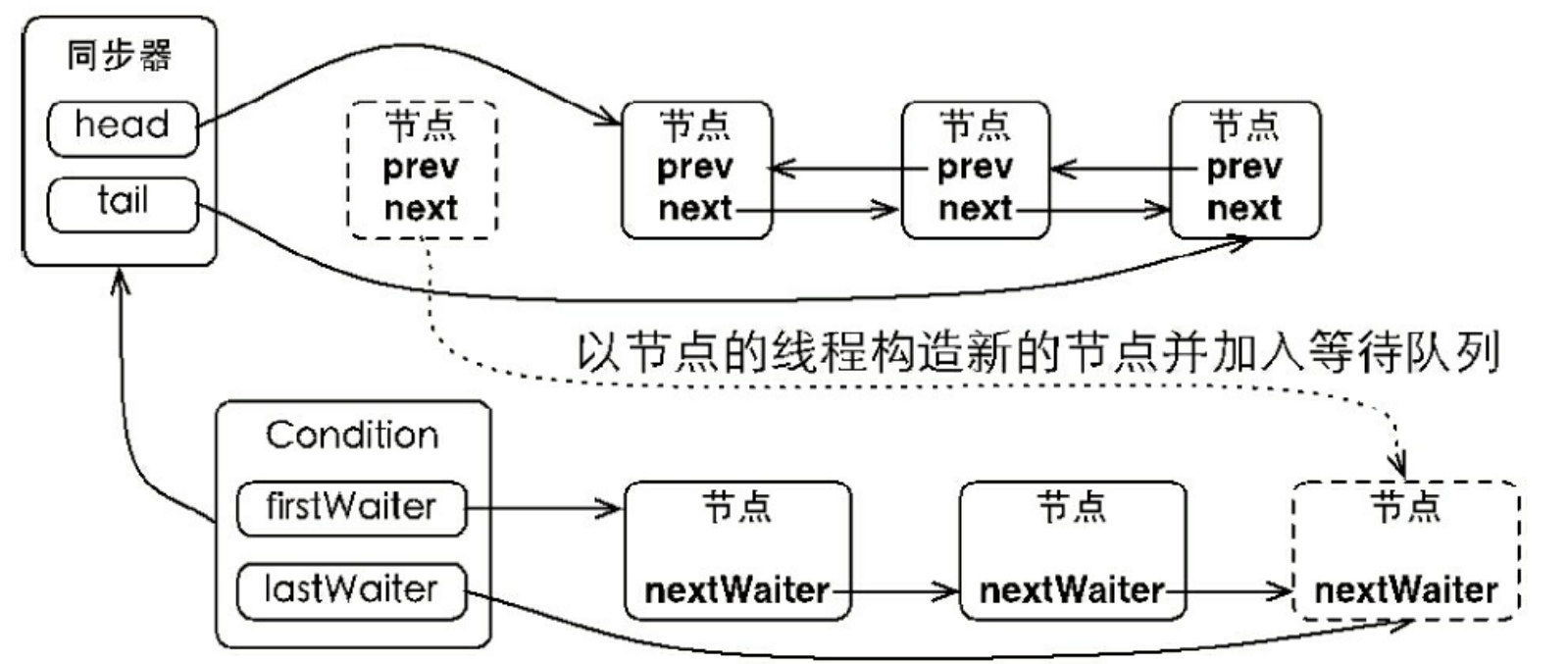


图5-11 当前线程加入等待队列

代码清单5-23 ConditionObject的signal方法

```
public final void signal() {
    if (!isHeldExclusively())
        throw new IllegalMonitorStateException();
    Node first = firstWaiter;
    if (first != null)
        doSignal(first);
}
```

调用该方法的前置条件是当前线程必须获取了锁，可以看到signal()方法进行了isHeldExclusively()检查，也就是当前线程必须是获取了锁的线程。接着获取等待队列的首节

点, 将其移动到同步队列并使用LockSupport唤醒节点中的线程。

节点从等待队列移动到同步队列的过程如图5-12所示。

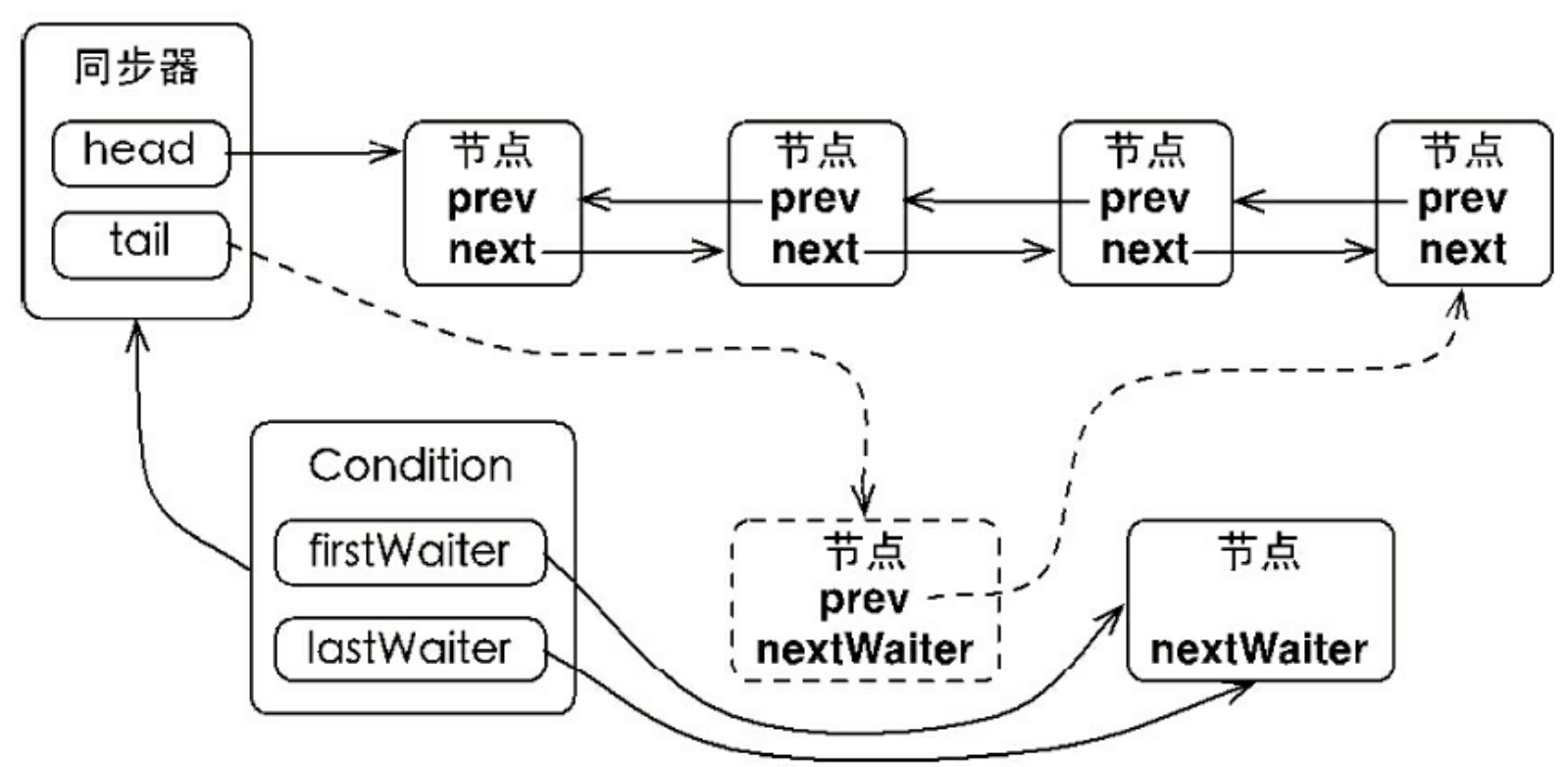


图5-12 节点从等待队列移动到同步队列

通过调用同步器的`enq(Node node)`方法, 等待队列中的头节点线程安全地移动到同步队列。当节点移动到同步队列后, 当前线程再使用LockSupport唤醒该节点的线程。

被唤醒后的线程, 将从`await()`方法中的while循环中退出 (`isOnSyncQueue(Node node)`方法返回true, 节点已经在同步队列中), 进而调用同步器的`acquireQueued()`方法加入到获取同步状态的竞争中。

成功获取同步状态(或者说锁)之后, 被唤醒的线程将从先前调用的`await()`方法返回, 此时该线程已经成功地获取了锁。

Condition的`signalAll()`方法, 相当于对等待队列中的每个节点均执行一次`signal()`方法, 效果就是将等待队列中所有节点全部移动到同步队列中, 并唤醒每个节点的线程。

5.7 本章小结

本章介绍了Java并发包中与锁相关的API和组件，通过示例讲述了这些API和组件的使用方式以及需要注意的地方，并在此基础上详细地剖析了队列同步器、重入锁、读写锁以及Condition等API和组件的实现细节，只有理解这些API和组件的实现细节才能够更加准确地运用它们。

第6章 Java并发容器和框架

Java程序员进行并发编程时，相比于其他语言的程序员而言要倍感幸福，因为并发编程大师Doug Lea不遗余力地为Java开发者提供了非常多的并发容器和框架。本章让我们一起来见识一下大师操刀编写的并发容器和框架，并通过每节的原理分析一起来学习如何设计出精妙的并发程序。

6.1 ConcurrentHashMap的实现原理与使用

ConcurrentHashMap是线程安全且高效的HashMap。本节让我们一起研究一下该容器是如何在保证线程安全的同时又能保证高效的操作。

6.1.1 为什么要使用ConcurrentHashMap

在并发编程中使用HashMap可能导致程序死循环。而使用线程安全的HashTable效率又非常低下，基于以上两个原因，便有了ConcurrentHashMap的登场机会。

(1) 线程不安全的HashMap

在多线程环境下，使用HashMap进行put操作会引起死循环，导致CPU利用率接近100%，所以在并发情况下不能使用HashMap。例如，执行以下代码会引起死循环。

```
final HashMap<String, String> map = new HashMap<String, String>(2);
    Thread t = new Thread(new Runnable() {
        @Override
        public void run() {
            for (int i = 0; i < 10000; i++) {
                new Thread(new Runnable() {
                    @Override
                    public void run() {
                        map.put(UUID.randomUUID().toString(), "");
                    }
                }, "ftf" + i).start();
            }
        }
    }, "ftf");
t.start();
t.join();
```

HashMap在并发执行put操作时会引起死循环，是因为多线程会导致HashMap的Entry链表形成环形数据结构，一旦形成环形数据结构，Entry的next节点永远不为空，就会产生死循环获取Entry。

(2) 效率低下的HashTable

HashTable容器使用synchronized来保证线程安全，但在线程竞争激烈的情况下HashTable的效率非常低下。因为当一个线程访问HashTable的同步方法，其他线程也访问HashTable的同步方法时，会进入阻塞或轮询状态。如线程1使用put进行元素添加，线程2不但不能使用put方

法添加元素，也不能使用get方法来获取元素，所以竞争越激烈效率越低。

(3)ConcurrentHashMap的锁分段技术可有效提升并发访问率

HashTable容器在竞争激烈的并发环境下表现出效率低下的原因是所有访问HashTable的线程都必须竞争同一把锁，假如容器里有多把锁，每一把锁用于锁容器其中一部分数据，那么当多线程访问容器里不同数据段的数据时，线程间就不会存在锁竞争，从而可以有效提高并发访问效率，这就是ConcurrentHashMap所使用的锁分段技术。首先将数据分成一段一段地存储，然后给每一段数据配一把锁，当一个线程占用锁访问其中一个段数据的时候，其他段的数据也能被其他线程访问。

6.1.2 ConcurrentHashMap的结构

通过ConcurrentHashMap的类图来分析ConcurrentHashMap的结构，如图6-1所示。

ConcurrentHashMap是由Segment数组结构和HashEntry数组结构组成。Segment是一种可重入锁(ReentrantLock)，在ConcurrentHashMap里扮演锁的角色；HashEntry则用于存储键值对数据。一个ConcurrentHashMap里包含一个Segment数组。Segment的结构和HashMap类似，是一种数组和链表结构。一个Segment里包含一个HashEntry数组，每个HashEntry是一个链表结构的元素，每个Segment守护着一个HashEntry数组里的元素，当对HashEntry数组的数据进行修改时，必须首先获得与它对应的Segment锁，如图6-2所示。

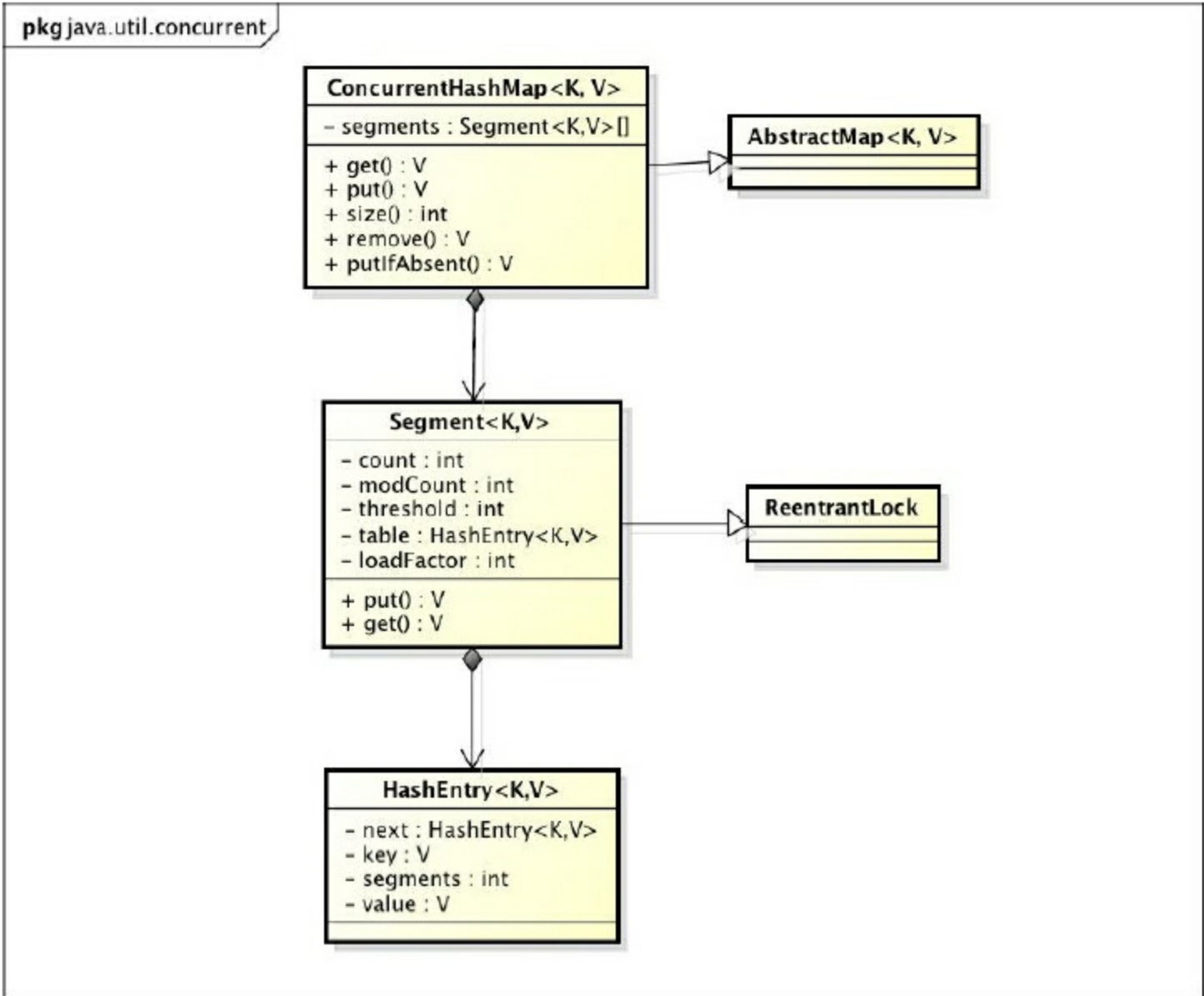


图6-1 ConcurrentHashMap的类图

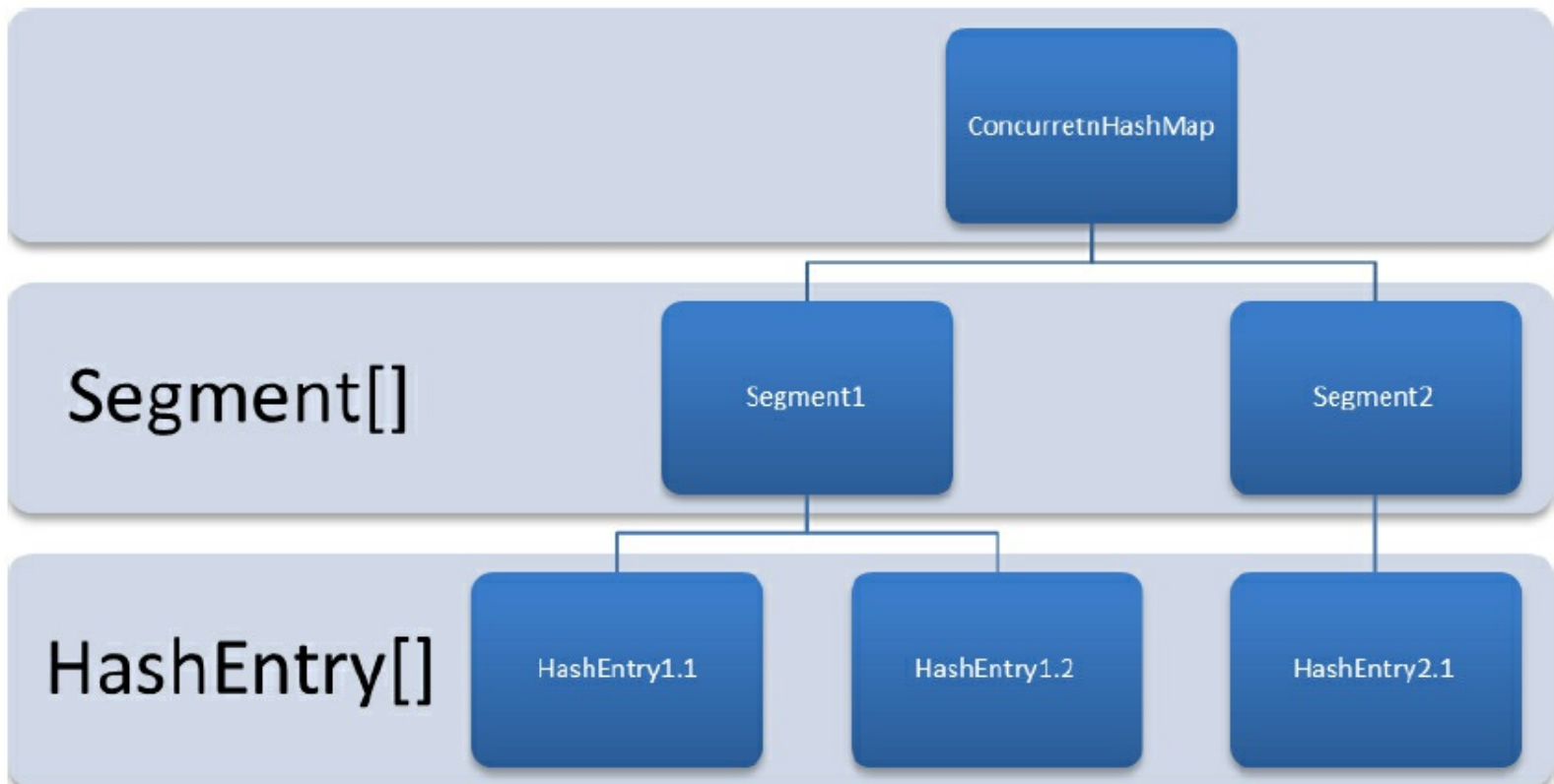


图6-2 ConcurrentHashMap的结构图

6.1.3 ConcurrentHashMap的初始化

ConcurrentHashMap初始化方法是通过initialCapacity、loadFactor和concurrencyLevel等几个参数来初始化segment数组、段偏移量segmentShift、段掩码segmentMask和每个segment里的HashEntry数组来实现的。

1.初始化segments数组

让我们来看一下初始化segments数组的源代码。

```
if (concurrencyLevel > MAX_SEGMENTS)
    concurrencyLevel = MAX_SEGMENTS;
int sshift = 0;
int ssize = 1;
while (ssize < concurrencyLevel) {
    ++sshift;
    ssize <=< 1;
}
segmentShift = 32 - sshift;
segmentMask = ssize - 1;
this.segments = Segment.newArray(ssize);
```

由上面的代码可知，segments数组的长度ssize是通过concurrencyLevel计算得出的。为了能通过按位与的散列算法来定位segments数组的索引，必须保证segments数组的长度是2的N次方(power-of-two size)，所以必须计算出一个大于或等于concurrencyLevel的最小的2的N次方值来作为segments数组的长度。假如concurrencyLevel等于14、15或16，ssize都会等于16，即容器里锁的个数也是16。

 **注意** concurrencyLevel的最大值是65535，这意味着segments数组的长度最大为65536，对应的二进制是16位。

2.初始化segmentShift和segmentMask

这两个全局变量需要在定位segment时的散列算法里使用，sshift等于ssize从1向左移位的次数，在默认情况下concurrencyLevel等于16，1需要向左移位移动4次，所以sshift等于4。segmentShift用于定位参与散列运算的位数，segmentShift等于32减sshift，所以等于28，这里之所以用32是因为ConcurrentHashMap里的hash()方法输出的最大数是32位的，后面的测试中我们可以看到这点。segmentMask是散列运算的掩码，等于ssize减1，即15，掩码的二进制各个位的值都是1。因为ssize的最大长度是65536，所以segmentShift最大值是16，segmentMask最大值是65535，对应的二进制是16位，每个位都是1。

3.初始化每个segment

输入参数initialCapacity是ConcurrentHashMap的初始化容量，loadfactor是每个segment的负载因子，在构造方法里需要通过这两个参数来初始化数组中的每个segment。

```
if (initialCapacity > MAXIMUM_CAPACITY)
    initialCapacity = MAXIMUM_CAPACITY;
int c = initialCapacity / ssize;
if (c * ssize < initialCapacity)
    ++c;
int cap = 1;
while (cap < c)
    cap <<= 1;
for (int i = 0; i < this.segments.length; ++i)
    this.segments[i] = new Segment<K,V>(cap, loadFactor);
```

上面代码中的变量cap就是segment里HashEntry数组的长度，它等于initialCapacity除以ssize的倍数c，如果c大于1，就会取大于等于c的2的N次方值，所以cap不是1，就是2的N次方。segment的容量threshold=(int)cap*loadFactor，默认情况下initialCapacity等于16，loadfactor等于0.75，通过运算cap等于1，threshold等于零。

6.1.4 定位Segment

既然ConcurrentHashMap使用分段锁Segment来保护不同段的数据，那么在插入和获取元素的时候，必须先通过散列算法定位到Segment。可以看到ConcurrentHashMap会首先使用Wang/Jenkins hash的变种算法对元素的hashCode进行一次再散列。

```
private static int hash(int h) {
    h += (h << 15) ^ 0xffffcd7d;
    h ^= (h >>> 10);
    h += (h << 3);
    h ^= (h >>> 6);
    h += (h << 2) + (h << 14);
    return h ^ (h >>> 16);
}
```

之所以进行再散列，目的是减少散列冲突，使元素能够均匀地分布在不同的Segment上，从而提高容器的存取效率。假如散列的质量差到极点，那么所有的元素都在一个Segment中，不仅存取元素缓慢，分段锁也会失去意义。笔者做了一个测试，不通过再散列而直接执行散列计算。

```
System.out.println(Integer.parseInt("0001111", 2) & 15);
System.out.println(Integer.parseInt("0011111", 2) & 15);
System.out.println(Integer.parseInt("0111111", 2) & 15);
System.out.println(Integer.parseInt("1111111", 2) & 15);
```

计算后输出的散列值全是15，通过这个例子可以发现，如果不进行再散列，散列冲突会非常严重，因为只要低位一样，无论高位是什么数，其散列值总是一样。我们再把上面的二进制数据进行再散列后结果如下(为了方便阅读，不足32位的高位补了0，每隔4位用竖线分割下)。

```
0100 | 0111 | 0110 | 0111 | 1101 | 1010 | 0100 | 1110
1111 | 0111 | 0100 | 0011 | 0000 | 0001 | 1011 | 1000
0111 | 0111 | 0110 | 1001 | 0100 | 0110 | 0011 | 1110
1000 | 0011 | 0000 | 0000 | 1100 | 1000 | 0001 | 1010
```

可以发现，每一位的数据都散列开了，通过这种再散列能让数字的每一位都参加到散列运算当中，从而减少散列冲突。ConcurrentHashMap通过以下散列算法定位segment。

```
final Segment<K,V> segmentFor(int hash) {  
    return segments[(hash >>> segmentShift) & segmentMask];  
}
```

默认情况下segmentShift为28, segmentMask为15, 再散列后的数最大是32位二进制数据，向右无符号移动28位，意思是让高4位参与到散列运算中，(hash>>>segmentShift)&segmentMask的运算结果分别是4、15、7和8, 可以看到散列值没有发生冲突。

6.1.5 ConcurrentHashMap的操作

本节介绍ConcurrentHashMap的3种操作——get操作、put操作和size操作。

1.get操作

Segment的get操作实现非常简单和高效。先经过一次再散列，然后使用这个散列值通过散列运算定位到Segment，再通过散列算法定位到元素，代码如下。

```
public V get(Object key) {
    int hash = hash(key.hashCode());
    return segmentFor(hash).get(key, hash);
}
```

get操作的高效之处在于整个get过程不需要加锁，除非读到的值是空才会加锁重读。我们知道HashTable容器的get方法是需要加锁的，那么ConcurrentHashMap的get操作是如何做到不加锁的呢？原因是它的get方法里将要使用的共享变量都定义成volatile类型，如用于统计当前Segment大小的count字段和用于存储值的HashEntry的value。定义成volatile的变量，能够在线程之间保持可见性，能够被多线程同时读，并且保证不会读到过期的值，但是只能被单线程写（有一种情况可以被多线程写，就是写入的值不依赖于原值），在get操作里只需要读不需要写共享变量count和value，所以可以不用加锁。之所以不会读到过期的值，是因为根据Java内存模型的happen before原则，对volatile字段的写入操作先于读操作，即使两个线程同时修改和获取volatile变量，get操作也能拿到最新的值，这是用volatile替换锁的经典应用场景。

```
transient volatile int count;
volatile V value;
```

在定位元素的代码里我们可以发现，定位HashEntry和定位Segment的散列算法虽然一样，都与数组的长度减去1再相“与”，但是相“与”的值不一样，定位Segment使用的是元素的hashCode通过再散列后得到的值的高位，而定位HashEntry直接使用再散列后的值。其目的

是避免两次散列后的值一样，虽然元素在Segment里散列开了，但是却没有在HashEntry里散列开。

```
hash >>> segmentShift) & segmentMask           // 定位Segment所使用的hash算法
int index = hash & (tab.length - 1);             // 定位HashEntry所使用的hash算法
```

2.put操作

由于put方法里需要对共享变量进行写入操作，所以为了线程安全，在操作共享变量时必须加锁。put方法首先定位到Segment，然后在Segment里进行插入操作。插入操作需要经历两个步骤，第一步判断是否需要Segment里的HashEntry数组进行扩容，第二步定位添加元素的位置，然后将其放在HashEntry数组里。

(1)是否需要扩容

在插入元素前会先判断Segment里的HashEntry数组是否超过容量(threshold)，如果超过阈值，则对数组进行扩容。值得一提的是，Segment的扩容判断比HashMap更恰当，因为HashMap是在插入元素后判断元素是否已经到达容量的，如果到达了就进行扩容，但是很有可能扩容之后没有新元素插入，这时HashMap就进行了一次无效的扩容。

(2)如何扩容

在扩容的时候，首先会创建一个容量是原来容量两倍的数组，然后将原数组里的元素进行再散列后插入到新的数组里。为了高效，ConcurrentHashMap不会对整个容器进行扩容，而只对某个segment进行扩容。

3.size操作

如果要统计整个ConcurrentHashMap里元素的大小，就必须统计所有Segment里元素的大小后求和。Segment里的全局变量count是一个volatile变量，那么在多线程场景下，是不是直接把

所有Segment的count相加就可以得到整个ConcurrentHashMap大小了呢？不是的，虽然相加时可以获取每个Segment的count的最新值，但是可能累加前使用的count发生了变化，那么统计结果就不准了。所以，最安全的做法是在统计size的时候把所有Segment的put、remove和clean方法全部锁住，但是这种做法显然非常低效。

因为在累加count操作过程中，之前累加过的count发生变化的几率非常小，所以ConcurrentHashMap的做法是先尝试2次通过不锁住Segment的方式来统计各个Segment大小，如果统计的过程中，容器的count发生了变化，则再采用加锁的方式来统计所有Segment的大小。

那么ConcurrentHashMap是如何判断在统计的时候容器是否发生了变化呢？使用modCount变量，在put、remove和clean方法里操作元素前都会将变量modCount进行加1，那么在统计size前后比较modCount是否发生变化，从而得知容器的大小是否发生变化。

6.2 ConcurrentLinkedQueue

在并发编程中，有时候需要使用线程安全的队列。如果要实现一个线程安全的队列有两种方式：一种是使用阻塞算法，另一种是使用非阻塞算法。使用阻塞算法的队列可以用一个锁（入队和出队用同一把锁）或两个锁（入队和出队用不同的锁）等方式来实现。非阻塞的实现方式则可以使用循环CAS的方式来实现。本节让我们一起来研究一下Doug Lea是如何使用非阻塞的方式来实现线程安全队列ConcurrentLinkedQueue的，相信从大师身上我们能学到不少并发编程的技巧。

ConcurrentLinkedQueue是一个基于链接节点的无界线程安全队列，它采用先进先出的规则对节点进行排序，当我们添加一个元素的时候，它会添加到队列的尾部；当我们获取一个元素时，它会返回队列头部的元素。它采用了“wait-free”算法（即CAS算法）来实现，该算法在Michael&Scott算法上进行了一些修改。

6.2.1 ConcurrentLinkedQueue的结构

通过ConcurrentLinkedQueue的类图来分析一下它的结构，如图6-3所示。

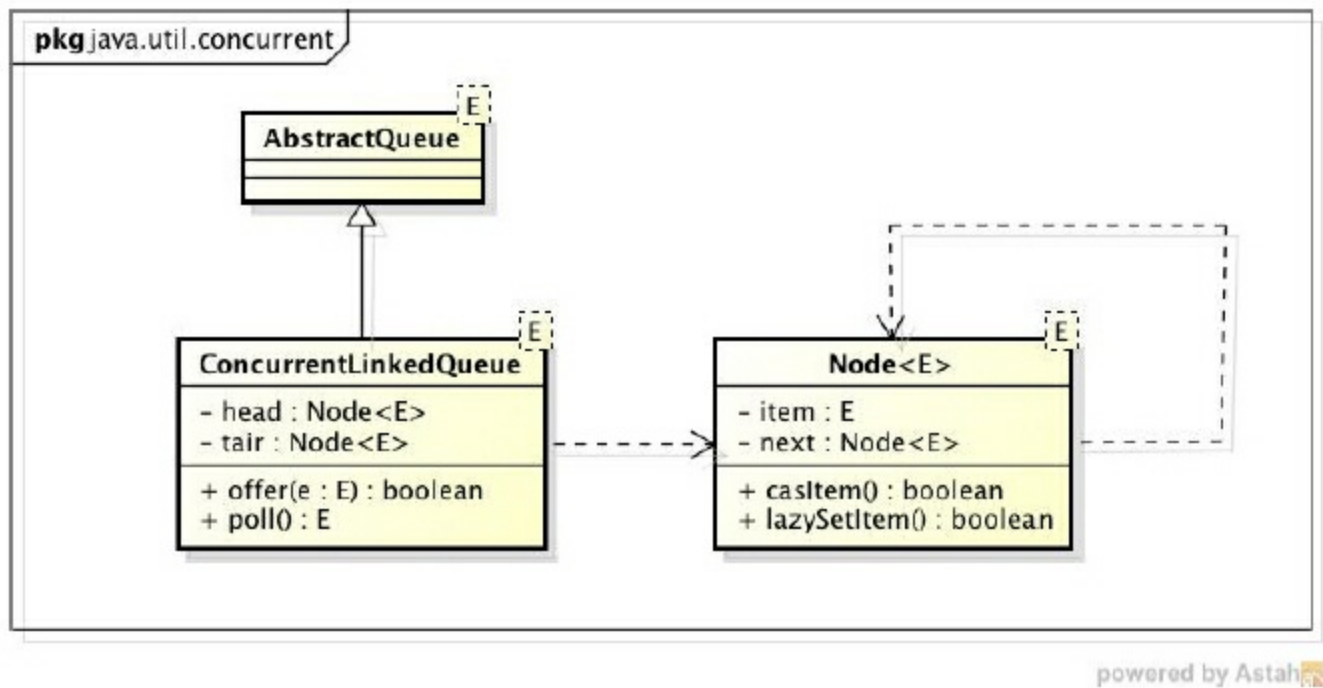


图6-3 ConcurrentLinkedQueue的类图

ConcurrentLinkedQueue由head节点和tail节点组成，每个节点(Node)由节点元素(item)和指向下一个节点(next)的引用组成，节点与节点之间就是通过这个next关联起来，从而组成一张链表结构的队列。默认情况下head节点存储的元素为空，tail节点等于head节点。

```
private transient volatile Node<E> tail = head;
```

6.2.2 入队列

本节将介绍入队列的相关知识。

1.入队列的过程

入队列就是将入队节点添加到队列的尾部。为了方便理解入队时队列的变化，以及head节点和tail节点的变化，这里以一个示例来展开介绍。假设我们想在一个队列中依次插入4个节点，为了帮助大家理解，每添加一个节点就做了一个队列的快照图，如图6-4所示。

图6-4所示的过程如下。

- 添加元素1。队列更新head节点的next节点为元素1节点。又因为tail节点默认情况下等于head节点，所以它们的next节点都指向元素1节点。
- 添加元素2。队列首先设置元素1节点的next节点为元素2节点，然后更新tail节点指向元素2节点。

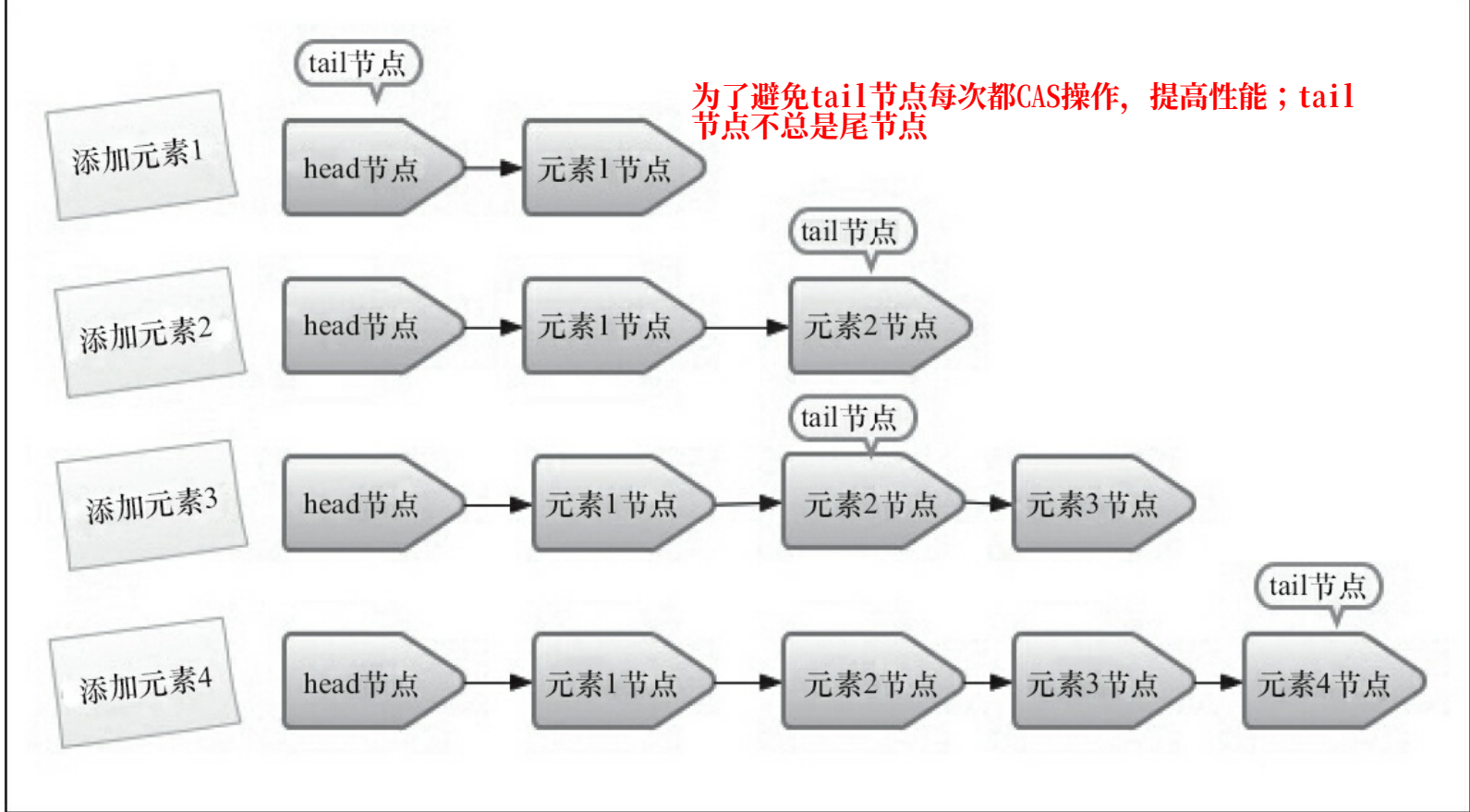


图6-4 队列添加元素的快照图

·添加元素3，设置tail节点的next节点为元素3节点。

·添加元素4，设置元素3的next节点为元素4节点，然后将tail节点指向元素4节点。

通过调试入队过程并观察head节点和tail节点的变化，发现入队主要做两件事情：**第一是**

将入队节点设置成当前队列尾节点的下一个节点；第二是更新tail节点，如果tail节点的next节点不为空，则将入队节点设置成tail节点，如果tail节点的next节点为空，则将入队节点设置成tail的next节点，所以tail节点不总是尾节点（理解这一点对于我们研究源码会非常有帮助）。

通过对上面的分析，我们从单线程入队的角度理解了入队过程，但是多个线程同时进行入队的情况就变得更加复杂了，因为可能会出现其他线程插队的情况。如果有一个线程正在入队，那么它必须先获取尾节点，然后设置尾节点的下一个节点为入队节点，但这时可能有另外一个线程插队了，那么队列的尾节点就会发生变化，这时当前线程要暂停入队操作，然后重新获取尾节点。让我们再通过源码来详细分析一下它是如何使用CAS算法来入队的。

```

public boolean offer(E e) {
    if (e == null) throw new NullPointerException();
    // 入队前, 创建一个入队节点
    Node<E> n = new Node<E>(e);
    retry:
    // 死循环, 入队不成功反复入队。
    for (;;) {
        // 创建一个指向tail节点的引用
        Node<E> t = tail;
        // p用来表示队列的尾节点, 默认情况下等于tail节点。
        Node<E> p = t;
        for (int hops = 0; ; hops++) {
            // 获得p节点的下一个节点。
            Node<E> next = succ(p);
            // next节点不为空, 说明p不是尾节点, 需要更新p后在将它指向next节点
            if (next != null) {
                // 循环了两次及以上, 并且当前节点还是不等于尾节点
                if (hops > HOPS && t != tail)
                    continue retry;
                p = next;
            }
            // 如果p是尾节点, 则设置p节点的next节点为入队节点。
            else if (p.casNext(null, n)) {
                /*如果tail节点有大于等于1个next节点, 则将入队节点设置成tail节点,
                更新失败了也没关系, 因为失败了表示有其他线程成功更新了tail节点*/
                if (hops >= HOPS)
                    casTail(t, n); // 更新tail节点, 允许失败
                return true;
            }
            // p有next节点, 表示p的next节点是尾节点, 则重新设置p节点
            else {
                p = succ(p);
            }
        }
    }
}

```

从源代码角度来看, 整个入队过程主要做两件事情: 第一是定位出尾节点; 第二是使用CAS算法将入队节点设置成尾节点的next节点, 如不成功则重试。

2. 定位尾节点

tail节点并不总是尾节点, 所以每次入队都必须先通过tail节点来找到尾节点。尾节点可能是tail节点, 也可能是tail节点的next节点。代码中循环体中的第一个if就是判断tail是否有next节点, 有则表示next节点可能是尾节点。获取tail节点的next节点需要注意的是p节点等于p的next节点的情况, 只有一种可能就是p节点和p的next节点都等于空, 表示这个队列刚初始化, 正准

备添加节点，所以需要返回head节点。获取p节点的next节点代码如下。

```
final Node<E> succ(Node<E> p) {
    Node<E> next = p.getNext();
    return (p == next)    head : next;
}
```

3.设置入队节点为尾节点

p.casNext(null, n)方法用于将入队节点设置为当前队列尾节点的next节点，如果p是null，表示p是当前队列的尾节点，如果不为null，表示有其他线程更新了尾节点，则需要重新获取当前队列的尾节点。

4.HOPS的设计意图

上面分析过对于先进先出的队列入队所要做的事情是将入队节点设置成尾节点，doug lea写的代码和逻辑还是稍微有点复杂。那么，我用以下方式来实现是否可行？

```
public boolean offer(E e) {
    if (e == null)
        throw new NullPointerException();
    Node<E> n = new Node<E>(e);
    for (;;) {
        Node<E> t = tail;
        if (t.casNext(null, n) && casTail(t, n)) {
            return true;
        }
    }
}
```

让tail节点永远作为队列的尾节点，这样实现代码量非常少，而且逻辑清晰和易懂。但是，这么做有个缺点，每次都需要使用循环CAS更新tail节点。如果能减少CAS更新tail节点的次数，就能提高入队的效率，所以doug lea使用hops变量来控制并减少tail节点的更新频率，并不是每次节点入队后都将tail节点更新成尾节点，而是当tail节点和尾节点的距离大于等于常量HOPS的值(默认等于1)时才更新tail节点，tail和尾节点的距离越长，使用CAS更新tail节点的次

数就会越少，但是距离越长带来的负面效果就是每次入队时定位尾节点的时间就越长，因为循环体需要多循环一次来定位出尾节点，但是这样仍然能提高入队的效率，因为从本质上来看它通过增加对volatile变量的读操作来减少对volatile变量的写操作，而对volatile变量的写操作开销要远远大于读操作，所以入队效率会有所提升。

```
private static final int HOPS = 1;
```



注意 入队方法永远返回true，所以不要通过返回值判断入队是否成功。

6.2.3 出队列

出队列的就是从队列里返回一个节点元素，并清空该节点对元素的引用。让我们通过每个节点出队的快照来观察一下head节点的变化，如图6-5所示。

从图中可知，并不是每次出队时都更新head节点，当head节点里有元素时，直接弹出head节点里的元素，而不会更新head节点。只有当head节点里没有元素时，出队操作才会更新head节点。这种做法也是通过hops变量来减少使用CAS更新head节点的消耗，从而提高出队效率。让我们再通过源码来深入分析下出队过程。

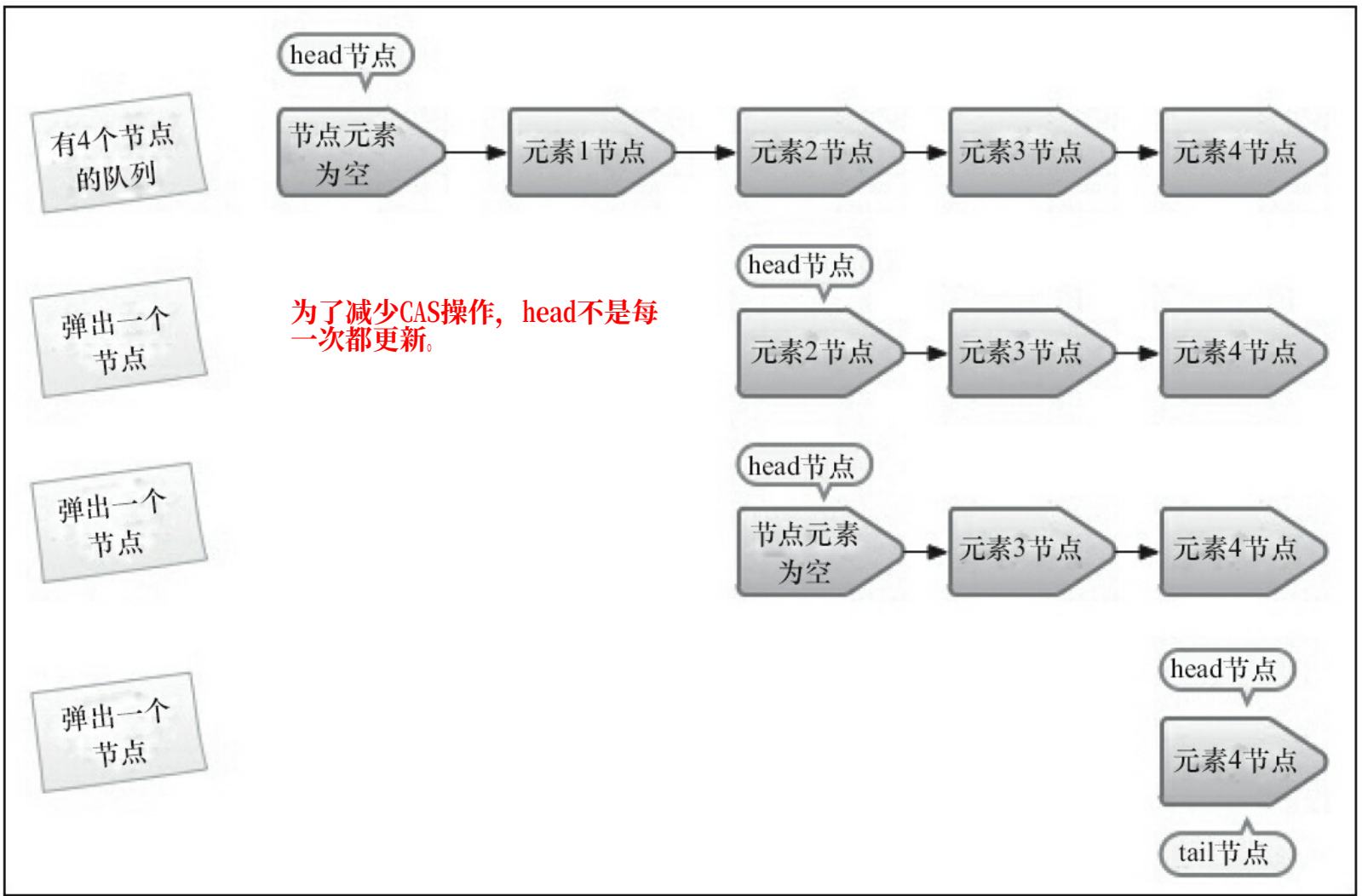


图6-5 队列出节点快照图

```
public E poll() {
    Node<E> h = head;
    // p表示头节点，需要出队的节点
```

```

Node<E> p = h;
for (int hops = 0;; hops++) {
    // 获取p节点的元素
    E item = p.getItem();
    // 如果p节点的元素不为空, 使用CAS设置p节点引用的元素为null,
    // 如果成功则返回p节点的元素。
    if (item != null && p.casItem(item, null)) {
        if (hops >= HOPS) {
            // 将p节点下一个节点设置成head节点
            Node<E> q = p.getNext();
            updateHead(h, (q != null) ? q : p);
        }
        return item;
    }
    // 如果头节点的元素为空或头节点发生了变化, 这说明头节点已经被另外
    // 一个线程修改了。那么获取p节点的下一个节点
    Node<E> next = succ(p);
    // 如果p的下一个节点也为空, 说明这个队列已经空了
    if (next == null) {
        // 更新头节点。
        updateHead(h, p);
        break;
    }
    // 如果下一个元素不为空, 则将头节点的下一个节点设置成头节点
    p = next;
}
return null;
}

```

首先获取头节点的元素, 然后判断头节点元素是否为空, 如果为空, 表示另外一个线程已经进行了一次出队操作将该节点的元素取走, 如果不为空, 则使用CAS的方式将头节点的引用设置成null, 如果CAS成功, 则直接返回头节点的元素, 如果不成功, 表示另外一个线程已经进行了一次出队操作更新了head节点, 导致元素发生了变化, 需要重新获取头节点。

6.3 Java中的阻塞队列

本节将介绍什么是阻塞队列，以及Java中阻塞队列的4种处理方式，并介绍Java 7中提供的7种阻塞队列，最后分析阻塞队列的一种实现方式。

6.3.1 什么是阻塞队列

阻塞队列(BlockingQueue)是一个支持两个附加操作的队列。**这两个附加的操作支持阻塞的插入和移除方法。**

1)支持阻塞的插入方法:意思是当队列满时, 队列会阻塞插入元素的线程, 直到队列不满。

2)支持阻塞的移除方法:意思是在队列为空时, 获取元素的线程会等待队列变为非空。

阻塞队列常用于生产者和消费者的场景, 生产者是向队列里添加元素的线程, 消费者是从队列里取元素的线程。阻塞队列就是生产者用来存放元素、消费者用来获取元素的容器。

在阻塞队列不可用时, 这两个附加操作提供了4种处理方式, 如表6-1所示。

表6-1 插入和移除操作的4中处理方式

方法 / 处理方式	抛出异常	返回特殊值	一直阻塞	超时退出
插入方法	add (e)	offer (e)	put (e)	offer (e, time, unit)
移除方法	remove()	poll()	take()	poll (time, unit)
检查方法	element()	peek()	不可用	不可用

抛出异常:当队列满时, 如果再往队列里插入元素, 会抛出IllegalStateException("Queue full")异常。当队列空时, 从队列里获取元素会抛出NoSuchElementException异常。

返回特殊值:当往队列插入元素时, 会返回元素是否插入成功, 成功返回true。如果是移除方法, 则是从队列里取出一个元素, 如果没有则返回null。

一直阻塞:当阻塞队列满时, 如果生产者线程往队列里put元素, 队列会一直阻塞生产者线程, 直到队列可用或者响应中断退出。当队列空时, 如果消费者线程从队列里take元素, 队列会阻塞住消费者线程, 直到队列不为空。

·**超时退出**: 当阻塞队列满时, 如果生产者线程往队列里插入元素, 队列会阻塞生产者线程一段时间, 如果超过了指定的时间, 生产者线程就会退出。

这两个附加操作的4种处理方式不方便记忆, 所以我找了一下这几个方法的规律。put和take分别尾首含有字母t, offer和poll都含有字母o。

 **注意** 如果是无界阻塞队列, 队列不可能会出现满的情况, 所以使用put或offer方法永远不会被阻塞, 而且使用offer方法时, 该方法永远返回true。

6.3.2 Java里的阻塞队列

JDK 7提供了7个阻塞队列，如下。

数组 + ReentrantLock

· `ArrayBlockingQueue`: 一个由数组结构组成的有界阻塞队列。

链表 + ReentrantLock

· `LinkedBlockingQueue`: 一个由链表结构组成的有界阻塞队列。最大容量是`Integer.MAX_VALUE`

数组实现的最小堆二叉堆 + ReentrantLock

· `PriorityBlockingQueue`: 一个支持优先级排序的无界阻塞队列。最大容量是`Integer.MAX_VALUE-8`

· `DelayQueue`: 一个使用优先级队列实现的无界阻塞队列。实现`PriorityQueue` + `ReentrantLock`

· `SynchronousQueue`: 一个不存储元素的阻塞队列。

· `LinkedTransferQueue`: 一个由链表结构组成的无界阻塞队列。

· `LinkedBlockingDeque`: 一个由链表结构组成的双向阻塞队列。

1. `ArrayBlockingQueue`

实现：数组 + ReentrantLock

`ArrayBlockingQueue`是一个用数组实现的有界阻塞队列。此队列按照先进先出(FIFO)的原则对元素进行排序。

ReentrantLock实现的阻塞操作：公平锁 或 非公平锁

默认情况下不保证线程公平的访问队列，所谓公平访问队列是指阻塞的线程，可以按照阻塞的先后顺序访问队列，即先阻塞线程先访问队列。非公平性是对先等待的线程是非公平的，当队列可用时，阻塞的线程都可以争夺访问队列的资格，有可能先阻塞的线程最后才访问队列。为了保证公平性，通常会降低吞吐量。我们可以使用以下代码创建一个公平的阻塞队列。

```
ArrayBlockingQueue fairQueue = new ArrayBlockingQueue(1000,true);
```

访问者的公平性是使用可重入锁实现的, 代码如下。

```
public ArrayBlockingQueue(int capacity, boolean fair) {
    if (capacity <= 0)
        throw new IllegalArgumentException();
    this.items = new Object[capacity];
    lock = new ReentrantLock(fair);
    notEmpty = lock.newCondition();
    notFull = lock.newCondition();
}
```

2. LinkedBlockingQueue

实现: ReentrantLock + 链表

LinkedBlockingQueue是一个用链表实现的有界阻塞队列。此队列的默认和最大长度为Integer.MAX_VALUE。此队列按照先进先出的原则对元素进行排序。

3. PriorityBlockingQueue

实现原理: ReentrantLock + object[]堆(数组实现的二叉堆), 队列最大值有限制

PriorityBlockingQueue是一个支持优先级的无界阻塞队列。默认情况下元素采取自然顺序升序排列。也可以自定义类实现compareTo()方法来指定元素排序规则, 或者初始化PriorityBlockingQueue时, 指定构造参数Comparator来对元素进行排序。需要注意的是不能保证同优先级元素的顺序。

4. DelayQueue

实现PriorityQueue + ReentrantLock

DelayQueue是一个支持延时获取元素的无界阻塞队列。队列使用PriorityQueue来实现。队列中的元素必须实现Delayed接口, 在创建元素时可以指定多久才能从队列中获取当前元素。只有在延迟期满时才能从队列中提取元素。在取元素的时, 会根据元素的getDelayed()获取wait的时间

DelayQueue非常有用, 可以将DelayQueue运用在以下应用场景。

DelayQueue每次获取元素都是PriorityQueue队列中第一个(堆中第一个), 如果每个元素延迟时间不一样时, 元素需要按延时进行排序重写compareTo()方法, 让延时时间最长的放在队列的末尾。

缓存系统的设计: 可以用DelayQueue保存缓存元素的有效期, 使用一个线程循环查询

DelayQueue, 一旦能从DelayQueue中获取元素时, 表示缓存有效期到了。

定时任务调度:使用DelayQueue保存当天将会执行的任务和执行时间,一旦从DelayQueue中获取到任务就开始执行,比如TimerQueue就是使用DelayQueue实现的。

(1) 如何实现Delayed接口

DelayQueue队列的元素必须实现Delayed接口。我们可以参考ScheduledThreadPoolExecutor里ScheduledFutureTask类的实现,一共有三步。

第一步:在对象创建的时候,初始化基本数据。使用time记录当前对象延迟到什么时候可以使用,使用sequenceNumber来标识元素在队列中的先后顺序。代码如下。

```
private static final AtomicLong sequencer = new AtomicLong(0);
ScheduledFutureTask(Runnable r, V result, long ns, long period) {
ScheduledFutureTask(Runnable r, V result, long ns, long period) {
    super(r, result);
    this.time = ns;
    this.period = period;
    this.sequenceNumber = sequencer.getAndIncrement();
}
```

第二步:实现getDelay方法,该方法返回当前元素还需要延时多长时间,单位是纳秒,代码如下。

```
public long getDelay(TimeUnit unit) {
    return unit.convert(time - now(), TimeUnit.NANOSECONDS);
}
```

通过构造函数可以看出延迟时间参数ns的单位是纳秒,自己设计的时候最好使用纳秒,因为实现getDelay()方法时可以指定任意单位,一旦以秒或分作为单位,而延时时间又精确不到纳秒就麻烦了。使用时请注意当time小于当前时间时,getDelay会返回负数。

第三步:实现compareTo方法来指定元素的顺序。例如,让延时时间最长的放在队列的末尾。实现代码如下。

```
public int compareTo(Delayed other) {
    if (other == this) // compare zero ONLY if same object
        return 0;
    if (other instanceof ScheduledFutureTask) {
        ScheduledFutureTask<> x = (ScheduledFutureTask<>)other;
        long diff = time - x.time;
        if (diff < 0)
            return -1;
        else if (diff > 0)
            return 1;
        else if (sequenceNumber < x.sequenceNumber)
            return -1;
        else
            return 1;
    }
    long d = (getDelay(TimeUnit.NANOSECONDS) -
              other.getDelay(TimeUnit.NANOSECONDS));
    return (d == 0) ? 0 : ((d < 0) ? -1 : 1);
}
```

(2) 如何实现延时阻塞队列

延时阻塞队列的实现很简单，当消费者从队列里获取元素时，如果元素没有达到延时时间，就阻塞当前线程。

```
long delay = first.getDelay(TimeUnit.NANOSECONDS);
if (delay <= 0)
    return q.poll();
else if (leader != null)
    available.await();
else {
    Thread thisThread = Thread.currentThread();
    leader = thisThread;
    try {
        available.awaitNanos(delay);
    } finally {
        if (leader == thisThread)
            leader = null;
    }
}
```

代码中的变量leader是一个等待获取队列头部元素的线程。如果leader不等于空，表示已经有线程在等待获取队列的头元素。所以，使用await()方法让当前线程等待信号。如果leader等于空，则把当前线程设置成leader，并使用awaitNanos()方法让当前线程等待接收信号或等待delay时间。

5.SynchronousQueue

SynchronousQueue是一个不存储元素的阻塞队列。每一个put操作必须等待一个take操作，否则不能继续添加元素。

它支持公平访问队列。默认情况下线程采用非公平性策略访问队列。使用以下构造方法可以创建公平性访问的SynchronousQueue，如果设置为true，则等待的线程会采用先进先出的顺序访问队列。

```
public SynchronousQueue(boolean fair) {  
    transferer = fair ? new TransferQueue() : new TransferStack();  
}
```

SynchronousQueue可以看成是一个传球手，负责把生产者线程处理的数据直接传递给消费者线程。队列本身并不存储任何元素，非常适合传递性场景。SynchronousQueue的吞吐量高于LinkedBlockingQueue和ArrayBlockingQueue。

6.LinkedTransferQueue

LinkedTransferQueue是一个由链表结构组成的无界阻塞TransferQueue队列。相对于其他阻塞队列，LinkedTransferQueue多了tryTransfer和transfer方法。

(1)transfer方法

如果当前有消费者正在等待接收元素(消费者使用take()方法或带时间限制的poll()方法时)，transfer方法可以把生产者传入的元素立刻transfer(传输)给消费者。如果没有消费者在等待接收元素，transfer方法会将元素存放在队列的tail节点，并等到该元素被消费者消费了才返回。transfer方法的关键代码如下。

```
Node pred = tryAppend(s, haveData);  
return awaitMatch(s, pred, e, (how == TIMED), nanos);
```

第一行代码是试图把存放当前元素的s节点作为tail节点。第二行代码是让CPU自旋等待消费者消费元素。因为自旋会消耗CPU, 所以自旋一定的次数后使用Thread.yield()方法来暂停当前正在执行的线程, 并执行其他线程。

(2)tryTransfer方法

tryTransfer方法是用来试探生产者传入的元素是否能直接传给消费者。如果没有消费者等待接收元素, 则返回false。和transfer方法的区别是tryTransfer方法无论消费者是否接收, 方法立即返回, 而transfer方法是必须等到消费者消费了才返回。

对于带有时间限制的tryTransfer(E e, long timeout, TimeUnit unit)方法, 试图把生产者传入的元素直接传给消费者, 但是如果消费者消费该元素则等待指定的时间再返回, 如果超时还没消费元素, 则返回false, 如果在超时时间内消费了元素, 则返回true。

7.LinkedBlockingDeque

双向队列设置了容量capacity后, 队列的大小就固定不变, 默认大小Integer.MAX_VALUE
LinkedBlockingDeque是一个由链表结构组成的双向阻塞队列。所谓双向队列指的是可以从队列的两端插入和移出元素。双向队列因为多了一个操作队列的入口, 在多线程同时入队时, 也就减少了一半的竞争。相比其他的阻塞队列, LinkedBlockingDeque多了addFirst、addLast、offerFirst、offerLast、peekFirst和peekLast等方法, 以First单词结尾的方法, 表示插入、获取(peek)或移除双端队列的第一个元素。以Last单词结尾的方法, 表示插入、获取或移除双端队列的最后一个元素。另外, 插入方法add等同于addLast, 移除方法remove等效于removeFirst。但是take方法却等同于takeFirst, 不知道是不是JDK的bug, 使用时还是用带有First和Last后缀的方法更清楚。

在初始化LinkedBlockingDeque时可以设置容量防止其过度膨胀。另外, 双向阻塞队列可以运用在“工作窃取”模式中。

6.3.3 阻塞队列的实现原理

如果队列是空的, 消费者会一直等待, 当生产者添加元素时, 消费者是如何知道当前队列有元素的呢? 如果让你来设计阻塞队列你会如何设计, 如何让生产者和消费者进行高效率的通信呢? 让我们先来看看JDK是如何实现的。

使用两个condition实现: notFull 和 notEmpty

使用通知模式实现。所谓通知模式, 就是当生产者往满的队列里添加元素时会阻塞住生产者, 当消费者消费了一个队列中的元素后, 会通知生产者当前队列可用。通过查看JDK源码发现ArrayBlockingQueue使用了Condition来实现, 代码如下。

```
private final Condition notFull;
private final Condition notEmpty;
public ArrayBlockingQueue(int capacity, boolean fair) {
    // 省略其他代码
    notEmpty = lock.newCondition();
    notFull = lock.newCondition();
}
public void put(E e) throws InterruptedException {
    checkNotNull(e);
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        while (count == items.length)
            notFull.await();
        insert(e);
    } finally {
        lock.unlock();
    }
}
public E take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        while (count == 0)
            notEmpty.await();
        return extract();
    } finally {
        lock.unlock();
    }
}
private void insert(E x) {
    items[putIndex] = x;
    putIndex = inc(putIndex);
    ++count;
    notEmpty.signal();
}
```

当往队列里插入一个元素时，如果队列不可用，那么阻塞生产者主要通过

`LockSupport.park(this)`来实现。

```
public final void await() throws InterruptedException {
    if (Thread.interrupted())
        throw new InterruptedException();
    Node node = addConditionWaiter();
    int savedState = fullyRelease(node);
    int interruptMode = 0;
    while (!isOnSyncQueue(node)) {
        LockSupport.park(this);
        if ((interruptMode = checkInterruptWhileWaiting(node)) != 0)
            break;
    }
    if (acquireQueued(node, savedState) && interruptMode != THROW_IE)
        interruptMode = REINTERRUPT;
    if (node.nextWaiter != null) // clean up if cancelled
        unlinkCancelledWaiters();
    if (interruptMode != 0)
        reportInterruptAfterWait(interruptMode);
}
```

继续进入源码，发现调用`setBlocker`先保存一下将要阻塞的线程，然后调用`unsafe.park`阻塞

当前线程。

```
public static void park(Object blocker) {
    Thread t = Thread.currentThread();
    setBlocker(t, blocker);
    unsafe.park(false, 0L);
    setBlocker(t, null);
}
```

`unsafe.park`是个native方法，代码如下。

```
public native void park(boolean isAbsolute, long time);
```

park这个方法会阻塞当前线程，只有以下4种情况中的一种发生时，该方法才会返回。

· 与park对应的unpark执行或已经执行时。“已经执行”是指unpark先执行，然后再执行park

的情况。

·线程被中断时。

·等待完time参数指定的毫秒数时。

·异常现象发生时, 这个异常现象没有任何原因。

继续看一下JVM是如何实现park方法: park在不同的操作系统中使用不同的方式实现, 在Linux下使用的是系统方法pthread_cond_wait实现。实现代码在JVM源码路径src/os/linux/vm/os_linux.cpp里的os::PlatformEvent::park方法, 代码如下。

```
void os::PlatformEvent::park() {
    int v ;
    for (;;) {
        v = _Event ;
        if (Atomic::cmpxchg (v-1, &_Event, v) == v) break ;
    }
    guarantee (v >= 0, "invariant") ;
    if (v == 0) {
        // Do this the hard way by blocking ...
        int status = pthread_mutex_lock(_mutex);
        assert_status(status == 0, status, "mutex_lock");
        guarantee (_nParked == 0, "invariant") ;
        ++ _nParked ;
        while (_Event < 0) {
            status = pthread_cond_wait(_cond, _mutex);
            // for some reason, under 2.7 lwp_cond_wait() may return ETIME ...
            // Treat this the same as if the wait was interrupted
            if (status == ETIME) { status = EINTR; }
            assert_status(status == 0 || status == EINTR, status, "cond_wait");
        }
        -- _nParked ;
        // In theory we could move the ST of 0 into _Event past the unlock(),
        // but then we'd need a MEMBAR after the ST.
        _Event = 0 ;
        status = pthread_mutex_unlock(_mutex);
        assert_status(status == 0, status, "mutex_unlock");
    }
    guarantee (_Event >= 0, "invariant") ;
}
}
```

pthread_cond_wait是一个多线程的条件变量函数, cond是condition的缩写, 字面意思可以

理解为线程在等待一个条件发生, 这个条件是一个全局变量。这个方法接收两个参数: 一个共享变量_cond, 一个互斥量_mutex。而unpark方法在Linux下是使用pthread_cond_signal实现的。park方法在Windows下则是使用WaitForSingleObject实现的。想知道pthread_cond_wait是如何实现的, 可以参考glibc-2.5的nptl/sysdeps/pthread/pthread_cond_wait.c。

线程会被封装成node, 添加到等待队列的尾部

当线程被阻塞队列阻塞时, 线程会进入WAITING(parking)状态。我们可以使用jstack dump阻塞的生产者线程看到这点, 如下。

```
"main" prio=5 tid=0x00007fc83c000000 nid=0x10164e000 waiting on condition [0x000000010
  java.lang.Thread.State: WAITING (parking)
    at sun.misc.Unsafe.park(Native Method)
      - parking to wait for  <0x0000000140559fe8> (a java.util.concurrent.lock
AbstractQueuedSynchronizer$ConditionObject)
    at java.util.concurrent.locks.LockSupport.park(LockSupport.java:186)
    at java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject
await(AbstractQueuedSynchronizer.java:2043)
    at java.util.concurrent.ArrayBlockingQueue.put(ArrayBlockingQueue.java:3
    at blockingqueue.ArrayBlockingQueueTest.main(ArrayBlockingQueueTest.java
```

6.4 Fork/Join框架

本节将会介绍Fork/Join框架的基本原理、算法、设计方式、应用与实现等。

6.4.1 什么是Fork/Join框架

分治思想

Fork/Join框架是Java 7提供的一个用于并行执行任务的框架，是一个把大任务分割成若干个小任务，最终汇总每个小任务结果后得到大任务结果的框架。

我们再通过Fork和Join这两个单词来理解一下Fork/Join框架。Fork就是把一个大任务切分为若干子任务并行的执行，Join就是合并这些子任务的执行结果，最后得到这个大任务的结果。比如计算 $1+2+\dots+10000$ ，可以分割成10个子任务，每个子任务分别对1000个数进行求和，最终汇总这10个子任务的结果。Fork/Join的运行流程如图6-6所示。

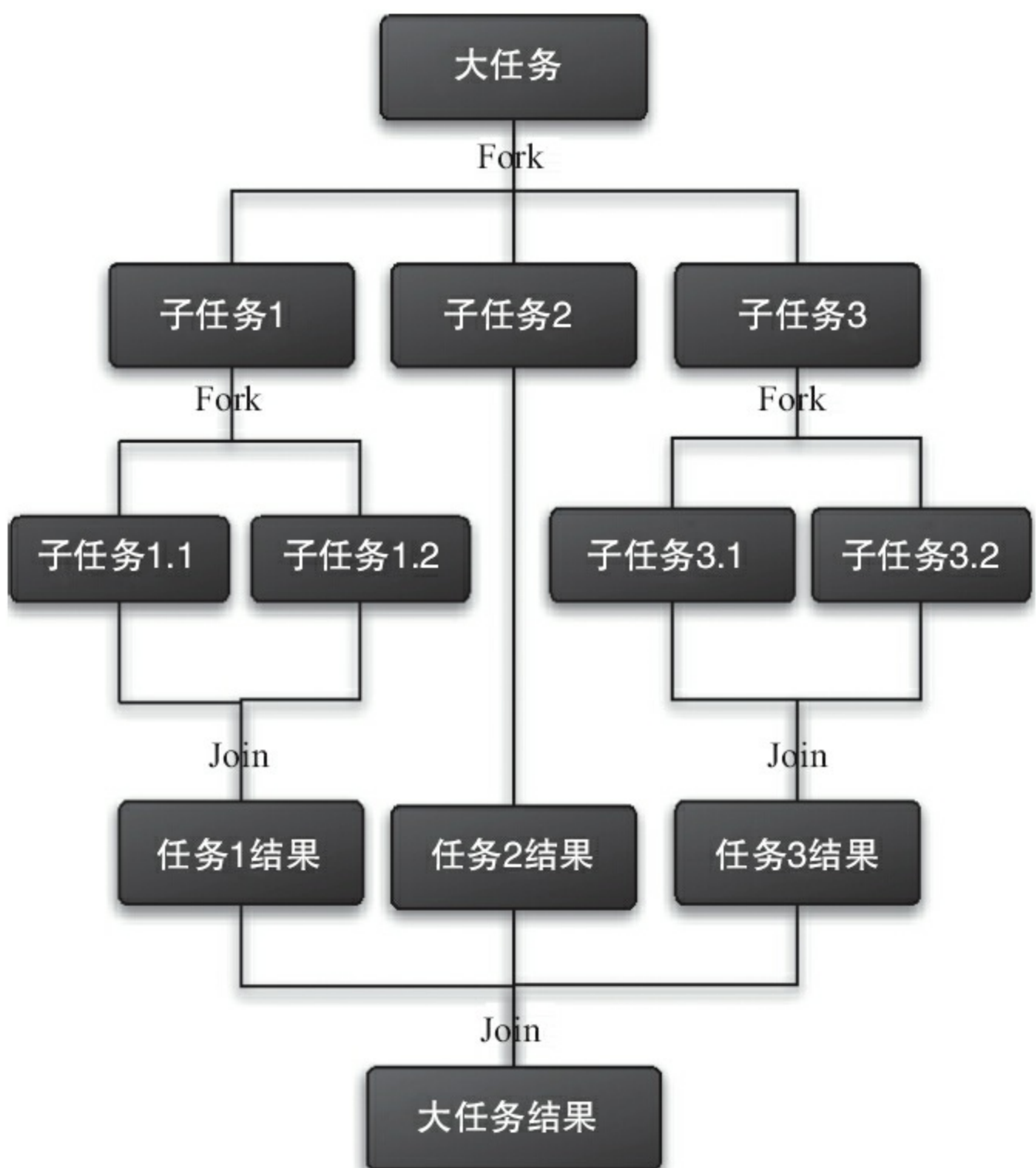


图6-6 Fork Join的运行流程图

6.4.2 工作窃取算法

工作窃取 (work-stealing) 算法是指某个线程从其他队列里窃取任务来执行。那么，为什么需要使用工作窃取算法呢？假如我们需要做一个比较大的任务，可以把这个任务分割为若干互不依赖的子任务，为了减少线程间的竞争，把这些子任务分别放到不同的队列里，并为每个队列创建一个单独的线程来执行队列里的任务，线程和队列一一对应。比如A线程负责处理A队列里的任务。但是，有的线程会先把自己队列里的任务干完，而其他线程对应的队列里还有任务等待处理。干完活的线程与其等着，不如去帮其他线程干活，于是它就去其他线程的队列里窃取一个任务来执行。而在这时它们会访问同一个队列，所以为了减少窃取任务线程和被窃取任务线程之间的竞争，通常会使用双端队列，被窃取任务线程永远从双端队列的头部拿任务执行，而窃取任务的线程永远从双端队列的尾部拿任务执行。

工作窃取的运行流程如图6-7所示。

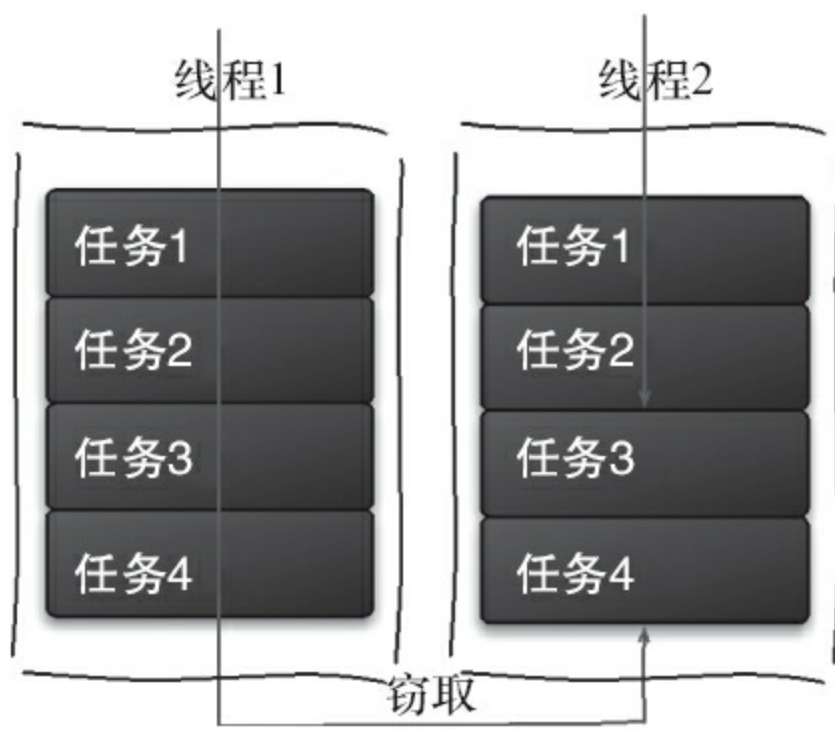


图6-7 工作窃取运行流程图

工作窃取算法的优点：充分利用线程进行并行计算，减少了线程间的竞争。

工作窃取算法的缺点:在某些情况下还是存在竞争, 比如双端队列里只有一个任务时。并且该算法会消耗了更多的系统资源, 比如创建多个线程和多个双端队列。

6.4.3 Fork/Join框架的设计

我们已经很清楚Fork/Join框架的需求了，那么可以思考一下，如果让我们来设计一个Fork/Join框架，该如何设计？这个思考有助于你理解Fork/Join框架的设计。

步骤1 分割任务。首先我们需要有一个fork类来把大任务分割成子任务，有可能子任务还是很大，所以还需要不停地分割，直到分割出的子任务足够小。

步骤2 执行任务并合并结果。分割的子任务分别放在双端队列里，然后几个启动线程分别从双端队列里获取任务执行。子任务执行完的结果都统一放在一个队列里，启动一个线程从队列里拿数据，然后合并这些数据。

Fork/Join使用两个类来完成以上两件事情。

①ForkJoinTask:我们要使用ForkJoin框架，必须首先创建一个ForkJoin任务。它提供在任务中执行fork()和join()操作的机制。通常情况下，我们不需要直接继承ForkJoinTask类，只需要继承它的子类，Fork/Join框架提供了以下两个子类。

·RecursiveAction: 用于没有返回结果的任务。

·RecursiveTask: 用于有返回结果的任务。

②ForkJoinPool:ForkJoinTask需要通过ForkJoinPool来执行。

任务分割出的子任务会添加到当前工作线程所维护的双端队列中，进入队列的头部。当一个工作线程的队列里暂时没有任务时，它会随机从其他工作线程的队列的尾部获取一个任务。

6.4.4 使用Fork/Join框架

让我们通过一个简单的需求来使用Fork/Join框架，需求是：计算 $1+2+3+4$ 的结果。

使用Fork/Join框架首先要考虑到的是如何分割任务，如果希望每个子任务最多执行两个数的相加，那么我们设置分割的阈值是2，由于是4个数字相加，所以Fork/Join框架会把这个任务fork成两个子任务，子任务一负责计算 $1+2$ ，子任务二负责计算 $3+4$ ，然后再join两个子任务的结果。因为是有结果的任务，所以必须继承RecursiveTask，实现代码如下。

```
package fj;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.Future;
import java.util.concurrent.RecursiveTask;
public class CountTask extends RecursiveTask<Integer> {
    private static final int THRESHOLD = 2; // 阈值
    private int start;
    private int end;
    public CountTask(int start, int end) {
        this.start = start;
        this.end = end;
    }
    @Override
    protected Integer compute() {
        int sum = 0;
        // 如果任务足够小就计算任务
        boolean canCompute = (end - start) <= THRESHOLD;
        if (canCompute) {
            for (int i = start; i <= end; i++) {
                sum += i;
            }
        } else {
            // 如果任务大于阈值，就分裂成两个子任务计算
            int middle = (start + end) / 2;
            CountTask leftTask = new CountTask(start, middle);
            CountTask rightTask = new CountTask(middle + 1, end);
            // 执行子任务
            leftTask.fork();
            rightTask.fork();
            // 等待子任务执行完，并得到其结果
            int leftResult=leftTask.join();
            int rightResult=rightTask.join();
            // 合并子任务
            sum = leftResult + rightResult;
        }
        return sum;
    }
}
```



```
public static void main(String[] args) {  
    ForkJoinPool forkJoinPool = new ForkJoinPool();  
    // 生成一个计算任务, 负责计算1+2+3+4  
    CountTask task = new CountTask(1, 4);  
    // 执行一个任务  
    Future<Integer> result = forkJoinPool.submit(task);  
    try {  
        System.out.println(result.get());  
    } catch (InterruptedException e) {  
    } catch (ExecutionException e) {  
    }  
}
```

通过这个例子, 我们进一步了解ForkJoinTask, ForkJoinTask与一般任务的主要区别在于它需要实现compute方法, 在这个方法里, 首先需要判断任务是否足够小, 如果足够小就直接执行任务。如果不够小, 就必须分割成两个子任务, 每个子任务在调用fork方法时, 又会进入compute方法, 看看当前子任务是否需要继续分割成子任务, 如果不需要继续分割, 则执行当前子任务并返回结果。使用join方法会等待子任务执行完并得到其结果。

6.4.5 Fork/Join框架的异常处理

ForkJoinTask在执行的时候可能会抛出异常，但是我们没办法在主线程里直接捕获异常，所以ForkJoinTask提供了isCompletedAbnormally()方法来检查任务是否已经抛出异常或已经被取消了，并且可以通过ForkJoinTask的getException方法获取异常。使用如下代码。

```
if(task.isCompletedAbnormally())
{
    System.out.println(task.getException());
}
```

getException方法返回Throwable对象，如果任务被取消了则返回CancellationException。如果任务没有完成或者没有抛出异常则返回null。

6.4.6 Fork/Join框架的实现原理

ForkJoinPool由ForkJoinTask数组和ForkJoinWorkerThread数组组成，ForkJoinTask数组负责

将存放程序提交给ForkJoinPool的任务，而ForkJoinWorkerThread数组负责执行这些任务。

(1) ForkJoinTask的fork方法实现原理

当我们调用ForkJoinTask的fork方法时，程序会调用ForkJoinWorkerThread的pushTask方法

异步地执行这个任务，然后立即返回结果。代码如下。

```
public final ForkJoinTask<V> fork() {
    ((ForkJoinWorkerThread) Thread.currentThread())
        .pushTask(this);
    return this;
}
```

pushTask方法把当前任务存放在ForkJoinTask数组队列里。然后再调用ForkJoinPool的

signalWork()方法唤醒或创建一个工作线程来执行任务。代码如下。

```
final void pushTask(ForkJoinTask<> t) {
    ForkJoinTask<>[] q; int s, m;
    if ((q = queue) != null) { // ignore if queue removed
        long u = (((s = queueTop) & (m = q.length - 1)) << ASHIFT) + ABASE;
        UNSAFE.putOrderedObject(q, u, t);
        queueTop = s + 1; // or use putOrderedInt
        if ((s -= queueBase) <= 2)
            pool.signalWork();
        else if (s == m)
            growQueue();
    }
}
```

(2) ForkJoinTask的join方法实现原理

Join方法的主要作用是阻塞当前线程并等待获取结果。让我们一起来看看ForkJoinTask的join

方法的实现，代码如下。

```

public final V join() {
    if (doJoin() != NORMAL)
        return reportResult();
    else
        return getRawResult();
}
private V reportResult() {
    int s; Throwable ex;
    if ((s = status) == CANCELLED)
        throw new CancellationException();
    if (s == EXCEPTIONAL && (ex = getThrowableException()) != null)
        UNSAFE.throwException(ex);
    return getRawResult();
}

```

首先, 它调用了doJoin()方法, 通过doJoin()方法得到当前任务的状态来判断返回什么结果, 任务状态有4种: 已完成(NORMAL)、被取消(CANCELLED)、信号(SIGNAL)和出现异常(EXCEPTIONAL)。

- 如果任务状态是已完成, 则直接返回任务结果。
- 如果任务状态是被取消, 则直接抛出CancellationException。
- 如果任务状态是抛出异常, 则直接抛出对应的异常。

让我们再来分析一下doJoin()方法的实现代码。

```

private int doJoin() {
    Thread t; ForkJoinWorkerThread w; int s; boolean completed;
    if ((t = Thread.currentThread()) instanceof ForkJoinWorkerThread) {
        if ((s = status) < 0)
            return s;
        if ((w = (ForkJoinWorkerThread)t).unpushTask(this)) {
            try {
                completed = exec();
            } catch (Throwable rex) {
                return setExceptionalCompletion(rex);
            }
            if (completed)
                return setCompletion(NORMAL);
        }
        return w.joinTask(this);
    }
    else
        return externalAwaitDone();
}

```

在doJoin()方法里, 首先通过查看任务的状态, 看任务是否已经执行完成, 如果执行完成, 则直接返回任务状态; 如果没有执行完, 则从任务数组里取出任务并执行。如果任务顺利执行完成, 则设置任务状态为NORMAL, 如果出现异常, 则记录异常, 并将任务状态设置为EXCEPTIONAL。

6.5 本章小结

本章介绍了Java中提供的各种并发容器和框架，并分析了该容器和框架的实现原理，从中我们能够领略到大师级的设计思路，希望读者能够充分理解这种设计思想，并在以后开发的并发程序时，运用上这些并发编程的技巧。

第7章 Java中的13个原子操作类

当程序更新一个变量时，如果多线程同时更新这个变量，可能得到期望之外的值，比如变量*i*=1，A线程更新*i*+1，B线程也更新*i*+1，经过两个线程操作之后可能*i*不等于3，而是等于2。因为A和B线程在更新变量*i*的时候拿到的*i*都是1，这就是线程不安全的更新操作，通常我们会使用synchronized来解决这个问题，synchronized会保证多线程不会同时更新变量*i*。

而Java从JDK 1.5开始提供了java.util.concurrent.atomic包(以下简称Atomic包)，这个包中的原子操作类提供了一种用法简单、性能高效、线程安全地更新一个变量的方式。

因为变量的类型有很多种，所以在Atomic包里一共提供了13个类，属于4种类型的原子更新方式，分别是原子更新基本类型、原子更新数组、原子更新引用和原子更新属性(字段)。
Atomic包里的类基本都是使用Unsafe实现的包装类。

7.1 原子更新基本类型类

使用原子的方式更新基本类型，Atomic包提供了以下3个类。

· AtomicBoolean: 原子更新布尔类型。

· AtomicInteger: 原子更新整型。

· AtomicLong: 原子更新长整型。

以上3个类提供的方法几乎一模一样，所以本节仅以AtomicInteger为例进行讲解，AtomicInteger的常用方法如下。

· `int addAndGet(int delta)`: 以原子方式将输入的数值与实例中的值(AtomicInteger里的value)相加，并返回结果。

· `boolean compareAndSet(int expect, int update)`: 如果输入的数值等于预期值，则以原子方式将该值设置为输入的值。

· `int getAndIncrement()`: 以原子方式将当前值加1，注意，这里返回的是自增前的值。

· `void lazySet(int newValue)`: 最终会设置成newValue，使用lazySet设置值后，可能导致其他线程在之后的一小段时间内还是可以读到旧的值。关于该方法的更多信息可以参考并发编程网翻译的一篇文章《AtomicLong.lazySet是如何工作的？》，文章地址是“<http://ifeve.com/how-does-atomiclong-lazyset-work/>”。

· `int getAndSet(int newValue)`: 以原子方式设置为newValue的值，并返回旧值。

AtomicInteger示例代码如代码清单7-1所示。

代码清单7-1 AtomicIntegerTest.java


```
import java.util.concurrent.atomic.AtomicInteger;
public class AtomicIntegerTest {
    static AtomicInteger ai = new AtomicInteger(1);
    public static void main(String[] args) {
        System.out.println(ai.getAndIncrement());
        System.out.println(ai.get());
    }
}
```

输出结果如下。

```
1
2
```

那么getAndIncrement是如何实现原子操作的呢？让我们一起分析其实现原理，getAndIncrement的源码如代码清单7-2所示。

代码清单7-2 AtomicInteger.java

```
public final int getAndIncrement() {
    for (;;) {
        int current = get();
        int next = current + 1;
        if (compareAndSet(current, next))
            return current;
    }
}
public final boolean compareAndSet(int expect, int update) {
    return unsafe.compareAndSwapInt(this, valueOffset, expect, update);
}
```

源码中for循环体的第一步先取得AtomicInteger里存储的数值，第二步对AtomicInteger的当前数值进行加1操作，关键的第三步调用compareAndSet方法来进行原子更新操作，该方法先检查当前数值是否等于current，等于意味着AtomicInteger的值没有被其他线程修改过，则将AtomicInteger的当前数值更新成next的值，如果不等compareAndSet方法会返回false，程序会进入for循环重新进行compareAndSet操作。

Atomic包提供了3种基本类型的原子更新，但是Java的基本类型里还有char、float和double

等。那么问题来了，如何原子的更新其他的基本类型呢？Atomic包里的类基本都是使用Unsafe实现的，让我们一起看一下Unsafe的源码，如代码清单7-3所示。

代码清单7-3 Unsafe.java

```
/**
 * 如果当前数值是expected, 则原子的将Java变量更新成x
 * @return 如果更新成功则返回true
 */
public final native boolean compareAndSwapObject(Object o,
                                                    long offset,
                                                    Object expected,
                                                    Object x);

public final native boolean compareAndSwapInt(Object o, long offset,
                                                int expected,
                                                int x);

public final native boolean compareAndSwapLong(Object o, long offset,
                                                long expected,
                                                long x);
```

通过代码，我们发现Unsafe只提供了3种CAS方法：compareAndSwapObject、compareAndSwapInt和compareAndSwapLong，再看AtomicBoolean源码，发现它是先把Boolean转换成整型，再使用compareAndSwapInt进行CAS，所以原子更新char、float和double变量也可以用类似的思路来实现。

AtomicBoolean的compareAndSet()实现源码：同理char、float、double可以使用compareAndSwapObject()方法实现

```
public final boolean compareAndSet(boolean expect, boolean update) {
    int e = expect ? 1 : 0;
    int u = update ? 1 : 0;
    return unsafe.compareAndSwapInt(this, valueOffset, e, u);
}
```

7.2 原子更新数组

通过原子的方式更新数组里的某个元素，Atomic包提供了以下4个类。

·AtomicIntegerArray: 原子更新整型数组里的元素。

·AtomicLongArray: 原子更新长整型数组里的元素。

·AtomicReferenceArray: 原子更新引用类型数组里的元素。

·AtomicIntegerArray类主要是提供原子的方式更新数组里的整型，其常用方法如下。

·int addAndGet(int i, int delta): 以原子方式将输入值与数组中索引i的元素相加。

·boolean compareAndSet(int i, int expect, int update): 如果当前值等于预期值，则以原子方式将数组位置i的元素设置成update值。

以上几个类提供的方法几乎一样，所以本节仅以AtomicIntegerArray为例进行讲解，AtomicIntegerArray的使用实例代码如代码清单7-4所示。

代码清单7-4 AtomicIntegerArrayTest.java

```
public class AtomicIntegerArrayTest {  
    static int[] value = new int[] { 1, 2 };  
    static AtomicIntegerArray ai = new AtomicIntegerArray(value);  
    public static void main(String[] args) {  
        ai.getAndSet(0, 3);  
        System.out.println(ai.get(0));  
        System.out.println(value[0]);  
    }  
}
```

以下是输出的结果。

需要注意的是，数组value通过构造方法传递进去，然后AtomicIntegerArray会将当前数组复制一份，所以当AtomicIntegerArray对内部的数组元素进行修改时，不会影响传入的数组。

7.3 原子更新引用类型

原子更新基本类型的AtomicInteger，只能更新一个变量，如果要原子更新多个变量，就需要使用这个原子更新引用类型提供的类。Atomic包提供了以下3个类。

AtomicReference: 原子更新引用类型。

AtomicReferenceFieldUpdater: 原子更新引用类型里的字段。

AtomicMarkableReference: 原子更新带有标记位的引用类型。可以原子更新一个布尔类型的标记位和引用类型。构造方法是AtomicMarkableReference(V initialRef, boolean initialMark)。

以上几个类提供的方法几乎一样，所以本节仅以AtomicReference为例进行讲解，AtomicReference的使用示例代码如代码清单7-5所示。

代码清单7-5 AtomicReferenceTest.java

```
public class AtomicReferenceTest {
    public static AtomicReference<user> atomicUserRef = new
        AtomicReference<user>();
    public static void main(String[] args) {
        User user = new User("conan", 15);
        atomicUserRef.set(user);
        User updateUser = new User("Shinichi", 17);
        atomicUserRef.compareAndSet(user, updateUser);
        System.out.println(atomicUserRef.get().getName());
        System.out.println(atomicUserRef.get().getOld());
    }
    static class User {
        private String name;
        private int old;
        public User(String name, int old) {
            this.name = name;
            this.old = old;
        }
        public String getName() {
            return name;
        }
        public int getOld() {
            return old;
        }
    }
}
```

```
    }  
    }  
}
```

代码中首先构建一个user对象，然后把user对象设置进AtomicReferenc中，最后调用compareAndSet方法进行原子更新操作，实现原理同AtomicInteger里的compareAndSet方法。代码执行后输出结果如下。

```
Shinichi  
17
```

7.4 原子更新字段类

如果需原子地更新某个类里的某个字段时，就需要使用原子更新字段类，Atomic包提供了以下3个类进行原子字段更新。

·AtomicIntegerFieldUpdater: 原子更新整型的字段的更新器。**字段必须是：不能是封装类型**
public + int + volatile

·AtomicLongFieldUpdater: 原子更新长整型字段的更新器。**字段必须是：不能是封装类型**
public + long + volatile

·AtomicStampedReference: 原子更新带有版本号的引用类型。该类将整数值与引用关联起来，可用于原子的更新数据和数据的版本号，可以解决使用CAS进行原子更新时可能出现的ABA问题。

要想原子地更新字段类需要两步。第一步，因为原子更新字段类都是抽象类，每次使用的时候必须使用静态方法newUpdater()创建一个更新器，并且需要设置想要更新的类和属性。第二步，更新类的字段(属性)必须使用public volatile修饰符。

以上3个类提供的方法几乎一样，所以本节仅以AtomicIntegerFieldUpdater为例进行讲解，AtomicIntegerFieldUpdater的示例代码如代码清单7-6所示。

代码清单7-6 AtomicIntegerFieldUpdaterTest.java

```
public class AtomicIntegerFieldUpdaterTest {
    // 创建原子更新器，并设置需要更新的对象类和对象的属性
    private static AtomicIntegerFieldUpdater<User> a = AtomicIntegerFieldUpdater.
        newUpdater(User.class, "old");
    public static void main(String[] args) {
        // 设置柯南的年龄是10岁
        User conan = new User("conan", 10);
        // 柯南长了一岁，但是仍然会输出旧的年龄
        System.out.println(a.getAndIncrement(conan));
        // 输出柯南现在的年龄
        System.out.println(a.get(conan));
    }
    public static class User {
        private String name;
        public volatile int old;
```

```
        public User(String name, int old) {  
            this.name = name;  
            this.old = old;  
        }  
        public String getName() {  
            return name;  
        }  
        public int getOld() {  
            return old;  
        }  
    }  
}
```

代码执行后输出如下。

```
10  
11
```

7.5 本章小结

本章介绍了JDK中并发包里的13个原子操作类以及原子操作类的实现原理, 读者需要熟悉这些类和使用场景, 在适当的场合下使用它。

第8章 Java中的并发工具类

在JDK的并发包里提供了几个非常有用的并发工具类。CountDownLatch、CyclicBarrier和

Semaphore工具类提供了一种并发流程控制的手段, Exchanger工具类则提供了在线程间交换数

据的一种手段。本章会配合一些应用场景来介绍如何使用这些工具类。

8.1 等待多线程完成的CountDownLatch

CountDownLatch允许一个或多个线程等待其他线程完成操作。

假如有这样一个需求:我们需要解析一个Excel里多个sheet的数据,此时可以考虑使用多线程,每个线程解析一个sheet里的数据,等到所有的sheet都解析完之后,程序需要提示解析完成。在这个需求中,要实现主线程等待所有线程完成sheet的解析操作,最简单的做法是使用join()方法,如代码清单8-1所示。

代码清单8-1 JoinCountDownLatchTest.java

```
public class JoinCountDownLatchTest {
    public static void main(String[] args) throws InterruptedException {
        Thread parser1 = new Thread(new Runnable() {
            @Override
            public void run() {
                // ...
            }
        });
        Thread parser2 = new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println("parser2 finish");
            }
        });
        parser1.start();
        parser2.start();
        parser1.join();
        parser2.join();
        System.out.println("all parser finish");
    }
}
```

Join的实现原理:

join用于让当前执行线程等待join线程执行结束。其实现原理是不停检查join线程是否存活,如果join线程存活则让当前线程永远等待。其中,wait(0)表示永远等待下去,代码片段如下。

join底层是用:wait()实现地

```
while (isAlive()) {
    wait(0);
}
```

直到join线程中止后，线程的this.notifyAll()方法会被调用，调用notifyAll()方法是在JVM里

实现的，所以在JDK里看不到，大家可以查看JVM源码。

在JDK 1.5之后的并发包中提供的CountDownLatch也可以实现join的功能，并且比join的功能更多，如代码清单8-2所示。

代码清单8-2 CountDownLatchTest.java

```
public class CountDownLatchTest {
    static CountDownLatch c = new CountDownLatch(2);
    public static void main(String[] args) throws InterruptedException {
        new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println(1);
                c.countDown();
                System.out.println(2);
                c.countDown();
            }
        }).start();
        c.await();
        System.out.println("3");
    }
}
```

CountDownLatch的构造函数接收一个int类型的参数作为计数器，如果你想等待N个点完成，这里就传入N。

底层是基于AQS实现的，传入的计数值N赋值给state，调用await()方法时，会判断state是否等于0，如果不等，加入AQS同步队列中，并阻塞；调用countDown()会对state减1，并判断state是否等于0，如果等则唤醒AQS同步队列中线程。

当我们调用CountDownLatch的countDown方法时，N就会减1，CountDownLatch的await方法会阻塞当前线程，直到N变成零。由于countDown方法可以用在任何地方，所以这里说的N个点，可以是N个线程，也可以是1个线程里的N个执行步骤。用在多个线程时，只需要把这个CountDownLatch的引用传递到线程里即可。

如果有某个解析sheet的线程处理得比较慢，我们不可能让主线程一直等待，所以可以使用另外一个带指定时间的await方法——await(long time, TimeUnit unit)，这个方法等待特定时间后，就会不再阻塞当前线程。join也有类似的方法。



注意 计数器必须大于等于0, 只是等于0时候, 计数器就是零, 调用await方法时不会

阻塞当前线程。CountDownLatch不可能重新初始化或者修改CountDownLatch对象的内部计数器的值。一个线程调用countDown方法happen-before, 另外一个线程调用await方法。

8.2 同步屏障CyclicBarrier

CyclicBarrier的字面意思是可循环使用(Cyclic)的屏障(Barrier)。它要做的事情是, 让一组线程到达一个屏障(也可以叫同步点)时被阻塞, 直到最后一个线程到达屏障时, 屏障才会开门, 所有被屏障拦截的线程才会继续运行。

8.2.1 CyclicBarrier简介

CyclicBarrier默认的构造方法是CyclicBarrier(int parties)，其参数表示屏障拦截的线程数量，每个线程调用await方法告诉CyclicBarrier我已经到达了屏障，然后当前线程被阻塞。示例代码如代码清单8-3所示。

代码清单8-3 CyclicBarrierTest.java

```
public class CyclicBarrierTest {
    staticCyclicBarrier c = new CyclicBarrier(2);
    public static void main(String[] args) {
        new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    c.await();
                } catch (Exception e) {
                }
                System.out.println(1);
            }
        }).start();
        try {
            c.await();
        } catch (Exception e) {
        }
        System.out.println(2);
    }
}
```

因为主线程和子线程的调度是由CPU决定的，两个线程都有可能先执行，所以会产生两种输出，第一种可能输出如下。

```
1
2
```

第二种可能输出如下。

```
2
1
```

如果把new CyclicBarrier(2)修改成new CyclicBarrier(3), 则主线程和子线程会永远等待,

因为没有第三个线程执行await方法, 即没有第三个线程到达屏障, 所以之前到达屏障的两个

线程都不会继续执行。 **CyclicBarrier同步屏障底层实也是基于AQS实现的,**
具体实现是: ReentrantLock + Condition

CyclicBarrier还提供一个更高级的构造函数CyclicBarrier(int parties, Runnable barrier-

Action), 用于在线程到达屏障时, 优先执行barrierAction, 方便处理更复杂的业务场景, 如代码

清单8-4所示。**注意执行顺序: CyclicBarrier(int parties, Runnable barrierAction)**
先执行所有子线程run()方法里c.await()前的代码 ---> c.await()达
到屏障数parties后 ---> 再执行完barrierAction线程 ---> 最后执行所有
线程await()方法后的代码。

代码清单8-4 CyclicBarrierTest2.java

```
import java.util.concurrent.CyclicBarrier;
public class CyclicBarrierTest2 {
    static CyclicBarrier c = new CyclicBarrier(2, new A());
    public static void main(String[] args) {
        new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    c.await();
                } catch (Exception e) {
                }
                System.out.println(1);
            }
        }).start();
        try {
            c.await();
        } catch (Exception e) {
        }
        System.out.println(2);
    }
    static class A implements Runnable {
        @Override
        public void run() {
            System.out.println(3);
        }
    }
}
```

因为CyclicBarrier设置了拦截线程的数量是2, 所以必须等代码中的第一个线程和线程A都执行完之后, 才会继续执行主线程, 然后输出2, 所以代码执行后的输出如下。

8.2.2 CyclicBarrier的应用场景

CyclicBarrier可以用于多线程计算数据，最后合并计算结果的场景。例如，用一个Excel保存了用户所有银行流水，每个Sheet保存一个账户近一年的每笔银行流水，现在需要统计用户的日均银行流水，先用多线程处理每个sheet里的银行流水，都执行完之后，得到每个sheet的日均银行流水，最后，再用barrierAction用这些线程的计算结果，计算出整个Excel的日均银行流水，如代码清单8-5所示。

代码清单8-5 BankWaterService.java

```
import java.util.Map.Entry;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.Executor;
import java.util.concurrent.Executors;
/**
 * 银行流水处理服务类
 *
 * @author ftf
 */
public class BankWaterService implements Runnable {
    /**
     * 创建4个屏障，处理完之后执行当前类的run方法
     */
    private CyclicBarrier c = new CyclicBarrier(4, this);
    /**
     * 假设只有4个sheet，所以只启动4个线程
     */
    private Executor executor = Executors.newFixedThreadPool(4);
    /**
     * 保存每个sheet计算出的银流结果
     */
    private ConcurrentHashMap<String, Integer> sheetBankWaterCount = new
        ConcurrentHashMap<String, Integer>();
    private void count() {
        for (inti = 0; i < 4; i++) {
            executor.execute(new Runnable() {
                @Override
                public void run() {
                    // 计算当前sheet的银流数据，计算代码省略
                    sheetBankWaterCount
                        .put(Thread.currentThread().getName(), 1);
                    // 银流计算完成，插入一个屏障
                }
            });
        }
    }
}
```


8.2.3 CyclicBarrier和CountDownLatch的区别

CyclicBarrier同步屏障可以重复使用，当await()达到屏障数后，计数值count又赋值成初值，进入到下一轮屏障计数。
CountDownLatch的计数器只能使用一次，而**CyclicBarrier**的计数器可以使用reset()方法重置。所以**CyclicBarrier**能处理更为复杂的业务场景。例如，如果计算发生错误，可以重置计数器，并让线程重新执行一次。

CyclicBarrier还提供其他有用的方法，比如getNumberWaiting方法可以获得Cyclic-Barrier阻塞的线程数量。isBroken()方法用来了解阻塞的线程是否被中断。代码清单8-5执行完之后会返回true，其中isBroken的使用代码如代码清单8-6所示。

代码清单8-6 CyclicBarrierTest3.java

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
public class CyclicBarrierTest3 {
    static CyclicBarrier c = new CyclicBarrier(2);
    public static void main(String[] args) throws InterruptedException,
        BrokenBarrierException {
        Thread thread = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    c.await();
                } catch (Exception e) {
                }
            }
        });
        thread.start();
        thread.interrupt();
        try {
            c.await();
        } catch (Exception e) {
            System.out.println(c.isBroken());
        }
    }
}
```

输出如下所示。

true

8.3 控制并发线程数的Semaphore

Semaphore(信号量)是用来控制同时访问特定资源的线程数量,它通过协调各个线程,以保证合理的使用公共资源。

多年以来,我都觉得从字面上很难理解Semaphore所表达的含义,只能把它比作是控制流量的红绿灯。比如××马路要限制流量,只允许同时有一百辆车在这条路上行使,其他的都必须在路口等待,所以前一百辆车会看到绿灯,可以开进这条马路,后面的车会看到红灯,不能驶入××马路,但是如果前一百辆中有5辆车已经离开了××马路,那么后面就允许有5辆车驶入马路,这个例子里说的车就是线程,驶入马路就表示线程在执行,离开马路就表示线程执行完成,看见红灯就表示线程被阻塞,不能执行。

1.应用场景

Semaphore可以用于做流量控制,特别是公用资源有限的应用场景,比如数据库连接。假如有一个需求,要读取几万个文件的数据,因为都是IO密集型任务,我们可以启动几十个线程并发地读取,但是如果读到内存后,还需要存储到数据库中,而数据库的连接数只有10个,这时我们必须控制只有10个线程同时获取数据库连接保存数据,否则会报错无法获取数据库连接。这个时候,就可以使用Semaphore来做流量控制,如代码清单8-7所示。

代码清单8-7 SemaphoreTest.java

```
public class SemaphoreTest {
    private static final int THREAD_COUNT = 30;
    private static ExecutorService threadPool = Executors
        .newFixedThreadPool(THREAD_COUNT);
    private static Semaphore s = new Semaphore(10);
    public static void main(String[] args) {
        for (inti = 0; i< THREAD_COUNT; i++) {
            threadPool.execute(new Runnable() {
                @Override
                public void run() {
                    try {
                        s.acquire();
```

```
        System.out.println("save data");
        s.release();
    } catch (InterruptedException e) {
    }
}
});
}
ThreadPool.shutdown();
}
```

在代码中，虽然有30个线程在执行，但是只允许10个并发执行。Semaphore的构造方法 Semaphore(int permits) 接受一个整型的数字，表示可用的许可证数量。Semaphore(10) 表示允许10个线程获取许可证，也就是最大并发数是10。Semaphore的用法也很简单，首先线程使用 Semaphore的acquire()方法获取一个许可证，使用完之后调用release()方法归还许可证。还可以用tryAcquire()方法尝试获取许可证。

2.其他方法

Semaphore还提供一些其他方法，具体如下。

· int availablePermits(): 返回此信号量中当前可用的许可证数。

· int getQueueLength(): 返回正在等待获取许可证的线程数。

· boolean hasQueuedThreads(): 是否有线程正在等待获取许可证。

· void reducePermits(int reduction): 减少reduction个许可证，是个protected方法。

· Collection getQueuedThreads(): 返回所有等待获取许可证的线程集合，是个protected方法。

8.4 线程间交换数据的Exchanger

Exchanger(交换者)是一个用于线程间协作的工具类。Exchanger用于进行线程间的数据交换。它提供一个同步点, 在这个同步点, 两个线程可以交换彼此的数据。这两个线程通过exchange方法交换数据, 如果第一个线程先执行exchange()方法, 它会一直等待第二个线程也执行exchange方法, 当两个线程都到达同步点时, 这两个线程就可以交换数据, 将本线程生产出来的数据传递给对方。

下面来看一下Exchanger的应用场景。

Exchanger可以用于遗传算法, 遗传算法里需要选出两个人作为交配对象, 这时候会交换两人的数据, 并使用交叉规则得出2个交配结果。Exchanger也可以用于校对工作, 比如我们需要将纸制银行流水通过人工的方式录入成电子银行流水, 为了避免错误, 采用AB岗两人进行录入, 录入到Excel之后, 系统需要加载这两个Excel, 并对两个Excel数据进行校对, 看看是否录入一致, 代码如代码清单8-8所示。

代码清单8-8 ExchangerTest.java

```
public class ExchangerTest {
    private static final Exchanger<String>exgr = new Exchanger<String>();
    private static ExecutorService threadPool = Executors.newFixedThreadPool(2);
    public static void main(String[] args) {
        threadPool.execute(new Runnable() {
            @Override
            public void run() {
                try {
                    String A = "银行流水A"; // A录入银行流水数据
                    exgr.exchange(A);
                } catch (InterruptedException e) {
                }
            }
        });
        threadPool.execute(new Runnable() {
            @Override
            public void run() {
                try {
                    String B = "银行流水B"; // B录入银行流水数据
                    String A = exgr.exchange("B");
                    System.out.println("A和B数据是否一致:" + A.equals(B) + ", A录入的是:"
```



```
        + A + ", B输入是:" + B);  
    } catch (InterruptedException e) {  
    }  
    }  
    });  
    threadPool.shutdown();  
    }  
}
```

如果两个线程有一个没有执行exchange()方法, 则会一直等待, 如果担心有特殊情况发生, 避免一直等待, 可以使用exchange(V x, long timeout, TimeUnit unit) 设置最大等待时长。

8.5 本章小结

本章配合一些应用场景介绍JDK中提供的几个并发工具类, 大家记住这个工具类的用途, 一旦有对应的业务场景, 不妨试试这些工具类。

第9章 Java中的线程池

Java中的线程池是运用场景最多的并发框架，几乎所有需要异步或并发执行任务的程序都可以使用线程池。在开发过程中，合理地使用线程池能够带来3个好处。

第一：**降低资源消耗**。通过重复利用已创建的线程降低线程创建和销毁造成的消耗。

第二：**提高响应速度**。当任务到达时，任务可以不需要等到线程创建就能立即执行。

第三：**提高线程的可管理性**。线程是稀缺资源，如果无限制地创建，不仅会消耗系统资源，还会降低系统的稳定性，使用线程池可以进行统一分配、调优和监控。但是，要做到合理利用线程池，必须对其实现原理了如指掌。

9.1 线程池的实现原理

当向线程池提交一个任务之后，线程池是如何处理这个任务的呢？本节来看一下线程池的主要处理流程，处理流程图如图9-1所示。

从图中可以看出，当提交一个新任务到线程池时，线程池的处理流程如下。

线程池实现原理:3个步骤(核心线程池，工作队列，线程池的线程是否都处于工作状态)

1)线程池判断核心线程池里的线程是否都在执行任务。如果不是，则创建一个新的工作线程来执行任务。corePoolSize大小如果核心线程池里的线程都在执行任务，则进入下个流程。

2)线程池判断workQueue是否已经满。如果工作队列没有满，则将新提交的任务存储在这个工作队列里。如果工作队列满了，则进入下个流程。

当前运行的线程是否超出maximumPoolSize
3)线程池判断线程池的线程是否都处于工作状态。如果没有，则创建一个新的工作线程来执行任务。如果已经满了，则交给饱和策略来处理这个任务。

ThreadPoolExecutor执行execute()方法的示意图，如图9-2所示。

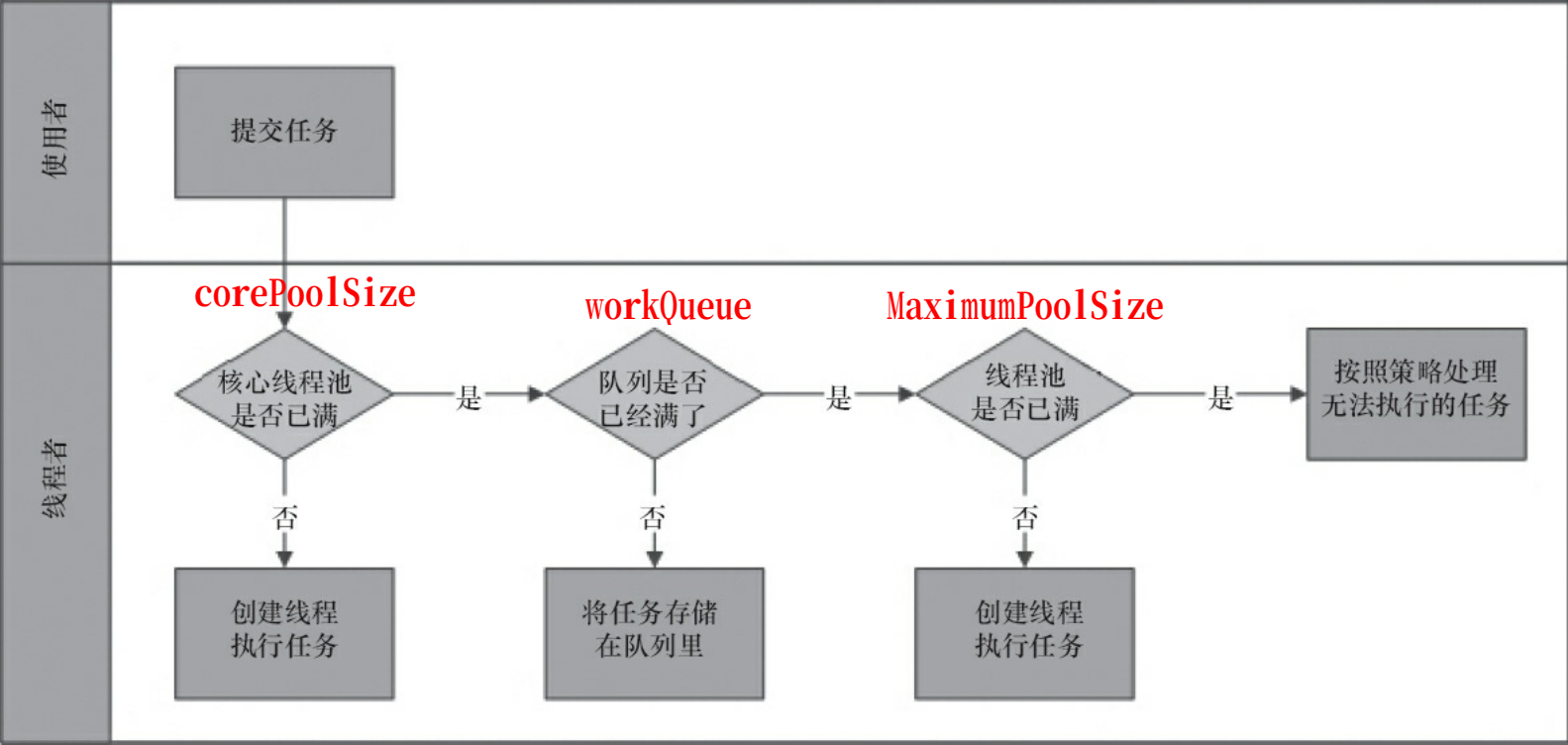


图9-1 线程池的主要处理流程

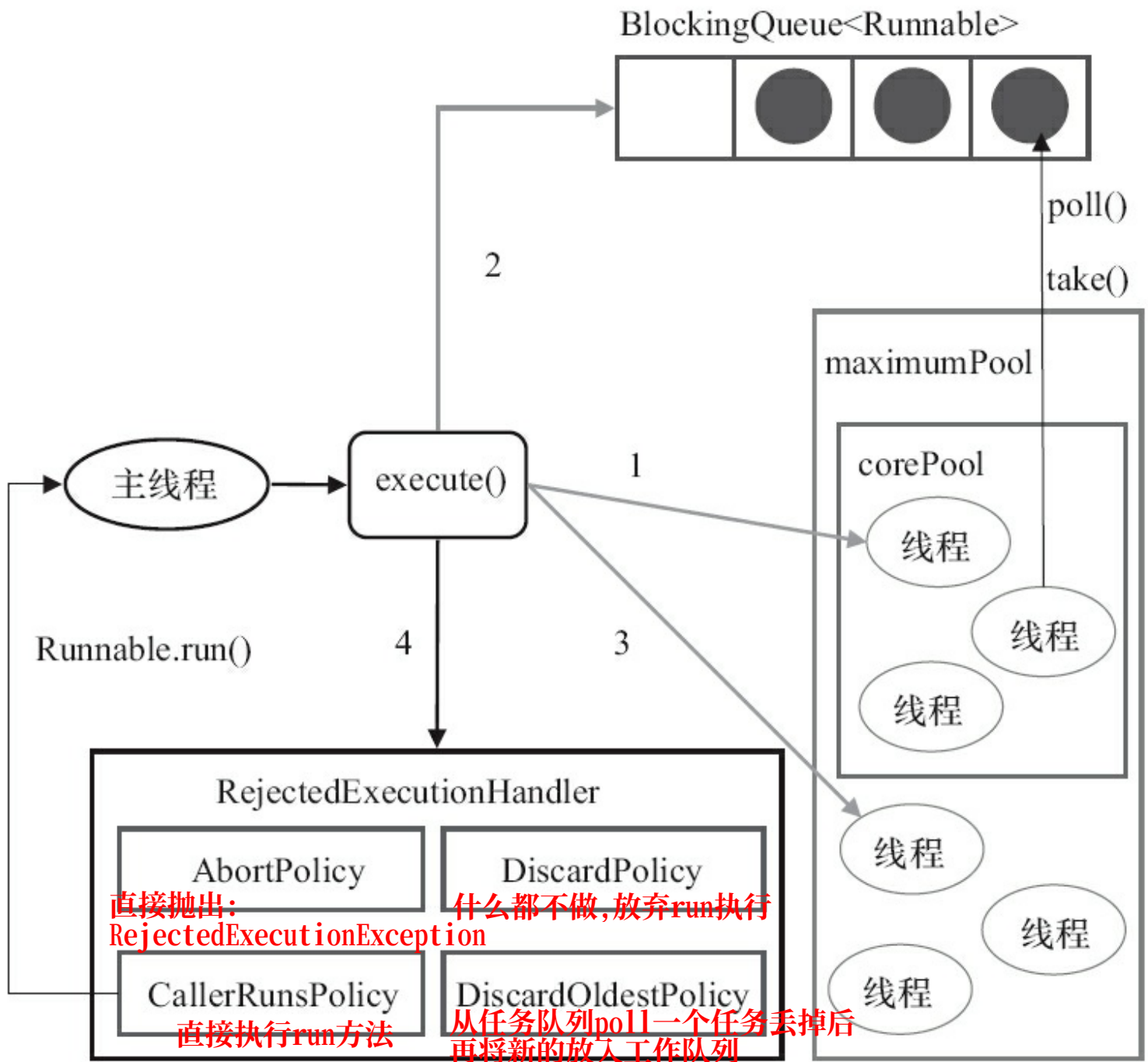


图9-2 ThreadPoolExecutor执行示意图

ThreadPoolExecutor执行execute方法分下面4种情况。

1) 如果当前运行的线程少于corePoolSize, 则创建新线程来执行任务 (注意, 执行这一步骤需要获取全局锁)。**mainLock全局锁(是ReentrantLock实现):** 因为要计算线程的状态, 如 `corePoolSize`, `maximumPoolSize` 等

2) 如果运行的线程等于或多于corePoolSize, 则将任务加入BlockingQueue。

3) 如果无法将任务加入BlockingQueue(队列已满), 则创建新的线程来处理任务 (注意, 执

行这一步骤需要获取全局锁)。mainLock全局锁(ReentrantLock实现)

4) 如果创建新线程将使当前运行的线程超出maximumPoolSize, 任务将被拒绝, 并调用RejectedExecutionHandler.rejectedExecution()方法。

ThreadPoolExecutor采取上述步骤的总体设计思路, 是为了在执行execute()方法时, 尽可能地避免获取全局锁(那将会是一个严重的可伸缩瓶颈)。在ThreadPoolExecutor完成预热之后(当前运行的线程数大于等于corePoolSize), 几乎所有的execute()方法调用都是执行步骤2, 而步骤2不需要获取全局锁。

源码分析:上面的流程分析让我们很直观地了解了线程池的工作原理, 让我们再通过源代码来看看是如何实现的, 线程池执行任务的方法如下。

```
public void execute(Runnable command) {
    if (command == null)
        throw new NullPointerException();
    // 如果线程数小于基本线程数, 则创建线程并执行当前任务
    if (poolSize >= corePoolSize || !addIfUnderCorePoolSize(command)) {
        // 如线程数大于等于基本线程数或线程创建失败, 则将当前任务放到工作队列中。
        if (runState == RUNNING && workQueue.offer(command)) {
            if (runState != RUNNING || poolSize == 0)
                ensureQueuedTaskHandled(command);
        }
        // 如果线程池不处于运行中或任务无法放入队列, 并且当前线程数量小于最大允许的线程数量,
        // 则创建一个线程执行任务。
        else if (!addIfUnderMaximumPoolSize(command))
            // 抛出RejectedExecutionException异常
            reject(command); // is shutdown or saturated
    }
}
```

工作线程:线程池创建线程时, 会将线程封装成工作线程Worker, Worker在执行完任务后, 还会循环获取工作队列里的任务来执行。我们可以从Worker类的run()方法里看到这点。

```
public void run() {
    try {
        Runnable task = firstTask;
        firstTask = null;
        while (task != null || (task = getTask()) != null) {
            runTask(task);
        }
    }
}
```

```

        task = null;
    }
    } finally {
        workerDone(this);
    }
}

```

ThreadPoolExecutor中线程执行任务的示意图如图9-3所示。

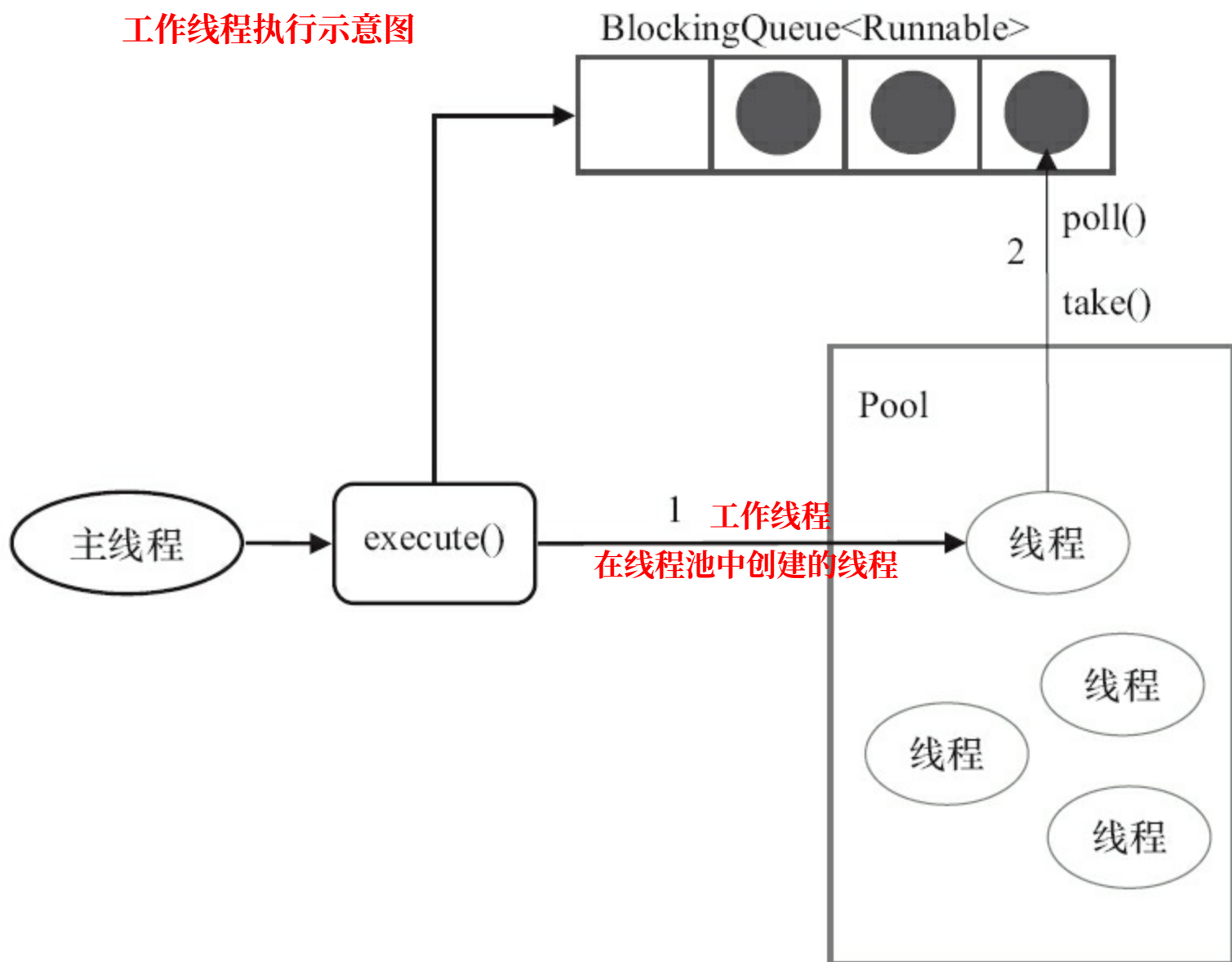


图9-3 ThreadPoolExecutor执行任务示意图

线程池中的线程执行任务分两种情况，如下。

工作线程:执行的两种情况

1) 在execute()方法中创建一个线程时，会让这个线程执行当前任务。

2) 这个线程执行完上图中1的任务后, 会反复从BlockingQueue获取任务来执行。

9.2 线程池的使用

9.2.1 线程池的创建

我们可以通过ThreadPoolExecutor来创建一个线程池。

```
new ThreadPoolExecutor(corePoolSize, maximumPoolSize, keepAliveTime,
    milliseconds, runnableTaskQueue, handler);
```

创建一个线程池时需要输入几个参数，如下。

1) corePoolSize(线程池的基本大小): 当提交一个任务到线程池时，线程池会创建一个线程来执行任务，即使其他空闲的基本线程能够执行新任务也会创建线程，等到需要执行的任务数大于线程池基本大小时就不再创建。如果调用了线程池的prestartAllCoreThreads()方法，线程池会提前创建并启动所有基本线程。

2) runnableTaskQueue(任务队列): 用于保存等待执行的任务的阻塞队列。可以选择以下几个阻塞队列。

BlockingQueue接口的实现(任务队列)

· ArrayBlockingQueue: 是一个基于数组结构的有界阻塞队列，此队列按FIFO(先进先出)原则对元素进行排序。

· LinkedBlockingQueue: 一个基于链表结构的阻塞队列，此队列按FIFO排序元素，吞吐量通常要高于ArrayBlockingQueue。静态工厂方法Executors.newFixedThreadPool()使用了这个队列。

· SynchronousQueue: 一个不存储元素的阻塞队列。每个插入操作必须等到另一个线程调用移除操作，否则插入操作一直处于阻塞状态，吞吐量通常要高于Linked-BlockingQueue，静态工厂方法Executors.newCachedThreadPool使用了这个队列。

· PriorityBlockingQueue: 一个具有优先级的无限阻塞队列。

3) maximumPoolSize(线程池最大数量): 线程池允许创建的最大线程数。如果队列满了, 并且已创建的线程数小于最大线程数, 则线程池会再创建新的线程执行任务。**值得注意的是, 如果使用了无界的任务队列这个参数就没什么效果。**

4) ThreadFactory: 用于设置创建线程的工厂, 可以通过线程工厂给每个创建出来的线程设置更有意义的名字。使用开源框架guava提供的ThreadFactoryBuilder可以快速给线程池里的线程设置有意义的名字, 代码如下。

ThreadFactory的作用: 在newThread()中创建线程池的所有线程。
流程: ThreadPoolExecutor提交一个runnable ---> new worker()
---> 在Worker中ThreadFactory创建一个thread ---> 再将runnable和thread封装成一个worker

```
new ThreadFactoryBuilder().setNameFormat("XX-task-%d").build();
```

5) RejectedExecutionHandler(饱和策略): 当队列和线程池都满了, 说明线程池处于饱和状态, 那么必须采取一种策略处理提交的新任务。这个策略默认情况下是AbortPolicy, 表示无法处理新任务时抛出异常。在JDK 1.5中Java线程池框架提供了以下4种策略。

饱和策略

- AbortPolicy: 直接抛出异常。**默认的饱和策略**
- CallerRunsPolicy: 只用调用者所在线程来运行任务。**直接就是: thread.run()方式执行, 调用者线程来执行。**
- DiscardOldestPolicy: 丢弃队列里最近的一个任务, 并执行当前任务。
- DiscardPolicy: 不处理, 丢弃掉。

当然, 也可以根据应用场景需要来实现RejectedExecutionHandler接口自定义策略。如记录日志或持久化存储不能处理的任务。

该时间是超过核心池大小的线程可以在终止前保持的空闲时间

keepAliveTime(线程活动保持时间): 线程池的工作线程空闲后, 保持存活的时间。所以, 如果任务很多, 并且每个任务执行的时间比较短, 可以调大时间, 提高线程的利用率。

TimeUnit(线程活动保持时间的单位): 可选的单位有天(DAYS)、小时(HOURS)、分钟(MINUTES)、毫秒(MILLISECONDS)、微秒(MICROSECONDS, 千分之一毫秒)和纳秒

(NANOSECONDS, 千分之一微秒)。

9.2.2 向线程池提交任务

向线程池提交任务：(1) `execute()` (2) `submit()`

可以使用两个方法向线程池提交任务，分别为`execute()`和`submit()`方法。

`execute()`方法用于提交不需要返回值的任务，所以无法判断任务是否被线程池执行成功。

通过以下代码可知`execute()`方法输入的任务是一个`Runnable`类的实例。

```
threadsPool.execute(new Runnable() {  
    @Override  
    public void run() {  
        // TODO Auto-generated method stub  
    }  
});
```

`submit()`方法用于提交需要返回值的任务。线程池会返回一个`future`类型的对象，通过这个`future`对象可以判断任务是否执行成功，并且可以通过`future`的`get()`方法来获取返回值，`get()`方法会阻塞当前线程直到任务完成，而使用`get(long timeout, TimeUnit unit)`方法则会阻塞当前线程一段时间后立即返回，这时候有可能任务没有执行完。

```
Future<Object> future = executor.submit(harReturnValuetask);  
try {  
    Object s = future.get();  
} catch (InterruptedException e) {  
    // 处理中断异常  
} catch (ExecutionException e) {  
    // 处理无法执行任务异常  
} finally {  
    // 关闭线程池  
    executor.shutdown();  
}
```

9.2.3 关闭线程池

可以通过调用线程池的shutdown或shutdownNow方法来关闭线程池。它们的原理是遍历线程池中的工作线程，然后逐个调用线程的interrupt方法来中断线程，所以无法响应中断的任务可能永远无法终止。但是它们存在一定的区别，shutdownNow首先将线程池的状态设置成STOP，然后尝试停止所有的正在执行或暂停任务的线程，并返回等待执行任务的列表，而shutdown只是将线程池的状态设置成SHUTDOWN状态，然后中断所有没有正在执行任务的线程。

关闭线程池：shutdown()和shutdownNow()的区别

shutdown()：线程池设置为SHUTDOWN状态，能把当前正在执行的任务和任务堵塞队列里的任务执行完。
shutdownNow()：线程池设置为STOP状态，把当前正在执行的任务执行完，任务堵塞队列里的任务不能执行。

[注意：任务正在执行sleep()时，调用shutdownNow()会抛出InterruptedException()异常，shutdown()不会]

只要调用了这两个关闭方法中的任意一个，isShutdown方法就会返回true。当所有的任务都已关闭后，才表示线程池关闭成功，这时调用isTerminated方法会返回true。至于应该调用哪一种方法来关闭线程池，应该由提交到线程池的任务特性决定，通常调用shutdown方法来关闭线程池，如果任务不一定要执行完，则可以调用shutdownNow方法。

9.2.4 合理地配置线程池

要想合理地配置线程池,就必须首先分析任务特性,可以从以下几个角度来分析。

- 任务的性质:CPU密集型任务、IO密集型任务和混合型任务。
- 任务的优先级:高、中和低。可以使用优先级队列PriorityBlockingQueue来处理
- 任务的执行时间:长、中和短。
- 任务的依赖性:是否依赖其他系统资源,如数据库连接。

性质不同的任务可以用不同规模的线程池分开处理。CPU密集型任务应配置尽可能小的线程,如配置 $N_{\text{cpu}}+1$ 个线程的线程池。由于IO密集型任务线程并不是一直在执行任务,则应配置尽可能多的线程,如 $2*N_{\text{cpu}}$ 。混合型的任务,如果可以拆分,将其拆分成一个CPU密集型任务和一个IO密集型任务,只要这两个任务执行的时间相差不是太大,那么分解后执行的吞吐量将高于串行执行的吞吐量。如果这两个任务执行时间相差太大,则没必要进行分解。可以通过`Runtime.getRuntime().availableProcessors()`方法获得当前设备的CPU个数。

优先级不同的任务可以使用优先级队列PriorityBlockingQueue来处理。它可以让优先级高的任务先执行。

 **注意** 如果一直有优先级高的任务提交到队列里,那么优先级低的任务可能永远不能执行。

执行时间不同的任务可以交给不同规模的线程池来处理,或者可以使用优先级队列,让执行时间短的任务先执行。

依赖数据库连接池的任务,因为线程提交SQL后需要等待数据库返回结果,等待的时间越长,则CPU空闲时间就越长,那么线程数应该设置得越大,这样才能更好地利用CPU。

建议使用有界队列。有界队列能增加系统的稳定性和预警能力，可以根据需要设大一点

儿，比如几千。有一次，我们系统里后台任务线程池的队列和线程池全满了，不断抛出抛弃任务的异常，通过排查发现是数据库出现了问题，导致执行SQL变得非常缓慢，因为后台任务线程池里的任务全是需要向数据库查询和插入数据的，所以导致线程池里的工作线程全部阻塞，任务积压在线程池里。如果当时我们设置成无界队列，那么线程池的队列就会越来越多，有可能会撑满内存，导致整个系统不可用，而不只是后台任务出现问题。当然，我们的系统所有的任务是用单独的服务器部署的，我们使用不同规模的线程池完成不同类型的任务，但是出现这样问题时也会影响到其他任务。

9.2.5 线程池的监控

如果在系统中大量使用线程池，则有必要对线程池进行监控，方便在出现问题时，可以根据线程池的使用状况快速定位问题。可以通过线程池提供的参数进行监控，在监控线程池的时候可以使用以下属性。

`taskCount`: 线程池需要执行的任务数量。
总任务数

`completedTaskCount`: 线程池在运行过程中已完成的任务数量，小于或等于`taskCount`。

`largestPoolSize`: 线程池里曾经创建过的最大线程数量。通过这个数据可以知道线程池是否曾经满过。如该数值等于线程池的最大大小，则表示线程池曾经满过。

`getPoolSize`: 线程池的线程数量。如果线程池不销毁的话，线程池里的线程不会自动销毁，所以这个大小只增不减。

`getActiveCount`: 获取活动的线程数。**正在执行任务的线程数**

通过扩展线程池进行监控。可以通过继承线程池来自定义线程池，重写线程池的`beforeExecute`、`afterExecute`和`terminated`方法，也可以在任务执行前、执行后和线程池关闭前执行一些代码来进行监控。例如，监控任务的平均执行时间、最大执行时间和最小执行时间等。这几个方法在线程池里是空方法。

```
protected void beforeExecute(Thread t, Runnable r) { }
```

9.3 本章小结

在工作中我经常发现, 很多人因为不了解线程池的实现原理, 把线程池配置错误, 从而导致了各种问题。本章介绍了为什么要使用线程池、如何使用线程池和线程池的使用原理, 相信阅读完本章之后, 读者能更准确、更有效地使用线程池。

第10章 Executor框架

在Java中，使用线程来异步执行任务。Java线程的创建与销毁需要一定的开销，如果我们为每一个任务创建一个新线程来执行，这些线程的创建与销毁将消耗大量的计算资源。同时，为每一个任务创建一个新线程来执行，这种策略可能会使处于高负荷状态的应用最终崩溃。

Java的线程既是工作单元，也是执行机制。从JDK 5开始，把工作单元与执行机制分离开来。工作单元包括Runnable和Callable，而执行机制由Executor框架提供。

10.1 Executor框架简介

10.1.1 Executor框架的两级调度模型

在HotSpot VM的线程模型中, Java线程(java.lang.Thread)被一对一映射为本地操作系统线程。Java线程启动时会创建一个本地操作系统线程;当该Java线程终止时, 这个操作系统线程也会被回收。操作系统会调度所有线程并将它们分配给可用的CPU。

Executor的两级调度模型:作用是将任务创建和任务执行分离开。

在上层, Java多线程程序通常把应用分解为若干个任务, 然后使用用户级的调度器(Executor框架)将这些任务映射为固定数量的线程;在底层, 操作系统内核将这些线程映射到硬件处理器上。这种两级调度模型的示意图如图10-1所示。

从图中可以看出, **应用程序通过Executor框架控制上层的调度;而下层的调度由操作系统内核控制, 下层的调度不受应用程序的控制。**

10.1.2 Executor框架的结构与成员

本文将分两部分来介绍Executor:Executor的结构和Executor框架包含的成员组件。

Executor两级调度模型

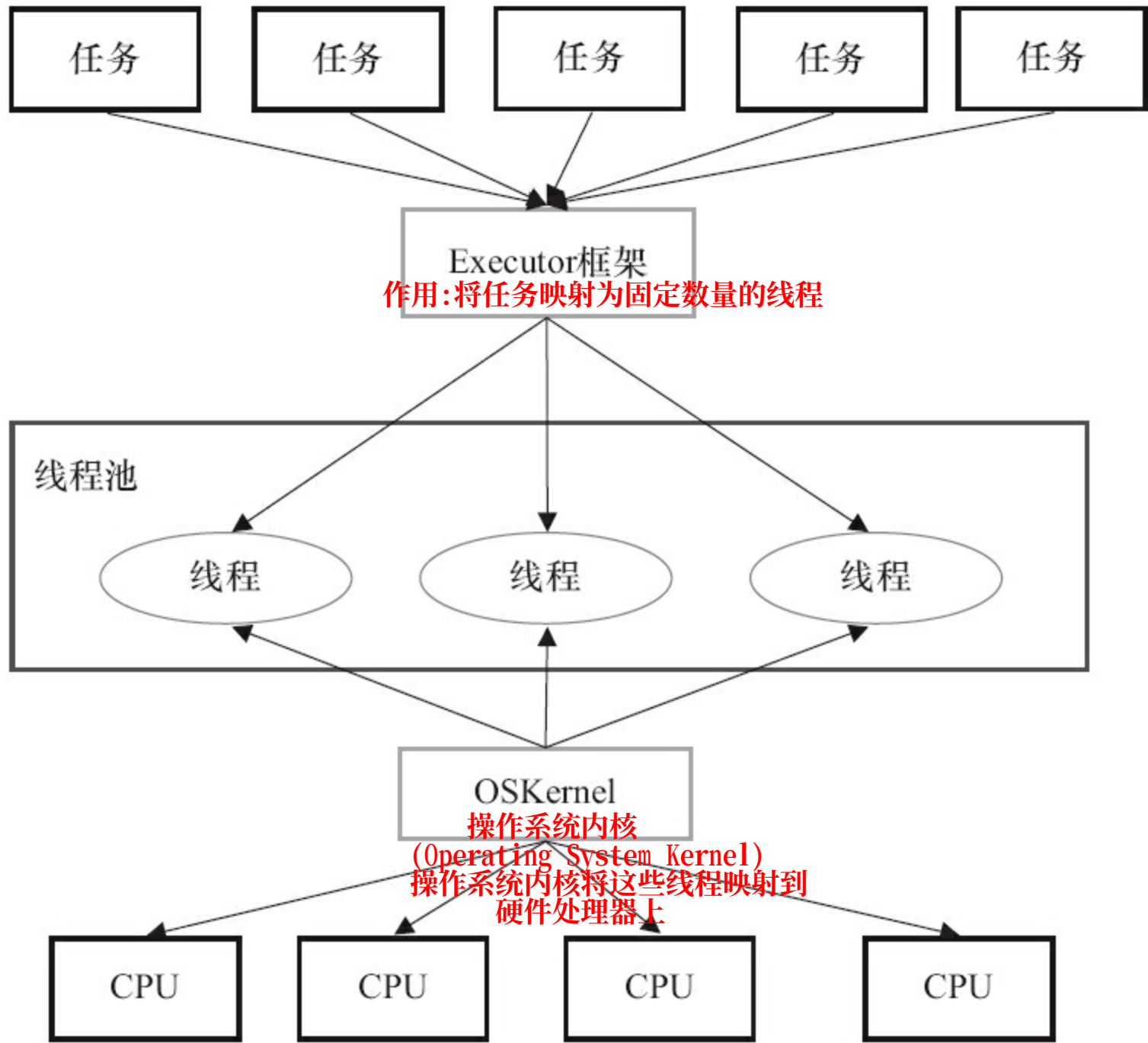


图10-1 任务的两级调度模型

1.Executor框架的结构

Executor框架主要由3大部分组成如下。

1. 任务。包括被执行任务需要实现的接口:Runnable接口或Callable接口。

2. 任务的执行。包括任务执行机制的核心接口Executor, 以及继承自Executor的ExecutorService接口。Executor框架有两个关键类实现了ExecutorService接口(ThreadPoolExecutor和ScheduledThreadPoolExecutor)。

3. 异步计算的结果。包括接口Future和实现Future接口的FutureTask类。

Executor框架包含的主要的类与接口如图10-2所示。

下面是这些类和接口的简介。

Executors的作用:

Executor是一个接口, 它是Executor框架的基础, 它将任务的提交与任务的执行分离开来。

ThreadPoolExecutor是线程池的核心实现类, 用来执行被提交的任务。

ScheduledThreadPoolExecutor是一个实现类, 可以在给定的延迟后运行命令, 或者定期执行命令。ScheduledThreadPoolExecutor比Timer更灵活, 功能更强大。

Future接口和实现Future接口的FutureTask类, 代表异步计算的结果。

Runnable接口和Callable接口的实现类, 都可以被ThreadPoolExecutor或ScheduledThreadPoolExecutor执行。

Executor框架的使用示意图如图10-3所示。

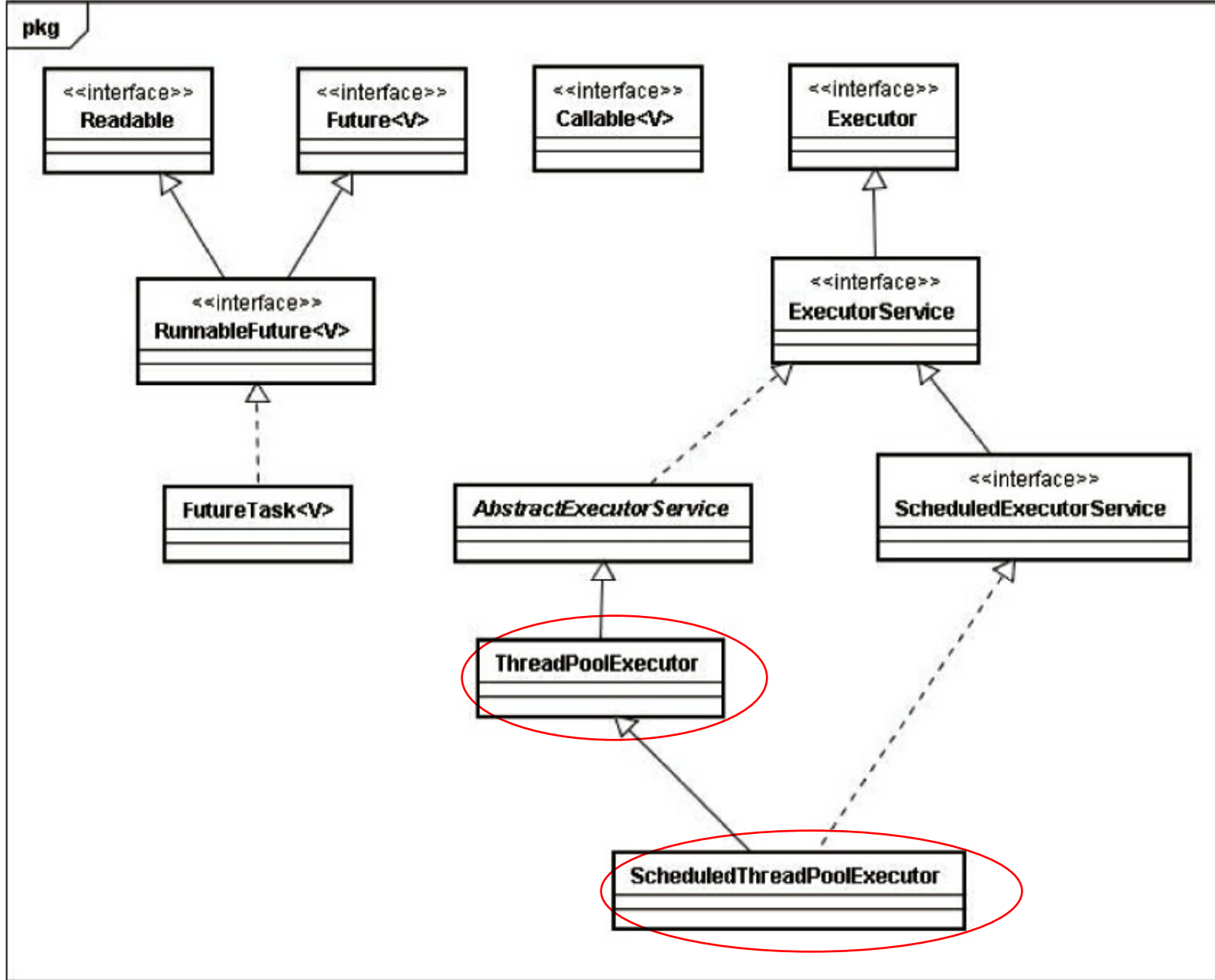


图10-2 Executor框架的类与接口

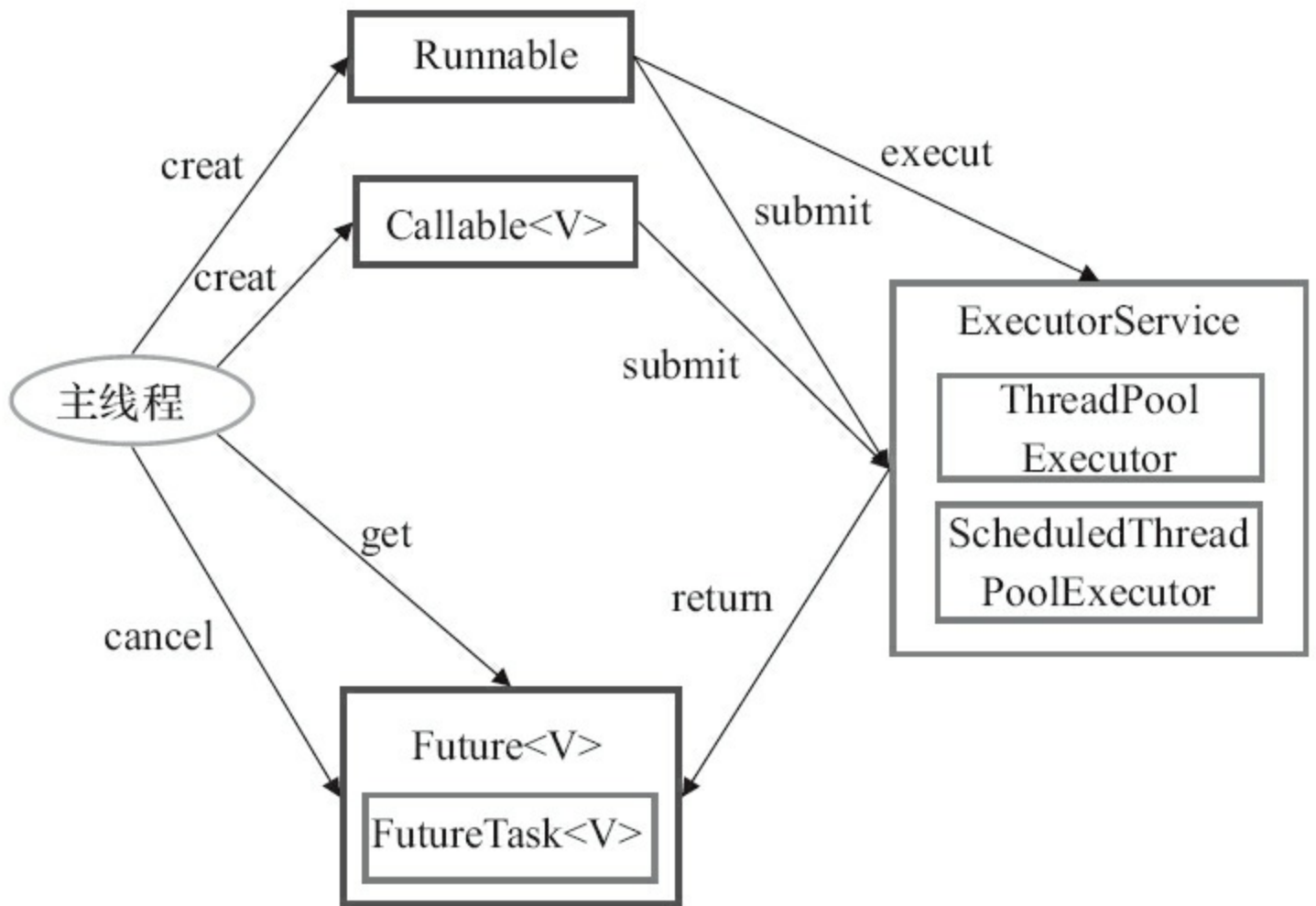


图10-3 Executor框架的使用示意图

主线程首先要创建实现Runnable或者Callable接口的任务对象。工具类Executors可以把一

个Runnable对象封装为一个Callable对象(Executors.callable(Runnable task)或

Executors.callable(Runnable task, Object result))。

然后可以把Runnable对象直接交给ExecutorService执行(ExecutorService.execute(Runnable command));或者也可以把Runnable对象或Callable对象提交给ExecutorService执行(ExecutorService.submit(Runnable task)或ExecutorService.submit(Callable<T>task))。

如果执行ExecutorService.submit(...), ExecutorService将返回一个实现Future接口的对象(到目前为止的JDK中,返回的是FutureTask对象)。由于FutureTask实现了Runnable,程序员也可以创建FutureTask,然后直接交给ExecutorService执行。

最后,主线程可以执行FutureTask.get()方法来等待任务执行完成。主线程也可以执行

FutureTask.cancel (boolean mayInterruptIfRunning) 来取消此任务的执行。

2.Executor框架的成员

本节将介绍Executor框架的主要成员: ThreadPoolExecutor、ScheduledThreadPoolExecutor、Future接口、Runnable接口、Callable接口和Executors。

(1) ThreadPoolExecutor

ThreadPoolExecutor通常使用工厂类Executors来创建。Executors可以创建3种类型的

ThreadPoolExecutor: SingleThreadExecutor、FixedThreadPool和CachedThreadPool。

→ Executor这三个静态方法底层都是: new ThreadPoolExecutor()

下面分别介绍这3种ThreadPoolExecutor。

1) FixedThreadPool。下面是Executors提供的, 创建使用固定线程数的FixedThreadPool的API。

```
public static ExecutorService newFixedThreadPool(int nThreads)
public static ExecutorService newFixedThreadPool(int nThreads, ThreadFactory threadFact
```

FixedThreadPool适用于为了满足资源管理的需求, 而需要限制当前线程数量的应用场景, 它适用于负载比较重的服务器。

2) SingleThreadExecutor。下面是Executors提供的, 创建使用单个线程的SingleThread-Executor的API。

```
public static ExecutorService newSingleThreadExecutor()
public static ExecutorService newSingleThreadExecutor(ThreadFactory threadFactory)
```

SingleThreadExecutor适用于需要保证顺序地执行各个任务;并且在任意时间点, 不会有多个线程是活动的应用场景。

3) **CachedThreadPool**。下面是Executors提供的, 创建一个会根据需要创建新线程的

CachedThreadPool的API。

查看底层代码:

`new ThreadPoolExecutor(0, Integer.MAX_VALUE, 60L, TimeUnit.SECONDS, new SynchronousQueue<Runnable>())`

使用的是 SynchronousQueue 任务队列, 队列永远处于满的状态。

```
public static ExecutorService newCachedThreadPool()  
public static ExecutorService newCachedThreadPool(ThreadFactory threadFactory)
```

CachedThreadPool是大小无界的线程池, 适用于执行很多的短期异步任务的小程序, 或者是负载较轻的服务器。

(2) ScheduledThreadPoolExecutor

ScheduledThreadPoolExecutor通常使用工厂类Executors来创建。 Executors可以创建2种类型的ScheduledThreadPoolExecutor, 如下。

· ScheduledThreadPoolExecutor。包含若干个线程的ScheduledThreadPoolExecutor。

· SingleThreadScheduledExecutor。只包含一个线程的ScheduledThreadPoolExecutor。

下面分别介绍这两种ScheduledThreadPoolExecutor。

下面是工厂类Executors提供的, 创建固定个数线程的ScheduledThreadPoolExecutor的API。

```
public static ScheduledExecutorService newScheduledThreadPool(int corePoolSize)  
public static ScheduledExecutorService newScheduledThreadPool(int corePoolSize, ThreadF
```

ScheduledThreadPoolExecutor适用于需要多个后台线程执行周期任务, 同时为了满足资源管理的需求而需要限制后台线程的数量的应用场景。 下面是Executors提供的, 创建单个线程的SingleThreadScheduledExecutor的API。

```
public static ScheduledExecutorService newSingleThreadScheduledExecutor()  
public static ScheduledExecutorService newSingleThreadScheduledExecutor  
(ThreadFactory threadFactory)
```

SingleThreadScheduledExecutor适用于需要单个后台线程执行周期任务，同时需要保证顺序地执行各个任务的应用场景。

(3) Future接口

Future接口和实现Future接口的FutureTask类用来表示异步计算的结果。当我们把Runnable接口或Callable接口的实现类提交(submit)给ThreadPoolExecutor或者invokeAll()方法ScheduledThreadPoolExecutor时，ThreadPoolExecutor或ScheduledThreadPoolExecutor会向我们返回一个FutureTask对象。下面是对应的API。

```
<T> Future<T> submit(Callable<T> task)      List<Future<T>> invokeAll(Collection<Callable<T>> tasks)
<T> Future<T> submit(Runnable task, T result)  批量执行tasks后,返回所有的Future
Future<> submit(Runnable task)
```

有一点需要读者注意，到目前最新的JDK 8为止，Java通过上述API返回的是一个FutureTask对象。但从API可以看到，Java仅仅保证返回的是一个实现了Future接口的对象。在将来的JDK实现中，返回的可能不一定是FutureTask。

(4) Runnable接口和Callable接口

Runnable接口和Callable接口的实现类，都可以被ThreadPoolExecutor或ScheduledThreadPoolExecutor执行。**区别：**它们之间的区别是Runnable不会返回结果，而Callable可以返回结果。**Runnable和Callable的区别：**(1)方法不一样，run()和call()。(2)有无返回结果。(3)能否抛出异常。

除了可以自己创建实现Callable接口的对象外，还可以使用工厂类Executors来把一个Runnable包装成一个Callable。

下面是Executors提供的，把一个Runnable包装成一个Callable的API。

```
public static Callable<Object> callable(Runnable task)      // 假设返回对象Callable1
```

下面是Executors提供的, 把一个Runnable和一个待返回的结果包装成一个Callable的API。

```
public static <T> Callable<T> callable(Runnable task, T result)           // 假设返回对象Call
```

前面讲过, 当我们把一个Callable对象(比如上面的Callable1或Callable2)提交给ThreadPoolExecutor或ScheduledThreadPoolExecutor执行时, submit(...)会向我们返回一个FutureTask对象。我们可以执行FutureTask.get()方法来等待任务执行完成。当任务成功完成后FutureTask.get()将返回该任务的结果。例如, 如果提交的是对象Callable1, FutureTask.get()方法将返回null; 如果提交的是对象Callable2, FutureTask.get()方法将返回result对象。

10.2 ThreadPoolExecutor详解

Executor框架最核心的类是ThreadPoolExecutor，它是线程池的实现类，主要由下列4个组件构成。

·corePool: 核心线程池的大小。

·maximumPool: 最大线程池的大小。

·BlockingQueue: 用来暂时保存任务的工作队列。

·RejectedExecutionHandler: 当ThreadPoolExecutor已经关闭或ThreadPoolExecutor已经饱和时(达到了最大线程池大小且工作队列已满)，execute()方法将要调用的Handler。

·通过Executor框架的工具类Executors，可以创建3种类型的ThreadPoolExecutor。

- FixedThreadPool。任务队列: 无界队列LinkedBlockingQueue; 饱和策略: 默认的AbortPolicy; corePoolSize和maximumPoolSize相同
- SingleThreadExecutor。任务队列: 无界队列LinkedBlockingQueue; 饱和策略: 默认的AbortPolicy; corePoolSize和maximumPoolSize都为1
- CachedThreadPool。任务队列: SynchronousQueue; 饱和策略: 默认的AbortPolicy; corePoolSize为0, maximumPoolSize为Integer.MAX

下面将分别介绍这3种ThreadPoolExecutor。

10.2.1 FixedThreadPool详解

FixedThreadPool被称为可重用固定线程数的线程池。下面是FixedThreadPool的源代码实现。

```
public static ExecutorService newFixedThreadPool(int nThreads) {  
    return new ThreadPoolExecutor(nThreads, nThreads,  
                                   0L, TimeUnit.MILLISECONDS,  
                                   new LinkedBlockingQueue<Runnable>());  
}
```

FixedThreadPool的corePoolSize和maximumPoolSize都被设置为创建FixedThreadPool时指定的参数nThreads。

当线程池中的线程数大于corePoolSize时, keepAliveTime为多余的空闲线程等待新任务的最长时间, 超过这个时间后多余的线程将被终止。这里把keepAliveTime设置为0L, 意味着多余的空闲线程会被立即终止。

FixedThreadPool的execute()方法的运行示意图如图10-4所示。

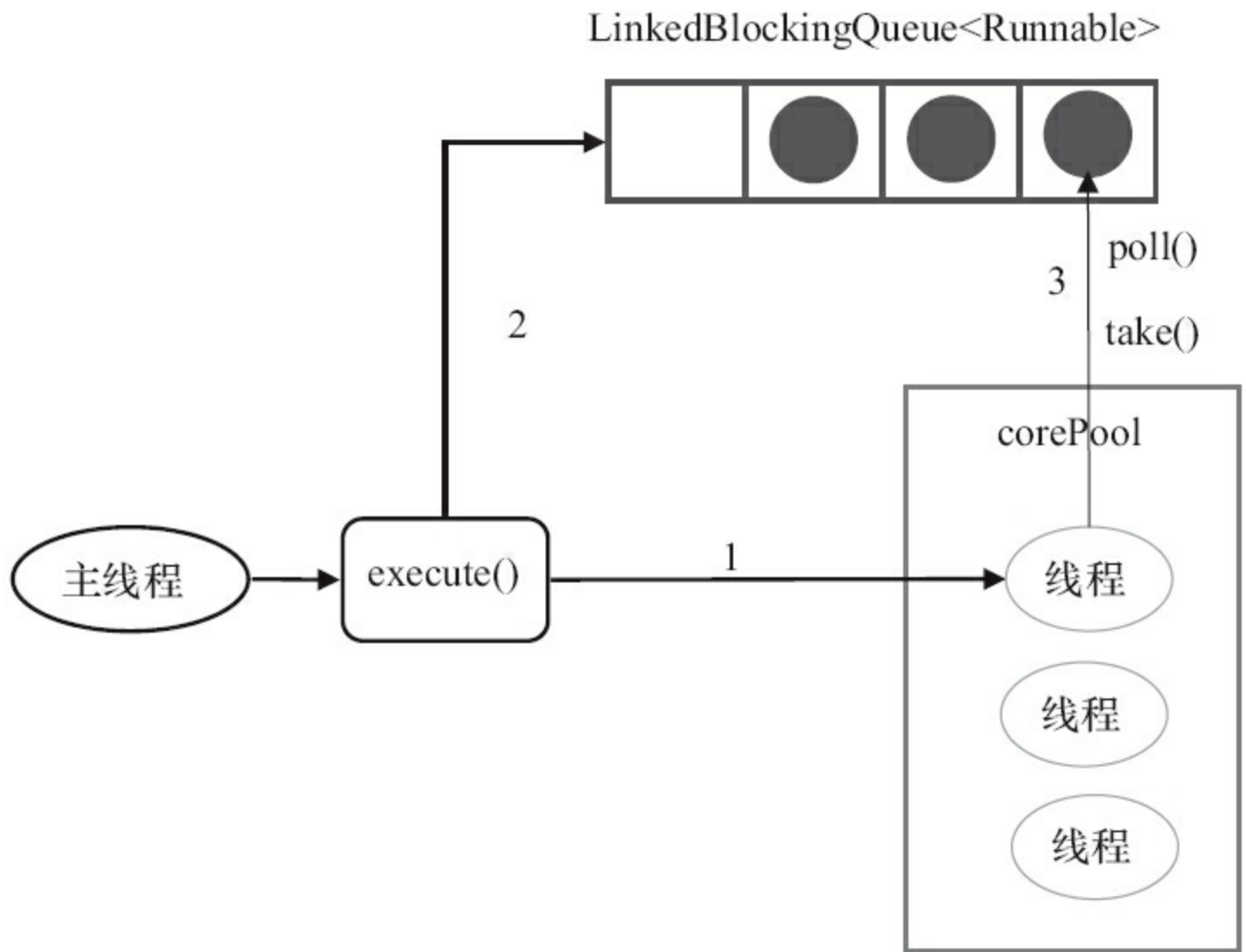


图10-4 FixedThreadPool的execute()的运行示意图

对图10-4的说明如下。

1) 如果当前运行的线程数少于`corePoolSize`，则创建新线程来执行任务。

2) 在线程池完成预热之后(当前运行的线程数等于`corePoolSize`)，将任务加入

`LinkedBlockingQueue`。

3) 线程执行完1中的任务后，会在循环中反复从`LinkedBlockingQueue`获取任务来执行。

FixedThreadPool使用无界队列`LinkedBlockingQueue`作为线程池的工作队列(队列的容量为

`Integer.MAX_VALUE`)。使用无界队列作为工作队列会对线程池带来如下影响。

无界队列的影响：

1) 当线程池中的线程数达到corePoolSize后, 新任务将在无界队列中等待, 因此线程池中的线程数不会超过corePoolSize。

2) 由于1, 使用无界队列时maximumPoolSize将是一个无效参数。

3) 由于1和2, 使用无界队列时keepAliveTime将是一个无效参数。

4) 由于使用无界队列, 运行中的FixedThreadPool(未执行方法shutdown()或shutdownNow())不会拒绝任务(不会调用RejectedExecutionHandler.rejectedExecution方法)。

10.2.2 SingleThreadExecutor详解

SingleThreadExecutor是使用单个worker线程的Executor。下面是SingleThreadExecutor的源代码实现。

```
public static ExecutorService newSingleThreadExecutor() {
    return new FinalizableDelegatedExecutorService
        (new ThreadPoolExecutor(1, 1,
                                0L, TimeUnit.MILLISECONDS,
                                new LinkedBlockingQueue<Runnable>()));
}
```

SingleThreadExecutor的corePoolSize和maximumPoolSize被设置为1。其他参数与FixedThreadPool相同。SingleThreadExecutor使用无界队列LinkedBlockingQueue作为线程池的工作队列（队列的容量为Integer.MAX_VALUE）。SingleThreadExecutor使用无界队列作为工作队列对线程池带来的影响与FixedThreadPool相同，这里就不赘述了。

SingleThreadExecutor的运行示意图如图10-5所示。

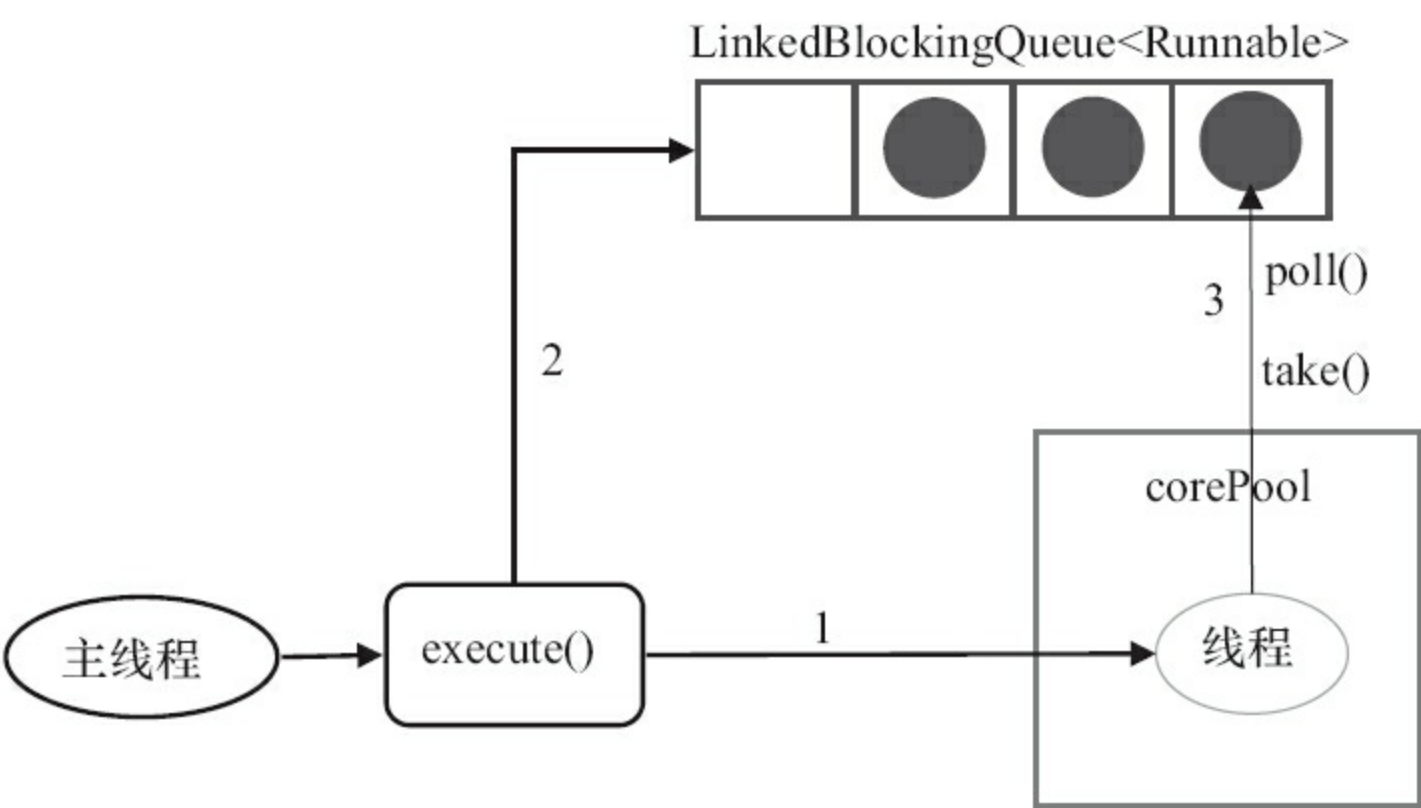


图10-5 SingleThreadExecutor的execute()的运行示意图

对图10-5的说明如下。

- 1) 如果当前运行的线程数少于corePoolSize(即线程池中无运行的线程), 则创建一个新线程来执行任务。
- 2) 在线程池完成预热之后(当前线程池中有一个运行的线程), 将任务加入Linked-BlockingQueue。
- 3) 线程执行完1中的任务后, 会在一个无限循环中反复从LinkedBlockingQueue获取任务来执行。

10.2.3 CachedThreadPool详解

CachedThreadPool是一个会根据需要创建新线程的线程池。下面是创建CachedThreadPool的源代码。

```
public static ExecutorService newCachedThreadPool() {  
    return new ThreadPoolExecutor(0, Integer.MAX_VALUE,  
                                   60L, TimeUnit.SECONDS,  
                                   new SynchronousQueue<Runnable>());  
}
```

CachedThreadPool的corePoolSize被设置为0, 即corePool为空; maximumPoolSize被设置为Integer.MAX_VALUE, 即maximumPool是无界的。这里把keepAliveTime设置为60L, 意味着CachedThreadPool中的空闲线程等待新任务的最长时间为60秒, 空闲线程超过60秒后将会被终止。

FixedThreadPool和SingleThreadExecutor使用无界队列LinkedBlockingQueue作为线程池的工作队列。CachedThreadPool使用没有容量的SynchronousQueue作为线程池的工作队列, 但CachedThreadPool的maximumPool是无界的。这意味着, 如果主线程提交任务的速度高于maximumPool中线程处理任务的速度时, CachedThreadPool会不断创建新线程。极端情况下, CachedThreadPool会因为创建过多线程而耗尽CPU和内存资源。

CachedThreadPool的execute()方法的执行示意图如图10-6所示。

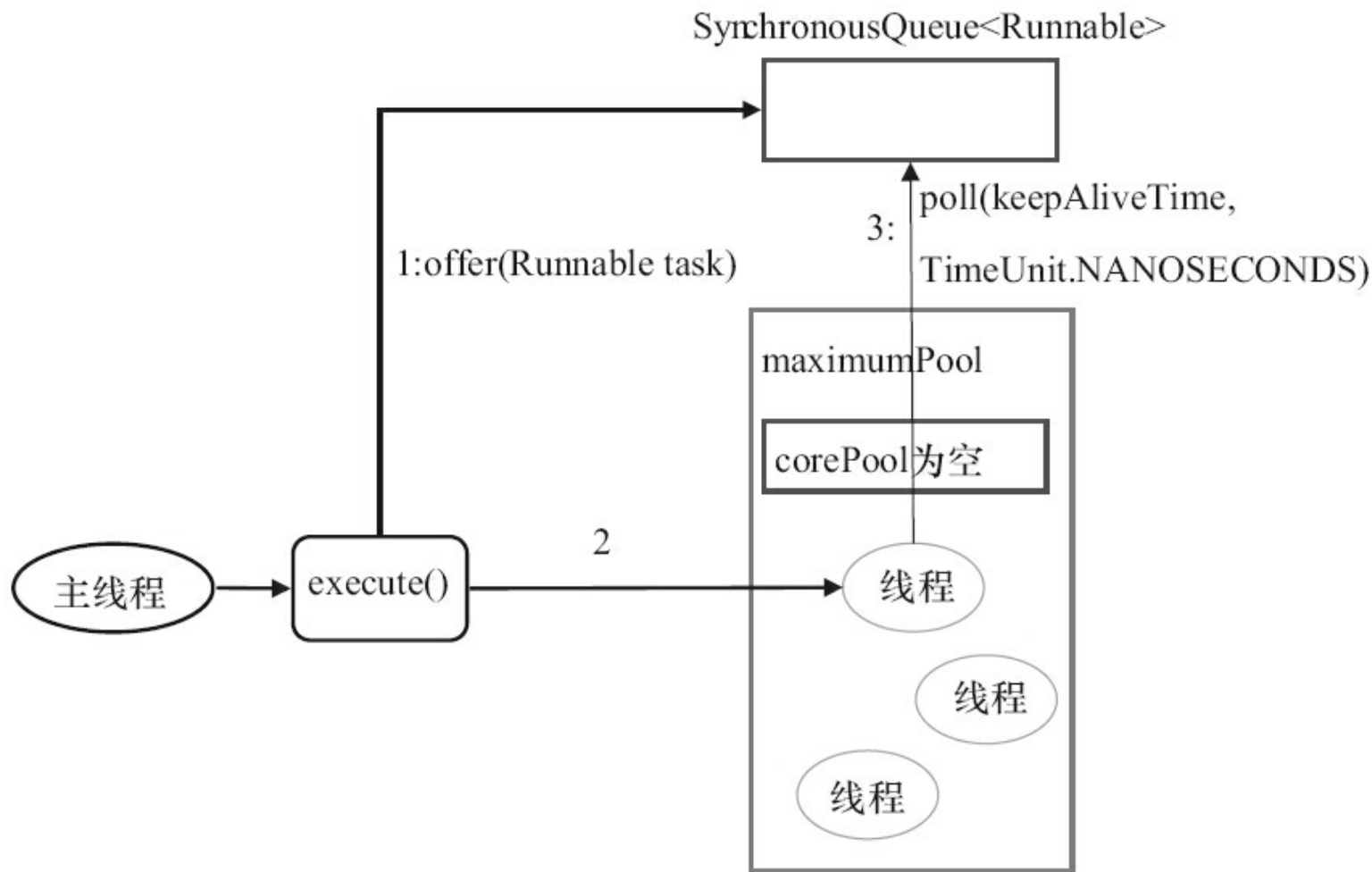


图10-6 CachedThreadPool的execute()的运行示意图

对图10-6的说明如下。

CachedThreadPool执行execute():顺序

1) 首先执行SynchronousQueue.offer(Runnable task)。如果当前maximumPool中有空闲线程正在执行SynchronousQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS)，那么主线程执行offer操作与空闲线程执行的poll操作配对成功，主线程把任务交给空闲线程执行，execute()方法执行完成；否则执行下面的步骤2)。

2) 当初始maximumPool为空，或者maximumPool中当前没有空闲线程时，将没有线程执行SynchronousQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS)。这种情况下，步骤1)将失败。此时CachedThreadPool会创建一个新线程执行任务，execute()方法执行完成。

3) 在步骤2)中新创建的线程将任务执行完后，会执行SynchronousQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS)。这个poll操作会让空闲线

程最多在SynchronousQueue中等待60秒钟。如果60秒钟内主线程提交了一个新任务(主线程执行步骤1)), 那么这个空闲线程将执行主线程提交的新任务; 否则, 这个空闲线程将终止。由于空闲60秒的空闲线程会被终止, 因此长时间保持空闲的CachedThreadPool不会使用任何资源。

前面提到过, SynchronousQueue是一个没有容量的阻塞队列。每个插入操作必须等待另一个线程的对应移除操作, 反之亦然。 CachedThreadPool使用SynchronousQueue, 把主线程提交的任务传递给空闲线程执行。CachedThreadPool中任务传递的示意图如图10-7所示。

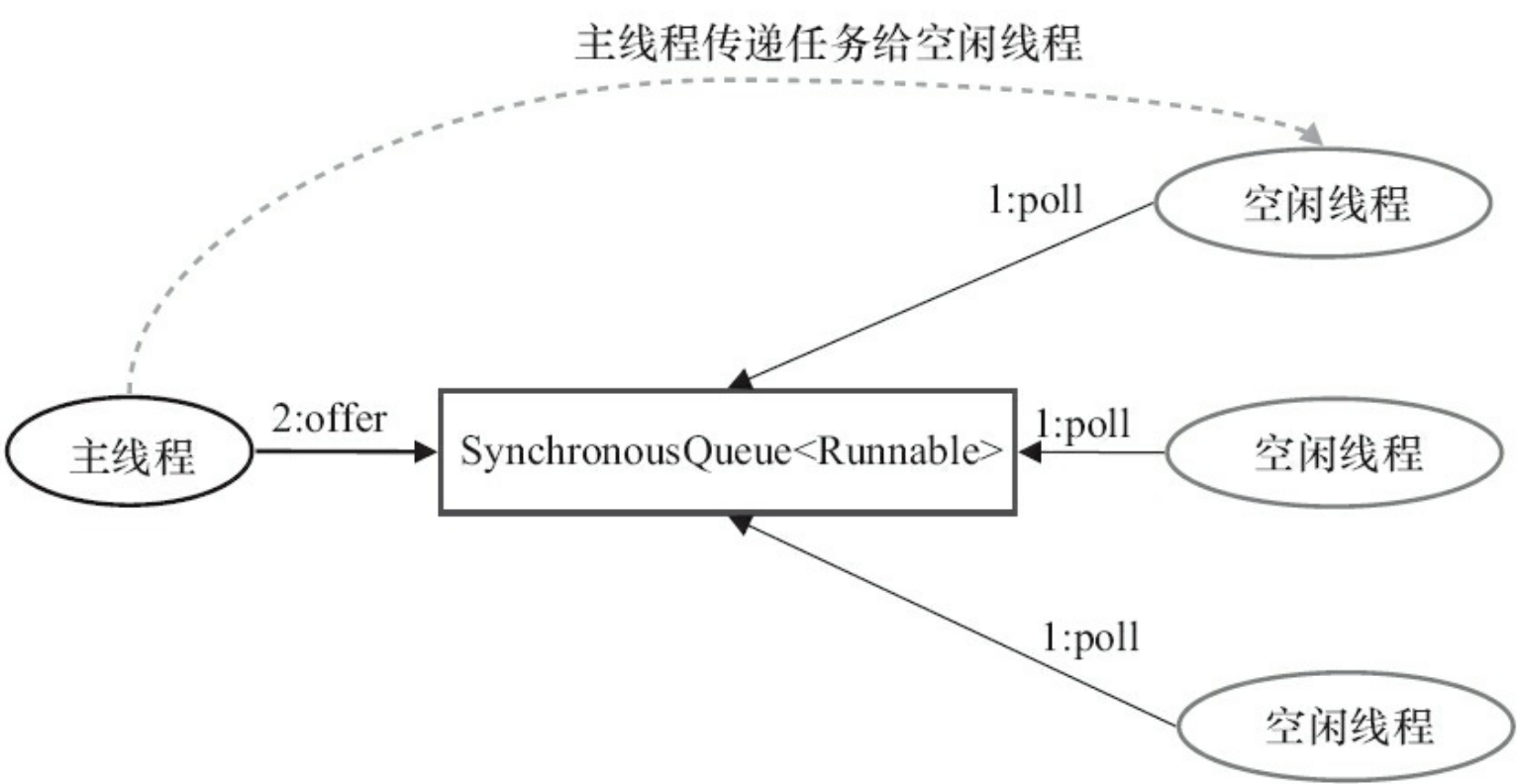


图10-7 CachedThreadPool的任务传递示意图

10.3 ScheduledThreadPoolExecutor详解

ScheduledThreadPoolExecutor继承自ThreadPoolExecutor。它主要用来在给定的延迟之后运

行任务，或者定期执行任务。ScheduledThreadPoolExecutor的功能与Timer类似，但

ScheduledThreadPoolExecutor功能更强大、更灵活。Timer对应的是单个后台线程，而

ScheduledThreadPoolExecutor可以在构造函数中指定多个对应的后台线程数。

10.3.1 ScheduledThreadPoolExecutor的运行机制

ScheduledThreadPoolExecutor的执行示意图(本文基于JDK 6)如图10-8所示。

DelayQueue是一个无界队列, 所以ThreadPoolExecutor的maximumPoolSize在ScheduledThreadPoolExecutor中没有什么意义(设置maximumPoolSize的大小没有什么效果)。

ScheduledThreadPoolExecutor的执行主要分为两大部分。

- 1) 当调用ScheduledThreadPoolExecutor的scheduleAtFixedRate()方法或者scheduleWithFixedDelay()方法时, 会向ScheduledThreadPoolExecutor的DelayQueue添加一个实现了RunnableScheduledFuture接口的ScheduledFutureTask。
ScheduledFutureTask是ScheduledThreadPoolExecutor的一个部类。
- 2) 线程池中的线程从DelayQueue中获取ScheduledFutureTask, 然后执行任务。

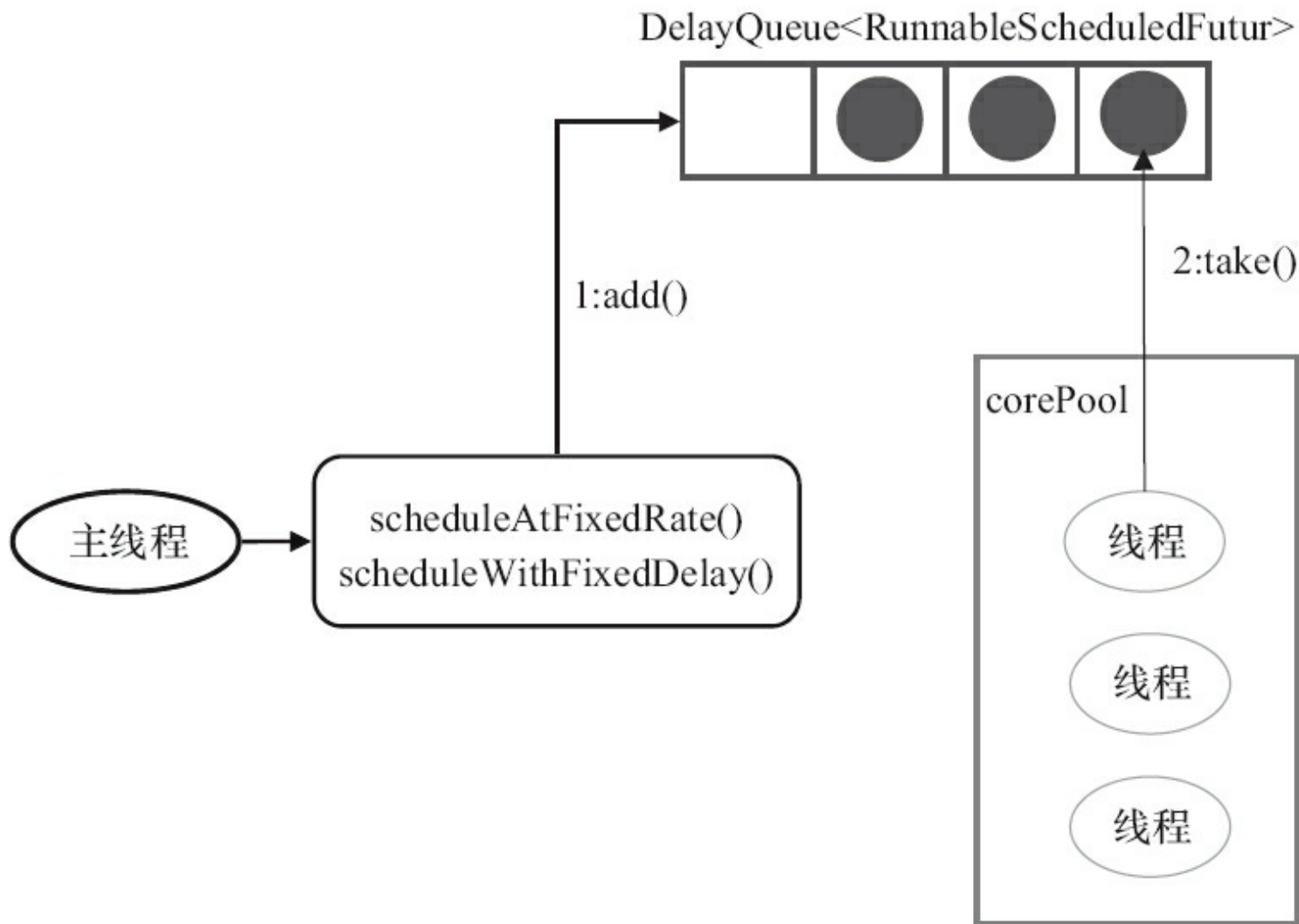


图10-8 `ScheduledThreadPoolExecutor`的任务传递示意图

`ScheduledThreadPoolExecutor`为了实现周期性的执行任务，对`ThreadPoolExecutor`做了如下的修改。

`scheduledThreadPoolExecutor`和`ThreadPoolExecutor`的区别：

·使用`DelayQueue`作为任务队列。

·获取任务的方式不同(后文会说明)。多一个时间比较;当任务的时间小于当前时间时执行任务

·执行周期任务后，增加了额外的处理(后文会说明)。

额外处理:Worker中的`coreThread`循环获取任务执行完后，再将修改`time`之后的`ScheduledFutureTask`放回`DelayQueue`中(判断标准是:是否周期循环执行)

10.3.2 ScheduledThreadPoolExecutor的实现

前面我们提到过, ScheduledThreadPoolExecutor会把待调度的任务(ScheduledFutureTask)放到一个DelayQueue中。

ScheduledFutureTask主要包含3个成员变量, 如下。

·long型成员变量time, 表示这个任务将要被执行的具体时间。

·long型成员变量sequenceNumber, 表示这个任务被添加到ScheduledThreadPoolExecutor中的序号。

·long型成员变量period, 表示任务执行的间隔周期。

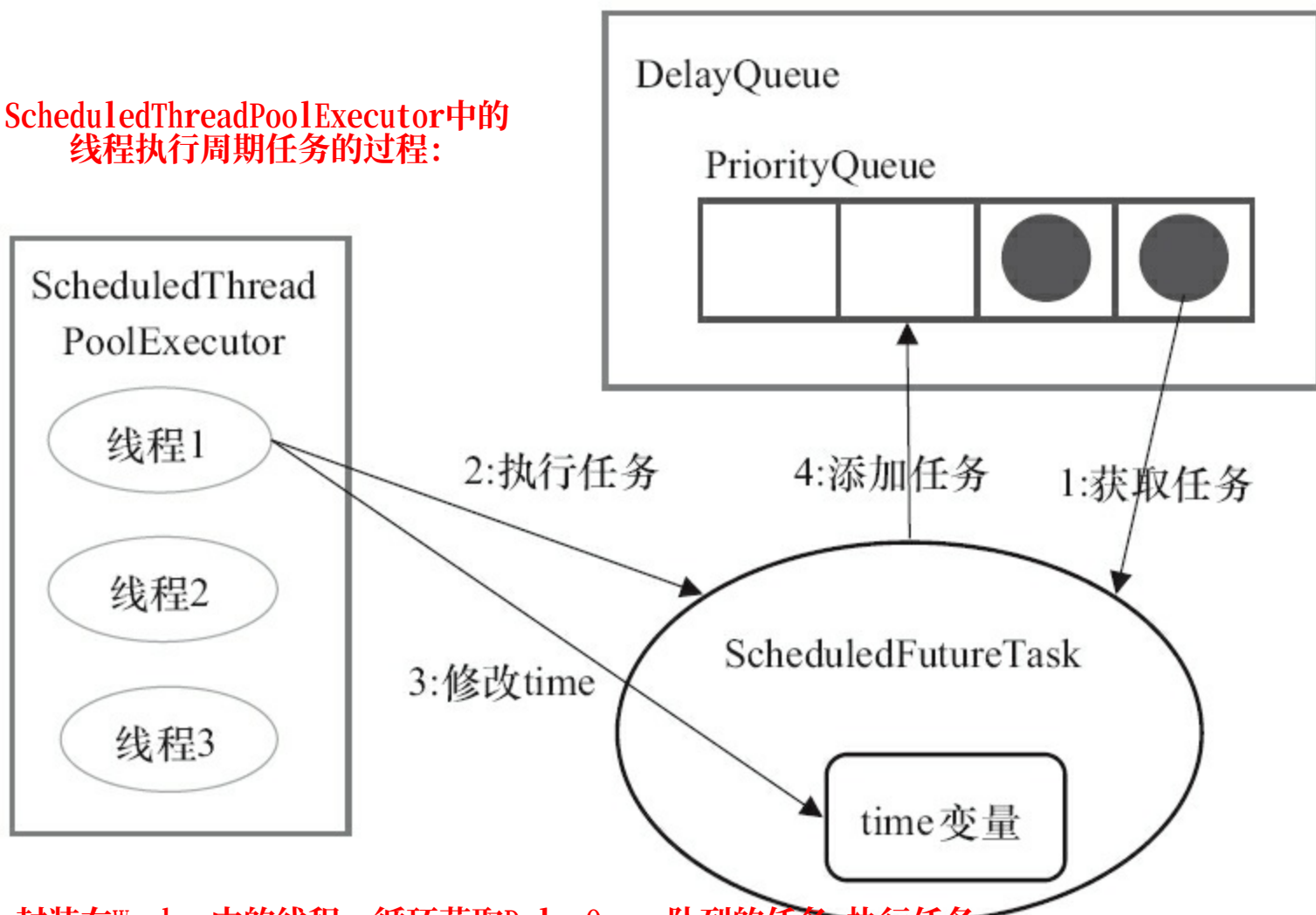
DelayQueue实现原理: 队列排序在DelayedWorkQueue中实现的

DelayQueue封装了一个PriorityQueue, 这个PriorityQueue会对队列中的Scheduled-FutureTask进行排序。排序时, time小的排在前面(时间早的任务将被先执行)。如果两个ScheduledFutureTask的time相同, 就比较sequenceNumber, sequenceNumber小的排在前面(也就是说, 如果两个任务的执行时间相同, 那么先提交的任务将被先执行)。

首先, 让我们看看ScheduledThreadPoolExecutor中的线程执行周期任务的过程。图10-9是ScheduledThreadPoolExecutor中的线程1执行某个周期任务的4个步骤。

ScheduledThreadPoolExecutor.schedule(command, delay, unit)时, 会将Runnable和延迟时间封装成一个ScheduledFutureTask;
execute()和submit()方法调用的是schedule()方法, 延迟时间为0

ScheduledThreadPoolExecutor中的线程执行周期任务的过程:



封装在Worker中的线程，循环获取DelayQueue队列的任务，执行任务

图10-9 ScheduledThreadPoolExecutor的任务执行步骤

下面是对这4个步骤的说明。

1. 过程

1) 线程1从DelayQueue中获取已到期的ScheduledFutureTask(`DelayQueue.take()`)。到期任务是指ScheduledFutureTask的time~~大于等于当前时间~~。小于等于当前时间

2) 线程1执行这个ScheduledFutureTask。

3) 线程1修改ScheduledFutureTask的time变量为下次将要被执行的时间。

4) 线程1把这个修改time之后的ScheduledFutureTask放回DelayQueue中(`DelayQueue.add()`)。在ScheduledFutureTask的run()方法中实现的

接下来，让我们看看上面的步骤1)获取任务的过程。下面是DelayQueue.take()方法的源代码

码实现。

```
public E take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();                // 1
    try {
        for (;;) {
            E first = q.peek();
            if (first == null) {
                available.await();            // 2.1
            } else {
                long delay = first.getDelay(TimeUnit.NANOSECONDS);
                if (delay > 0) {
                    long tl = available.awaitNanos(delay);    // 2.2
                } else {
                    E x = q.poll();            // 2.3.1
                    assert x != null;
                    if (q.size() != 0)
                        available.signalAll();    // 2.3.2
                    return x;
                }
            }
        }
    } finally {
        lock.unlock();                        // 3
    }
}
```

图10-10是DelayQueue.take()的执行示意图。

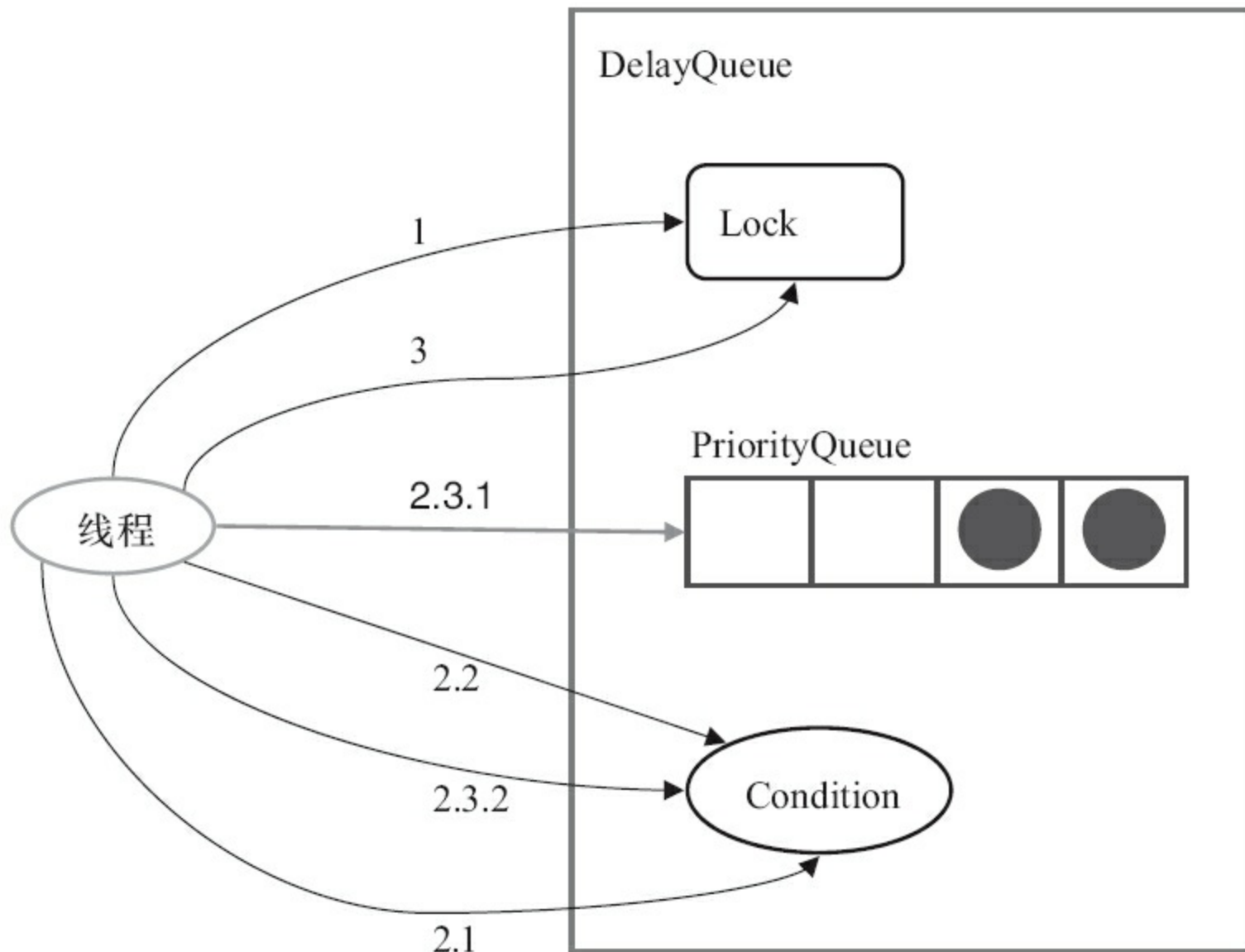


图10-10 ScheduledThreadPoolExecutor获取任务的过程

如图所示, 获取任务分为3大步骤。

2. 获取任务

1) 获取Lock。

2) 获取周期任务。

· 如果PriorityQueue为空, 当前线程到Condition中等待; 否则执行下面的2.2。

· 如果PriorityQueue的头元素的time时间比当前时间大, 到Condition中等待到time时间; 否则执行下面的2.3。

· 获取PriorityQueue的头元素(2.3.1); 如果PriorityQueue不为空, 则唤醒在Condition中等待

的所有线程(2.3.2)。

3)释放Lock。

ScheduledThreadPoolExecutor在一个循环中执行步骤2, 直到线程从PriorityQueue获取到一个元素之后(执行2.3.1之后), 才会退出无限循环(结束步骤2)。

最后, 让我们看看ScheduledThreadPoolExecutor中的线程执行任务的步骤4, 把

ScheduledFutureTask放入DelayQueue中的过程。下面是DelayQueue.add()的源代码实现。

```
public boolean offer(E e) {
    final ReentrantLock lock = this.lock;
    lock.lock();                // 1
    try {
        E first = q.peek();
        q.offer(e);              // 2.1
        if (first == null || e.compareTo(first) < 0)
            available.signalAll(); // 2.2
        return true;
    } finally {
        lock.unlock();           // 3
    }
}
```

图10-11是DelayQueue.add()的执行示意图。

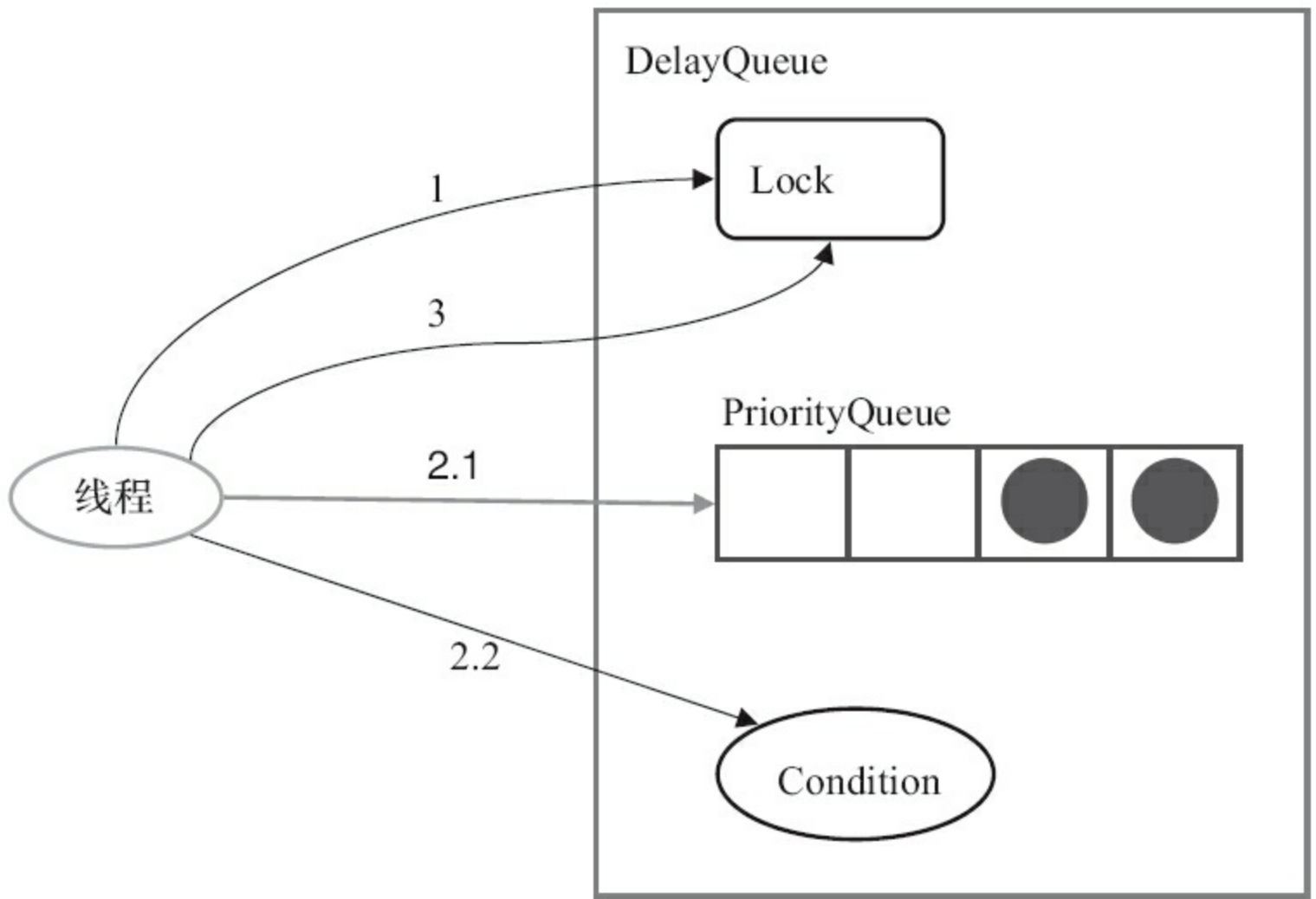


图10-11 ScheduledThreadPoolExecutor添加任务的过程

如图所示，添加任务分为3大步骤。

3. 添加任务

1) 获取Lock。

2) 添加任务。

· 向PriorityQueue添加任务。

· 如果在上面2.1中添加的任务是PriorityQueue的头元素，唤醒在Condition中等待的所有线程。

3) 释放Lock。

10.4 FutureTask详解

Future接口和实现Future接口的FutureTask类, 代表异步计算的结果。

10.4.1 FutureTask简介

FutureTask除了实现Future接口外，还实现了Runnable接口。因此，FutureTask可以交给Executor执行，也可以由调用线程直接执行(FutureTask.run())。根据FutureTask.run()方法被执行的时机，FutureTask可以处于下面3种状态。
直接使用future.run()是方法调用，不是多线程

- 1) 未启动。FutureTask.run()方法还没有被执行之前，FutureTask处于未启动状态。当创建一个FutureTask，且没有执行FutureTask.run()方法之前，这个FutureTask处于未启动状态。
- 2) 已启动。FutureTask.run()方法被执行的过程中，FutureTask处于已启动状态。
- 3) 已完成。FutureTask.run()方法执行完后正常结束，或被取消(FutureTask.cancel(...))，或执行FutureTask.run()方法时抛出异常而异常结束，FutureTask处于已完成状态。

图10-12是FutureTask的状态迁移的示意图。

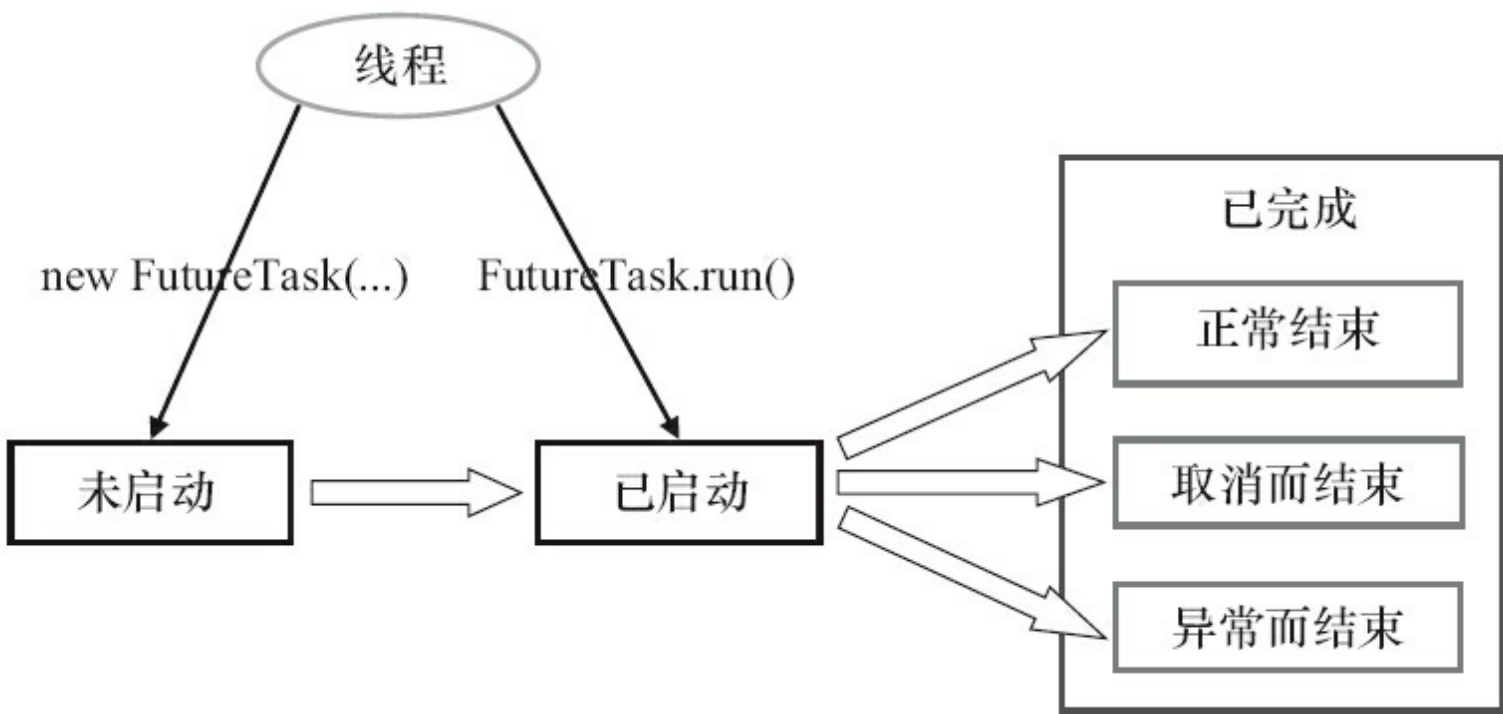


图10-12 FutureTask的状态迁移示意图

当FutureTask处于未启动或已启动状态时，执行FutureTask.get()方法将导致调用线程阻塞；

当FutureTask处于已完成状态时，执行FutureTask.get()方法将导致调用线程立即返回结果或抛出异常。

当FutureTask处于未启动状态时，执行FutureTask.cancel()方法将导致此任务永远不会被执行；当FutureTask处于已启动状态时，执行FutureTask.cancel(true)方法将以中断执行此任务线程的方式来试图停止任务；当FutureTask处于已启动状态时，执行FutureTask.cancel(false)方法将不会对正在执行此任务的线程产生影响(让正在执行的任务运行完成)；当FutureTask处于已完成状态时，执行FutureTask.cancel(...)方法将返回false。

图10-13是get方法和cancel方法的执行示意图。

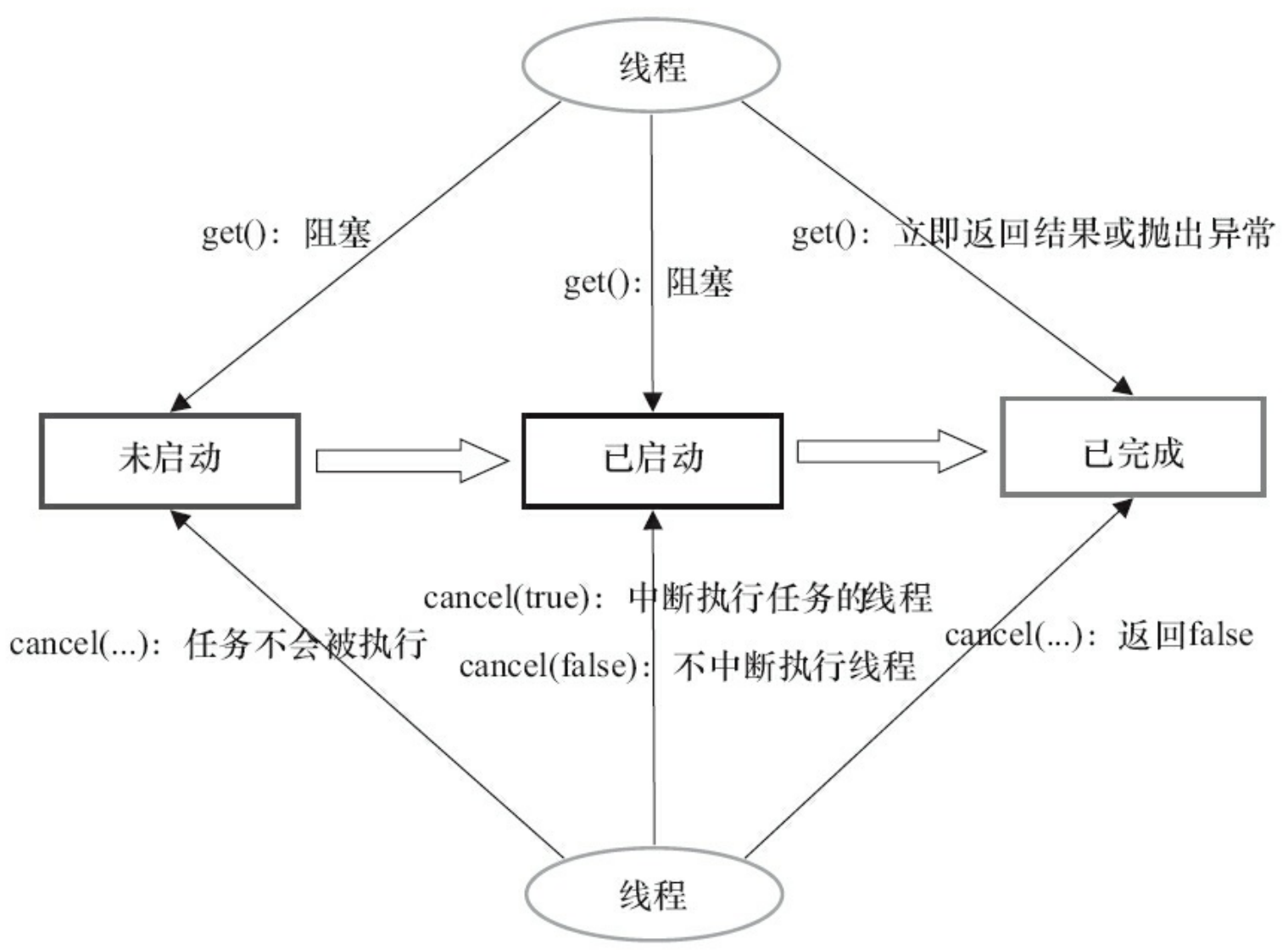


图10-13 FutureTask的get和cancel的执行示意图

10.4.2 FutureTask的使用

可以把FutureTask交给Executor执行;也可以通过ExecutorService.submit(...)方法返回一个FutureTask, 然后执行FutureTask.get()方法或FutureTask.cancel(...)方法。除此以外, 还可以单独使用FutureTask。

当一个线程需要等待另一个线程把某个任务执行完后它才能继续执行, 此时可以使用FutureTask。假设有多个线程执行若干任务, 每个任务最多只能被执行一次。当多个线程试图同时执行同一个任务时, 只允许一个线程执行任务, 其他线程需要等待这个任务执行完后才能继续执行。下面是对应的示例代码。

```
private final ConcurrentMap<Object, Future<String>> taskCache =
    new ConcurrentHashMap<Object, Future<String>>();
private String executionTask(final String taskName)
    throws ExecutionException, InterruptedException {
    while (true) {
        Future<String> future = taskCache.get(taskName);           // 1.1,2.1
        if (future == null) {
            Callable<String> task = new Callable<String>() {
                public String call() throws InterruptedException {
                    return taskName;
                }
            };
            FutureTask<String> futureTask = new FutureTask<String>(task);
            future = taskCache.putIfAbsent(taskName, futureTask); // 1.3
            if (future == null) {
                future = futureTask;
                futureTask.run();
            }
        }
        try {
            return future.get();                                     // 1.5
        } catch (CancellationException e) {
            taskCache.remove(taskName, future);
        }
    }
}
```

上述代码的执行示意图如图10-14所示。

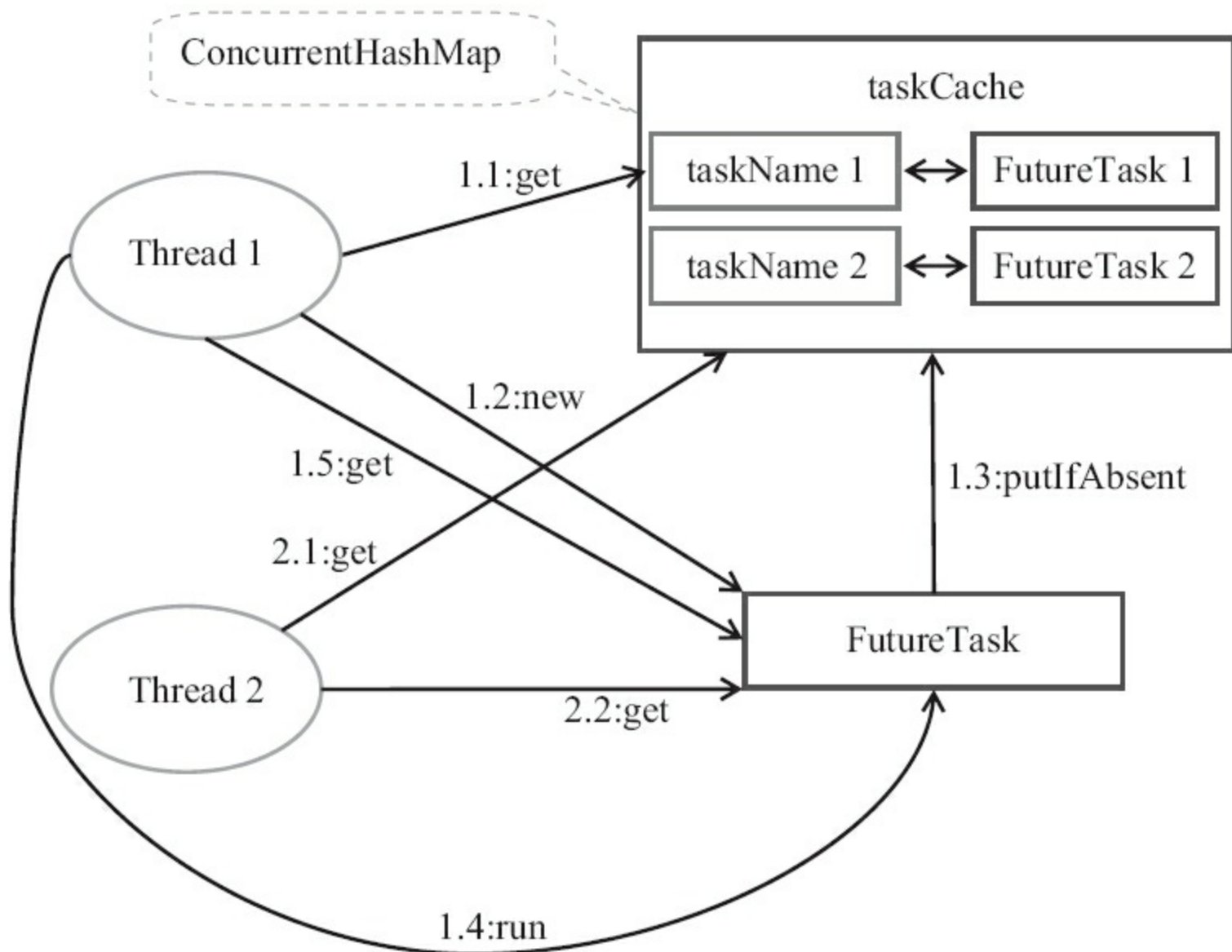


图10-14 代码的执行示意图

当两个线程试图同时执行同一个任务时，如果Thread 1执行1.3后Thread 2执行2.1，那么接下来Thread 2将在2.2等待，直到Thread 1执行完1.4后Thread 2才能从2.2(`FutureTask.get()`)返回。

10.4.3 FutureTask的实现

FutureTask的实现基于AbstractQueuedSynchronizer(以下简称为AQS)。java.util.concurrent中的很多可阻塞类(比如ReentrantLock)都是基于AQS来实现的。AQS是一个同步框架,它提供通用机制来原子性管理同步状态、阻塞和唤醒线程,以及维护被阻塞线程的队列。JDK 6中AQS被广泛使用,基于AQS实现的同步器包括:ReentrantLock、Semaphore、ReentrantReadWriteLock、CountDownLatch和FutureTask。

每一个基于AQS实现的同步器都会包含两种类型的操作,如下。

- 至少一个acquire操作。这个操作阻塞调用线程,除非/直到AQS的状态允许这个线程继续执行。FutureTask的acquire操作作为get()/get(long timeout, TimeUnit unit)方法调用。

- 至少一个release操作。这个操作改变AQS的状态,改变后的状态可允许一个或多个阻塞线程被解除阻塞。FutureTask的release操作包括run()方法和cancel(...)方法。

基于“复合优先于继承”的原则,FutureTask声明了一个内部私有的继承于AQS的子类Sync,对FutureTask所有公有方法的调用都会委托给这个内部子类。

AQS被作为“模板方法模式”的基础类提供给FutureTask的内部子类Sync,这个内部子类只需要实现状态检查和状态更新的方法即可,这些方法将控制FutureTask的获取和释放操作。具体来说,Sync实现了AQS的tryAcquireShared(int)方法和tryReleaseShared(int)方法,Sync通过这两个方法来检查和更新同步状态。

FutureTask的设计示意图如图10-15所示。

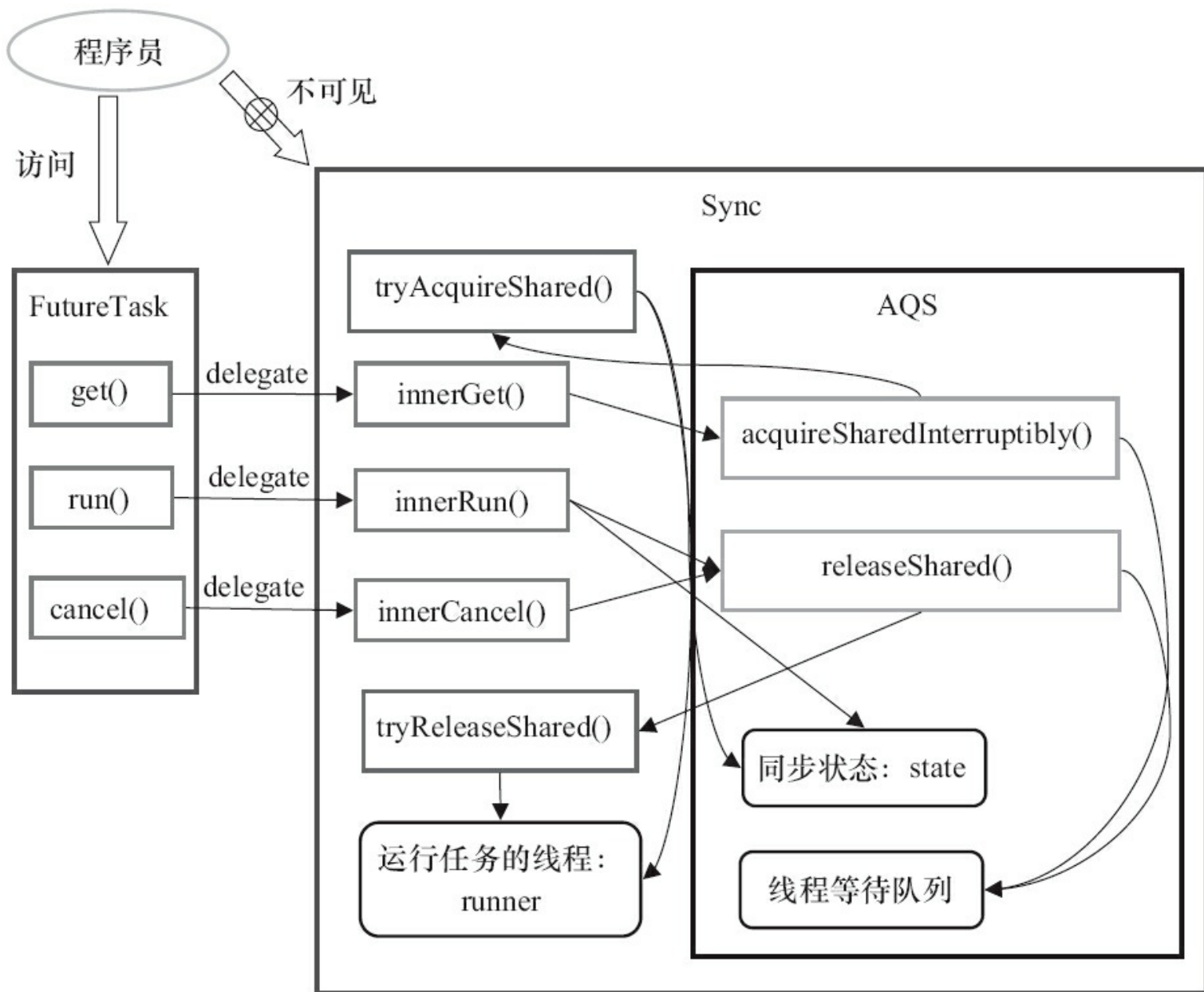


图10-15 FutureTask的设计示意图

如图所示，Sync是FutureTask的内部私有类，它继承自AQS。创建FutureTask时会创建内部私有的成员对象Sync，FutureTask所有的的公有方法都直接委托给了内部私有的Sync。

FutureTask.get()方法会调用AQS.acquireSharedInterruptibly(int arg)方法，这个方法的执行过程如下。

1) 调用AQS.acquireSharedInterruptibly(int arg)方法，这个方法首先会回调在子类Sync中实现的tryAcquireShared()方法来判断acquire操作是否可以成功。acquire操作可以成功的条件为：state为执行完成状态RAN或已取消状态CANCELLED，且runner不为null。

2) 如果成功则`get()`方法立即返回。如果失败则到线程等待队列中去等待其他线程执行`release`操作。

3) 当其他线程执行`release`操作(比如`FutureTask.run()`或`FutureTask.cancel(...)`)唤醒当前线程后, 当前线程再次执行`tryAcquireShared()`将返回正值1, 当前线程将离开线程等待队列并唤醒它的后继线程(这里会产生级联唤醒的效果, 后面会介绍)。

4) 最后返回计算的结果或抛出异常。

`FutureTask.run()`的执行过程如下。

1) 执行在构造函数中指定的任务(`Callable.call()`)。

2) 以原子方式来更新同步状态(调用`AQS.compareAndSetState(int expect, int update)`, 设置`state`为执行完成状态`RAN`)。如果这个原子操作成功, 就设置代表计算结果的变量`result`的值为`Callable.call()`的返回值, 然后调用`AQS.releaseShared(int arg)`。

3) `AQS.releaseShared(int arg)`首先会回调在子类`Sync`中实现的`tryReleaseShared(arg)`来执行`release`操作(设置运行任务的线程`runner`为`null`, 然后返回`true`); `AQS.releaseShared(int arg)`, 然后唤醒线程等待队列中的第一个线程。

4) 调用`FutureTask.done()`。

当执行`FutureTask.get()`方法时, 如果`FutureTask`不是处于执行完成状态`RAN`或已取消状态`CANCELLED`, 当前执行线程将到`AQS`的线程等待队列中等待(见下图的线程A、B、C和D)。当某个线程执行`FutureTask.run()`方法或`FutureTask.cancel(...)`方法时, 会唤醒线程等待队列的第一个线程(见图10-16所示的线程E唤醒线程A)。

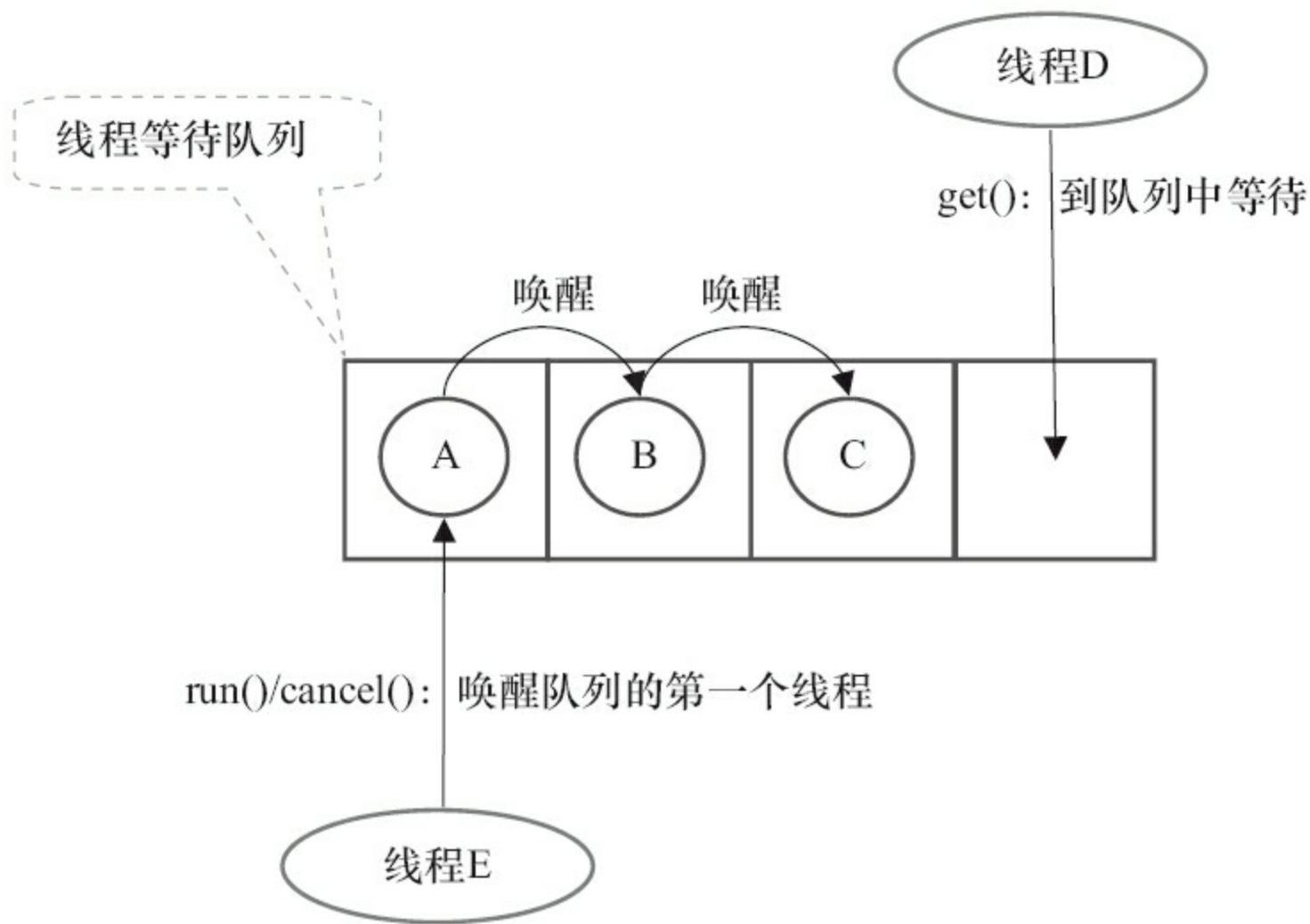


图10-16 FutureTask的级联唤醒示意图

假设开始时FutureTask处于未启动状态或已启动状态，等待队列中已经有3个线程(A、B和C)在等待。此时，线程D执行get()方法将导致线程D也到等待队列中去等待。

当线程E执行run()方法时，会唤醒队列中的第一个线程A。线程A被唤醒后，首先把自己从队列中删除，然后唤醒它的后继线程B，最后线程A从get()方法返回。线程B、C和D重复A线程的处理流程。最终，在队列中等待的所有线程都被级联唤醒并从get()方法返回。

10.5 本章小结

本章介绍了Executor框架的整体结构和成员组件。希望读者阅读本章之后，能够对Executor框架有一个比较深入的理解，同时也希望本章内容有助于读者更熟练地使用Executor框架。

第11章 Java并发编程实践

当你在进行并发编程时，看着程序的执行速度在自己的优化下运行得越来越快，你会觉得越来越有成就感，这就是并发编程的魅力。但与此同时，并发编程产生的问题和风险可能也会随之而来。本章先介绍几个并发编程的实战案例，然后再介绍如何排查并发编程造成的问题。

11.1 生产者和消费者模式

在并发编程中使用生产者和消费者模式能够解决绝大多数并发问题。该模式通过平衡生产线程和消费线程的工作能力来提高程序整体处理数据的速度。

在线程世界里，生产者就是生产数据的线程，消费者就是消费数据的线程。在多线程开发中，如果生产者处理速度很快，而消费者处理速度很慢，那么生产者就必须等待消费者处理完，才能继续生产数据。同样的道理，如果消费者的处理能力大于生产者，那么消费者就必须等待生产者。为了解决这种生产消费能力不均衡的问题，便有了生产者和消费者模式。

什么是生产者和消费者模式

生产者和消费者模式是通过一个容器来解决生产者和消费者的强耦合问题。生产者和消费者彼此之间不直接通信，而是通过阻塞队列来进行通信，所以生产者生产完数据之后不用等待消费者处理，直接扔给阻塞队列，消费者不找生产者要数据，而是直接从阻塞队列里取，阻塞队列就相当于一个缓冲区，平衡了生产者和消费者的处理能力。

这个阻塞队列就是用来给生产者和消费者解耦的。纵观大多数设计模式，都会找一个第三者出来进行解耦，如工厂模式的第三者是工厂类，模板模式的第三者是模板类。在学习一些设计模式的过程中，先找到这个模式的第三者，能帮助我们快速熟悉一个设计模式。

11.1.1 生产者消费者模式实战

我和同事一起利用业余时间开发的Yuna工具中使用了生产者和消费者模式。我先介绍下Yuna^[1]工具，在阿里巴巴很多同事都喜欢通过邮件分享技术文章，因为通过邮件分享很方便，大家在网上看到好的技术文章，执行复制→粘贴→发送就完成了一次分享，但是我发现技术文章不能沉淀下来，新来的同事看不到以前分享的技术文章，大家也很难找到以前分享过的技术文章。为了解决这个问题，我们开发了一个Yuna工具。

我们申请了一个专门用来收集分享邮件的邮箱，比如share@alibaba.com，大家将分享的文章发送到这个邮箱，让大家每次都抄送到这个邮箱肯定很麻烦，所以我们的做法是将这个邮箱地址放在部门邮件列表里，所以分享的同事只需要和以前一样向整个部门分享文章就行。Yuna工具通过读取邮件服务器里该邮箱的邮件，把所有分享的邮件下载下来，包括邮件的附件、图片和邮件回复。因为我们可能会从这个邮箱里下载到一些非分享的文章，所以我们要求分享的邮件标题必须带有一个关键字，比如“内贸技术分享”。下载完邮件之后，通过confluence的Web Service接口，把文章插入到confluence里，这样新同事就可以在confluence里看以前分享过的文章了，并且Yuna工具还可以自动把文章进行分类和归档。

为了快速上线该功能，当时我们花了3天业余时间快速开发了Yuna 1.0版本。在1.0版本中并没有使用生产者消费模式，而是使用单线程来处理，因为当时只需要处理我们一个部门的邮件，所以单线程明显够用，整个过程是串行执行的。在一个线程里，程序先抽取全部的邮件，转化为文章对象，然后添加全部的文章，最后删除抽取过的邮件。代码如下。

```
public void extract() {
    logger.debug("开始" + getExtractorName() + "。。");
    // 抽取邮件
    List<Article> articles = extractEmail();
    // 添加文章
    for (Article article : articles) {
        addArticleOrComment(article);
    }
    // 清空邮件
    cleanEmail();
    logger.debug("完成" + getExtractorName() + "。。");
}
```

```
}
```

Yuna工具在推广后，越来越多的部门使用这个工具，处理的时间越来越慢，Yuna是每隔5分钟进行一次抽取的，而当邮件多的时候一次处理可能就花了几分钟，于是我在Yuna 2.0版本里使用了生产者消费者模式来处理邮件，首先生产者线程按一定的规则去邮件系统里抽取邮件，然后存放在阻塞队列里，消费者从阻塞队列里取出文章后插入到confluence里。代码如下。

```
public class QuickEmailToWikiExtractor extends AbstractExtractor {
    private ThreadPoolExecutor      threadsPool;
    private ArticleBlockingQueue<ExchangeEmailShallowDTO> emailQueue;
    public QuickEmailToWikiExtractor() {
        emailQueue= new ArticleBlockingQueue<ExchangeEmailShallowDTO>();
        int corePoolSize = Runtime.getRuntime().availableProcessors() * 2;
        threadsPool = new ThreadPoolExecutor(corePoolSize, corePoolSize, 10l, TimeUnit
            SECONDS,
            new LinkedBlockingQueue<Runnable>(2000));
    }
    public void extract() {
        logger.debug("开始" + getExtractorName() + "。。");
        long start = System.currentTimeMillis();
        // 抽取所有邮件放到队列里
        new ExtractEmailTask().start();
        // 把队列里的文章插入到Wiki
        insertToWiki();
        long end = System.currentTimeMillis();
        double cost = (end - start) / 1000;
        logger.debug("完成" + getExtractorName() + ",花费时间:" + cost + "秒");
    }
    /**
     * 把队列里的文章插入到Wiki
     */
    private void insertToWiki() {
        // 登录Wiki, 每间隔一段时间需要登录一次
        confluenceService.login(RuleFactory.USER_NAME, RuleFactory.PASSWORD);
        while (true) {
            // 2秒内取不到就退出
            ExchangeEmailShallowDTO email = emailQueue.poll(2, TimeUnit.SECONDS);
            if (email == null) {
                break;
            }
            threadsPool.submit(new insertToWikiTask(email));
        }
    }
    protected List<Article> extractEmail() {
        List<ExchangeEmailShallowDTO> allEmails = getEmailService().queryAllEmails();
        if (allEmails == null) {
            return null;
        }
        for (ExchangeEmailShallowDTO exchangeEmailShallowDTO : allEmails) {
            emailQueue.offer(exchangeEmailShallowDTO);
        }
    }
}
```

```
    }  
    return null;  
}  
/**  
 * 抽取邮件任务  
 *  
 * @author tengfei.fangtf  
 */  
public class ExtractEmailTask extends Thread {  
    public void run() {  
        extractEmail();  
    }  
}  
}
```

代码的执行逻辑是，生产者启动一个线程把所有邮件全部抽取到队列中，消费者启动CPU*2个线程数处理邮件，从之前的单线程处理邮件变成了现在的多线程处理，并且抽取邮件的线程不需要等处理邮件的线程处理完再抽取新邮件，所以使用了生产者和消费者模式后，邮件的整体处理速度比以前要快了几倍。

[1] Yuna取名自我非常喜欢的一款RPG游戏《最终幻想》中女主角的名字。

11.1.2 多生产者和多消费者场景

在多核时代，多线程并发处理速度比单线程处理速度更快，所以可以使用多个线程来生产数据，同样可以使用多个消费线程来消费数据。而更复杂的情况是，消费者消费的数据，有可能需要继续处理，于是消费者处理完数据之后，它又要作为生产者把数据放在新的队列里，交给其他消费者继续处理，如图11-1所示。



图11-1 多生产者消费者模式

我们在一个长连接服务器中使用了这种模式，生产者1负责将所有客户端发送的消息存放在阻塞队列1里，消费者1从队列里读消息，然后通过消息ID进行散列得到N个队列中的一个，然后根据编号将消息存放在到不同的队列里，每个阻塞队列会分配一个线程来消费阻塞队列

里的数据。如果消费者2无法消费消息, 就将消息再抛回到阻塞队列1中, 交给其他消费者处理。

以下是消息总队列的代码。

```
/**
 * 总消息队列管理
 *
 * @author tengfei.fangtf
 */
public class MsgQueueManager implements IMsgQueue{
    private static final Logger          LOGGER
= LoggerFactory.getLogger(MsgQueueManager.class);
    /**
     * 消息总队列
     */
    public final BlockingQueue<Message> messageQueue;
    private MsgQueueManager() {
        messageQueue = new LinkedTransferQueue<Message>();
    }
    public void put(Message msg) {
        try {
            messageQueue.put(msg);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    public Message take() {
        try {
            return messageQueue.take();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        return null;
    }
}
```

启动一个消息分发线程。在这个线程里子队列自动去总队列里获取消息。

```
/**
 * 分发消息, 负责把消息从大队列塞到小队列里
 *
 * @author tengfei.fangtf
 */
static class DispatchMessageTask implements Runnable {
    @Override
    public void run() {
        BlockingQueue<Message> subQueue;
        for (;;) {
```

```

// 如果没有数据, 则阻塞在这里
Message msg = MsgQueueFactory.getMessageQueue().take();
// 如果为空, 则表示没有Session机器连接上来,
// 需要等待, 直到有Session机器连接上来
while ((subQueue = getInstance().getSubQueue()) == null) {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
// 把消息放到小队列里
try {
    subQueue.put(msg);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}
}
}

```

使用散列(hash)算法获取一个子队列, 代码如下。

```

/**
 * 均衡获取一个子队列。
 *
 * @return
 */
public BlockingQueue<Message> getSubQueue() {
    int errorCount = 0;
    for (;;) {
        if (subMsgQueues.isEmpty()) {
            return null;
        }
        int index = (int) (System.nanoTime() % subMsgQueues.size());
        try {
            return subMsgQueues.get(index);
        } catch (Exception e) {
            // 出现错误表示, 在获取队列大小之后, 队列进行了一次删除操作
            LOGGER.error("获取子队列出现错误", e);
            if ((++errorCount) < 3) {
                continue;
            }
        }
    }
}
}

```

使用的时候, 只需要往总队列里发消息。

```

// 往消息队列里添加一条消息

```



```
    IMessageQueue messageQueue = MsgQueueFactory.getMessageQueue();
    Packet msg = Packet.createPacket(Packet64FrameType.
        TYPE_DATA, "{}".getBytes(), (short) 1);
    messageQueue.put(msg);
```

11.1.3 线程池与生产者消费者模式

Java中的线程池类其实就是一种生产者和消费者模式的实现方式，但是我觉得其实现方式更加高明。生产者把任务丢给线程池，线程池创建线程并处理任务，如果将要运行的任务数大于线程池的基本线程数就把任务扔到阻塞队列里，这种做法比只使用一个阻塞队列来实现生产者和消费者模式显然要高明很多，因为消费者能够处理直接就处理掉了，这样速度更快，而生产者先存，消费者再取这种方式显然慢一些。

我们的系统也可以使用线程池来实现多生产者和消费者模式。例如，创建N个不同规模的Java线程池来处理不同性质的任务，比如线程池1将数据读到内存之后，交给线程池2里的线程继续处理压缩数据。线程池1主要处理IO密集型任务，线程池2主要处理CPU密集型任务。

本节讲解了生产者和消费者模式，并给出了实例。读者可以在平时的工作中思考一下哪些场景可以使用生产者消费者模式，我相信这种场景应该非常多，特别是需要处理任务时间比较长的场景，比如上传附件并处理，用户把文件上传到系统后，系统把文件丢到队列里，然后立刻返回告诉用户上传成功，最后消费者再去队列里取出文件处理。再如，调用一个远程接口查询数据，如果远程服务接口查询时需要几十秒的时间，那么它可以提供一个申请查询的接口，这个接口把要申请查询任务放数据库中，然后该接口立刻返回。然后服务器端用线程轮询并获取申请任务进行处理，处理完之后发消息给调用方，让调用方再来调用另外一个接口取数据。

11.2 线上问题定位

有时候, 有很多问题只有在线上或者预发环境才能发现, 而线上又不能调试代码, 所以线上问题定位就只能看日志、系统状态和dump线程, 本节只是简单地介绍一些常用的工具, 以帮助大家定位线上问题。

1) 在Linux命令行下使用TOP命令查看每个进程的情况, 显示如下。

```
top - 22:27:25 up 463 days, 12:46, 1 user, load average: 11.80, 12.19, 11.79
Tasks: 113 total, 5 running, 108 sleeping, 0 stopped, 0 zombie
Cpu(s): 62.0%us, 2.8%sy, 0.0%ni, 34.3%id, 0.0%wa, 0.0%hi, 0.7%si, 0.2%st
Mem: 7680000k total, 7665504k used, 14496k free, 97268k buffers
Swap: 2096472k total, 14904k used, 2081568k free, 3033060k cached
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
31177 admin 18 0 5351m 4.0g 49m S 301.4 54.0 935:02.08 java
31738 admin 15 0 36432 12m 1052 S 8.7 0.2 11:21.05 nginx-proxy
```

我们的程序是Java应用, 所以只需要关注COMMAND是Java的性能数据, COMMAND表示启动当前进程的命令, 在Java进程这一行里可以看到CPU利用率是300%, 不用担心, 这个是当前机器所有核加在一起的CPU利用率。

2) 再使用top的交互命令数字1查看每个CPU的性能数据。

```
top - 22:24:50 up 463 days, 12:43, 1 user, load average: 12.55, 12.27, 11.73
Tasks: 110 total, 3 running, 107 sleeping, 0 stopped, 0 zombie
Cpu0 : 72.4%us, 3.6%sy, 0.0%ni, 22.7%id, 0.0%wa, 0.0%hi, 0.7%si, 0.7%st
Cpu1 : 58.7%us, 4.3%sy, 0.0%ni, 34.3%id, 0.0%wa, 0.0%hi, 2.3%si, 0.3%st
Cpu2 : 53.3%us, 2.6%sy, 0.0%ni, 34.1%id, 0.0%wa, 0.0%hi, 9.6%si, 0.3%st
Cpu3 : 52.7%us, 2.7%sy, 0.0%ni, 25.2%id, 0.0%wa, 0.0%hi, 19.5%si, 0.0%st
Cpu4 : 59.5%us, 2.7%sy, 0.0%ni, 31.2%id, 0.0%wa, 0.0%hi, 6.6%si, 0.0%st
Mem: 7680000k total, 7663152k used, 16848k free, 98068k buffers
Swap: 2096472k total, 14904k used, 2081568k free, 3032636k cached
```

命令行显示了CPU4, 说明这是一个5核的虚拟机, 平均每个CPU利用率在60%以上。如果这里显示CPU利用率100%, 则很有可能程序里写了一个死循环。这些参数的含义, 可以对比表11-1来查看。

表11-1 CPU参数含义

参 数	描 述
us	用户空间占用 CPU 百分比
1.0% sy	内核空间占用 CPU 百分比
0.0% ni	用户进程空间内改变过优先级的进程占用 CPU 百分比
98.7% id	空闲 CPU 百分比
0.0% wa	等待输入 / 输出的 CPU 时间百分比

3)使用top的交互命令H查看每个线程的性能信息。

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31558	admin	15	0	5351m	4.0g	49m	S	12.2	54.0	10:08.31	java
31561	admin	15	0	5351m	4.0g	49m	R	12.2	54.0	9:45.43	java
31626	admin	15	0	5351m	4.0g	49m	S	11.9	54.0	13:50.21	java
31559	admin	15	0	5351m	4.0g	49m	S	10.9	54.0	5:34.67	java
31612	admin	15	0	5351m	4.0g	49m	S	10.6	54.0	8:42.77	java
31555	admin	15	0	5351m	4.0g	49m	S	10.3	54.0	13:00.55	java
31630	admin	15	0	5351m	4.0g	49m	R	10.3	54.0	4:00.75	java
31646	admin	15	0	5351m	4.0g	49m	S	10.3	54.0	3:19.92	java
31653	admin	15	0	5351m	4.0g	49m	S	10.3	54.0	8:52.90	java
31607	admin	15	0	5351m	4.0g	49m	S	9.9	54.0	14:37.82	java

在这里可能会出现3种情况。

·第一种情况，某个线程CPU利用率一直100%，则说明是这个线程有可能有死循环，那么请记住这个PID。

·第二种情况，某个线程一直在TOP 10的位置，这说明这个线程可能有性能问题。

·第三种情况，CPU利用率高的几个线程在不停变化，说明并不是由某一个线程导致CPU偏高。

如果是第一种情况，也有可能是GC造成，可以用jstat命令看一下GC情况，看看是不是因为持久代或年老代满了，产生Full GC，导致CPU利用率持续飙高，命令和回显如下。

```
sudo /opt/java/bin/jstat -gcutil 31177 1000 5
S0 S1 E O P YGC YGCT FGC FGCT GCT
```

```
0.00 1.27 61.30 55.57 59.98 16040 143.775 30 77.692 221.467
0.00 1.27 95.77 55.57 59.98 16040 143.775 30 77.692 221.467
1.37 0.00 33.21 55.57 59.98 16041 143.781 30 77.692 221.474
1.37 0.00 74.96 55.57 59.98 16041 143.781 30 77.692 221.474
0.00 1.59 22.14 55.57 59.98 16042 143.789 30 77.692 221.481
```

还可以把线程dump下来，看看究竟是哪个线程、执行什么代码造成的CPU利用率高。执行以下命令，把线程dump到文件dump17里。执行如下命令。

```
sudo -u admin /opt/taobao/java/bin/jstack 31177 > /home/tengfei.fangtf/dump17
```

dump出来内容的类似下面内容。

```
"http-0.0.0.0-7001-97" daemon prio=10 tid=0x000000004f6a8000 nid=0x555e in Object.
wait() [0x0000000052423000]
  java.lang.Thread.State: WAITING (on object monitor)
    at java.lang.Object.wait(Native Method)
    - waiting on  (a org.apache.tomcat.util.net.AprEndpoint$Worker)
    at java.lang.Object.wait(Object.java:485)
    at org.apache.tomcat.util.net.AprEndpoint$Worker.await(AprEndpoint.java:1464)
    - locked  (a org.apache.tomcat.util.net.AprEndpoint$Worker)
    at org.apache.tomcat.util.net.AprEndpoint$Worker.run(AprEndpoint.java:1489)
    at java.lang.Thread.run(Thread.java:662)
```

dump出来的线程ID(nid)是十六进制的，而我们用TOP命令看到的线程ID是十进制的，所以要用printf命令转换一下进制。然后用十六进制的ID去dump里找到对应的线程。

```
printf "%x\n" 31558
```

输出:7b46。

11.3 性能测试

因为要支持某个业务，有同事向我们提出需求，希望系统的某个接口能够支持2万的QPS，因为我们的应用部署在多台机器上，要支持两万的QPS，我们必须先要知道该接口在单机上能支持多少QPS，如果单机能支持1千QPS，我们需要20台机器才能支持2万的QPS。需要注意的是，要支持的2万的QPS必须是峰值，而不能是平均值，比如一天当中有23个小时QPS不足1万，只有一个小时的QPS达到了2万，我们的系统也要支持2万的QPS。

我们先进行性能测试。我们使用公司同事开发的性能测试工具进行测试，该工具的原理是，用户写一个Java程序向服务器端发起请求，这个工具会启动一个线程池来调度这些任务，可以配置同时启动多少个线程、发起请求次数和任务间隔时长。将这个程序部署在多台机器上执行，统计出QPS和响应时长。我们在10台机器上部署了这个测试程序，每台机器启动了100个线程进行测试，压测时长为半小时。注意不能压测线上机器，我们压测的是开发服务器。

测试开始后，首先登录到服务器里查看当前有多少台机器在压测服务器，因为程序的端口是12200，所以使用netstat命令查询有多少台机器连接到这个端口上。命令如下。

```
$ netstat -nat | grep 12200 -c
10
```

通过这个命令可以知道已经有10台机器在压测服务器。QPS达到了1400，程序开始报错获取不到数据库连接，因为我们的数据库端口是3306，用netstat命令查看已经使用了多少个数据库连接。命令如下。

```
$ netstat -nat | grep 3306 -c
12
```

增加数据库连接到20，QPS没上去，但是响应时长从平均1000毫秒下降到700毫秒，使用TOP命令观察CPU利用率，发现已经90%多了，于是升级CPU，将2核升级成4核，和线上的机器

保持一致。再进行压测，CPU利用率下去了达到了75%，QPS上升到了1800。执行一段时间后响应时长稳定在200毫秒。

增加应用服务器里线程池的核心线程数和最大线程数到1024，通过ps命令查看下线程数是否增长了，执行的命令如下。

```
$ ps -eLf | grep java -c
1520
```

再次压测，QPS并没有明显的增长，单机QPS稳定在1800左右，响应时长稳定在200毫秒。

我在性能测试之前先优化了程序的SQL语句。使用了如下命令统计执行最慢的SQL，左边的是执行时长，单位是毫秒，右边的是执行的语句，可以看到系统执行最慢的SQL是queryNews和queryNewIds，优化到几十毫秒。

```
$ grep Y /home/admin/logs/xxx/monitor/dal-rw-monitor.log |awk -F',' '{print $7$5}' |
sort -nr|head -20
1811 queryNews
1764 queryNews
1740 queryNews
1697 queryNews
679 queryNewIds
```

性能测试中使用的其他命令

1)查看网络流量。

```
$ cat /proc/net/dev
Inter-| Receive | Transmit
face |bytes packets errs drop fifo frame compressed multicast|bytes packets
errs drop fifo colls carrier compressed
lo:242953548208 231437133 0 0 0 0 0 0 242953548208 231437133 0 0 0 0 0 0
eth0:153060432504 446365779 0 0 0 0 0 0 108596061848 479947142 0 0 0 0 0 0
bond0:153060432504 446365779 0 0 0 0 0 0 108596061848 479947142 0 0 0 0 0 0
```

2)查看系统平均负载。

```
$ cat /proc/loadavg
0.00 0.04 0.85 1/1266 22459
```

3)查看系统内存情况。

```
$ cat /proc/meminfo
MemTotal: 4106756 kB
MemFree: 71196 kB
Buffers: 12832 kB
Cached: 2603332 kB
SwapCached: 4016 kB
Active: 2303768 kB
Inactive: 1507324 kB
Active(anon): 996100 kB
部分省略
```

4)查看CPU的利用率。

```
cat /proc/stat
cpu 167301886 6156 331902067 17552830039 8645275 13082 1044952 33931469 0
cpu0 45406479 1992 75489851 4410199442 7321828 12872 688837 5115394 0
cpu1 39821071 1247 132648851 4319596686 379255 67 132447 11365141 0
cpu2 40912727 1705 57947971 4418978718 389539 78 110994 8342835 0
cpu3 41161608 1211 65815393 4404055191 554651 63 112672 9108097 0
```


11.4 异步任务池

Java中的线程池设计得非常巧妙，可以高效并发执行多个任务，但是在某些场景下需要对线程池进行扩展才能更好地服务于系统。例如，如果一个任务仍进线程池之后，运行线程池的程序重启了，那么线程池里的任务就会丢失。另外，线程池只能处理本机的任务，在集群环境下不能有效地调度所有机器的任务。所以，需要结合线程池开发一个异步任务处理池。图11-2为异步任务池设计图。

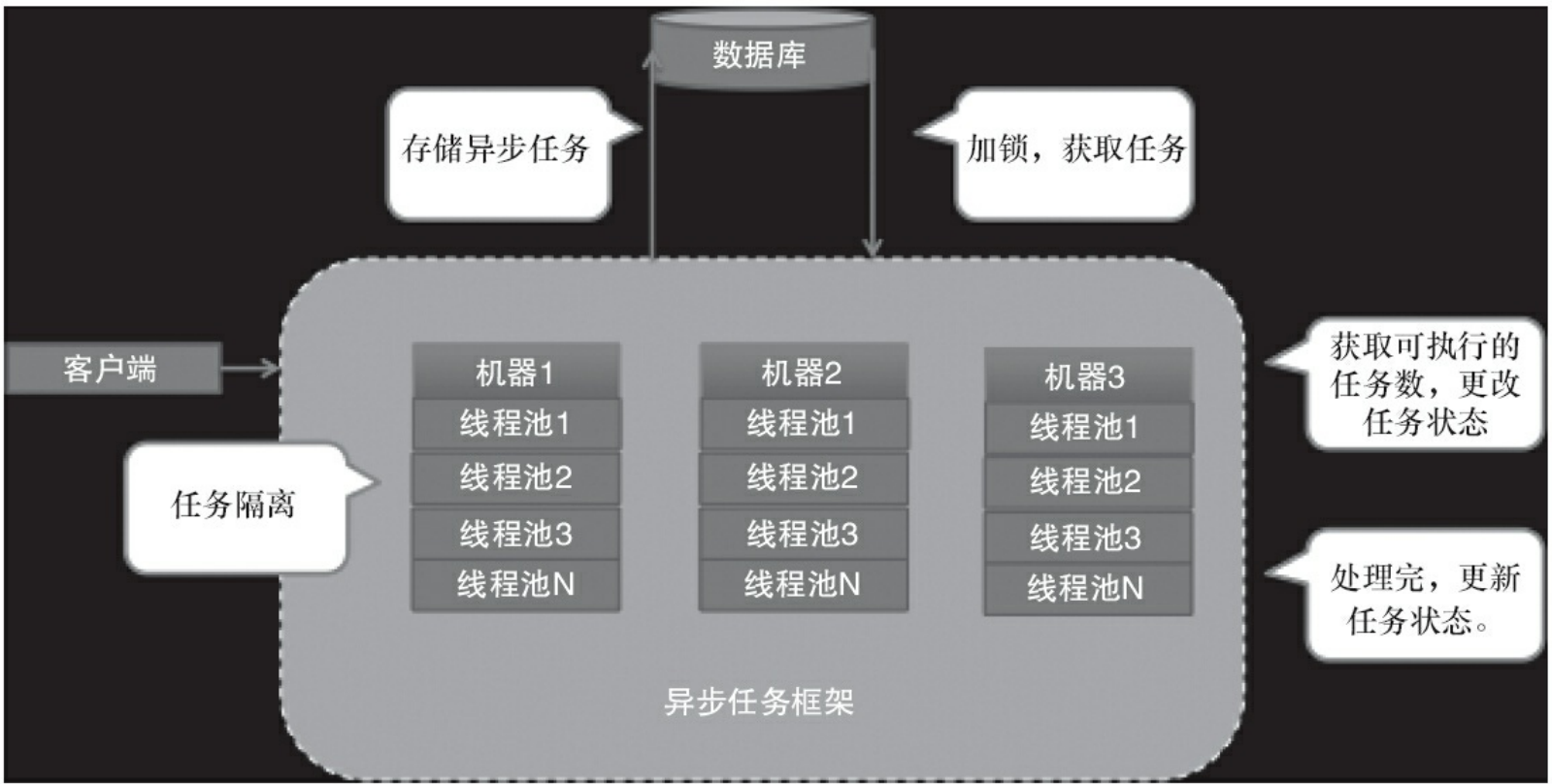


图11-2 异步任务池设计图

任务池的主要处理流程是，每台机器会启动一个任务池，每个任务池里有多个线程池，当某台机器将一个任务交给任务池后，任务池会先将这个任务保存到数据中，然后某台机器上的任务池会从数据库中获取待执行的任务，再执行这个任务。

每个任务有几种状态，分别是创建(NEW)、执行中(EXECUTING)、RETRY(重试)、挂起(SUSPEND)、中止(TEMINER)和执行完成(FINISH)。

·创建:提交给任务池之后的状态。

·执行中:任务池从数据库中拿到任务执行时的状态。

·重试:当执行任务时出现错误,程序显式地告诉任务池这个任务需要重试,并设置下一次执行时间。

·挂起:当一个任务的执行依赖于其他任务完成时,可以将这个任务挂起,当收到消息后,再开始执行。

·中止:任务执行失败,让任务池停止执行这个任务,并设置错误消息告诉调用端。

·执行完成:任务执行结束。

任务池的任务隔离。异步任务有很多种类型,比如抓取网页任务、同步数据任务等,不同类型的任务优先级不一样,但是系统资源是有限的,如果低优先级的任务非常多,高优先级的任务就可能得不到执行,所以必须对任务进行隔离执行。使用不同的线程池处理不同的任务,或者不同的线程池处理不同优先级的任务,如果任务类型非常少,建议用任务类型来隔离,如果任务类型非常多,比如几十个,建议采用优先级的方式来隔离。

任务池的重试策略。根据不同的任务类型设置不同的重试策略,有的任务对实时性要求高,那么每次的重试间隔就会非常短,如果对实时性要求不高,可以采用默认的重试策略,重试间隔随着次数的增加,时间不断增长,比如间隔几秒、几分钟到几小时。每个任务类型可以设置执行该任务类型线程池的最小和最大线程数、最大重试次数。

使用任务池的注意事项。任务必须无状态:任务不能在执行任务的机器中保存数据,比如某个任务是处理上传的文件,任务的属性里有文件的上传路径,如果文件上传到机器1,机器2获取到了任务则会处理失败,所以上传的文件必须存在其他的集群里,比如OSS或SFTP。

异步任务的属性。包括任务名称、下次执行时间、已执行次数、任务类型、任务优先级和

执行时的报错信息(用于快速定位问题)。

11.5 本章小结

本章介绍了使用生产者和消费者模式进行并发编程、线上问题排查手段和性能测试实战，以及异步任务池的设计。并发编程的实战需要大家平时多使用和测试，才能在项目中发挥作用。